

Introduction to Computer Architecture

Chapter 1



Introduction to Computer Architecture

- ★ Why should we take this class?
- ★ What is Computer Architecture?
- ★ Instruction Set Architecture
- ★ Computer Organization
- ★ Design Tradeoffs/Design Goals
- ★ Requirements and Technology Trends
- ★ Cost and Trends
- ★ How to design?



Why should we take this class?

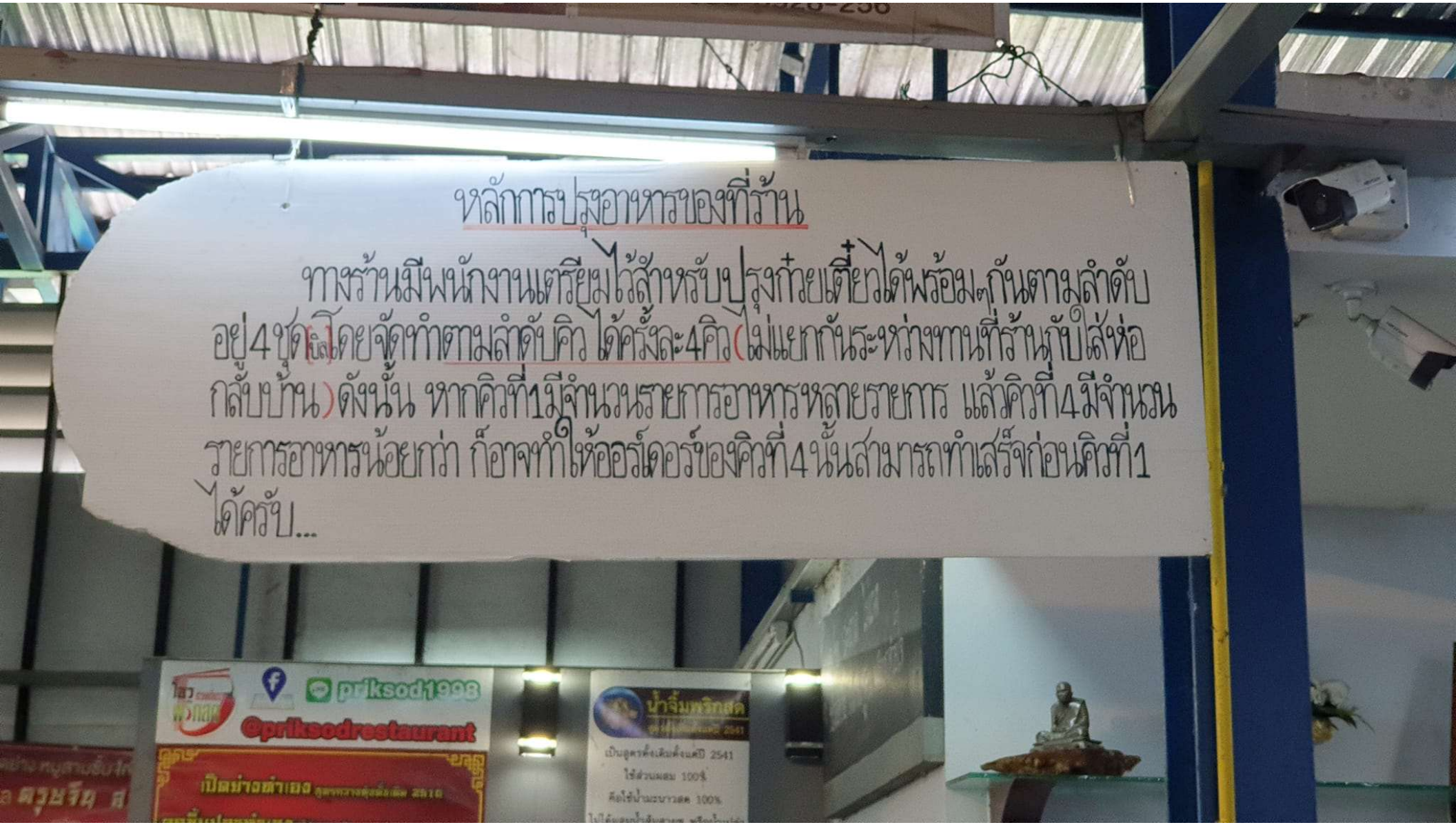
- ★ To design the next great architecture? …. Well … Maybe ….
  - Architecture (especially desktop/server/laptop) has largely converged.
  - Dictated by big name in the market
  - How many of you will be an architect at the processor companies? (e.g. Intel, AMD, ARM)
- ★ Current Instruction set architecture (ISA) abstractions include enormous organizational innovation
  - Differences in Intel Core I3, I5, and I7 are organizations (number of cores, Cache, clock/Turbo boost, Hyper-Threading).



Why should we take this class? (ctd)

- ★ We are Computer Engineering.
  Design, Analysis, Implementation concepts are vital to us.
- ★ To deal with systems design challenges.
  (To be able to choose appropriate architecture.)

- ★ To be a better developer.


หลักการปรุงอาหารของที่ร้าน
ทางร้านมีพนักงานเตรียมไว้สำหรับปรุงก๋วยเตี๋ยวได้พร้อมๆกันตามลำดับ
อยู่4ชุด(ที่)โดยจัดทำตามลำดับคิว ได้ครั้งละ4คิว(ไม่แยกกันระหว่างทานที่ร้านกับใส่ห่อ
กลับบ้าน)ดังนั้น หากคิวที่1มีจำนวนรายการอาหารหลายรายการ แล้วคิวที่4มีจำนวน
รายการอาหารน้อยกว่า ก็อาจทำให้ออร์เดอร์ของคิวที่4นั้นสามารถทำเสร็จก่อนคิวที่1
ได้ครับ...
priksod1998
Epriksodrestaurant
น้ำจิ้มพริกสด



Why one program is faster than another?

- ★ Same algorithm and same programming language
- ★ Two programmers (A and B)
- ★ Same machine
- ★ One can run faster.

Why?

- ★ Different data structure?
- ★ Different compiler?
- ★ Different optimization?
- ★ -------- Good knowledge of architecture means better program --------



Which one is faster? Why?

Same program, different data structure

```
unsigned short a[2],b[2],c[2];
unsigned short i=0;
a[0]=10+i;
a[1]=20+i;
b[0]=15+i;
b[1]=25+i;
c[0]=a[0]+b[0];
c[1]=a[1]+b[1];
printf("c1 = %d, c2 = %d\n",c[0],c[1]);
```

56 LOC

```
union i16 {
        unsigned int x;
        unsigned short h[2];
};

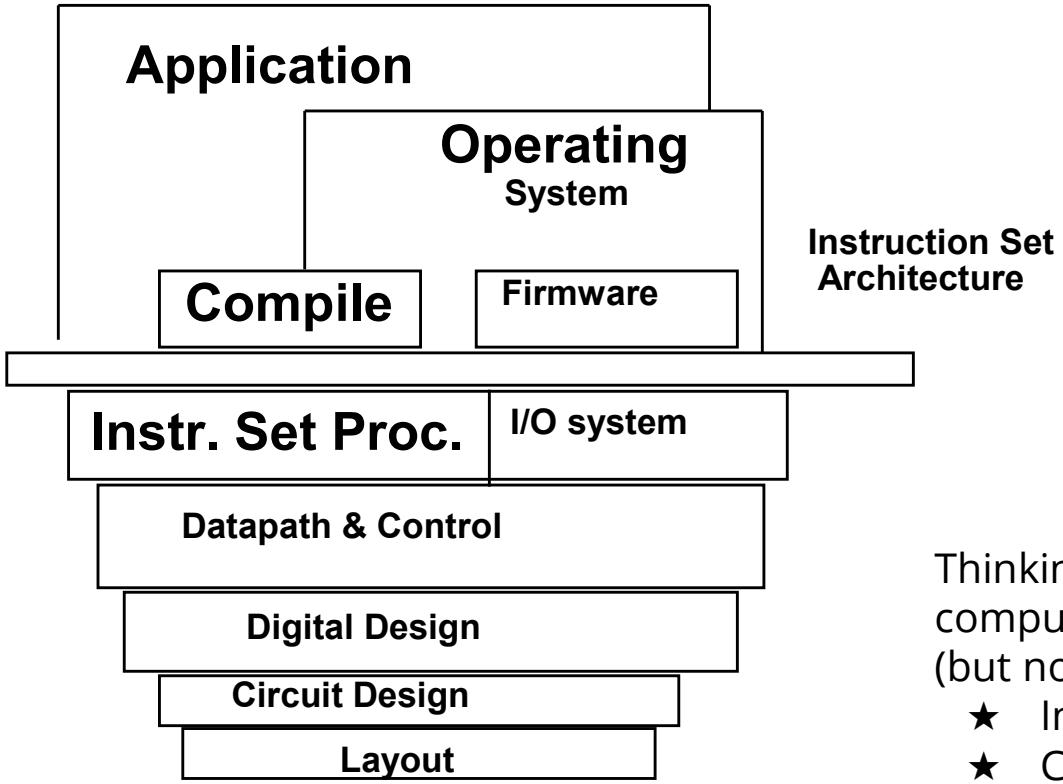
union i16 a;
union i16 b;
union i16 c;
unsigned short i=0;
a.h[0]=10+i;
a.h[1]=20+i;
b.h[0]=15+i;
b.h[1]=25+i;
c.x=a.x+b.x;
printf("c1 = %d, c2 = %d\n",c.h[0],c.h[1]);
```

52 LOC

Depending on version of compilers and operating systems, the result may vary.



What is Computer Architecture?



- ★ Two words
  - Computer
  - Architecture - structure (usually building), concepts
- ★ Many levels of abstraction
- ★ Design, Measurement , and Evaluation

Thinking about a document explaining a computer Architecture. The book should include (but not limited to):

- ★ Instruction Set Architecture
- ★ Computer Organization
- ★ Application Binary Interface



Instruction Set Architecture

- ★ Abstraction of machine for software
  - Nowaday, we usually program in high-level programing languages (e.g. C). Compiler would translate to intermediate code for optimization. Eventually, the code will be translated to binary code.
    Programmers know nothing about underlying hardware.
- ★ Abstraction of software for hardware
  - Legacy 8088 software still works on latest Intel core I7 architecture.

```
int average (int a, int b) {
    return (a+b)/2;
}
```

Compiler

Intermediate/Assembly Code

```
<average>:
   0:   55                      push   %rbp
   1:   48 89 e5                mov    %rsp,%rbp
   4:   89 7d fc                mov    %edi,-0x4(%rbp)
   7:   89 75 f8                mov    %esi,-0x8(%rbp)
   a:   8b 55 fc                mov    -0x4(%rbp),%edx
   d:   8b 45 f8                mov    -0x8(%rbp),%eax
  10:   01 d0                   add    %edx,%eax
  12:   89 c2                   mov    %eax,%edx
  14:   c1 ea 1f                shr    $0x1f,%edx
  17:   01 d0                   add    %edx,%eax
  19:   d1 f8                   sar    %eax
  1b:   5d                      pop    %rbp
  1c:   c3                      retq
```

Assembler

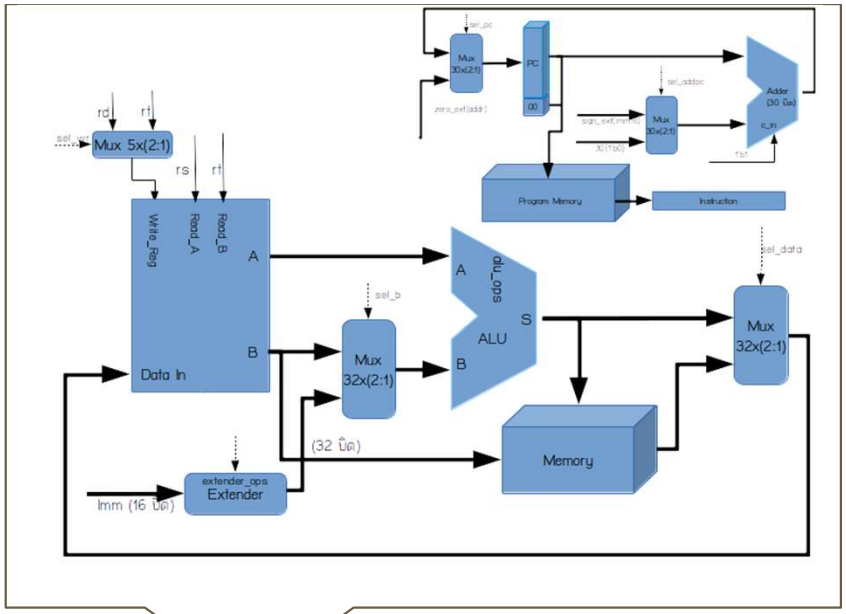
Binary/Machine Code

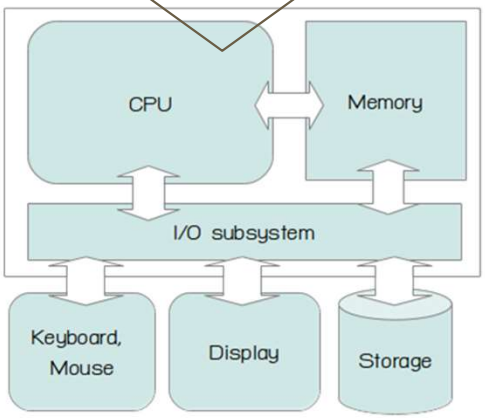
```
5548 89e5 897d fc89 75f8 8b55 fc8b 45f8
01d0 89c2 c1ea 1f01 d0d1 f85d c3
```



Computer Organization

- ★ ALU
- ★ Control Unit
- ★ Register
- ★ Bus
- ★ Memory subsystem
- ★ I/O subsystem
  - ○ Keyboard, Mouse
  - ○ Display
  - ○ Storage
  - ○ ...







Why do we need temporary storage (register)?

- ★ How to do 2 + 3 in grade school?

- ★ Remember/Put 2 in your mind (temporary storage)
- ★ Put up 3 fingers
- ★ Count 3, 4, 5
- ★ The answer is 5.

Does having more registers beneficial for the calculation?



Design Tradeoffs/Design Goals

- ★ Performance ? at any cost ?
- ★ Power ?
- ★ Design Tradeoffs:
  - ○ Cost vs. Performance (vs. Power)
  - ○ Need for modeling and measurement

- ★ Goals
  - ○ Functional requirements
    - ■ Market & application driven
  - ○ Performance Goals
  - ○ Cost constraints
  - ○ Power constraints
  - ○ … (you name it) …
- ★ Involvements
  - ○ ISA (software interface)
  - ○ Organization (CPU internals, memory, buses, ….)
  - ○ Hardware - logic design, packaging, …



Requirements and Technology Trends

- Requirements
  - Application area
    - General purpose
    - Scientific
    - Commercial
    - Multimedia
  - Operating system requirements
    - Memory Management
    - Security, Protection
    - Context switching
    - Interrupts
  - Standards
    - IEEE floating-point ?
    - Bus, I/O (PCI, USB, ...)
    - Support for programming languages
- Time to market ?
- Cost?

- Software Trends
  - Require more memory (2x per year)
    - 1 bit of address
  - Use of compilers
    - (ISA for compiler, not programmer)
    - Optimization
    - Scheduling
- Hardware Trends
  - IC/Transistor technology - density, size, performance
  - DRAM - capacity 4x per 3 years, slow performance improvement
  - Disk - capacity 4x per 3 years



Tradeoffs

★ Power



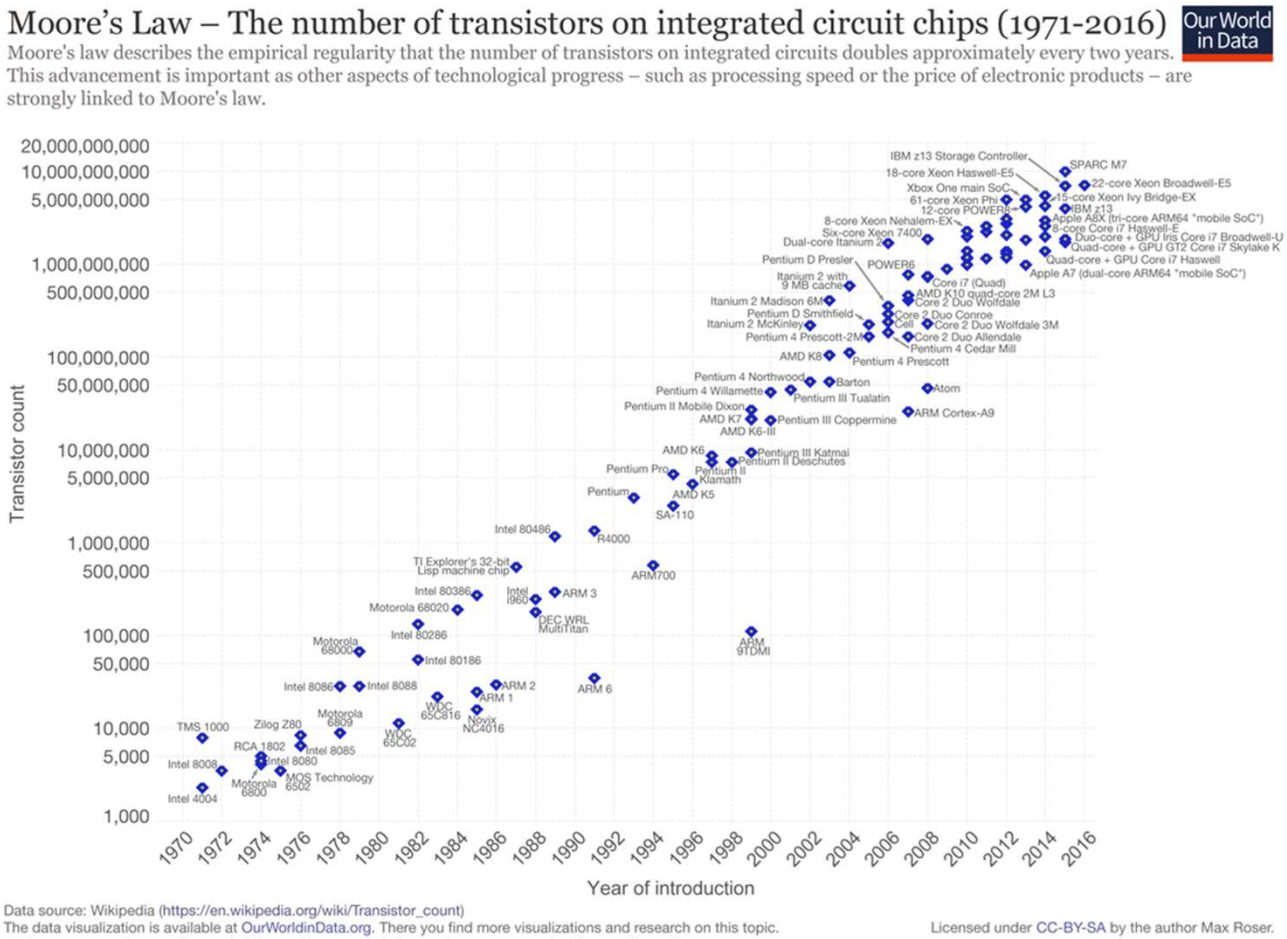


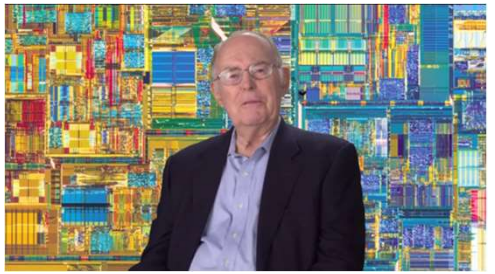
★ Performance





Moore's Law





- ★ An observation by Gordon Moore (the co-founder of Fairchild Semiconductor and CEO of Intel)
- ★ the number of transistors in a dense integrated circuit doubles about every two years (18 months)

Images and charts are taken from wikipedia.



Cost and Trends (DRAM)

- ★ DRAM cost drops 40% per year
- ★ Commoditization lowers the cost

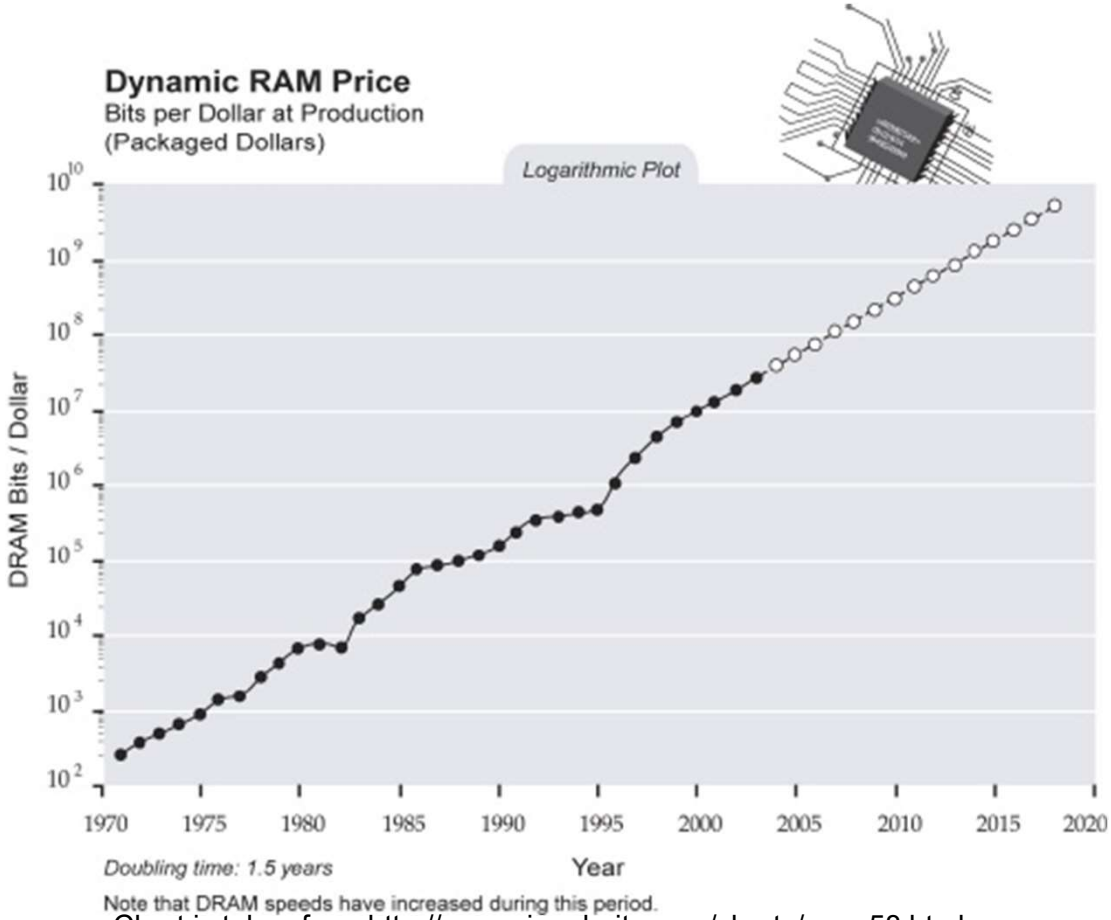


Chart is taken from http://www.singularity.com/charts/page58.html



Cost and Trends (Watts per MIPS)

- Watts per MIPS (Millions Instructions Per Second) is getting better



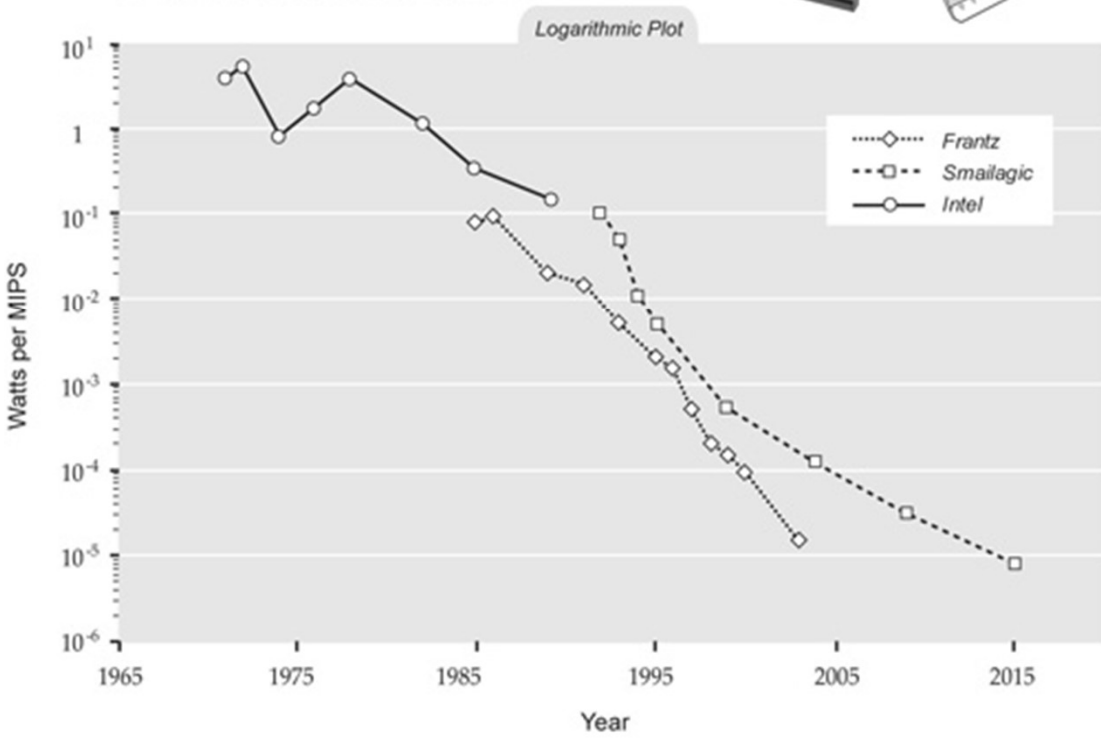


Chart is taken from http://www.singularity.com/charts/page129.html



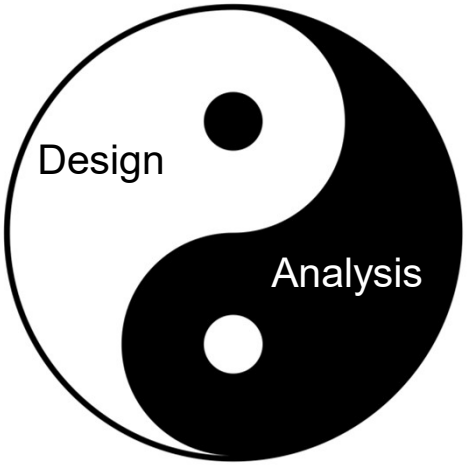
Cost of Components

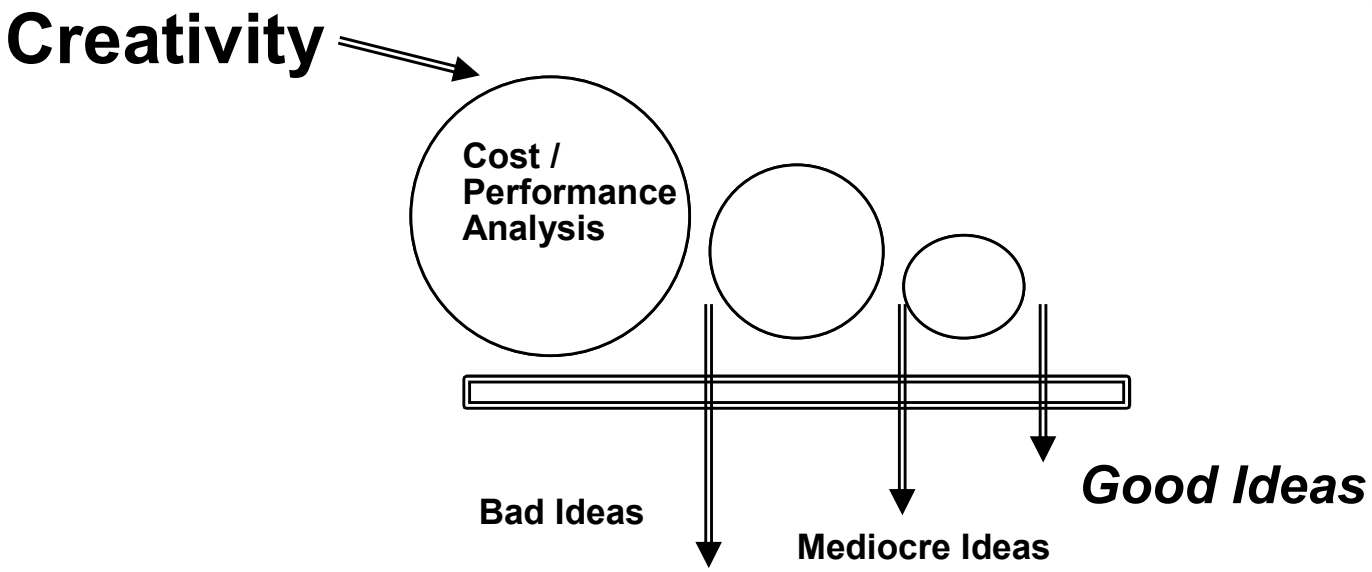
- Price of a notebook computer
  - 6% for Case
  - 25% for Circuit/Processor
  - 10% for RAM
  - 5% for video system
  - 5% for I/O, PCB
  - 4% for keyboard, mouse
  - 20% for monitor
  - 20% for storage
  - 25% for battery
- 20%+ to 30%+ margin



How to design

- ★ Measurement and evaluation
- ★ Iterative process through possible designs







End of Chapter 1





Performance: Measurement and Analysis

Chapter 2



Performance Measurement and Analysis

- ★ Performance
  - ○ Speedup
- ★ Performance Measurement
  - ○ Response Time and Throughput
  - ○ Measurement Issues
- ★ Benchmarks
  - ○ SPEC Benchmark Suites (SPEC.org)
- ★ Performance Comparison
- ★ CPU Time
- ★ Amdahl's Law
- ★ Performance Improvement



Performance

- ★ Performance is usually context dependent (relative).
- ★ Performance is the inverse function of Execution Time.
- ★ How much faster is X than Y?
  - Depending on execution time

$$Performance = \frac{1}{Execution\ Time}$$

$$Speedup(n) = \frac{Performance_X(1/seconds)}{Performance_Y(1/seconds)}$$

$$Speedup(n) = \frac{Performance_x}{Performance_y} = \frac{\frac{1}{Execution\ Time_x}}{\frac{1}{Execution\ Time_y}} = \frac{Execution\ Time_Y}{Execution\ Time_X}$$

Hint:
1.High performance means short time. Low performance means long time.
2. X is n time faster than Y. n >1 means faster. n<1 means slower.



Performance (ctd.)

- ★ Computer A runs a program in 20 seconds.
- ★ Computer B runs a program in 25 seconds.
- ★ How much faster is A than B?

$$Speedup = \frac{[Execution\ Time_Y]}{[Execution\ Time_X]}$$

$$Speedup = \frac{[Execution\ Time_B]}{[Execution\ Time_A]} = \frac{25}{20} = 1.25$$



Performance Measurement

$$Elapse\ Time = CPU\ Time + I/O\ Time + Memory\ Time$$

- ★ Response Time
  - Elapsed Time - Wall clock
  - CPU Time - Only CPU time (without the wait time) … we will revisit this later.
  - From O.S. class, we use: user time, real time, sys time.
- ★ Throughput
  - Tasks per time unit
- ★ Units for commercialization (Not a real performance)
  - MIPS
    - Millions of instructions per second
  - FLOPS
    - Floating Point Operations per second



Response Time

- Timer using clock on the wall (in second)
  - Can be measure by Unix time command.
  - For Windows use ptime (require powershell).

```
$ time gcc -o a.out test.c

real    0m0.180s

user    0m0.040s

sys             0m0.012s
```



Throughput

- ★ Number of tasks that can be completed in a unit of time.
- ★ Can be referred as parallelism at task level.

Computer A performs 5 tasks in 10 minutes. All tasks also finish at the same time.

Throughput 0.5 task per minute

Response Time 10 minutes.

Computer B performs each task in 3 minutes. However, the task must be executed sequentially.

Throughput 0.33 task per minute

Response Time 3 minutes.

Consider transferring 20 TB of data between two building using high-speed fiber-optic cable (1Gb/s), this can take up to 40 hours. However, a man carrying 20x1TB hard drive in a backpack can walk between the building in 15 minutes.

Though fiber-optic cable is faster, but can we say that a man has higher throughput?



Measurement Issues

- ★ What to measure
  - What program(s) to run?
- ★ Interference
  - Loaded / Unloaded jobs on multi-programmed system
- ★ Reproducibility
  - Can the measurement be repeated?
- ★ Comparability
  - A is good at multimedia processor, but is poor at others.
  - B is poor at multimedia processor, but is good at others.
  - Which one is better?



Benchmarks

- ★ Real benchmark - using real applications
  - Scripted application to reproduce interactive or multi-user behaviors
- ★ Microbenchmark - using a part of software
- ★ Kernels - only execute certain aspects of performance
- ★ Toy benchmark - small programs (e.g. quick sort, merge sort). Widely used in the past.
- ★ Synthetics benchmarks - try to mimic behavior of real programs (e.g. scientific workloads)
  - Wheatstone
  - Dhrystone

A unified set is required for comparison.



SPEC Benchmark Suites

- ★ System Performance Evaluation Cooperation (SPEC.org) - An organization that create benchmark suites.
  - SPEC CPU 2017 - for CPU
  - SPECviewpref - for graphics calculation
  - SPECjbb2015 - for java virtual machine
  - SPEC SFS 2014 - for file server
  - .... visit SPEC.org for more details

SPECrate 2017 Integer	SPECspeed 2017 Integer	Language [1]	KLOC [2]	Application Area
500.perlbench_r	600.perlbench_s	C	362	Perl interpreter
502.gcc_r	602.gcc_s	C	1,304	GNU C compiler
505.mcf_r	605.mcf_s	C	3	Route planning
520.omnetpp_r	620.omnetpp_s	C++	134	Discrete Event simulation - computer network
523.xalancbmk_r	623.xalancbmk_s	C++	520	XML to HTML conversion via XSLT
525.x264_r	625.x264_s	C	96	Video compression
531.deepsjeng_r	631.deepsjeng_s	C++	10	Artificial Intelligence: alpha-beta tree search (Chess)
541.leela_r	641.leela_s	C++	21	Artificial Intelligence: Monte Carlo tree search (Go)
548.exchange2_r	648.exchange2_s	Fortran	1	Artificial Intelligence: recursive solution generator (Sudoku)
557.xz_r	657.xz_s	C	33	General data compression



Performance Comparison

- How do we compare machines using collections of execution times?
- Choices
  - Summation
  - Arithmetic Mean/Weighted Arithmetic Mean
  - Harmonic Mean/Weighted Harmonic Mean
  - Geometric Mean/Weighted Geometric Mean

When to use each one?

$$\frac{\sum_{i=1}^{n} x_i}{n}$$ Arithmetic Mean

$$\frac{\sum_{i=1}^{n} w_i x_i}{n}$$ Weighted Arithmetic Mean

$$\sqrt[n]{\prod_{i=1}^{n} x_i}$$ Geometric Mean



(Weighted) Arithmetic Mean

- ★ Which one is the fastest?
- ★ Which one will you buy?

	Computer A (secs)	Computer B (secs)	Computer C (secs)
Program 1 (80%)	10	4	5
Program 2 (20%)	2	10	5
Total	12	14	10
Arithmetic Mean	6	7	5
Weighted Arithmetic Mean	8.4	5.2	5



Geometric Mean and Speed Up

- ★ Since performance measure is context dependent (relative), it is usually measured as a ratio (speed up).
- ★ For a ratio (no unit), Geometric mean should be used.

B is a base line.

	A (secs)	B (secs)	A/B (X)	C (secs)	A/C (Y)	C/B	X/Y
Program 1	10	4	2.5	5	2	1.25	1.25
Program 2	2	10	0.2	5	0.4	0.5	0.5
Arithmetic Mean	6	7	?	5	?	?	?
Geometric Mean			**0.71**		**0.89**	**0.79**	**0.79**

$\sqrt[n]{\prod x_i}$ Geometric Mean



Reporting Benchmark

- ★ With speed up and geometric mean, we can ignore workload.
- ★ For example,
  Improvement from 5 sec to 3 sec
  is equal to
  Improvement from 500 sec to 300 sec.
- ★ Nonetheless, Geometric mean (as well as speed up) cannot predict actual performance.



CPU Time

★ If we ignore the I/O (wait) time, CPU time can be calculated.

$$CPU\ Time = Instruction \times CPI_{average} \times Cycle\ Time$$

$$Cycle\ Time(seconds) = \frac{1}{Clock\ Rate(Hz)}$$

$$CPI_{average} = \sum_{i=1}^{n} CPI_i \times Weight_i$$

CPI = Clock Per Instruction



CPI and Cycle Time

- ★ Processor is based on a clock rate (usually in MHz or GHz). We can easily calculate a period of each clock cycle (Cycle Time).
- ★ For RISC processor, the cycle per instruction is usually fixed (e.g. 1 cycle for each instruction).

$$Cycle\ Time(seconds) = \frac{1}{Clock\ Rate(Hz)}$$



Average CPI

★ If each instruction class has different CPI, we can use weight to find an average CPI.

$$CPI_{average} = \sum_{i=1}^{n} CPI_i \times Weight_i$$

	CPI	Rate
class A	5	30%
class B	3	50%
class C	8	20%

$$CPI_{average} = (5 \times 0.3) + (3 \times 0.5) + (8 \times 0.2)$$

$$\therefore \quad CPI_{average} = 4.6$$



CPU TIME calculation

★ A 1 GHz processor execute 2000 millions instructions in 20 seconds. If the architect can double the clock to 2 GHz in exchange for lower average CPI by a factor of 1.2, how long does it take for this program to execute on a new architecture.

$$Clock\ Cycle\ Time = \frac{1}{(1\times10^{9})}\ seconds$$

$$CPU\ Time\,(seconds) = Instruction \times CPI_{\ average} \times Cycle\ Time$$

$$CPI_{\text{เดิม}} = \frac{(20\ seconds)\times(1\times10^{9}\ seconds\ per\ cycle)}{(2000\times10^{6}\ instructions)}$$

$$CPI_{\text{เดิม}} = 10$$

$$CPI_{\text{ใหม่}} = 10\times1.2 = 12$$

$$CPU\ TIME_{\text{ใหม่}} = \frac{(2000\times10^{6}\times1.2\times10)}{(2\times10^{9})}\ seconds$$

$\therefore$ CPU TIME ใหม่ = 12 วินาที



MIPS

Despite the lower MIPS,
B is faster than A.

- ★ Bad unit for performance measured. Used by several architects.
- ★ Computer A is 10 MIPS running at 33 MHz. Computer B is 9 MIPS running at 20 MHz. If a same program uses 12,000,000 instructions and 10,000,000 instructions on A and B respectively, which computer is faster?

$$Cycle\ Time_A(seconds) = \frac{1}{Clock\ Rate} = \frac{1}{33\times10^6(Hz)}$$

$$CPU\ Time_A = Instruction\ Count_A \times CPI_A \times CycleTime_A$$

$$CPU\ Time_A = 12\times10^6 \times \left(\frac{1}{10\times10^6 \times \frac{1}{33\times10^6}}\right) \times \frac{1}{3\times10^6}(seconds)$$

$$CPU\ Time_A = \frac{12\times10^6}{10\times10^6} = 1.2(seconds)$$

$$Cycle\ Time_B(seconds) = \frac{1}{Clock\ Rate} = \frac{1}{20\times10^6(Hz)}$$

$$CPU\ Time_B = 10\times10^6 \times \left(\frac{1}{9\times10^6 \times \frac{1}{20\times10^6}}\right) \times \frac{1}{20\times10^6}(seconds)$$

$$CPU\ Time_B = \frac{10\times10^6}{9\times10^6} = 1.11(seconds)$$

$$Speedup = \frac{1.2(seconds)}{1.1(seconds)} = 1.09$$



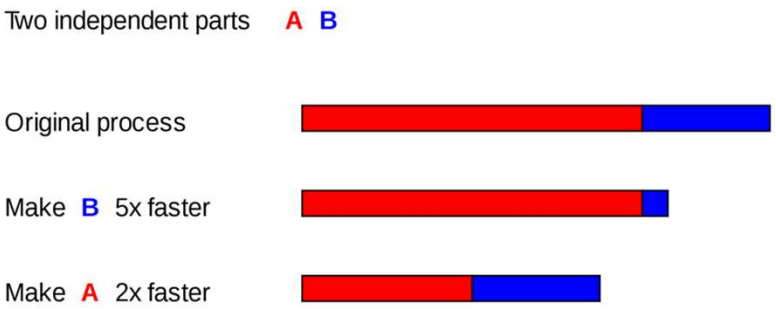
Amdahl's Law

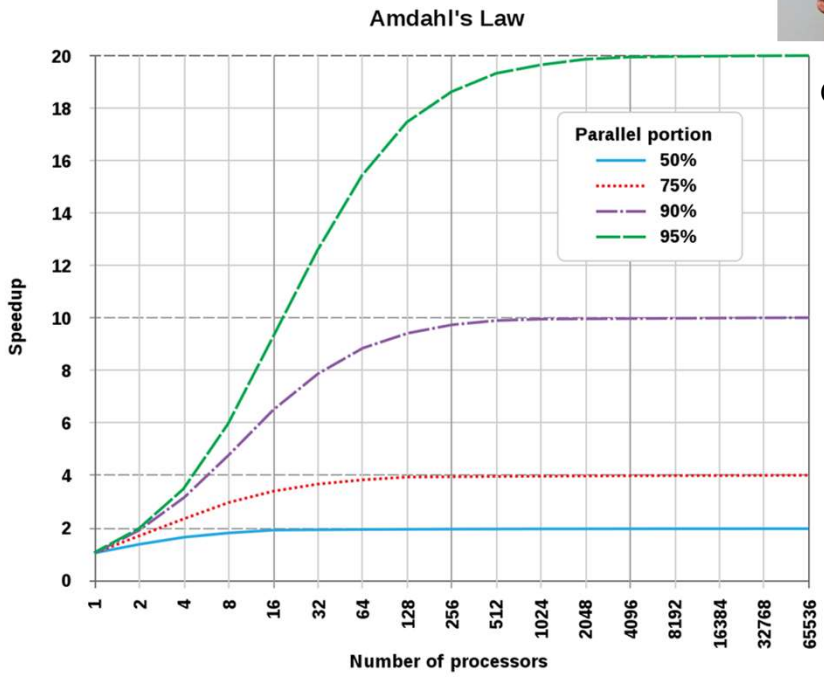


Gene M. Amdahl
1922-2015

★ Law of diminish return

To do grocery shopping, there are two parts: traveling (A), shopping (B).
Which optimization will be faster?
1. Travel 2 times faster
2. Shop 5 times faster





Images are taken from wikipedia



Amdahl's Law (ctd.)

$$Overall\ Speedup = \frac{1}{(1-P)+\frac{P}{n}}$$

★ If time required for buying meal (select and pay) is accounting for ⅓ of the meal time, what is the ideal speedup when we can buying 3-time faster.

$$Overall\ Speedup = \frac{1}{(1-(\frac{1}{3}))+(\frac{(\frac{1}{3})}{3})} = \frac{1}{\frac{2}{3}+\frac{1}{9}} = \frac{1}{\frac{6+1}{9}} = \frac{9}{7} \approx 1.29$$



Performance Improvement

- ★ With CPU Time function as a guideline, we can improve performance by minimizing the function. $CPU\ Time = Instruction \times CPI_{average} \times Cycle\ Time$
- ★ Reducing no. of Instruction
  - Redesign ISA?
  - Compiler Optimization?
- ★ Reducing (average) CPI
  - Redesign ISA?
  - Redesign internal CPU organization?
- ★ Reducing Cycle Time
  - Redesign internal CPU organization?
  - Improve transistor technology to make it faster?



Exercises





CPI and SpeedUp

- Suppose that we want to enhance the processor used for Mail Server. The new server is 10 times faster on computation in the mail server application.
  Assuming that the original processor is busy with computation 40% of the time and is waiting for I/O 60% of the time, what is the overall speed up gained by incorporating the enhancement?



CPI and SpeedUp

- If we can make all FP instructions in the graphics processor run faster by a factor of 1.6; FP instructions are responsible for half of the execution time for the application, calculate the speedup.



CPI and SpeedUp

- ★ Given the following measurements:
  - Frequency of FP operations = 25%
  - Average CPI of FP operations = 4.0
  - Average CPI of other instructions = 1.33
- ★ If we can decrease the average CPI of all FP operations to 2.5, calculate the speedup.



End of Chapter 2





Instruction Set Architecture

Chapter 3



Instruction Set Architecture

- ★ Computer Architecture class in the past.
- ★ Instruction Set Architecture
  - History, Matrix, Design Issues
- ★ Memory Organization
  - Endian
  - Alignment
- ★ Addressing Mode
- ★ Instruction Format
- ★ Roles of Compiler in Optimization
- ★ Parameter Passing
- ★ Exception



Computer Architecture Class in the past

- ★ 50s and 60s - Computer Arithmetic
- ★ 70s and 80s - Instruction Set Design and ISA for Compiler
  - Key evidences: Pascal, Ada, Stack Frame, etc
- ★ 90s - CPU Organization, Memory Architecture, Instruction Set Extension
  - Key evidences: Intel Architecture, Multimedia Extension, Nvidia (1993), ATI (1985)
- ★ 2000s - Computer Arithmetic, I/O system, Parallelism
  - Key evidences: multi-core processor, 64-bit architecture, PCI-express, USB, Thunderbolt
- ★ 2010s - GPU? (deep learning), Virtualization?, Distributed Architecture?
  - See trends around you.



Instruction Set Architecture

- ★ What do you need to know in order to program a computer machine in assembly programming language (i.e. to build system software such as O.S. , Compiler, Assembler, etc)?
  - Instructions or Operators - (Logical) Register Transfer Language, CPI, format
  - Operands, Registers and flags (status) - How the operators interact with data
  - Memory Reference - Addressing, Referencing and Dereferencing memory
  - Instruction Format - How an instruction is translated into binary code.
  - Interrupt and Exception Handler
- ★ This is basically called “Instruction Set Architecture”.
  (Reminder. ISA is abstraction between hardware and software.)
- ★ Note that we currently ignore external components.



Instruction Set Architecture (ctd.)

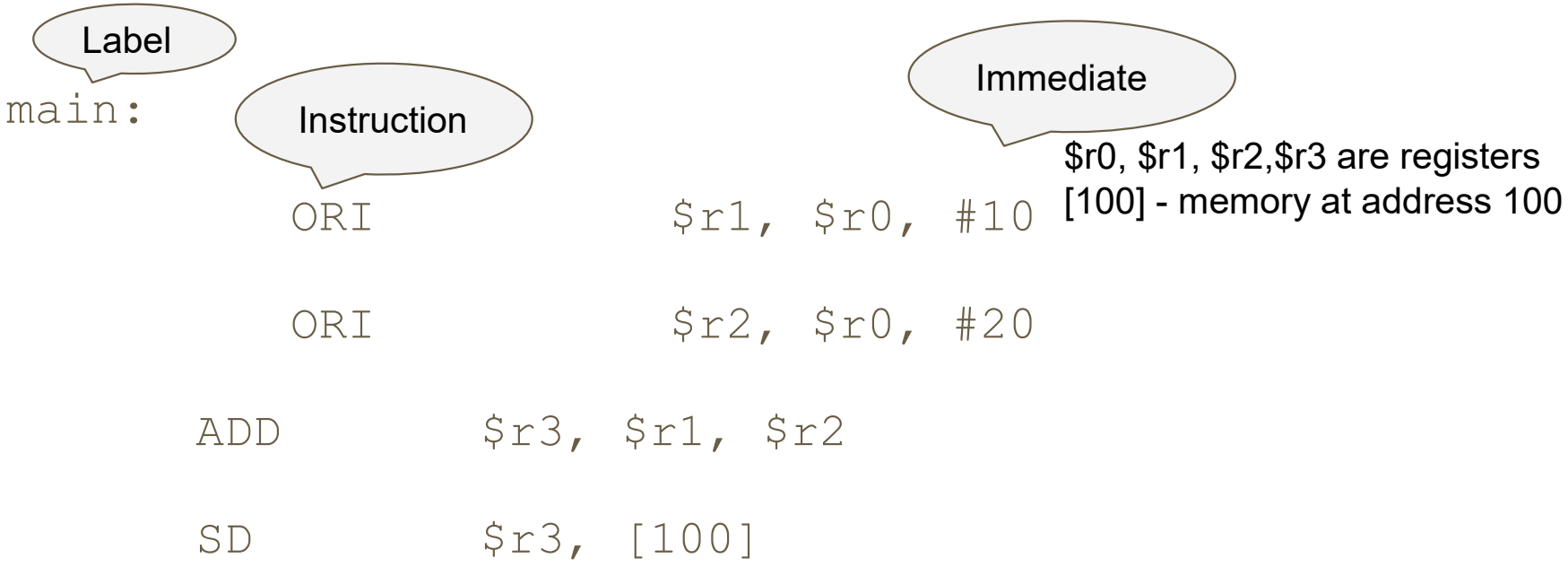
- ★ In 1964, IBM introduces the IBM 360 architecture, which eliminated 7 different IBM instruction sets.
  - Before that, programs are not portable.

- ★ The instruction set architecture serves as the interface between software and hardware.
- ★ It provides the mechanism by which the software tells the hardware what should be done.



Anatomy of Assembly Programming Language

- ★ We will quickly visit assembly programming language.
- ★ C=10+20;





Design Issues

- ★ Where are operands stored?
  - Accumulator, stack, registers, memory
- ★ How many explicit operands?
  - 0, 1, 2, or 3
- ★ How is the operand be specified?
  - addressing mode (immediate, direct, indirect, displacement)



ISA Matrix

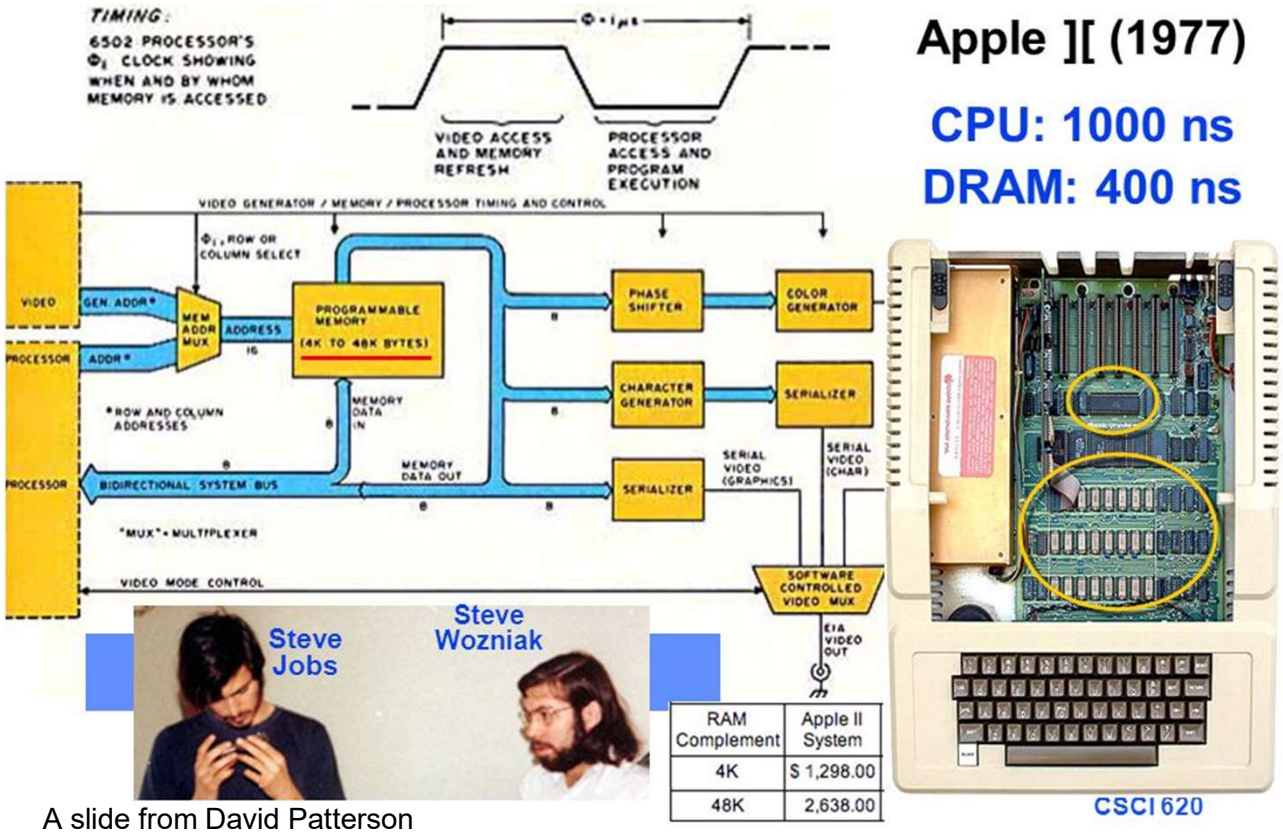
- ★ Properties of preferred ISA
  - Compatibility
    - Portable software
  - Generality / Completeness
    - Support a wide rage of operations
  - Orthogonal
    - Few special cases, all operands modes available with any data/instruction type
  - Regularity
    - No overloading for the meanings of instruction fields
  - Ease of Compilation
  - East of Implementation



History of ISA

- ★ Historically, memory (DRAM) was fast comparing to CPU.
- ★ Transistor is much more expensive.
- ★ Keep in mind that back then, technology are limited.

1977: At one time, DRAM was faster than microprocessors

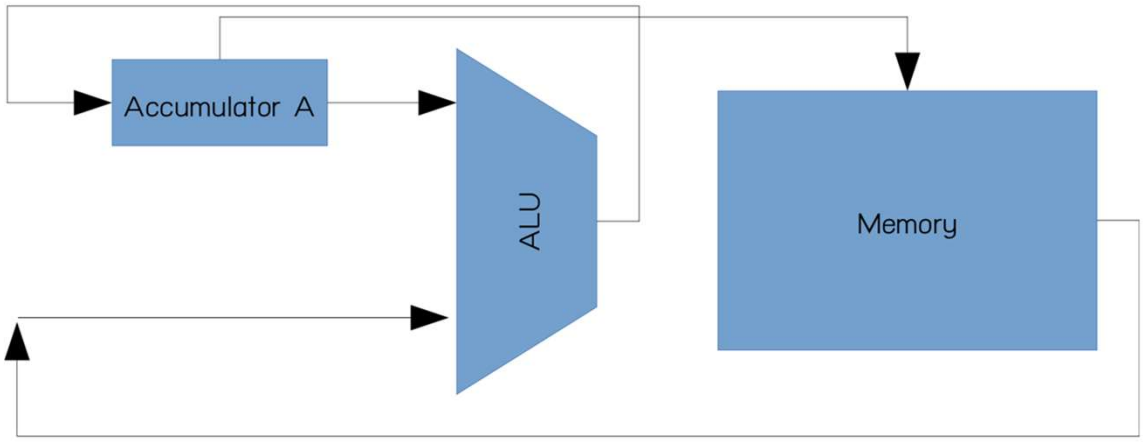


A slide from David Patterson



Accumulator Architecture

- ★ Before 1960
- ★ 1 address. Accumulator is an implicit source and/or target
- ★ Only one register (accumulator) is enough for every calculations.
- ★ Remember your grade-school adder.



x=(a*b)+(a–(b*c))

```
LD      $A, [a]   ;     $A=a
MUL     $A, [b]   ;     $A=a*b
SD      $A, [x]   ;     x=$A
LD      $A, [b]   ;     $A=b
MUL     $A, [c]   ;     $A=b*c
LD      $A, [a]   ;     $A=a
SUB     $A, [y]   ;     $A=a-(b*c)
SD      $A, [y]   ;     y=$A
LD      $A, [x]   ;     $A=x
ADD     $A, [y]   ;     $A=(a*b)+(a-
SD      $A, [x]   ;     x=$a
```



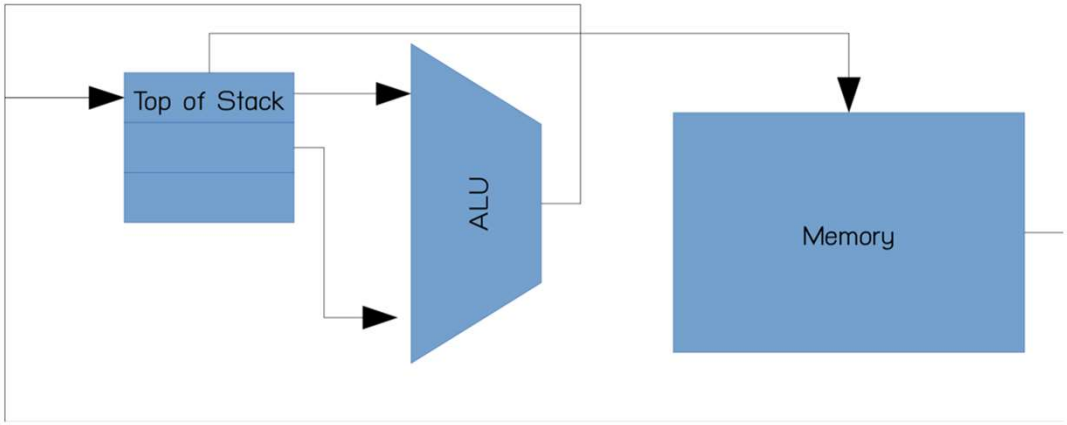
Accumulator Architecture (ctd.)

- 11 instructions
- Pros
  - (Very) low hardware requirements
  - Easy to design and understand
- Cons
  - Bottleneck (Accumulator)
  - Bad for parallelism (including pipelining)
  - High memory traffic



Stack Architecture

- ★ 1960s to 1970s
- ★ 0 address. Operands are the top of stack.



x=(a*b)+(a–(b*c))

```
PUSH    a     ; (top of stack) a
PUSH    b     ; (top of stack)  b , a
MUL           ; (top of stack) (a*b)
PUSH    a     ; (top of stack) a, (a*b)
PUSH    b     ; (top of stack) b, a, (a*b)
PUSH    c     ; (top of stack) c, b, a, (a*b)
MUL           ; (top of stack) (b*c), a, (a*b)
SUB           ; (top of stack) (a - (b*c) , (a*b)
ADD           ; (top of stack) (a*b) + (a - (b*c))
POP     x     ;
```



Stack Architecture (ctd.)

- ★ 10 instructions
- ★ Pros
  - ○ Low hardware requirements
  - ○ Good code density
  - ○ Easy for compiler
- ★ Cons
  - ○ Bottleneck (stack)
  - ○ Required data is not always at the top of stack
  - ○ Difficult for optimization (compiler)
  - ○ Bad for parallelism (including pipelining)



Memory-Memory Architecture

x=(a*b)+(a–(b*c))

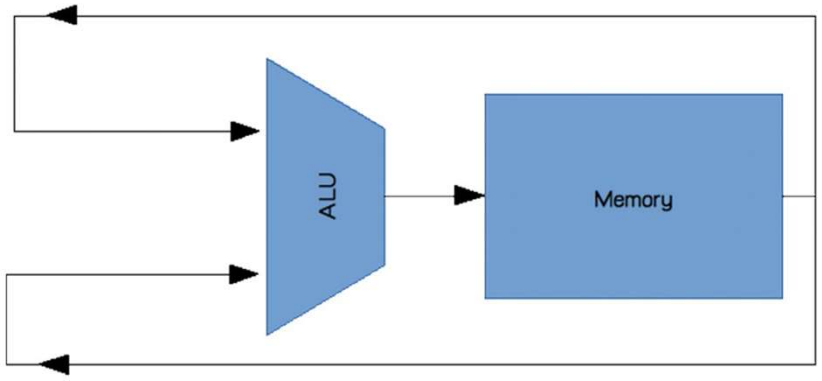
- ★ 1970s to 1980s
- ★ 2-3 address. Do every calculations on memory

3 operands

```
MUL       k, a, b   ; k = a*b
MUL       i, b, c   ; I = b*c
SUB       j, a, i   ; j = a-i = a - (b*c)
ADD       x, k, j   ; x = k+j = (a*b) + (a - (b*c))
```

2 operands

```
LD    x,a    ; x = a
MUL    x, b    ; x = x*b = a*b
LD    i, b    ; i = b
MUL    i, c    ; i = i*c = b*c
LD    j, a    ; j = a
SUB    j, i    ; j = j - i = a - (b*c)
ADD    x, j    ; x = x+j = (a*b) + (a - (b*c))
```





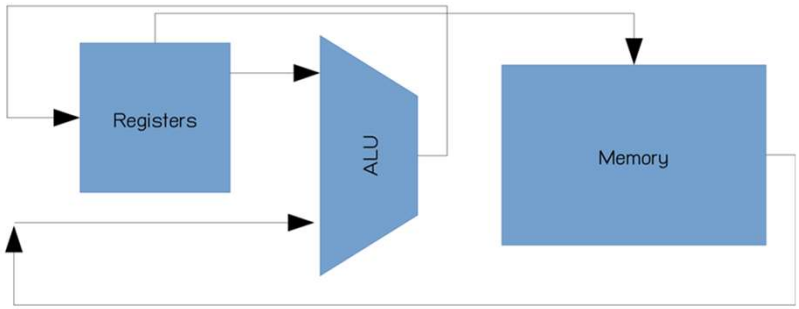
Memory-Memory Architecture (ctd.)

- ★ 4 instructions (3-operand version), 7 instructions (2-operand version)
- ★ Pros
  - Fewer instructions
  - Easy for compilers
- ★ Cons
  - (Very) high memory traffic
  - Variable number of CPI (due to memory access)
  - For 2-operand version, more data movements are required.



Register-Memory Architecture

- ★ 1970s to present
- ★ 2 address
- ★ Like accumulator, but more registers.



x=(a*b)+(a–(b*c))

```
LD          $r1, a     ; $r1 = a
MUL         $r1, b     ; $r1 = $r1*b = a*b
LD          $r2, b     ; $r2 = b
MUL         $r2, c     ; $r2 = $r2*c = b*c
SD          $r2, j     ; j = $r2
LD     $r3, a     ; $r3 = a
SUB         $r3, j     ; $r3 = $r3-j = a - (b*c)
SD          $r3, j     ; j = $r3
ADD         $r1, j     ; $r1 = $r1-j
SD          $r1, x
```



Register-Memory Architecture (ctd.)

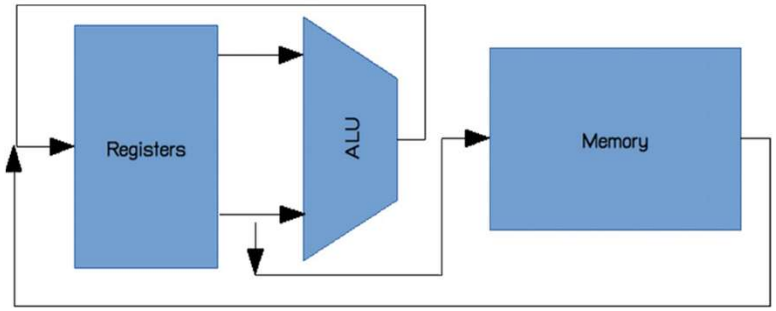
- ★ 10 instructions
- ★ Pros
  - ○ (Some) Data can be accessed without loading
  - ○ Easy to encode (instruction format)
  - ○ Good code density
- ★ Cons
  - ○ Poor orthogonal (operands are not equivalent)
  - ○ Variable number of CPI
  - ○ (May) limit number of registers



Register-Register Architecture

x=(a*b)+(a–(b*c))

- ★ 1960s to present
- ★ 3 address
- ★ Operands are registers only.
- ★ Aka. Load-Store architecture




```
LD    $r1, a    ; $r1 = a

LD    $r2, b    ; $r2 = b

LD    $r3, c    ; $r3 = c

MUL    $r4, $r1, $r2    ; $r4 = $r1 * $r2 = a*b

MUL    $r5, $r2, $r3    ; $r5 = $r2 * $r3 = b*c

SUB    $r6, $r1, $r5    ; $r6 = $r1 - $r5 = a - (b*c)

ADD    $r7, $r4, $r6    ; $r7 = $r4 + $r6 = (a*b) + (a - (b*c)

SD    $r7, x
```

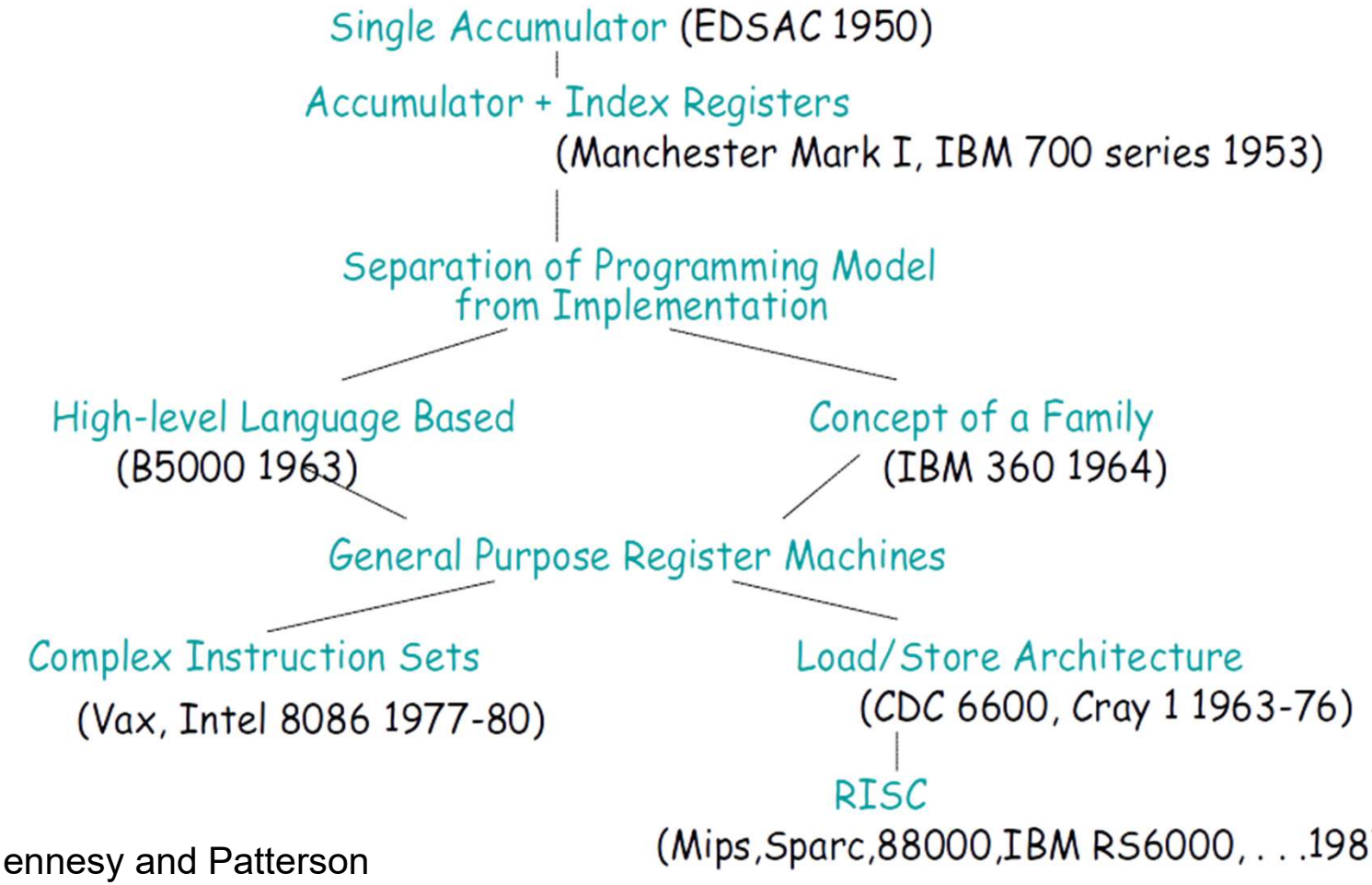


Register-Register Architecture (ctd.)

- ★ 8 instructions
- ★ Pros
  - ○ Simple to encode (fixed-length instruction format)
  - ○ Similar CPI
  - ○ Easy to parallelism (including pipelining)
- ★ Cons
  - ○ Higher instruction count
  - ○ Performance depending on good compiler



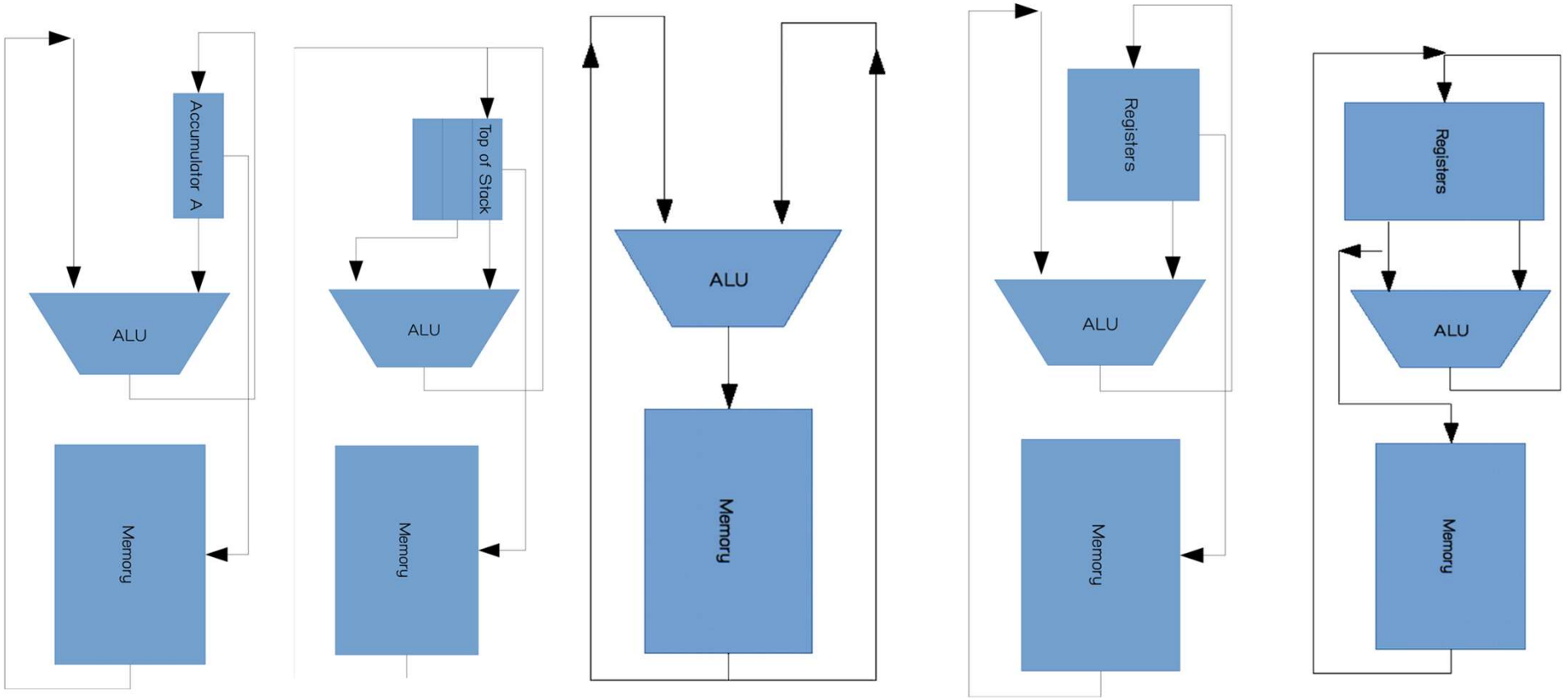
Evolution of ISA



Taken from Hennesy and Patterson



Comparison of Architecture





Comparison of Architecture (ctd.)

- ★ 2 Paradigms
  - CISC - an instruction is powerful (complex).
  - RISC - an instruction is a basic for building block.
- ★ With the emerging of instruction-level parallelism, RISC wins.
  (We will revisit this in chapter 6.)

- ★ We are in post RISC era. Intel ISA is considered CISC, but it is translated into internal RISC code before performing parallelism.
  The distinction is no longer clear.
- ★ ARM stands for Acorn RISC machine.
  Better at power consumption. (See your mobile devices)



Memory Organization

- ★ Memory is basically a big array indexed by address.
- ★ The smallest unit is a bit. However, we always use byte addressing.
  To get/set a bit of a byte, use bit mask operation (AND, OR)
- ★ How to store/retrieve multiple-byte data (e.g. short, int, long, ...) ?
  - ○ Ordering (endian)
  - ○ Parallelism (alignment)

Note: In standard C, the *sizeof* function would show the following:

- ★ `sizeof(char) - 1 byte`
- ★ `sizeof(short) - 2 byte(s)`
- ★ `sizeof(int) - 4 byte(s)`
- ★ `sizeof(long) - 8 byte(s)`
- ★ `sizeof(float) - 4 byte(s)`
- ★ `sizeof(double) - 8 byte(s)`
- ★ `sizeof(long double) - 16 byte(s)`



Endian

- ★ Multiple-byte data can simply be stored in a byte addressing memory.
- ★ Base address can be used as a reference. (A variable using 0x0040 and 0x0041 would be referred with 0x0040.)
- ★ How the direction should be stored.
- ★ Little Endian
  - Low-order byte stores at the lowest address
  - Used by Intel
- ★ Big Endian
  - High-order byte stores at the lowest address
  - Used by IBM PowerPC, Sun
- ★ Note that modern processor (ie. ARM from ARM6 onwards) have the option of operating in either little-endian or big-endian mode.



Endian (ctd.)

- ★ In the past, transferring binary data between Mac PowerPC (big endian) and Windows PC (little endian) requires a careful conversion.
- ★ Storing 0x1A2B3C4D

Address	00	01	02	03
Big Endian	1A	2B	3C	4D
Little Endian	4D	3C	2B	1A

- ★ Sometime, programs may store file in a specific endian. (e.g. photoshop store file in big endian regardless of the underlying architecture.)



Endian and performance

- ★ When the size of memory bus is smaller than the actual data, we usually read from lower address to higher address.
- ★ For arithmetic operation, little endian is becoming more natural.
  - Adding the low order byte first allows you to properly handle carry.
- ★ For comparison (more than/less than), big endian is faster.
  - The sign bit is always at the high-order byte

Example. Two 32-bit numbers on 8-bit architecture.

Adding two numbers, little endian allows us to read and to add in order.

```
0x 12 34 56 78
0x 98 76 54 32

0x AA AA AA AA
```

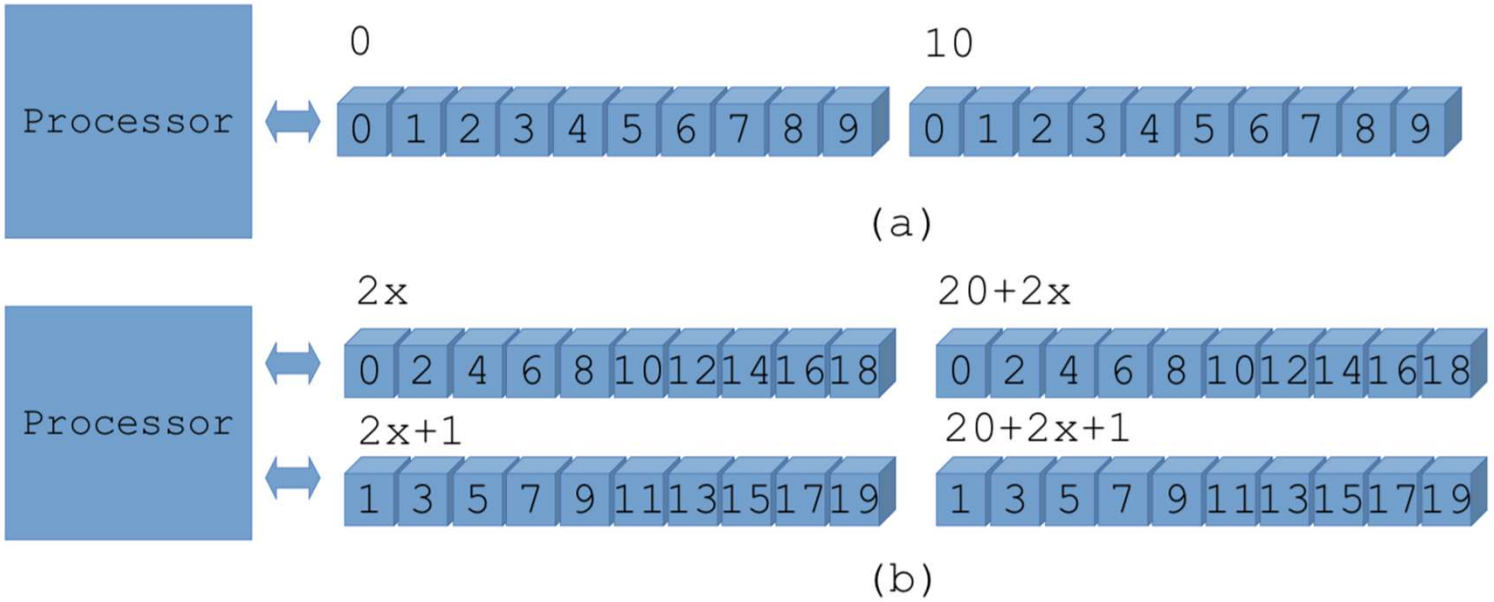
Comparing two numbers, big endian allows us to answer immediately.

```
0x 12 xx xx xx
0x 98 xx xx xx    ; greater
```



Memory Alignment

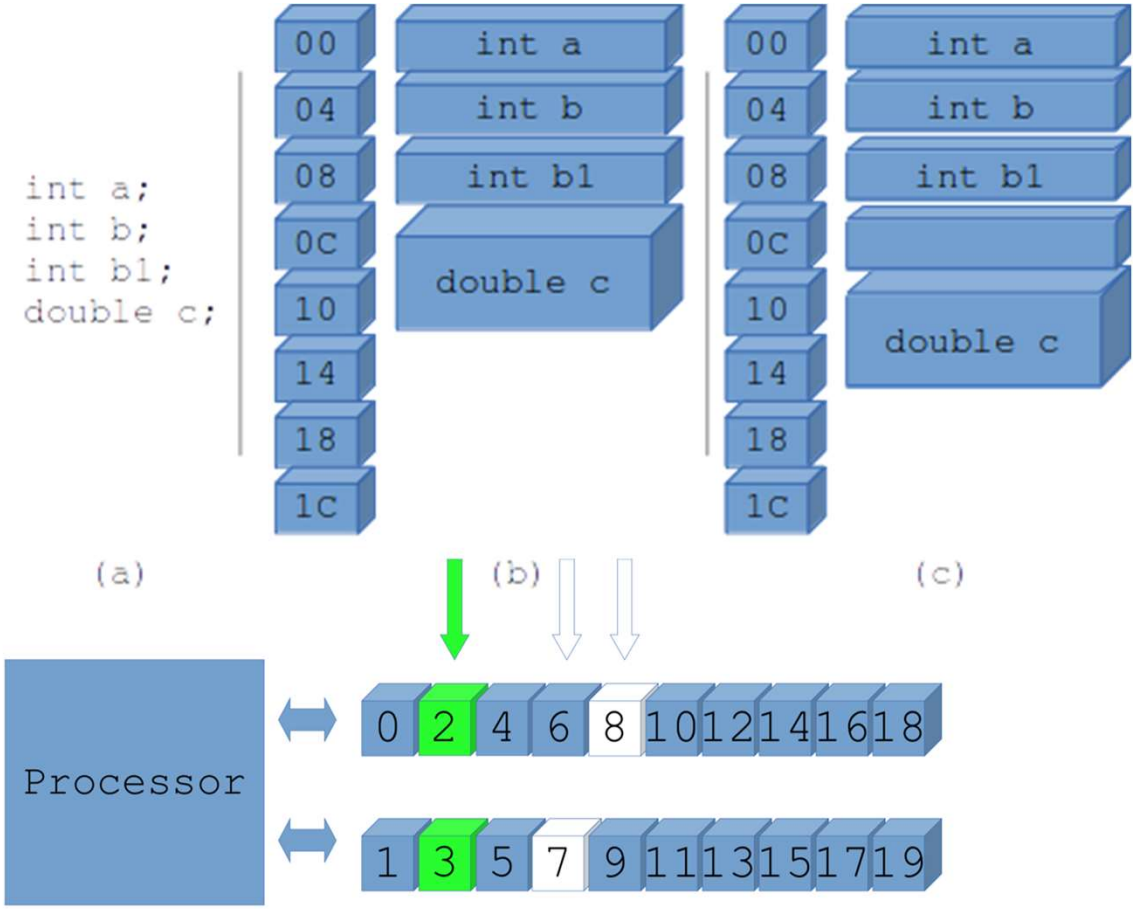
- ★ Parallelism in the memory front allows reading multiple bytes at a time.
- ★ If a read gives us two bytes, we can read a short in one cycle.





Restricted vs. Unrestricted Alignment

- ★ Restricted alignment means software must confirm that data can be read at the minimum number of cycles.
- ★ Restricted Alignment means Base Addr. (A) mod size (s) =0





Unrestricted Alignment

- For unrestricted alignment,
  - Software is simple.
  - Hardware must detect and fix the misalignment.
  - Usually take two (or more) memory accesses.
  - Expensive logic?
  - Performance (Slow)?
  - Sometimes required for backwards compatibility with old hardware.



Restricted Alignment

- ★ With restricted alignment,
  - ○ Burden is on the software (compiler).
  - ○ Hardware detect misalignment and traps.
  - ○ No extra time.
- ★ For performance's sake, restricted alignment wins.
- ★ Make the common case fast.



Addressing Mode

★ Allow a program to easily refer to data in memory

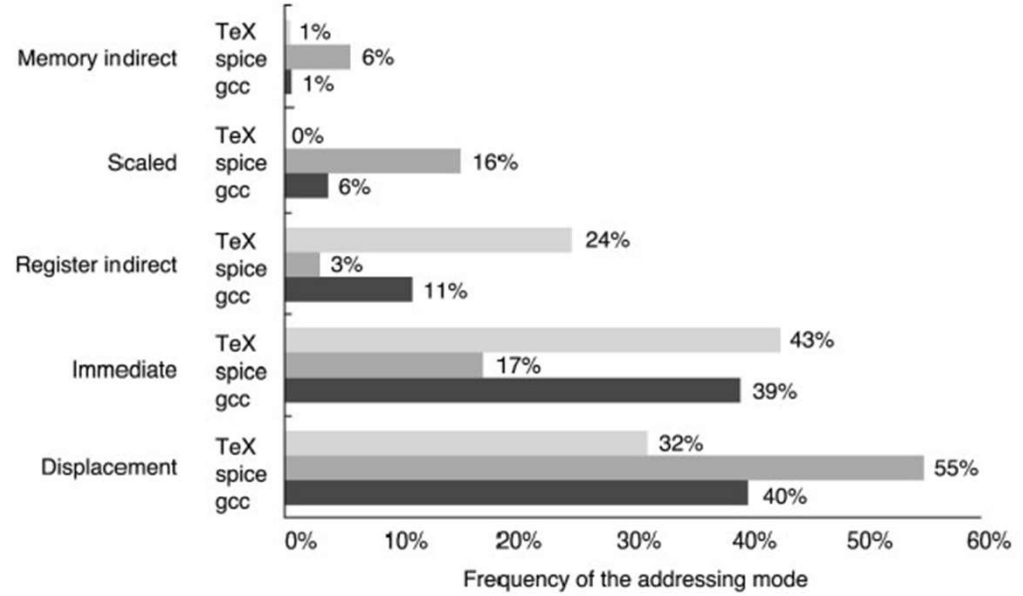
1.	Immediate	LDI $r1, #4	A = 4
2.	Register Direct	ADD $r1, $r2, $r3	A = B + C
3.	Displacement	LD $r1, 100($r2)	A = D[100] หรือ A = G.age
4.	Register Indirect	LD $r1, ($r2)	A = *p
5.	Index	LD $r1, ($r2 + $r3)	A = D[i]
6.	Direct	LD $r1, 1000	
7.	Memory Indirect	LD $r1, @($r2)	A = **p
8.	AutoIncrement	LD $r1, ($r2)+	A = D[i++]
9.	AutoDrecrement	LD $r1, ($r2)-	A = D[i--]
10.	Scale	LD $r1, ($r2)[$r3*4]	F = E[i]

```
unsigned char A, B, C;
char D[200];
int E[200];
int F;
char *p;
struct G {
  char name[100];
  unsigned char age;
}
```



What do we really need?

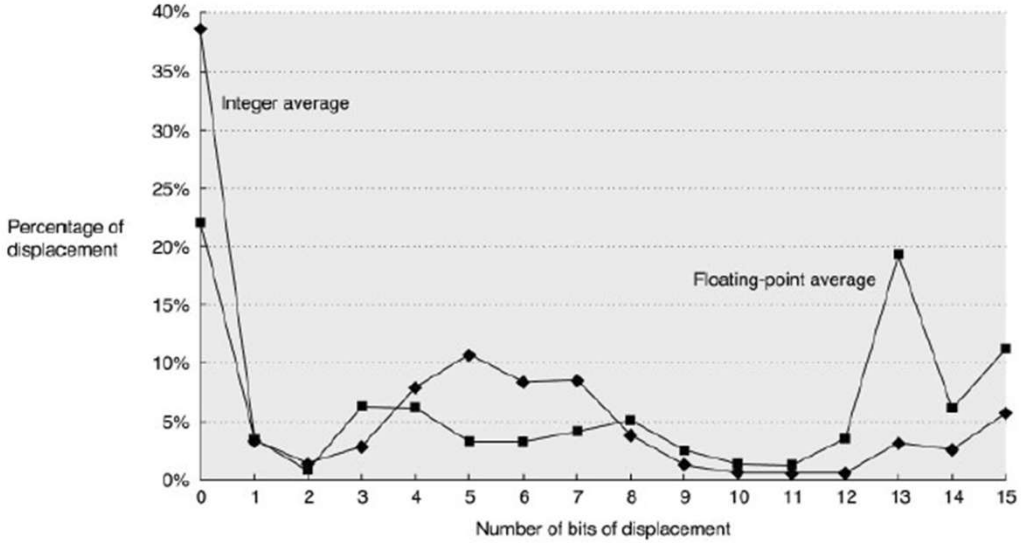
- ★ Most addressing modes are available in modern architecture.
- ★ Technically, register indirect can replace most modes by first calculating address and put it in a register.



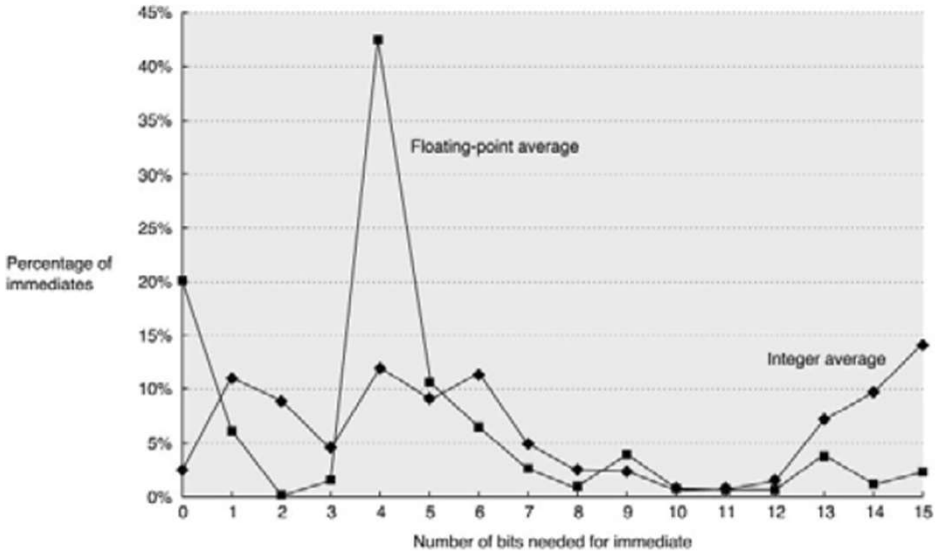
© 2003 Elsevier Science (USA). All rights reserved.



Displacement Value/Immediate Value



© 2003 Elsevier Science (USA). All rights reserved.



© 2003 Elsevier Science (USA). All rights reserved.

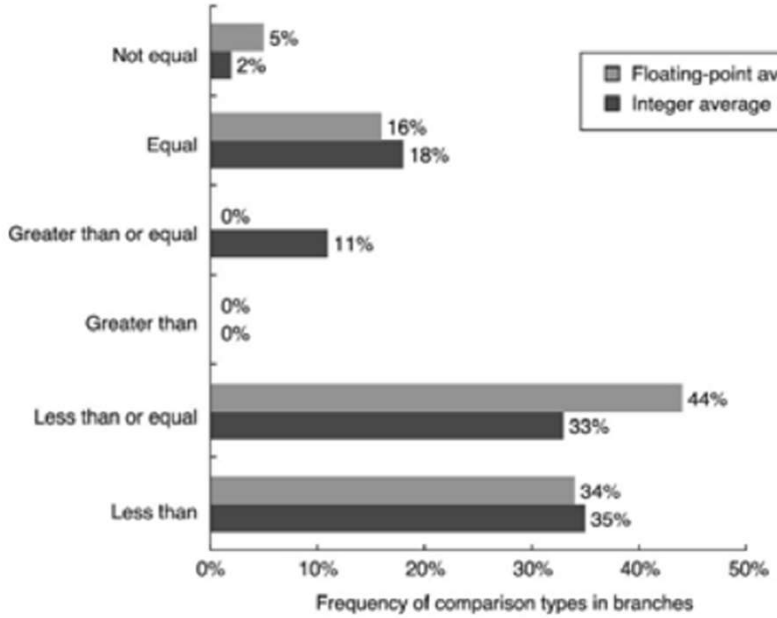


Classes/Types of Instructions

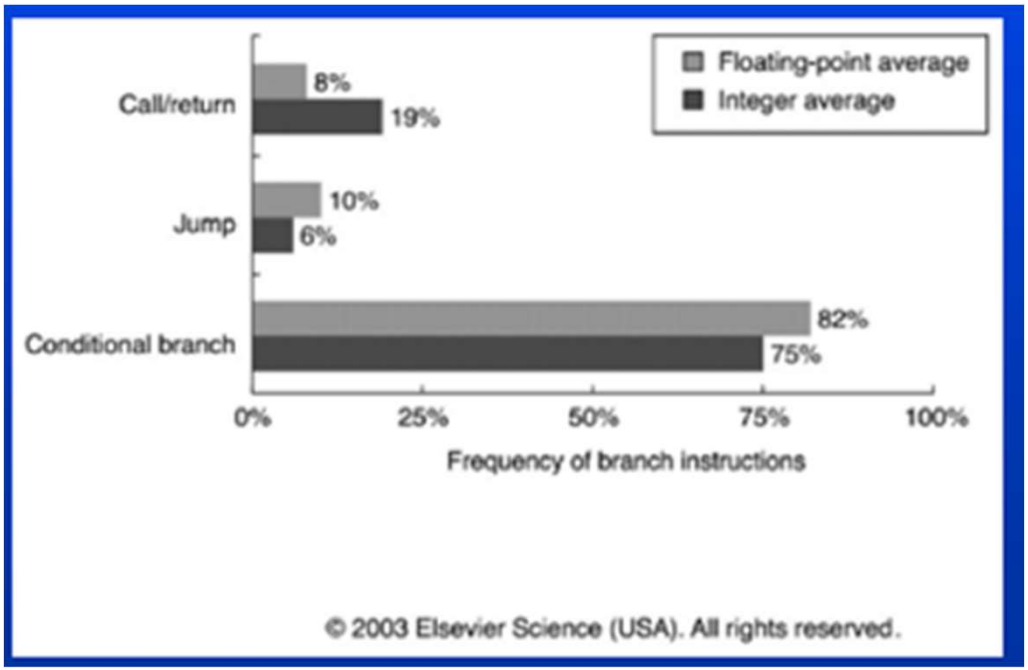
- ★ Data Movement/Transfer Instructions
  - Move data between memory locations, registers, I/O (optional)
  - E.g. LW, SW
- ★ Arithmetics Instructions
  - Mathematical operations between 2 operands
  - E.g. Add, Sub, Mul, Div
- ★ Logic Instructions
  - Logical operations
  - E.g. And, Or, Xor, Com (Not)
- ★ Control Instructions
  - Change control flow (PC <- [xxxx]. Normally, PC<- PC+1)
  - E.g. JMP, BEQ
- ★ Extended Instructions (Multimedia, Encryption)



Control Flow



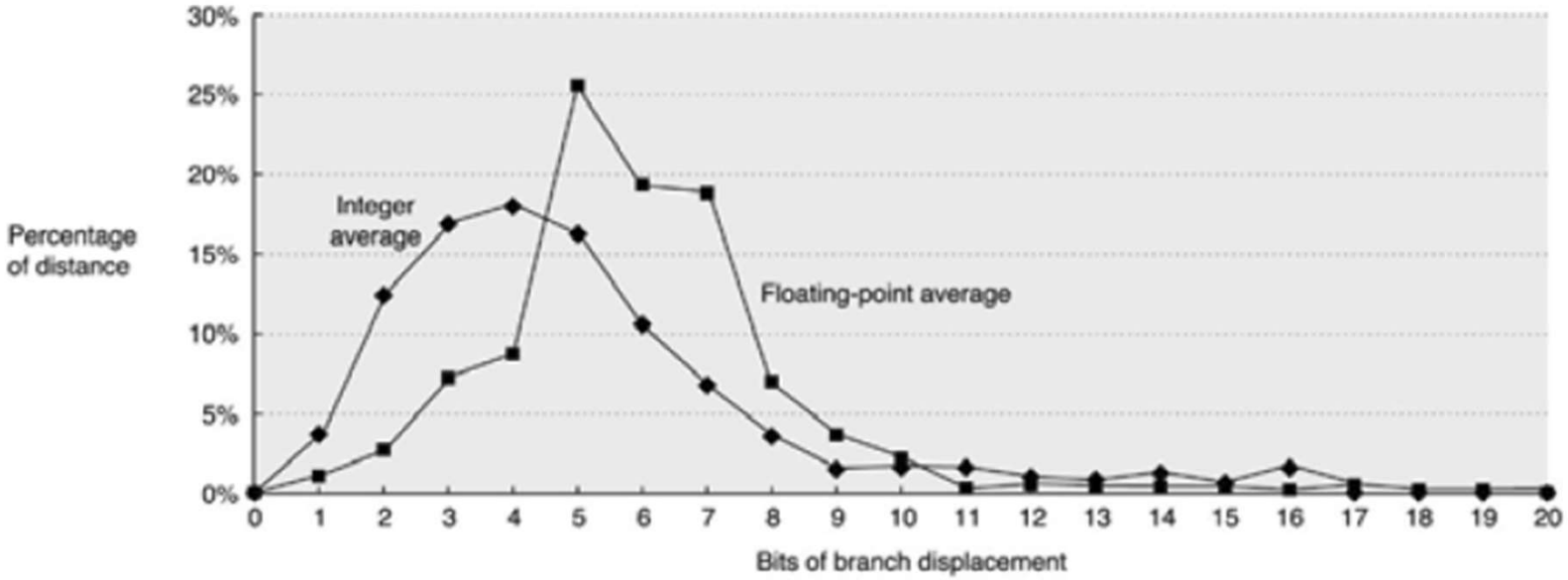
© 2003 Elsevier Science (USA). All rights reserved.



© 2003 Elsevier Science (USA). All rights reserved.



Jump Distances



© 2003 Elsevier Science (USA). All rights reserved.



Addressing Modes in Control Flow

- ★ Usually displacement is relative to PC (why?)
- ★ For Some addresses, we do not known statically (eg. shared library).
  - Use register containing address.
  - (In really, there is a symbol table for this.)
- ★ Relocatable Program
- ★ Dynamic binding

```
Loop:
          ....
     ;BEQ loop
     BEQ -120


;CALL _printf
CALL $r14
```



Trends in Addressing Mode

- ★ For a new ISA design,
  - Support for displacement, immediate, register indirect addressing modes
  - 12-16 bits of displacement field
  - 8-16 bits of immediate field
- ★ For complex addressing mode, let the compilers transform it to simple addressing mode.
- ★ Hardware only focuses on simpler modes.



Sub Routine

- A call to a subroutine means
  - **Passing parameter**
  - **Save current status**
  - Save the return address
  - Call subroutine (Change PC to the address subroutine)
  - ..... (do the subroutine work)
  - ..... Save return data
- A return from a subroutine means
  - Restore status
  - Load return address to PC
- Usually defined in application binary interface (ABI)



Parameter Passing

- ★ Register
- ★ Stack Frame

```
int min(a,b) {
          if(a>b) {
                    return b;
          }
          return a;
}

min(5,4);
```

```
_min:
          CMP       $r3, $r4
          BLE       _less
          MOV       $r1, $r4
          RET
_less:
          MOV       $r1, $r3
          RET


     ...
     LI   $r3, #5
     LI   $r4, #4
     CALL _min
```

```
_min:
          LD        $r3, [$r14+4]
          LD        $r4, [$r14+8]
          BLE       _less
          MOV       $r1, $r4
          RET
_less:
          MOV       $r1, $r3
          RET


     ...
     PUSH #5
     PUSH #4
     CALL _min
```

Assuming that $r14 is stack



Caller Save vs. Callee Save

- ★ Assume that a routine is using $r1.
- ★ In reality, caller do not know which registers are used by the routine.
  - Intel x86 provides PUSHA and POPA to save all registers. (Simpler, but slower)
- ★ Callee save has more benefit?

Caller Save

```
ADD         $r1, $r1, $r2
PUSH        $r1

CALL        SUB

POP         $r1
ADD         $r2, $r2, $r3
            ...

SUB:


ANI         $r1, $r1, #00
ADD         $r1, $r7, $r8


RET
```

Callee Save

```
ADD     $r1, $r1, $r2


CALL     SUB


ADD     $r2, $r2, $r3
    ...

SUB:
PUSH      $r1

ANI     $r1, $r1, #00
ADD     $r1, $r7, $r8

POP       $r1
RET
```



Exceptions

★ What is an exception?

- ★ Unprogrammed Control Flow
- ★ Classified by sources
  - Hardware/External - Interrupt
  - Software - trap
  - Intel uses interrupt for both hardware and software exceptions
- ★ When an exception occurs,
  - Save status
  - Save current instruction (PC)
  - Call the interrupt handles
  - … (do the handler routine)
  - Restore status
  - Restore saved address (PC)

The mechanism is similar to that of the subroutine.



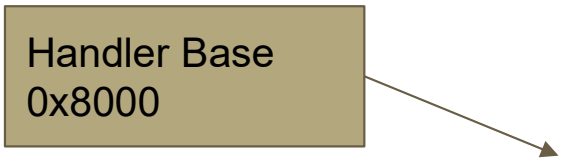
Where is the handlers?

- ★ Vector Table
  - Address in the table
- ★ Handler Table
  - Code in the table
- ★ Fixed Entry
  - Let the software check status and make a decision



Vector Table

- ★ Address in the table
- ★ Address =
  Base +
  (Source no. * address size)
- ★ Assume 32-bit address,
  interrupt 8
- ★ Address=mem[0x8000+(8*4)]
- ★ JUMP to address



+56	0x……
+52	0x…….
…	
+8	0x00004000
+4	
+0	0x88000004



Handler Table

- ★ Code in the table
- ★ Handler =
  Base + (Source no. * handler size)
- ★ Assume 16 bytes handler size,
  interrupt 8
- ★ Address=0x8000+(8*16)
- ★ JUMP Address

+240	IRET
+224	IRET
...	
+32	MOVE $..... .. IRET
+16	
+0	IRET

Handler Base
0x8000



Fixed Entry

- ★ Regardless of the interrupt source, CPU always jump to 0x1000.
- ★ A program check interrupt source register and make a decision from that.



Instruction Format

- ★ Encoding of instructions affects:
  - Size of code
  - Decoding work (of the processor)
- ★ Depends on:
  - Range of addressing modes
  - Degree of independence between opcodes and modes
- ★ We want:
  - As many registers and addressing modes as possible
  - Reduce the size of instructions (and programs)
  - Ease of decoding and implementing in hardware.



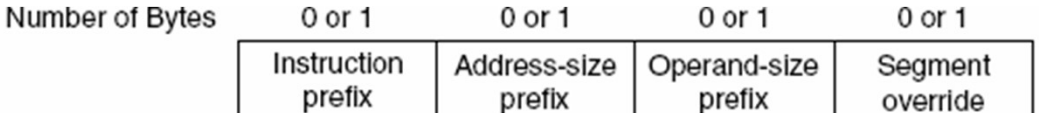
Three (Basic) Variations

- ★ Variable instruction length
  - Bits in the first fetch tell the length
  - Use address specifier
  - Support several modes and operations
  - Smaller code?
- ★ Fixed instruction length
  - Small number of addressing mode
  - Opcode imply addressing mode
  - Larger code?
- ★ Hybrid approach
  - Arm thumbs support java byte code on chip (at higher CPI)

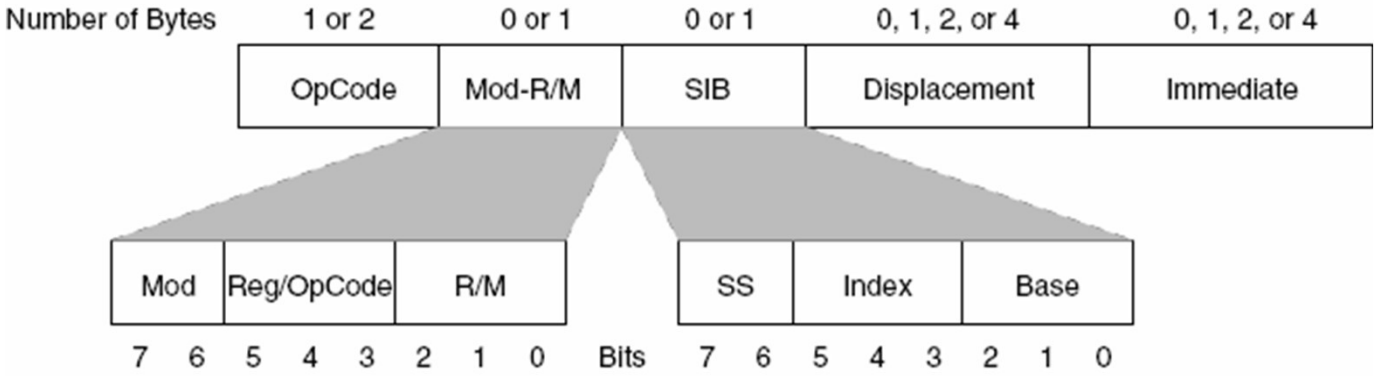


Variable Instruction Length

- A sample from Intel x86



(a) Optional instruction prefixes



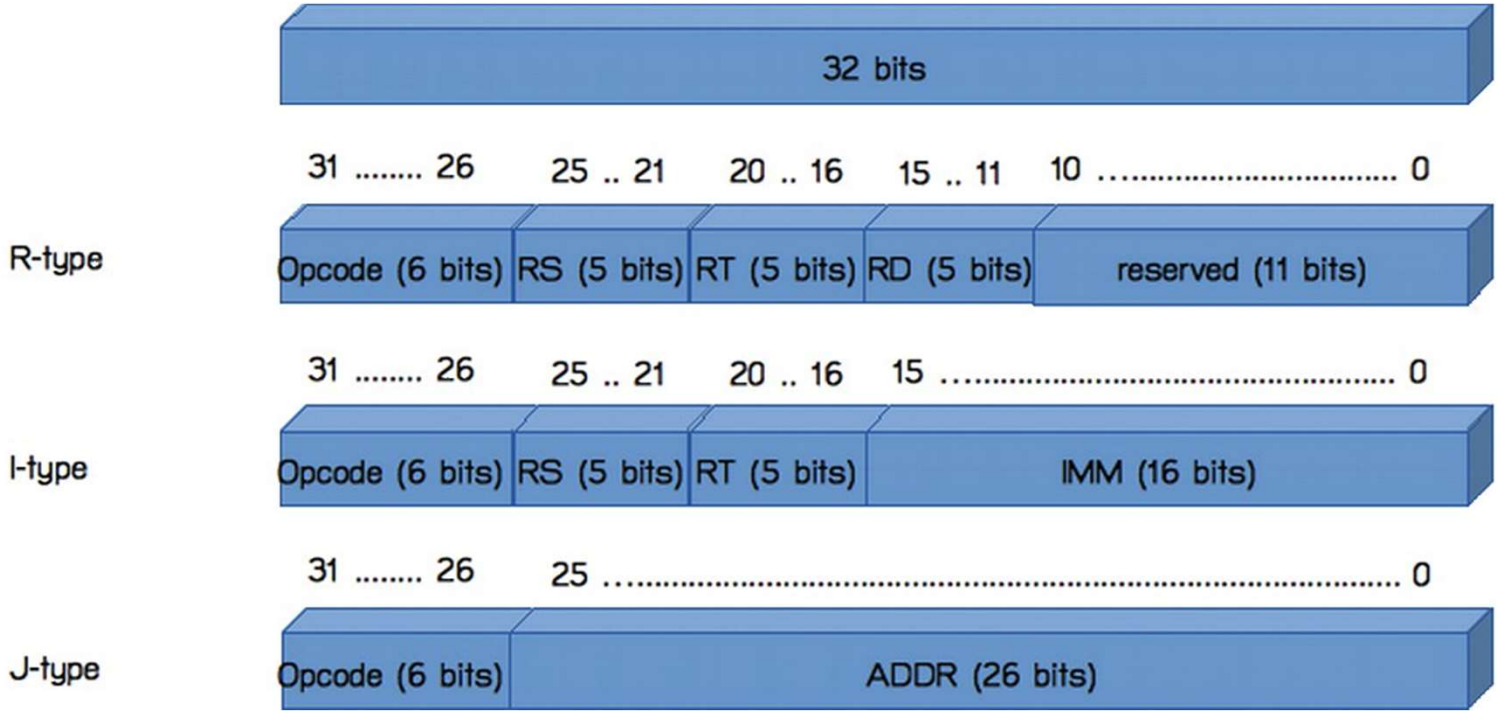
(b) General instruction format

Taken from http://www.c-jump.com/CIS77/CPU/x86/lecture.html



Fixed Instruction Length

★ A sample from nano LADA



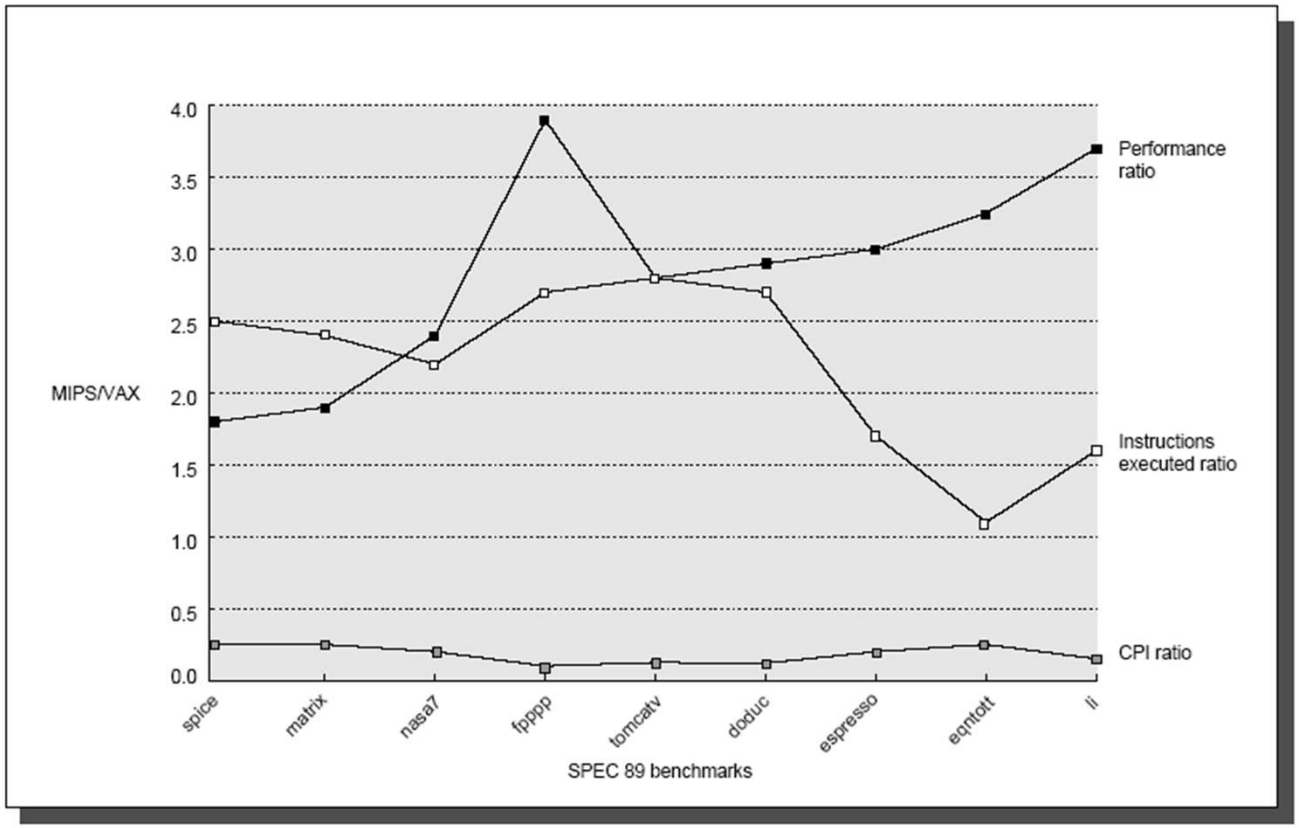


Trends

- ★ For performance, fixed Instruction Length wins.
- ★ However, we have previous design. (We will revisit this in PostRISC.)

MIPS code is usually twice the size of VAX code.
MIPS CPI is ⅙ of VAX CPI.

Which one is faster?





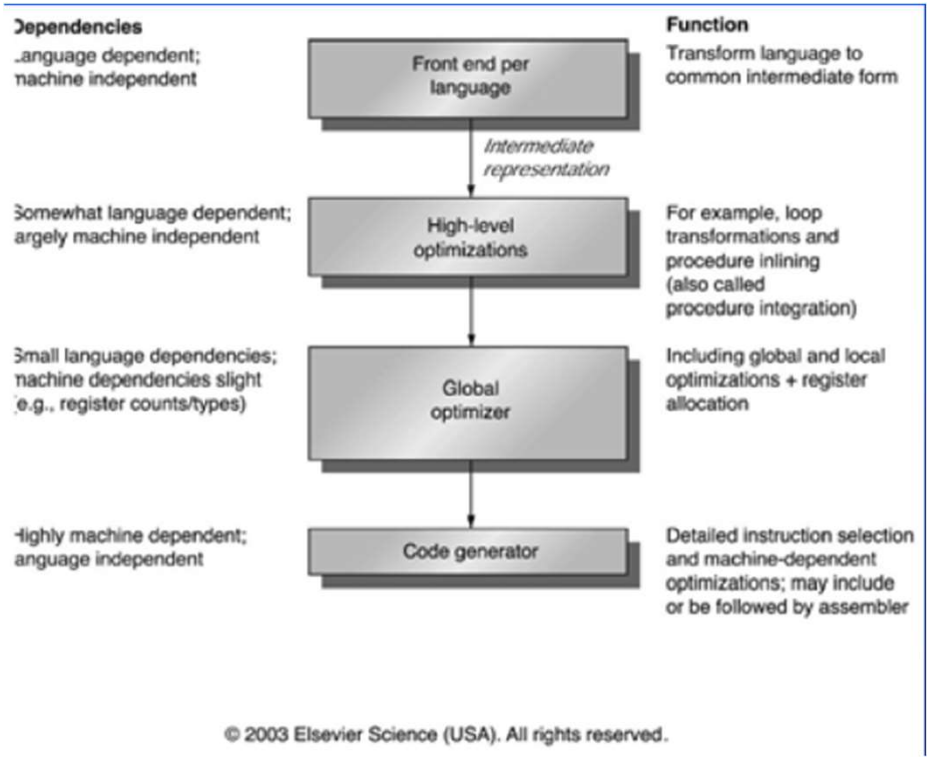
Roles of Compiler in Optimization

- ★ Instruction set architecture is the target of compiler.
- ★ Instruction set architecture should allow compilers to generate efficient code.
- ★ Goals of Compiler
  - ○ Correctness
  - ○ Speed (of code)
  - ○ Compile time
  - ○ Debugging support



Compilation Steps

- ★ Compile Language specific to intermediate code
- ★ High-level Optimization
- ★ Global Optimization
- ★ Code generator





Optimizations

- ★ High-level
  - Procedure integration (procedure in-lining)
- ★ Local (within a basic block)
  - Common expression elimination
  - Constant propagation
  - Stack height reduction
- ★ Global (beyond a basic block)
  - Global common expression elimination
  - Copy propagation
  - Code motion
- ★ Machine-Dependent
  - Strength reduction (shift for multiplication)
  - Branch offset optimization
  - Pipeline scheduling

Note: A basic block is a single sequence of code with no jump or branch.



Examples of Optimizations

Copy propagation

```
// before
m = 60;
h = 60 * m;
d = 24 * h;
days = seconds / d;
```

```
// after
days = seconds / 86400;
```

Common-expression elimination

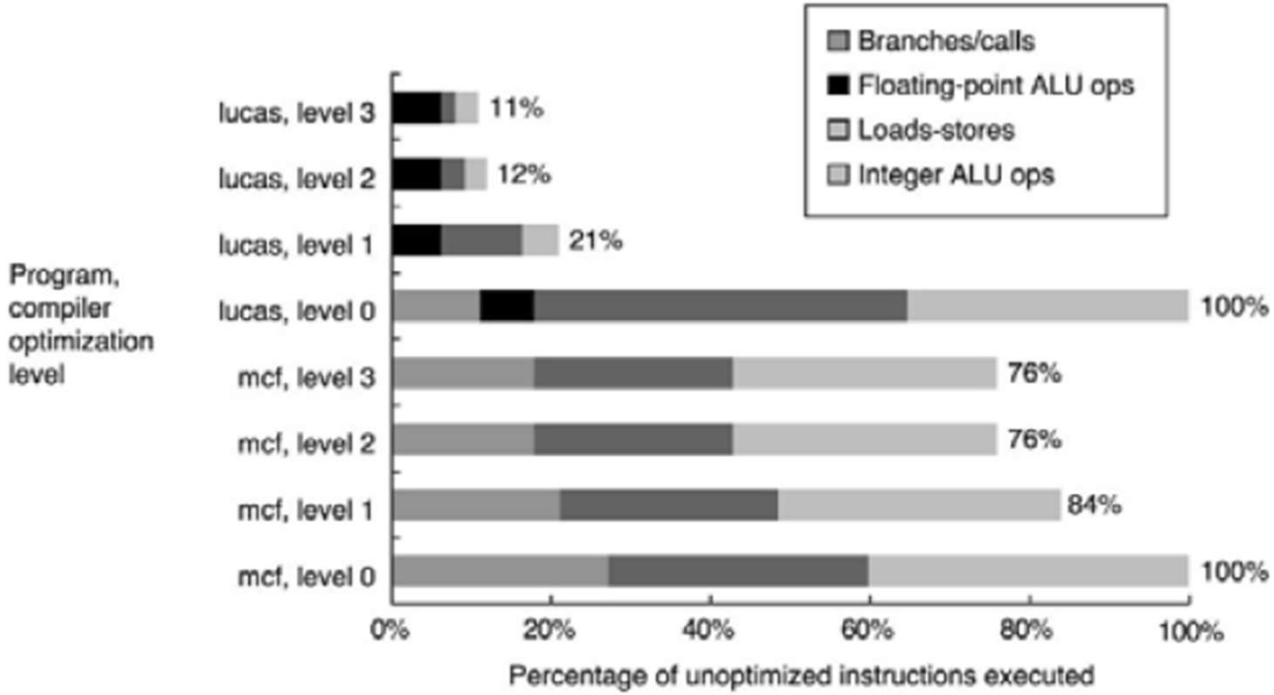
```
// before
a = b + c + d;
y = b + c + e;
```

```
// after
x = b + c;
a = x + d;
y = x + e;
```



Effectiveness of Optimization

- ★ Programs are locally simple, but globally complex.
- ★ Most bugs in compilers are from optimizations.
- ★ Alias (Pointer) prevents allocating registers to variables



https://gcc.gnu.org/bugzilla/buglist.cgi?component=c%2B%2B&order=changeddate%20DESC%2Cbug_status%2Cpriority%2Cassigned_to%2Cbug_id&product=gcc&query_format=advanced&resolution=---

https://en.wikipedia.org/wiki/Undefined_behavior

© 2003 Elsevier Science (USA). All rights reserved.



How to help compiler writer

- ★ Make the common case fast?
- ★ Desirable ISA properties (from compiler's point of view)
  - Regularity
    - Orthogonality among addressing mode, data types, and operations
  - Simplicity
    - Provide primitives, not solutions
  - Simplify tradeoffs
  - Allow compile-time binding



Exercises





Which one is the fastest?

- ★ Hardware-specific optimization.
- ★ Let's clear a register
  - CLR R1
  - MOV R1,0
  - SUB R1, R1, R1
  - XOR R1, R1, R1
- Which one is the fastest?
- ★ Is it funny?

For most architecture, the recommended method is XOR.



Optimize the following code

- Optimize the following code:
  - for(x=0;x<=100;x++) {
  - hs=3600;
  - ds=hs*24;
  - h=a[x]/hs;
  - d=a[x]/ds
  - }



Optimize the following code

- Consider this three statements:
  - A=B+C;
  - B=A+C;
  - D=A-B;
- Use the technique of copy propagation to transform the code sequence to the point where no operand is a computed value. Note the instances in which the transformation has reduced the computational work of a statement and those cases where the work has increased. What does this suggest about the technical challenge faced in trying to satisfy the desire for optimizing compilers?



Memory Allocation

- With restricted alignment, draw a diagram showing the memory allocation for following variables
  - char a;
  - short b;
  - int c;
  - char d;
  - long e;
  - double f;



Assignment 1

For more information, see the announcement.



End of Chapter 3





Single Cycle Processor

Chapter 4



Single Cycle Processor

- ★ Design Concept
- ★ Design Steps
- ★ Instruction Set Architecture (nanoLADA)
- ★ Components
  - ○ Register Files
  - ○ Extender
  - ○ Memory
- ★ Data Path
- ★ Control Signal
- ★ Critical Path

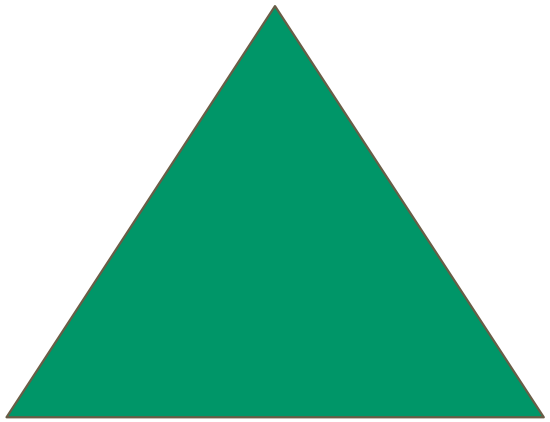


Design Concept

- ★ Performance (CPU Time) is a function of
  - Instruction Count
  - Cycle per Instruction
  - Clock Cycle Time

Depends on:
Programs
ISA
Compiler

Instruction Count



Depends on:
Programs
ISA
Compiler?
CPU Organization

Cycle Per Instruction

Depends on:
CPU Organization
VLSI Technology

Clock Cycle Time



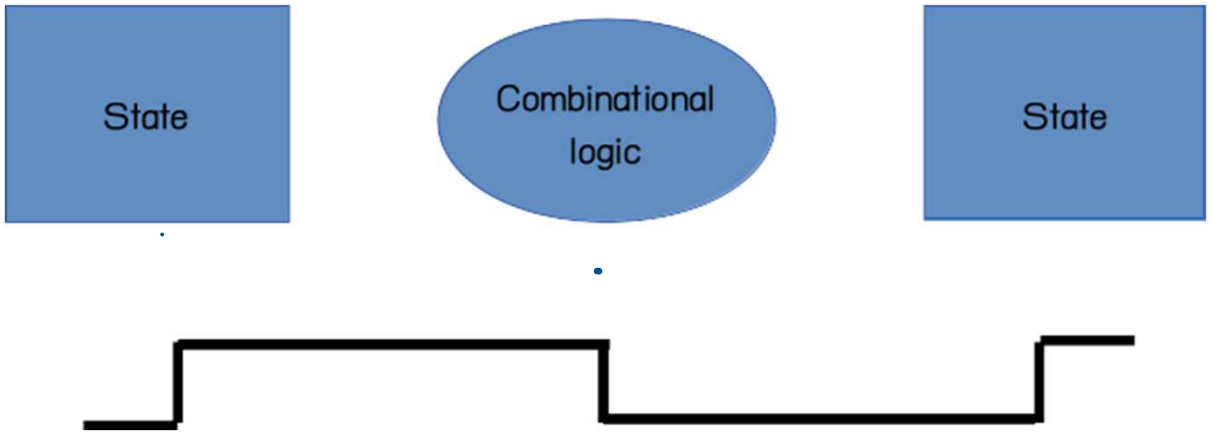
Design Concept (ctd.)

- ★ Processor Design (DataPath and Control) will determine:
  - ○ Cycle Per Instruction
  - ○ Clock Cycle Time
- ★ We will (try to) design a single-cycle processor
  - ○ 1 cycle per instruction (simple)
  - ○ Very long cycle time (slow)
- ★ In hope that you will understand the internal organization of the processor



Design Concept (ctd.)

- ★ Single-Cycle Processor (CPU = 1)
  - Start an instruction every cycle.
  - Related operation for an instruction is a combinational logic.





Design Steps

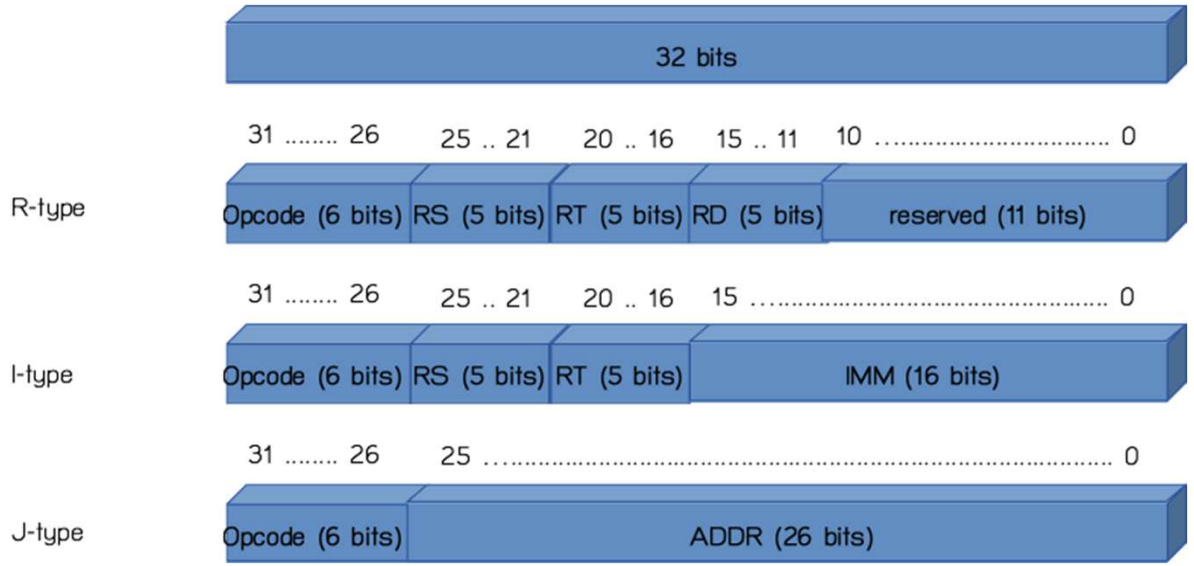
1. Analyze the ISA to determine datapath requirements
   a. Definition of each instruction on Register Transfer Language (RTL)
   b. Storage Elements (registers, and more)
2. Select set of datapath components and establish clocking methodology
3. Assemble datapath meeting the requirements
4. Analyze implementation of each instruction to determine setting of control points that effects the register transfer.
5. Assemble the control logic

Compare to to software design?



Instruction Format (nanoLADA)

- ★ Fixed-Length
- ★ 3 formats
- ★ R-Type
  - 6-bit opcode
  - 5-bit RS, 5-bit RT, 5-bit RD
  - 11-bit reserve
- ★ I-Type
  - 6-bit opcode
  - 5-bit RS, 5-bit RT
  - 16-bit immediate
- ★ J-Type
  - 6-bit opcode
  - 26-bit address/displacement





Step 1: Requirements (nanoLADA)

- ★ Instructions are explained with RTL.
- ★ ORI rt, rs, imm ; (I-type) ; Opcode 010000
  - R[rt] ← R[rs] | zero_ext(imm);
  - PC ← PC + 4
- ★ ORUI rt, rs, imm ; (I-type) ; Opcode 010001
  - R[rt] ← R[rs] | zero_pad(imm);
  - PC ← PC + 4
- ★ ADD rd, rs, rt ; (R-type) ; Opcode 000001
  - R[rd] ← R[rs] + R[rt];
  - PC ← PC + 4



Step 1: Requirements (nanoLADA) (ctd.)

- ★ LW rt, imm(rs) ; (I-type) ; Opcode 011000
  - R[rt] ← MEM[R[rs] + sign_ext(imm)];
  - PC ← PC + 4
- ★ SW rt, imm(rs) ; (I-type) ; Opcode 011100
  - MEM[R[rs] + sign_ext(imm)] ← R[rt];
  - PC ← PC + 4



Step 1: Requirements (nanoLADA) (ctd.)

- ★ BEQrs, rt, imm*4 ; (I-type) ; Opcode 100100
  - If (R[rs] == R[rt]) then
  - PC ← PC + 4 + (sign_ext(imm) * 4)
  - else
  - PC ← PC + 4
- ★ JMP addr*4 ; (J-type) ; Opcode 110000
  - PC ← (addr * 4)



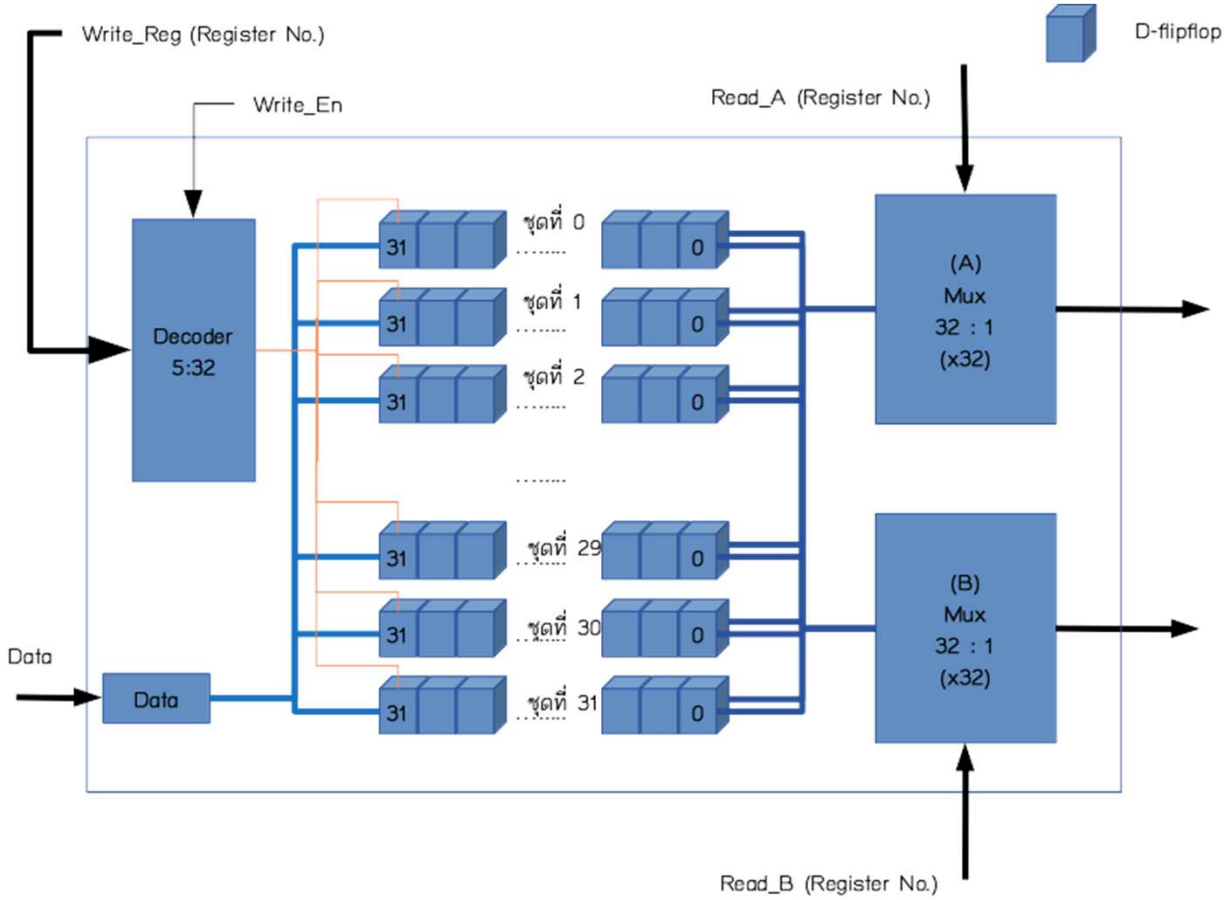
Step 2: Components for Datapath

- ★ Register file (bits? size?)
  - ○ Indexed by RS, RT, RD
- ★ Special register
  - ○ PC
- ★ ALU
  - ○ Supports for ADD, OR
- ★ ADDER (for PC)
- ★ EXTENDER



Register File

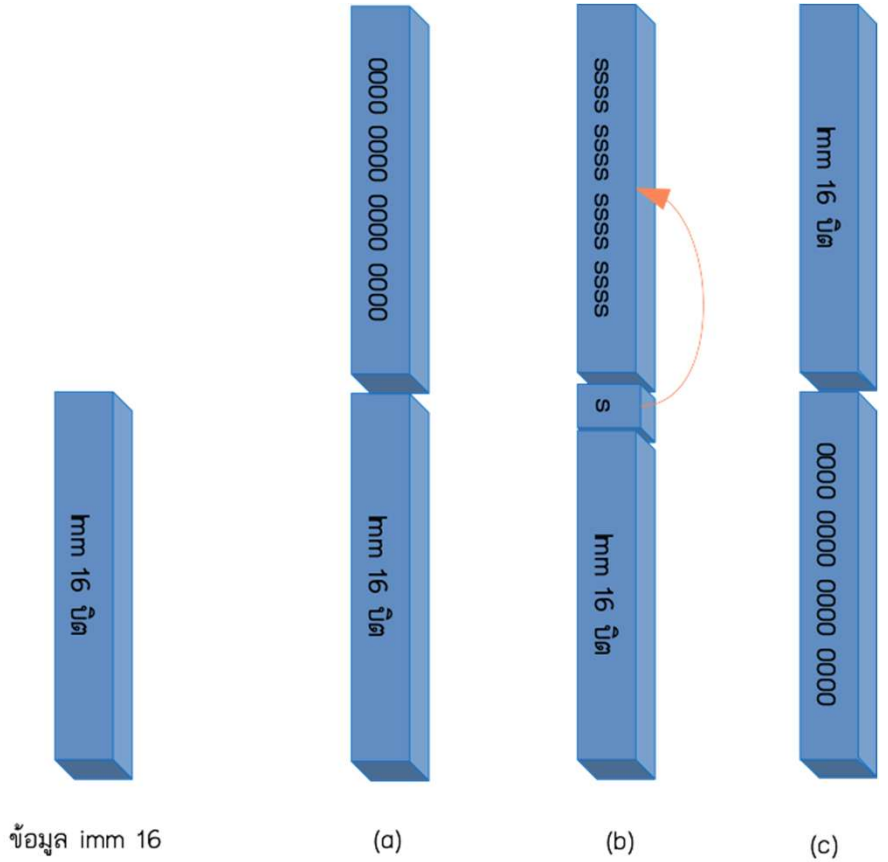
- ★ 32 (5-bit address) registers
  - Each with 32 bit
- ★ Reading from 2 ports (Read_A, Read_B)
- ★ Writing to one port at Write_Reg
- ★ Can be implemented with 32x32 D-flipflop, 2x muliplexors, and decoder





Extender

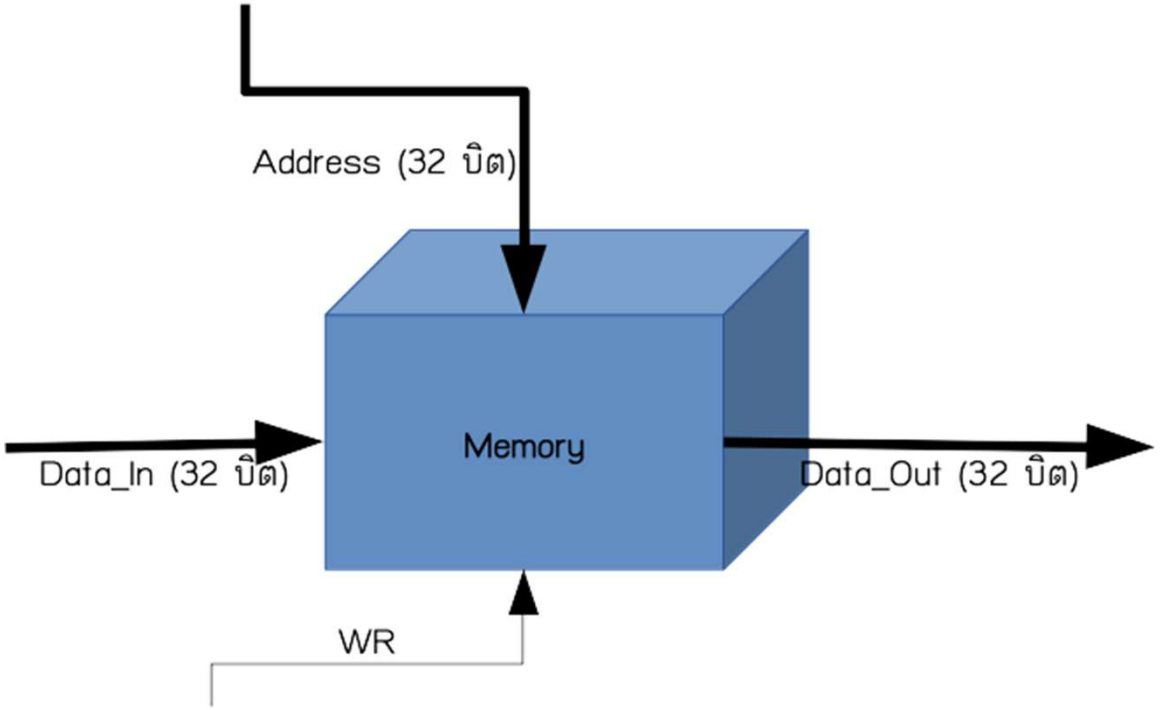
- ★ Extend 16-bit immediate to 32-bit data
- ★ 3 modes
- ★ Sign Extender (b)
  - Extended with sign bit
- ★ Zero Padding (c)
  - Padded with 16'b0
- ★ Zero Extender (a)
  - Extended with 16'b0





Memory

- ★ A memory
- ★ 32-bit in, out
- ★ 32-bit address
- ★ One port for read
- ★ One port for write





ALU

- Let's make an ALU that can
  - Add, Or, Sub

alu_ops	operations
00	S<- A+B
01	S<- A|B
10	S<- A-B

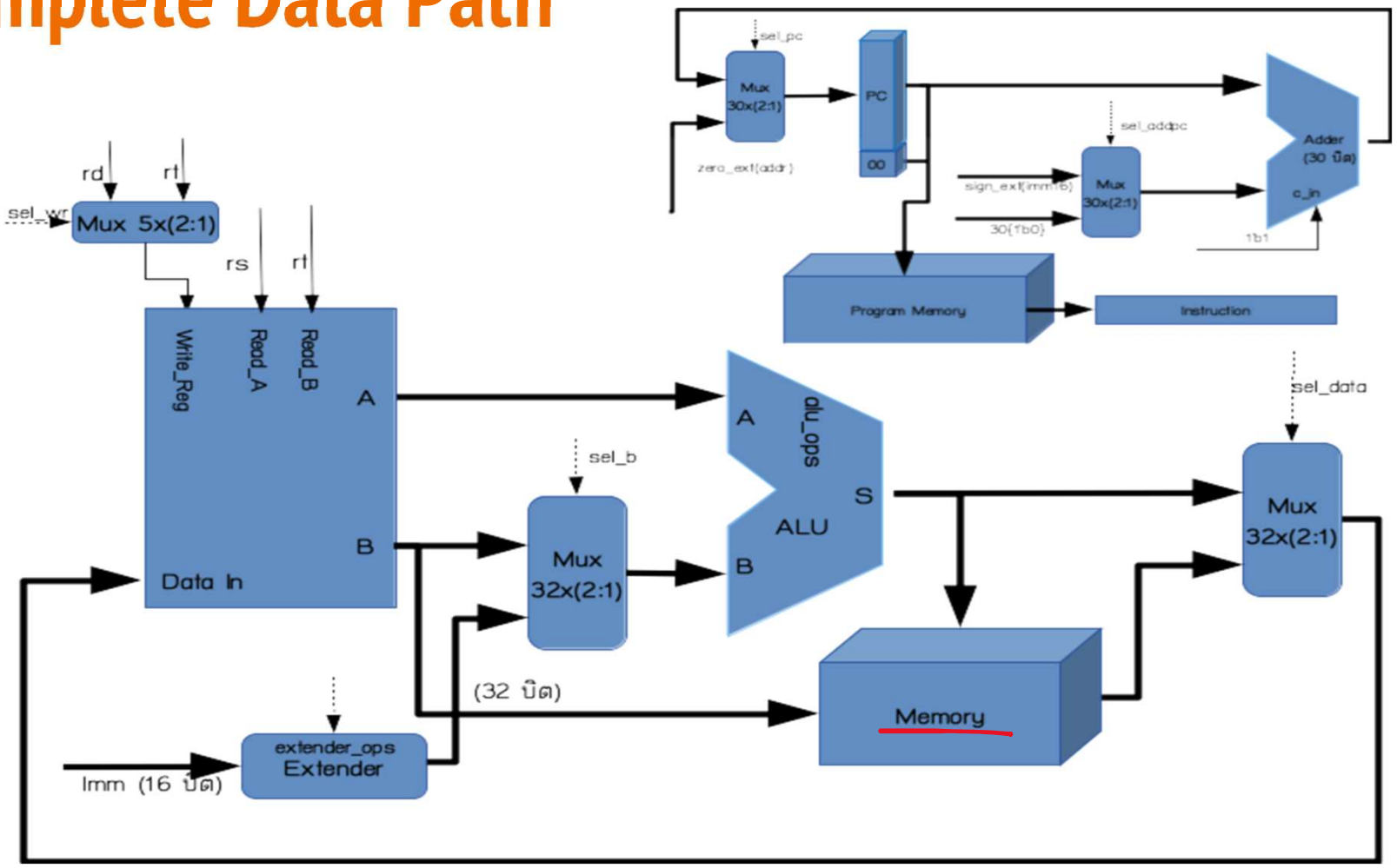


Step 3: Assembly Data Path

- ★ To assembly data path, simple add one instruction at a time
- ★ For each instruction added, modify the data path to support the RTL of the instruction.



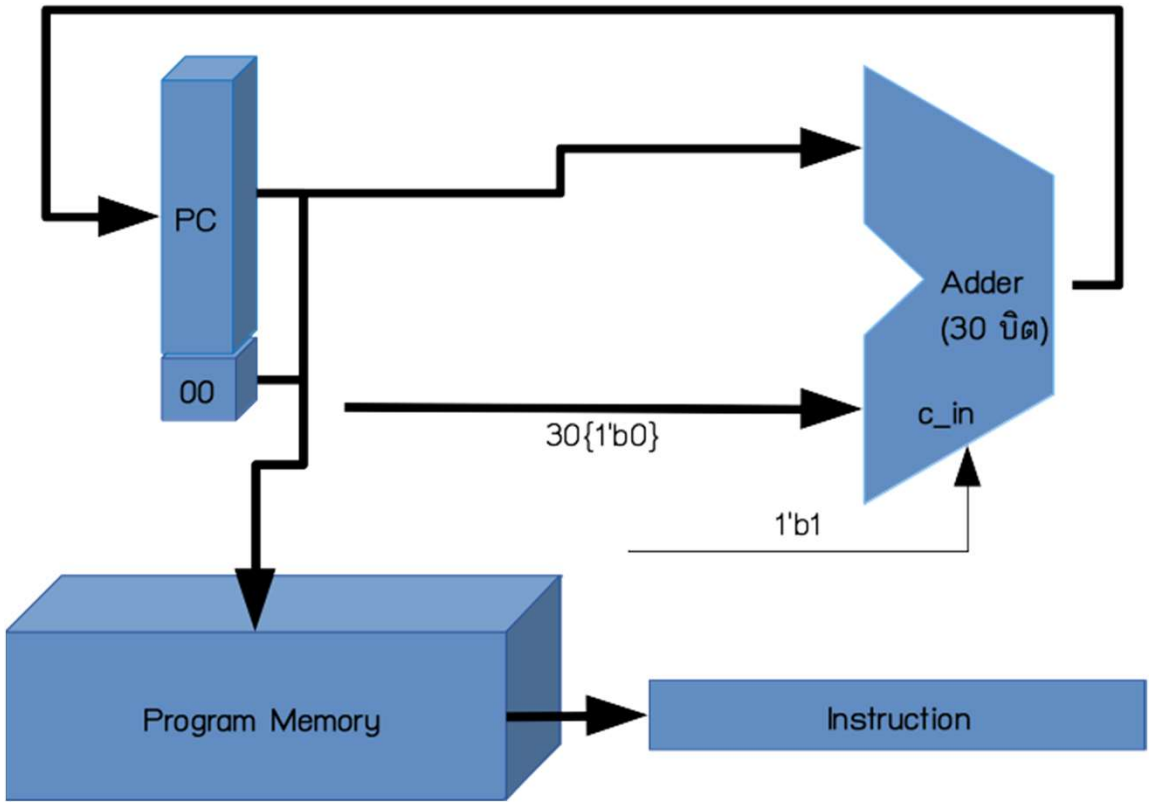
Complete Data Path





Fetch and PC

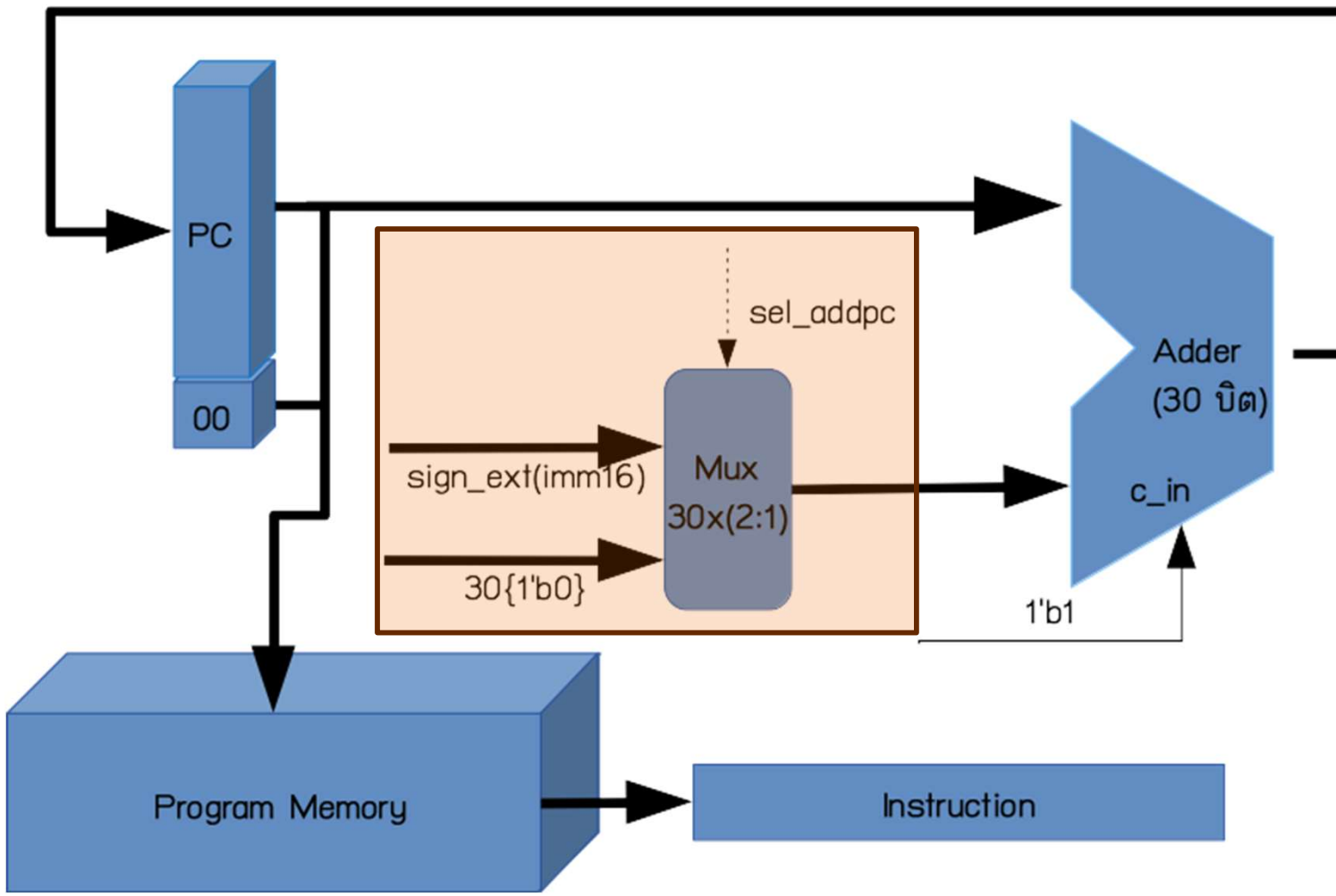
- ★ Fetch is usually ignored by RTL.
  - Instruction<- MEM[PC]
  - PC<- PC+4
- ★ Note.
  PC<-PC+4
  also means
  PC<-{PC[31:2]+1 , 2'b00}
  We can use 30-bit adder for it.





BEQ

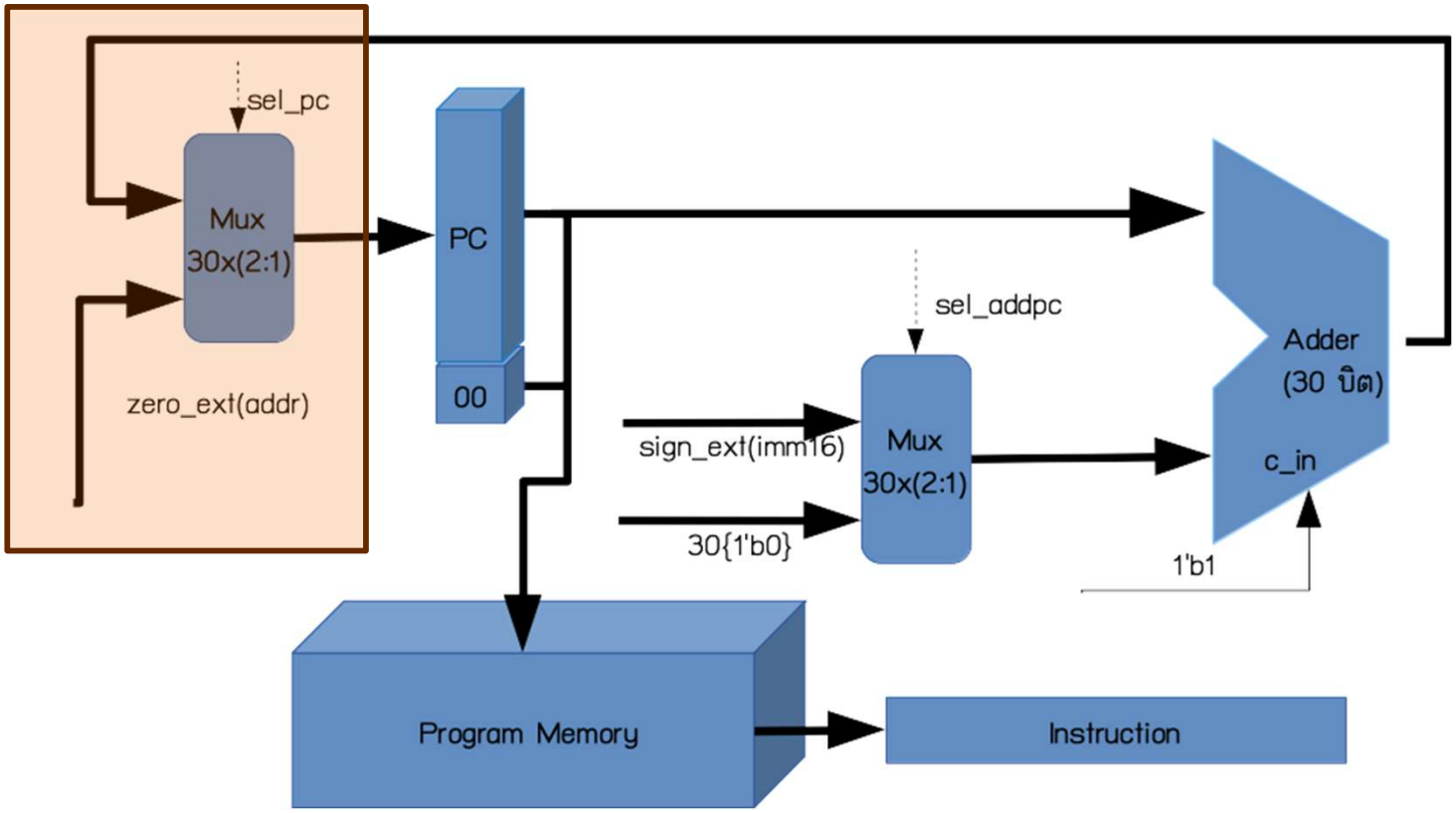
- ★ BEQrs, rt, imm*4 ; (I-type)
  - If (R[rs] == R[rt]) then
  - PC ← PC + 4 + (sign_ext(imm) * 4)
  - else
  - PC ← PC + 4





JMP

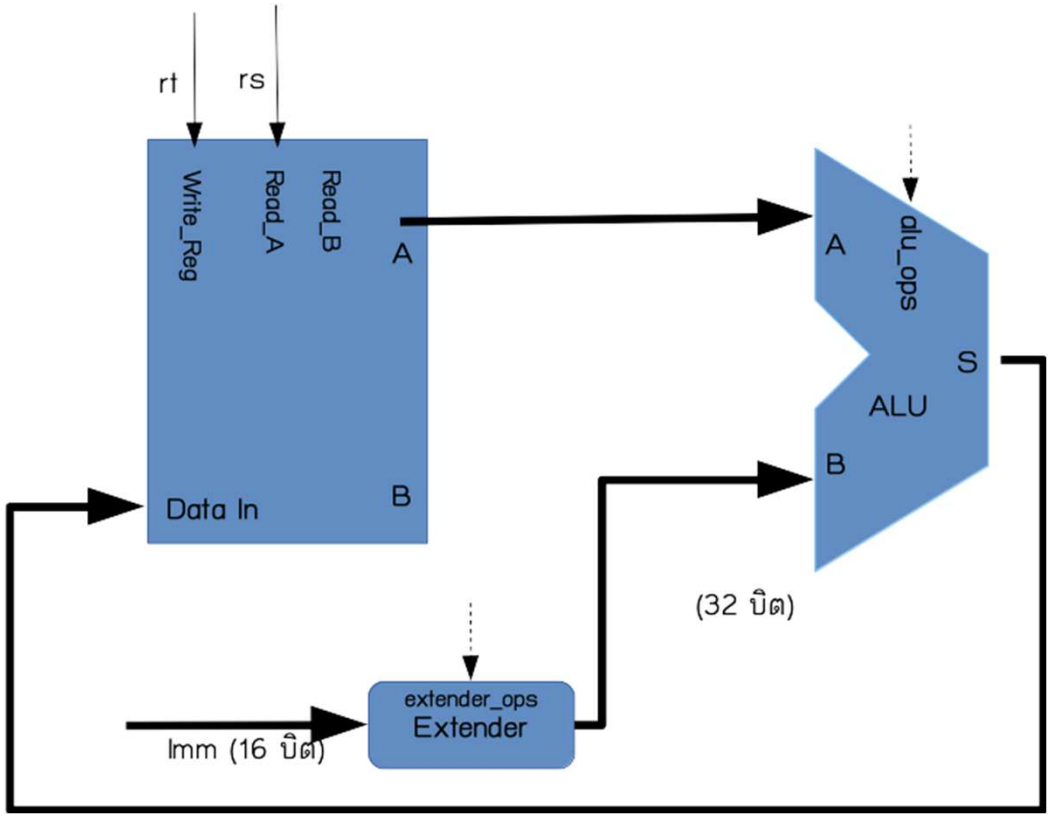
★ JMP addr*4 ; (J-type)
- PC ← (addr * 4)





ORI, ORUI

- ORI rt, rs, imm ; (I-type)
  - R[rt] ← R[rs] | zero_ext(imm);
    PC ← PC + 4
- ORUI rt, rs, imm
  ; (I-type)
  - R[rt] ← R[rs] | zero_pad(imm);
    PC ← PC + 4
- Same data path, only different extender operation.

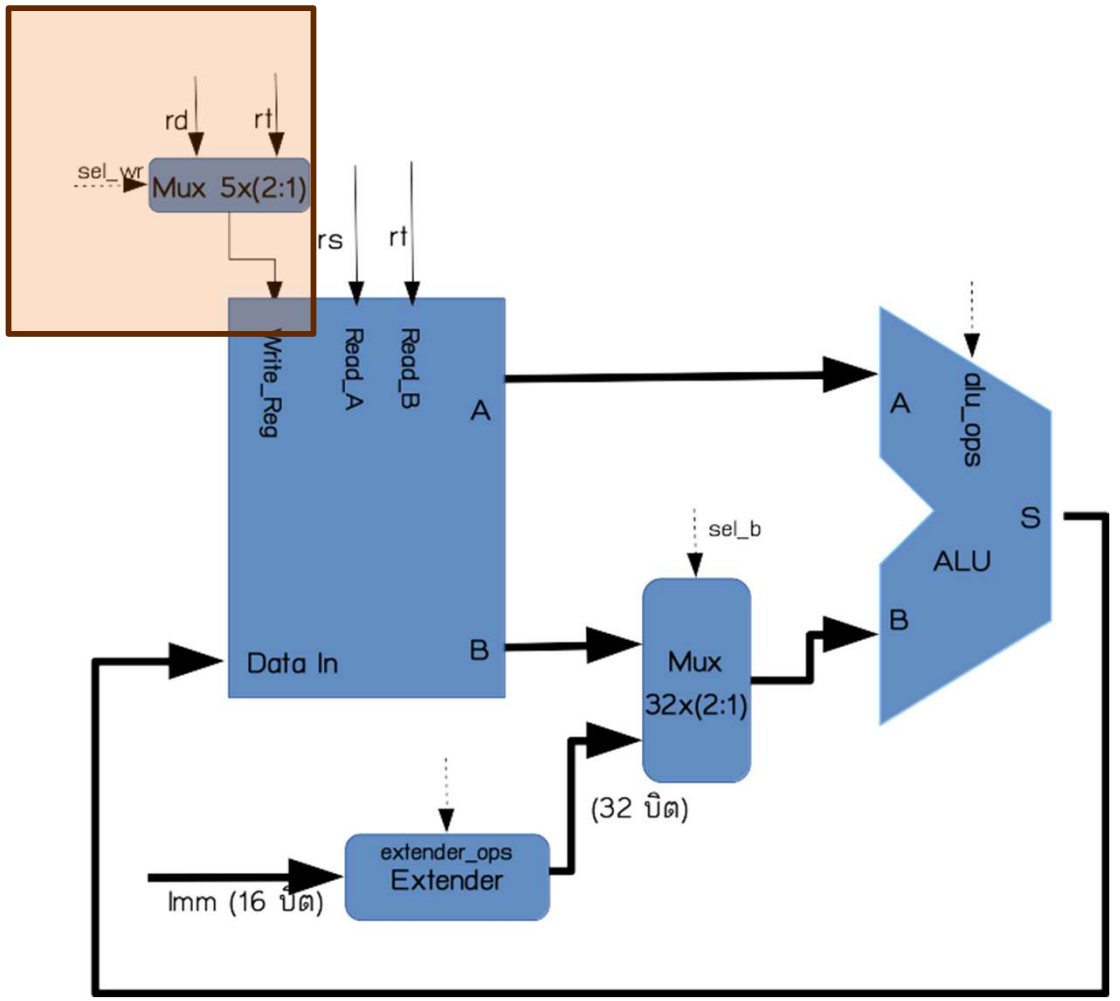




Add

★ ADD rd, rs, rt ; (R-type)

- R[rd] ← R[rs] + R[rt];
  PC ← PC + 4





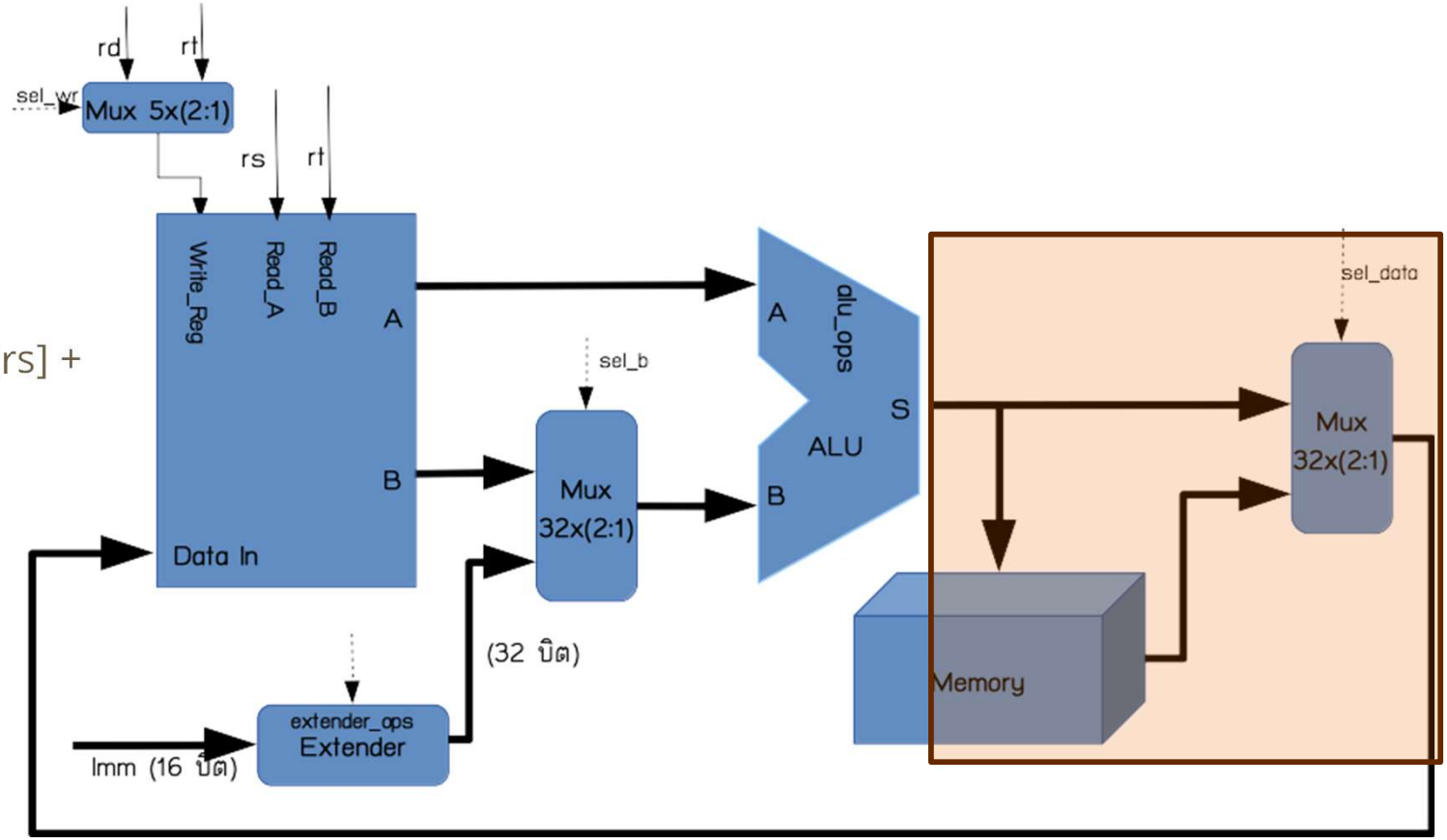
LW

★ LW rt, imm(rs)

; (I-type)

- R[rt] ← MEM[R[rs] + sign_ext(imm)];

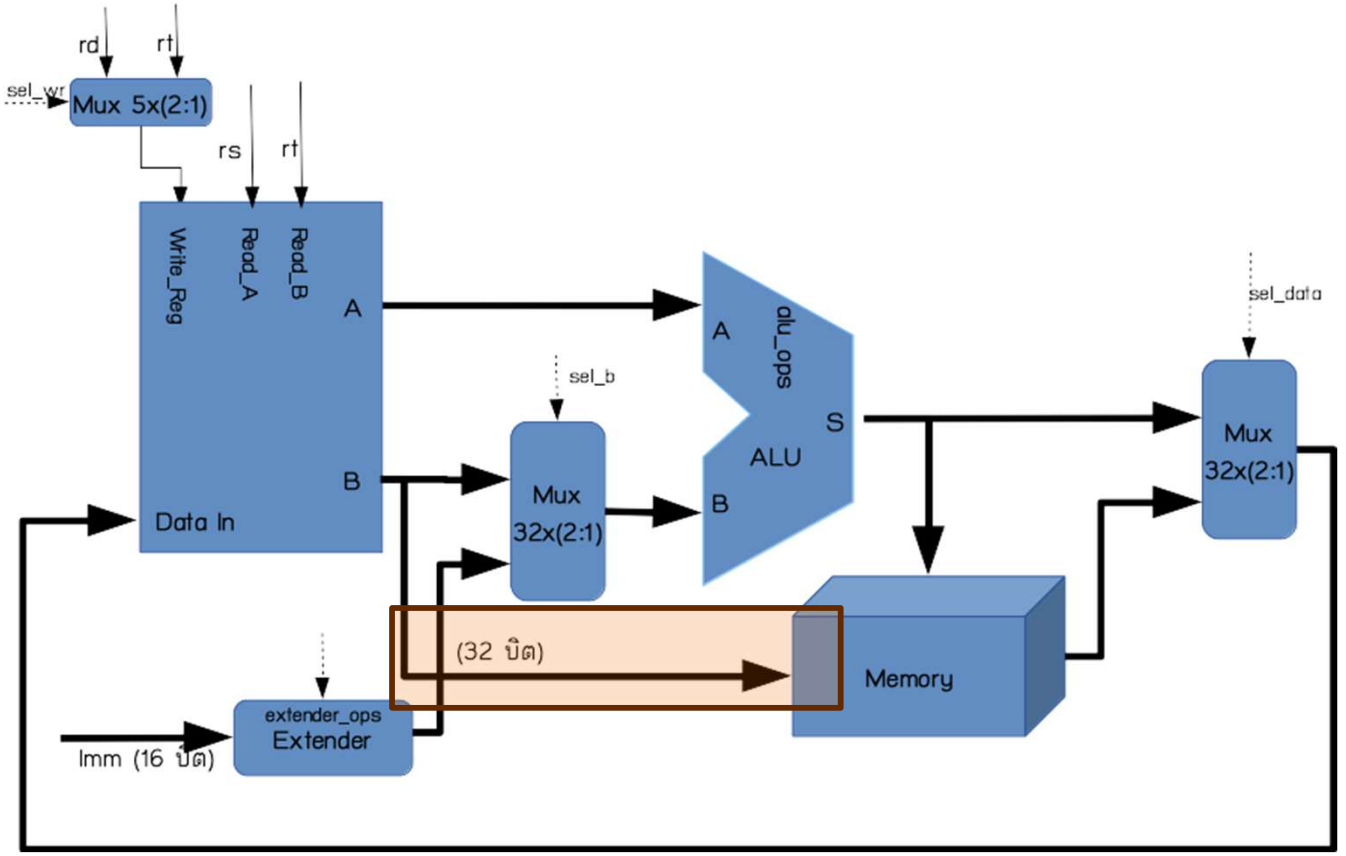
  PC ← PC + 4





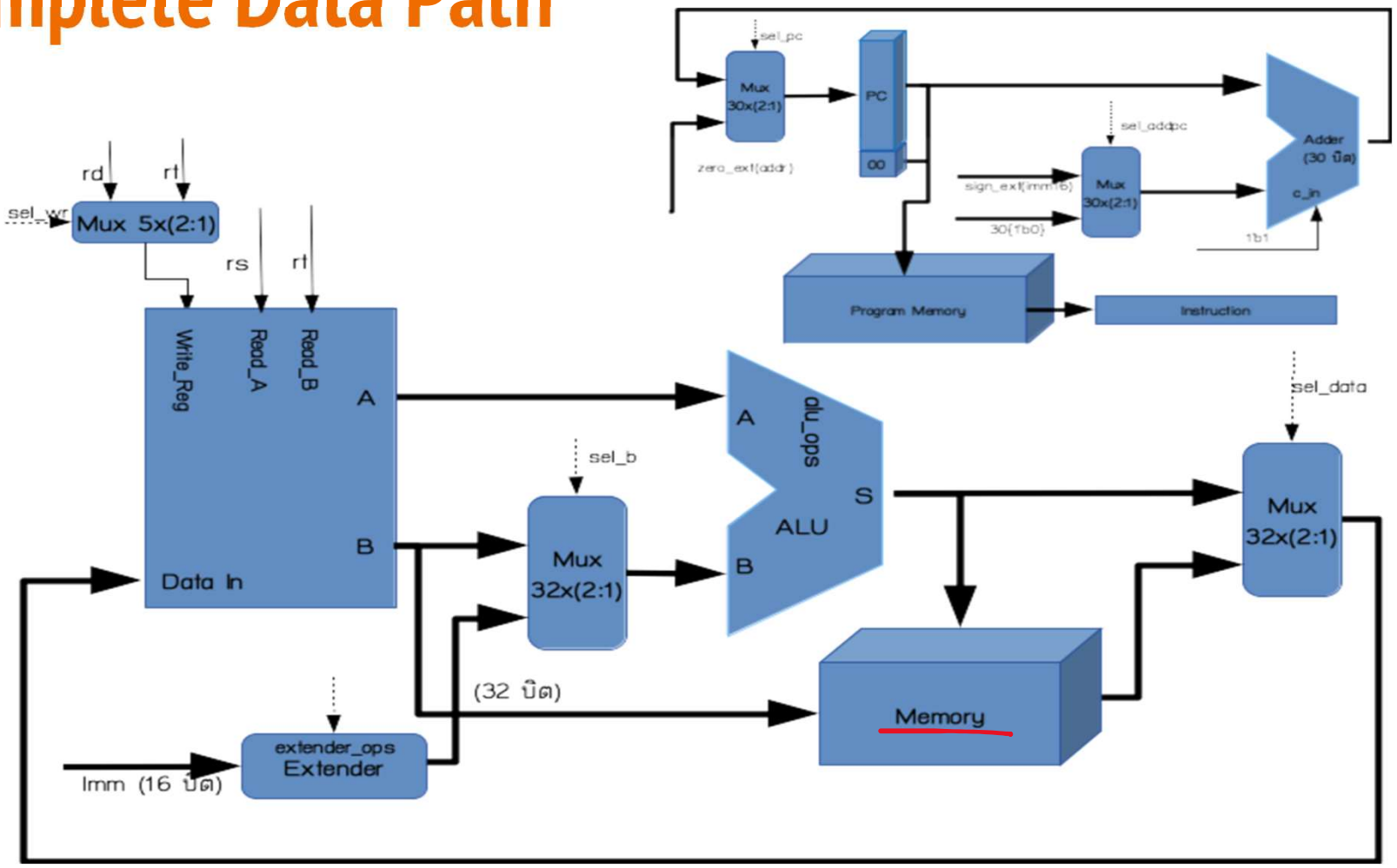
SW

- ★ SW rt, imm(rs) ; (I-type)
  - MEM[R[rs] + sign_ext(imm)] ← R[rt];
  - PC ← PC + 4





Complete Data Path





Step 4: RTL > Control (for the given datapath)

- Control Signals
  - sel_pc
  - sel_addpc
  - sel_wr
  - sel_b
  - sel_data
  - reg_wr
  - Mem_wr
  - alu_ops



Step 5: Control

Opcode	Zero (ALU)	sel_pc 0 – PC 1 - addr	sel_addpc 0 – 0 1 - addr	sel_wr 0 – rd 1 – rt	sel_b 0 – R[rt] 1 - imm	sel_data 0 – S 1 - M	reg_wr 1 – wr	mem_wr 1 – wr	alu_ops
ORI 010000	x	0	0	1	0	0	0	0	01
ORUI 010001	x	0	0	1	0	0	0	0	01
ADD 000001	x	0	0	0	0	0	0	0	00
LW 011000	x	0	0	1	1	1	0	0	00
SW 01100	x	0	0	x	1	x	0	1	00
BEQ 100100	0 1	0 0	0 1	x	x	x	0	0	10
JMP 110000	x	1	x	x	x	x	0	0	00



Control Implementation

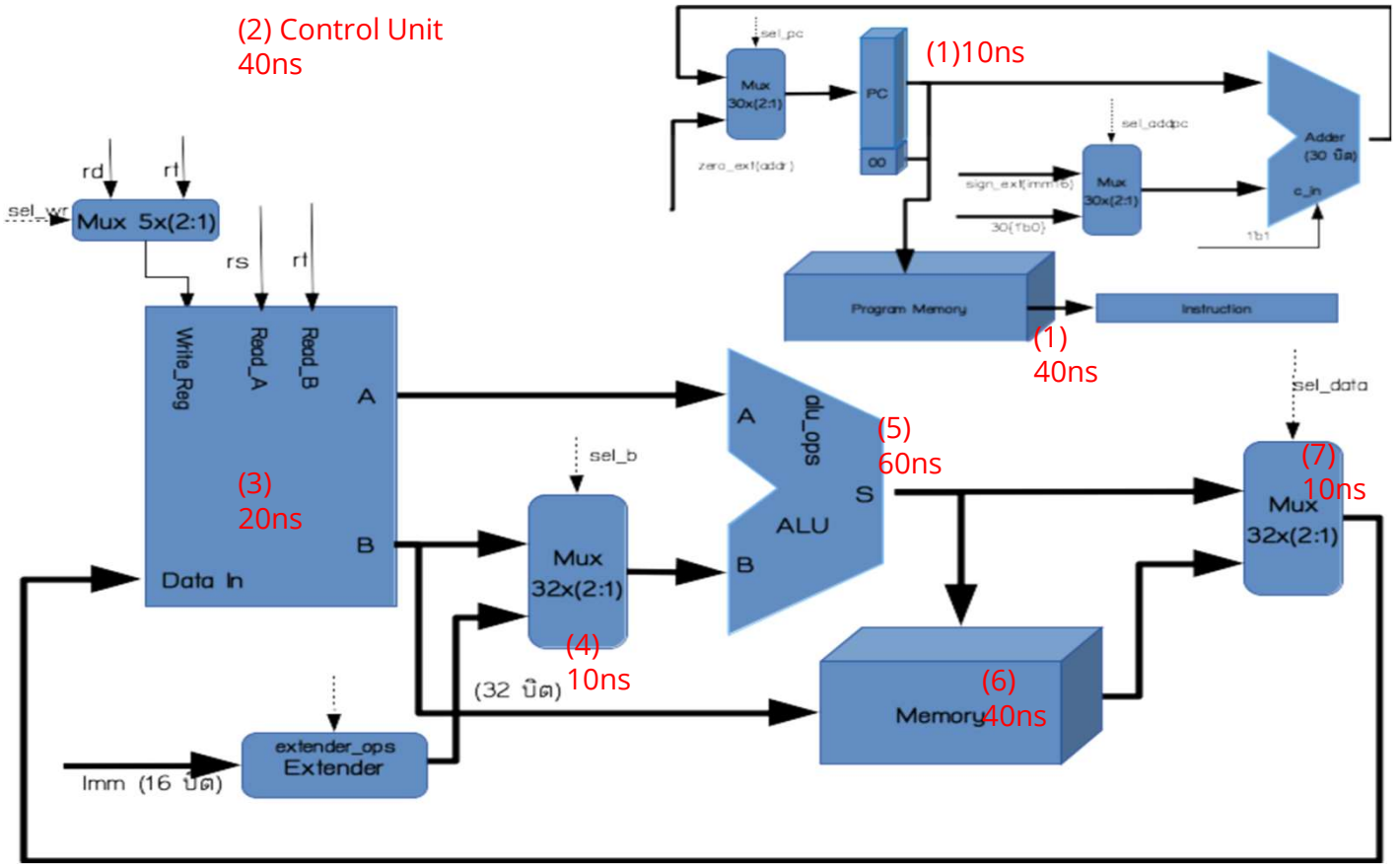
- ★ Combinational Logic
- ★ ROM
- ★ PLA



Critical Path

(1) 10 + 40 + (2) 40 + (3) 20 + (4) 10 + (5) 60 + (6) 40 + (7) 10

- ★ Longest path?
- ★ Instruction ← MEM[PC]
  - PC 10ns
  - MEM 40ns
- ★ Control/Decode 40ns
- ★ Register 20ns
- ★ Extender 10ns
- ★ MUX 10ns
- ★ ALU 60ns
- ★ MUX 10ns





Summary

- ★ The simplicity of nanoLADA makes it easy.
  - Few instructions
  - Fixed-Length instruction set
  - Immediate and Location (Displacement) share the same field.
- ★ Single Cycle datapath
  - CPI=1
  - Cycle Time - Long
- ★ How to make it faster?



Given an ISA, can you design datapath?





Exercises





Performance

★ For the given single cycle processor, if a benchmark contains 2 millions instructions, calculate the CPU time.
Assuming that the critical path is 230 ns.

$$clock\ rate = \frac{10^9}{230} = 4.35 \times 10^6\ Hz$$

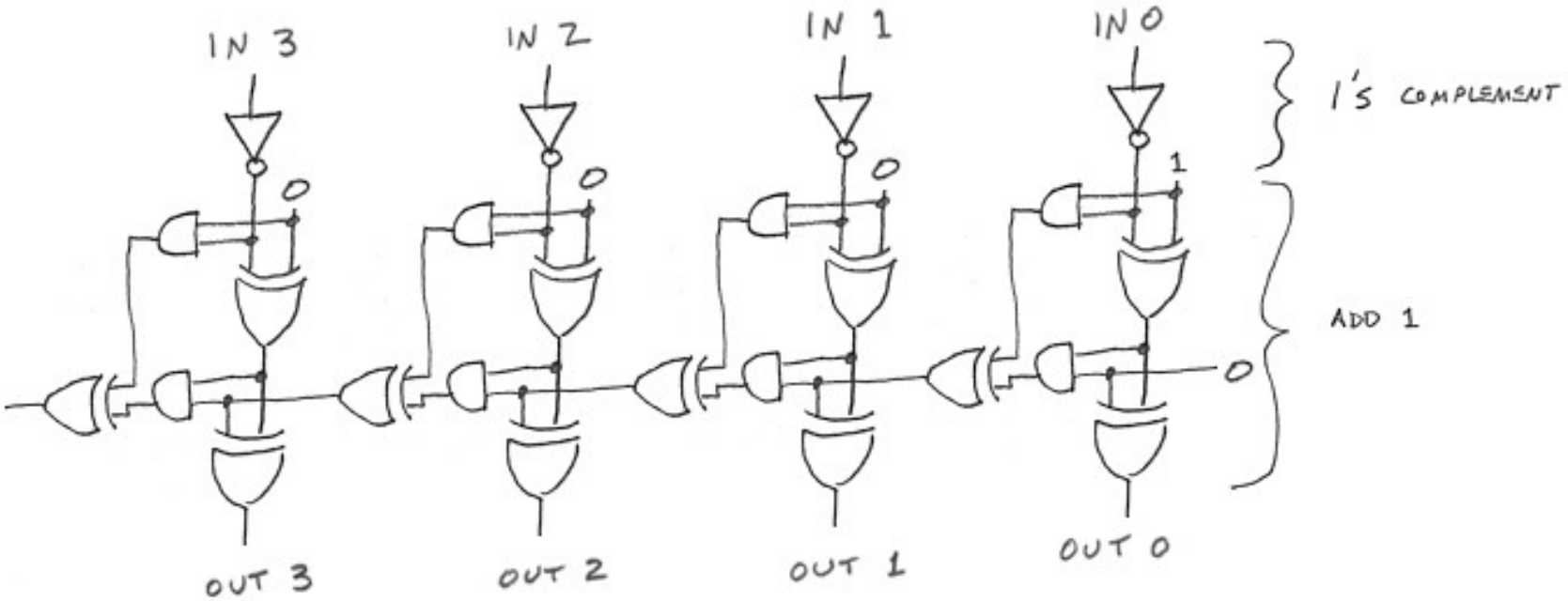


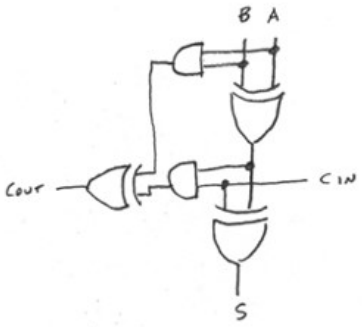
ALU

★ Please design an ALU that supports the following operations

alu_ops	Operation
00	S ← A + B ; z ← (S==0)
01	S ← A | B ; z ← (S==0)
10	S ← A – B ; z ← (S==0)







https://cs.stackexchange.com/questions/51034/twos-complement-using-only-logic-gates



PLA Implementation

★ How to implement the control signal (in p.26) with PLA?

G1 represents ORI (010000) - $G1=x5^{-}x4x3^{-}x2^{-}x1^{-}x0^{-}$

G2 represents ORUI (010001) - $G2=x5^{-}x4x3^{-}x2^{-}x1^{-}x0$

G3 represents ADD (000001) - $G3=x5^{-}x4^{-}x3^{-}x2^{-}x1^{-}x0$

G4 represents LW (011000) - $G4=x5^{-}x4x3x2^{-}x1^{-}x0^{-}$

G5 represents SW (011100) - $G5=x5^{-}x4x3x2x1^{-}x0^{-}$

G6 represents BEQ (100100) - $G6=x5x4^{-}x3^{-}x2x1^{-}x0^{-}$

G7 represents JMP (110000) - $G7=x5x4x3^{-}x2^{-}x1^{-}x0^{-}$

$sel_pc=G7$

$sel_addpc=G6+zero$

$sel_wr=G1+G2+G4$

$sel_b=G1+G2+G4+G5$

$sel_data=G4$

$reg_wr=G1+G2+G3+G4$

$mem_wr=G5$



Adding scale operation

- Scale LD $r1, ($r2)[$r3*4]



End of Chapter 4





Multiple Cycle Processor

Chapter 5



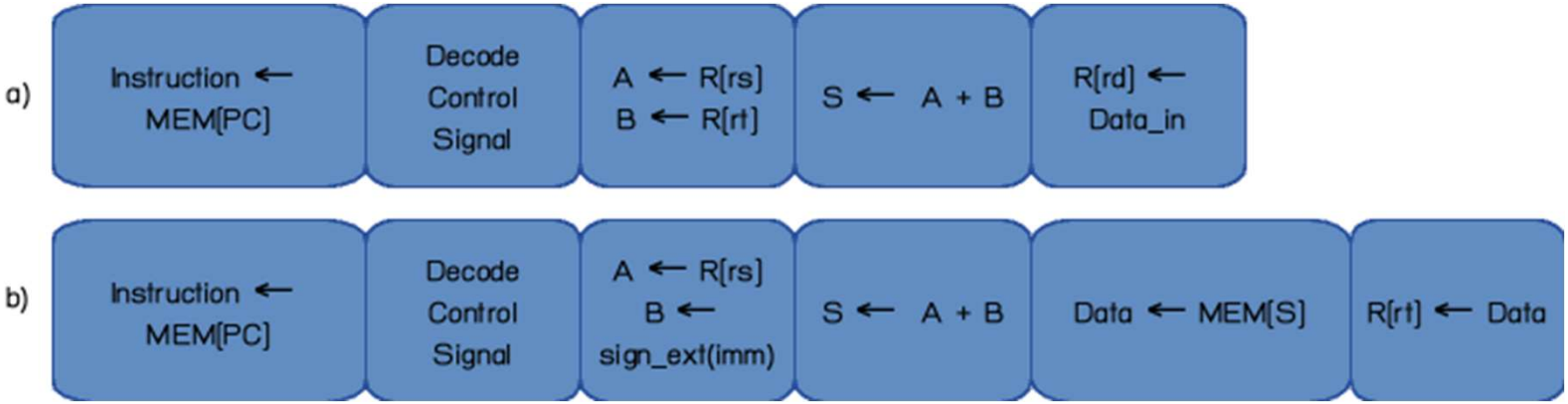
Multiple Cycle Processor

- ★ What is wrong with Single Cycle?
- ★ Memory Access Time
- ★ How to reduce cycle time
- ★ Data Path Partitioning
- ★ Logical RTL vs. Physical RTL
- ★ Control Model
- ★ Implementing the Control
- ★ Performance Evaluation



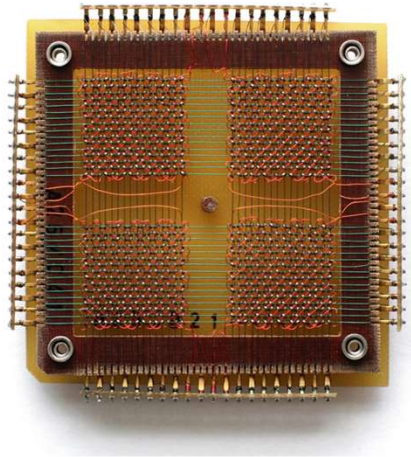
What is wrong with Single Cycle?

- ★ Long Cycle Time
- ★ All instructions take as much time as the slowest.
- ★ Single Cycle forces all instructions to wait for the critical path.
- ★ Real memory is not so nice as our idealized memory (take several cycles).

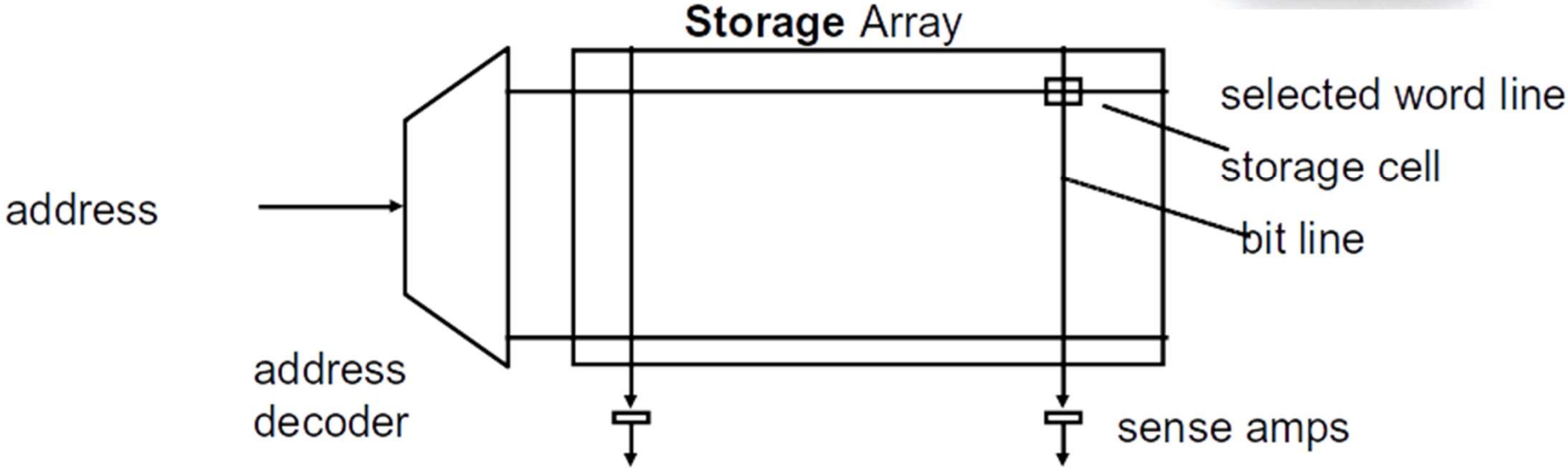




Memory Access Time



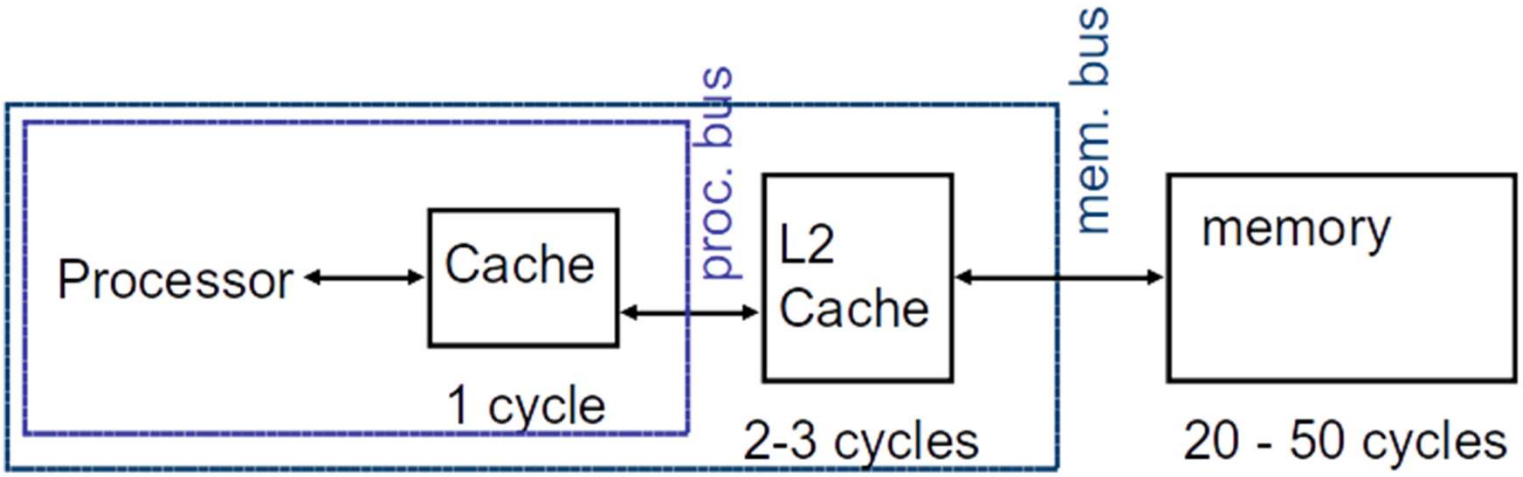
- Physical rules of memory:
  - Fast memories are small.
  - Large memories are slow.





Memory Access Time (ctd.)

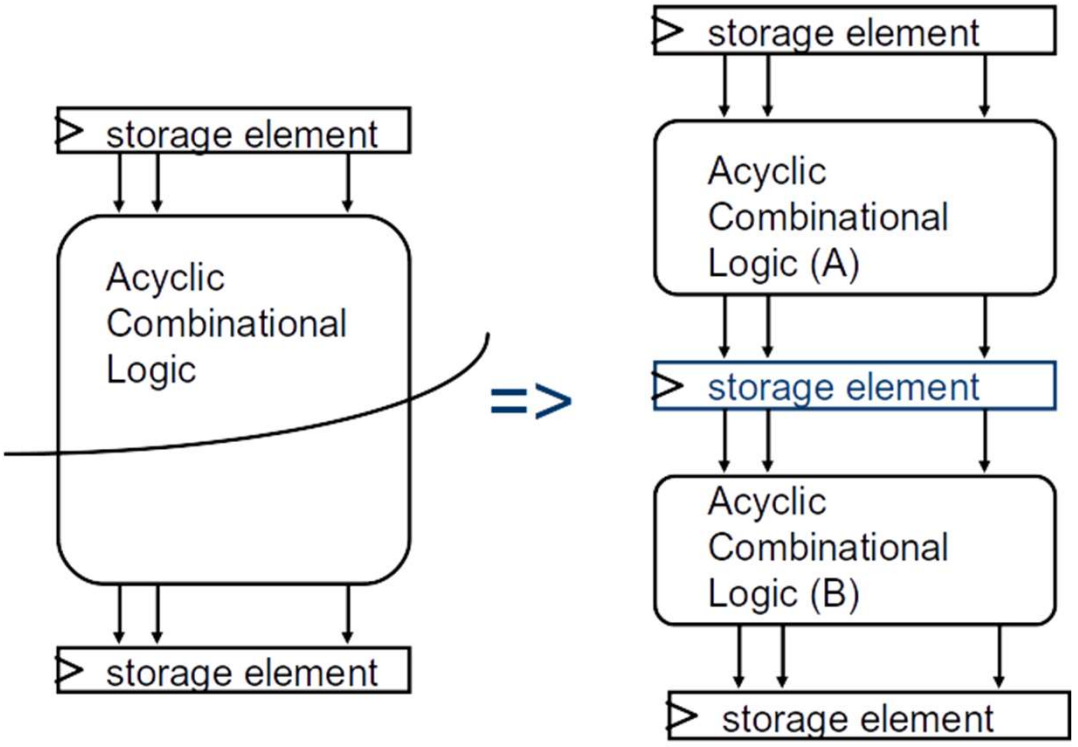
- ★ With memory hierarchy (CACHE), we may assume that read/write can be done in one cycle (for now).
- ★ We will revisit this concept later in chapter 8.





How to reduce cycle time?

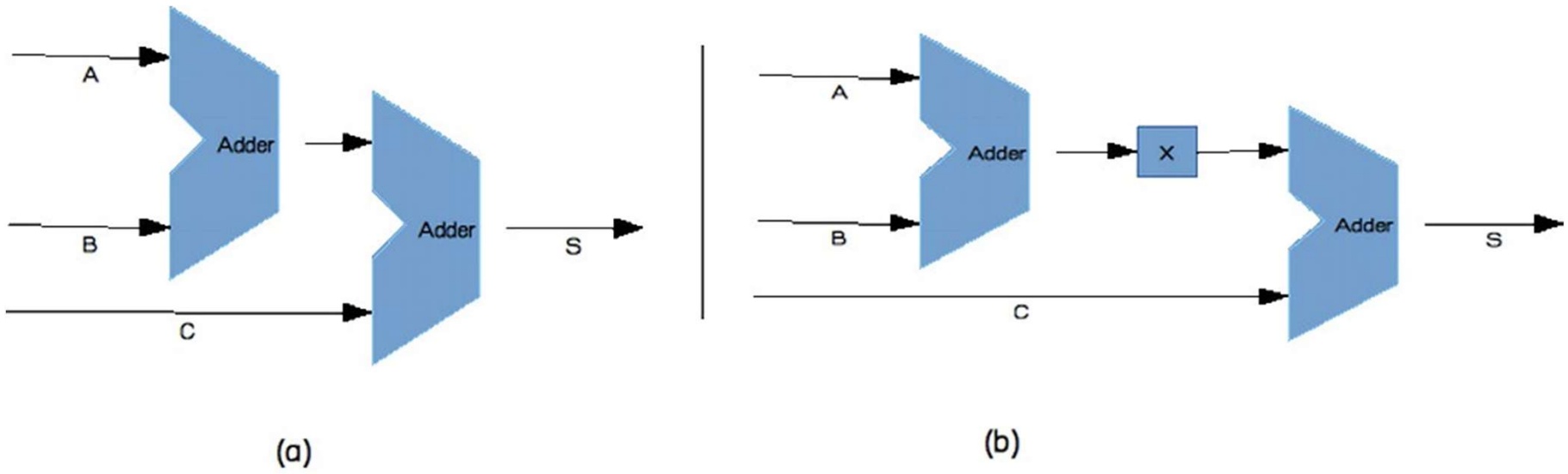
- ★ Partition combinational dependency graph and insert intermediate register (latch).
- ★ Use several (fast) cycles, rather than one slow cycle.
- ★ Some cycles can be skip for certain instructions.





Datapath Partitioning

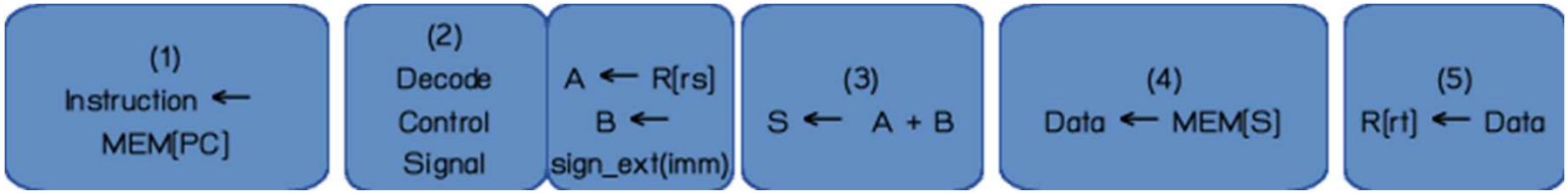
★ Add intermediate register(s) in between.





Basic Limits on Cycle Time

- ★ Each part should take similar amount of times
- ★ For our design, 5 parts
  - Fetch (Instruction←MEM[PC]; PC←PC+4)
  - Decode (A← ; B←)
  - Execution (S← A+B)
  - Memory Access (Data ← MEM[S])
  - Write back (R[rt]←Data)





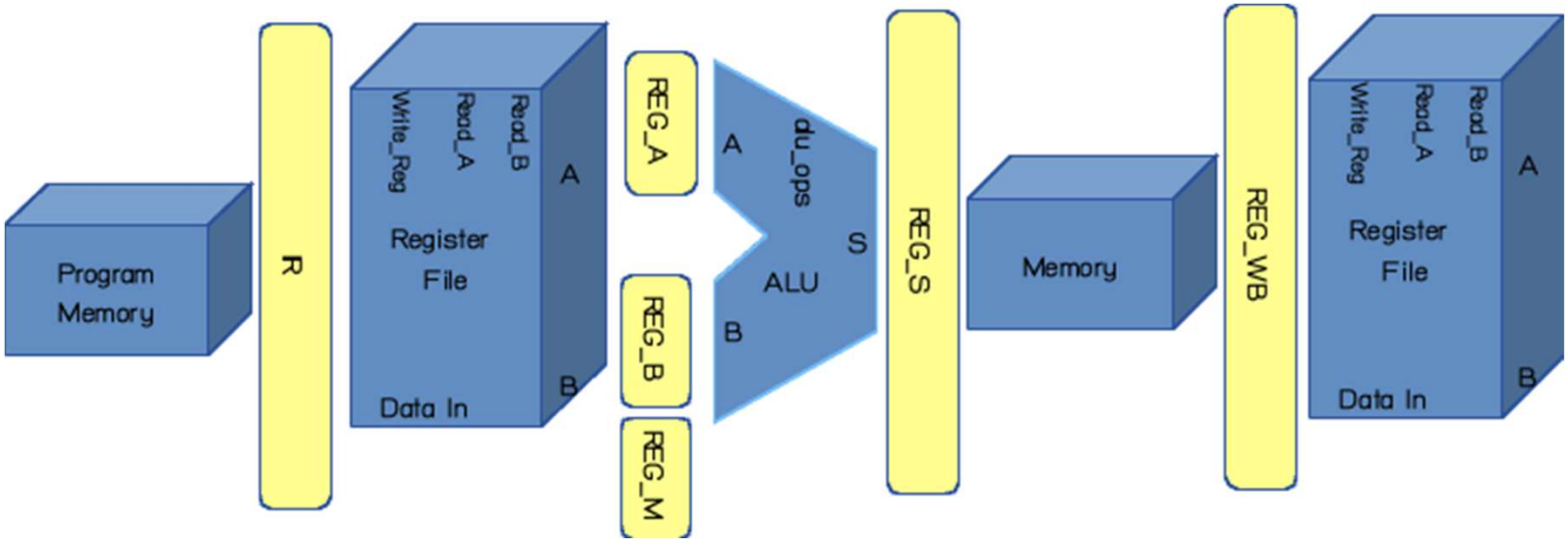
Potential Speedup

- ★ Assume that cycle time of the original single cycle processor is 210ns. If multiple cycle can reduce the cycle time to 45ns, calculate the new average CPI that will give us at least the same performance as the original one.



New Datapath

★ Add registers between smallest steps





Our Design Steps

1. ISA Analysis
2. Components design
3. Assembly Datapath
4. Datapath + Logical RTs → Physical RTs
5. Physical RTs → Control



ORI

- Logical RTL:
  - R[rt] ← R[rs] | zero_ext(imm); PC ← PC + 4

Cycle	Physical RTL
1	IR ← MEM[PC] PC ← PC + 4
2	REG_A ← R[rs] REG_B ← zero_ext(imm)
3	REG_WR ← REG_S ← REG_A | REG_B
4	R[rt] ← REG_WR



ORUI

- Logical RTL:
  - R[rt] ← R[rs] | zero_pad(imm); PC ← PC + 4

Cycle	Physical RTL
1	IR ← MEM[PC] PC ← PC + 4
2	REG_A ← R[rs] REG_B ← zero_pad(imm)
3	REG_WR ← REG_S ← REG_A | REG_B
4	R[rt] ← REG_WR



Add

- Logical RTL:
  - R[rd] ← R[rs] + R[rt]; PC ← PC + 4

Cycle	Physical RTL
1	IR ← MEM[PC] PC ← PC + 4
2	REG_A ← R[rs] REG_B ← R[rt]
3	REG_WR ← REG_S ← REG_A + REG_B
4	R[rt] ← REG_WR



LW

★ Logical RTL:

- R[rt] ← MEM[R[rs] + sign_ext(imm)]; PC ← PC + 4

Cycle	Physical RTL
1	IR ← MEM[PC] PC ← PC + 4
2	REG_A ← R[rs] REG_B ← sign_ext(imm)
3	REG_S ← REG_A + REG_B
4	REG_WR ← MEM[REG_S]
5	R[rd] ← REG_WR



SW

- Logical RTL:
  - MEM[R[rs] + sign_ext(imm)] ← R[rt]; PC ← PC + 4

Cycle	Physical RTL
1	IR ← MEM[PC] PC ← PC + 4
2	REG_A ← R[rs] REG_B ← sign_ext(imm) REG_M ← R[rt]
3	REG_S ← REG_A + REG_B
4	MEM[REG_S] ← REG_M



BEQ

- Logical RTL:
  - If (R[rs] == R[rt]) then
  - PC ← PC + 4 + (sign_ext(imm) * 4)
  - else PC ← PC + 4

Cycle	Physical RTL
1	IR ← MEM[PC] PC ← PC + 4
2	(decode)
3	If (R[rs] == R[rt]) THEN PC ← PC + (sign_ext(imm) * 4) ENDIF



JMP

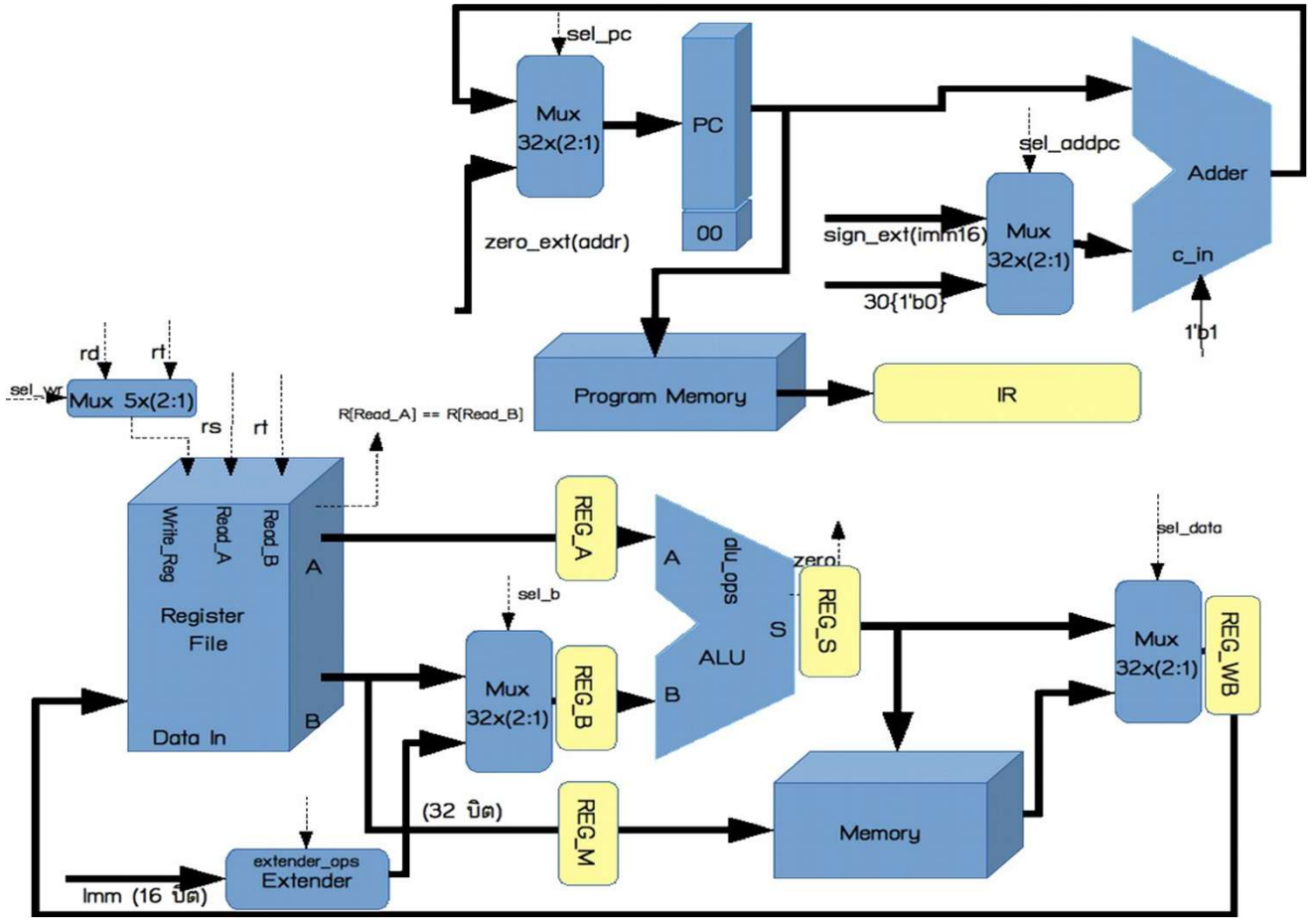
- Logical RTL:
  - PC ← (addr * 4)

Cycle	Physical RTL
1	IR ← MEM[PC] PC ← PC + 4
2	(decode)
3	PC ← addr * 4



Complete Data Path

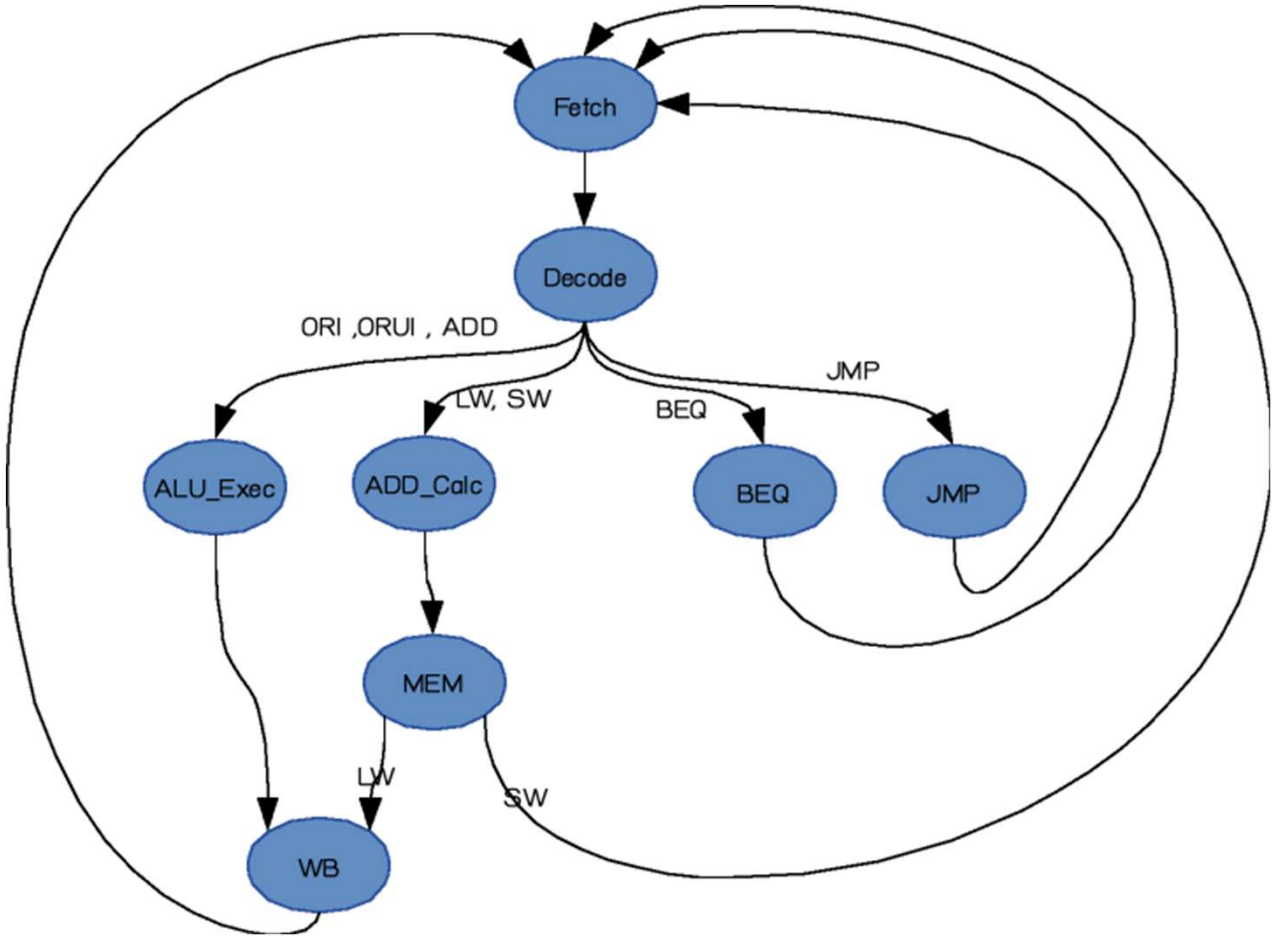
- We may:
  - ALU for adding PC
  - Combine Memory





Control Path

- ★ State Diagram





Control Signal

State	Signal	Next State
Fetch	IR ← MEM[PC]	Decode
Decode	IF (op[5..4] == 01) THEN ; I-Type sel_wr ← 1 sel_b ← 1 ENDIF (wait for decoder)	ALU_Exec ADD_Calc BEQ JMP
ALU_Exec	IF (opcode in {ORI, ORUI}) THEN alu_ops ← OR ELSE alu_ops ← ADD ENDIF	
ADD_Calc	alu_ops ← ADD	MEM
MEM	sel_data ← 1 IF (op[2] == 1) THEN mem_wr ← 1 ENDIF	WB
WB	reg_wr=1	Fetch
BEQ	If (R[Read_A] == R[Read_B]) THEN sel_addpc ← 1 END IF	Fetch
JMP	sel_pc ← 1	Fetch



Control Implementations





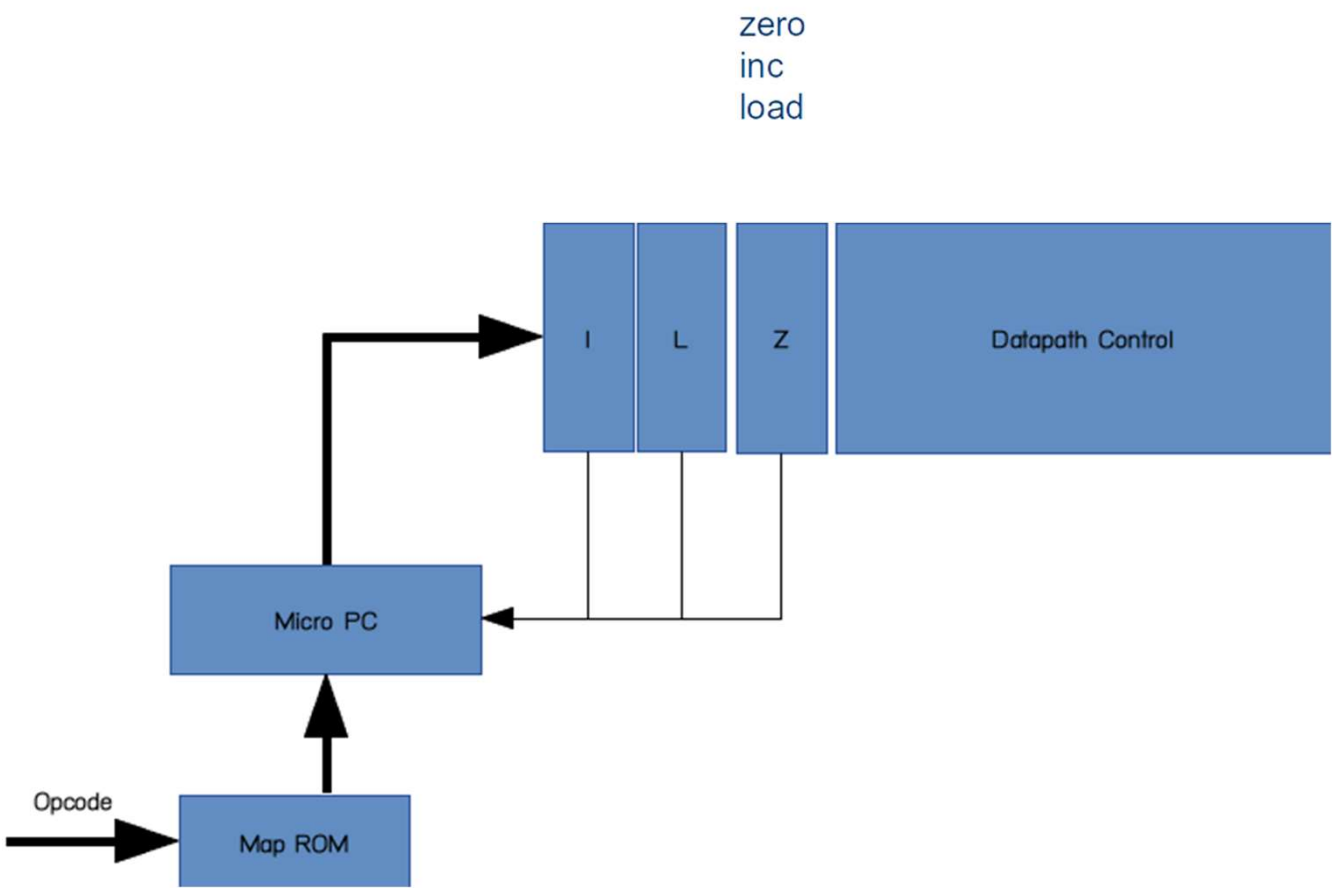
Sequential Circuits

- ★ State assignment
- ★ Truth Table (Next-state function)



Microprogram

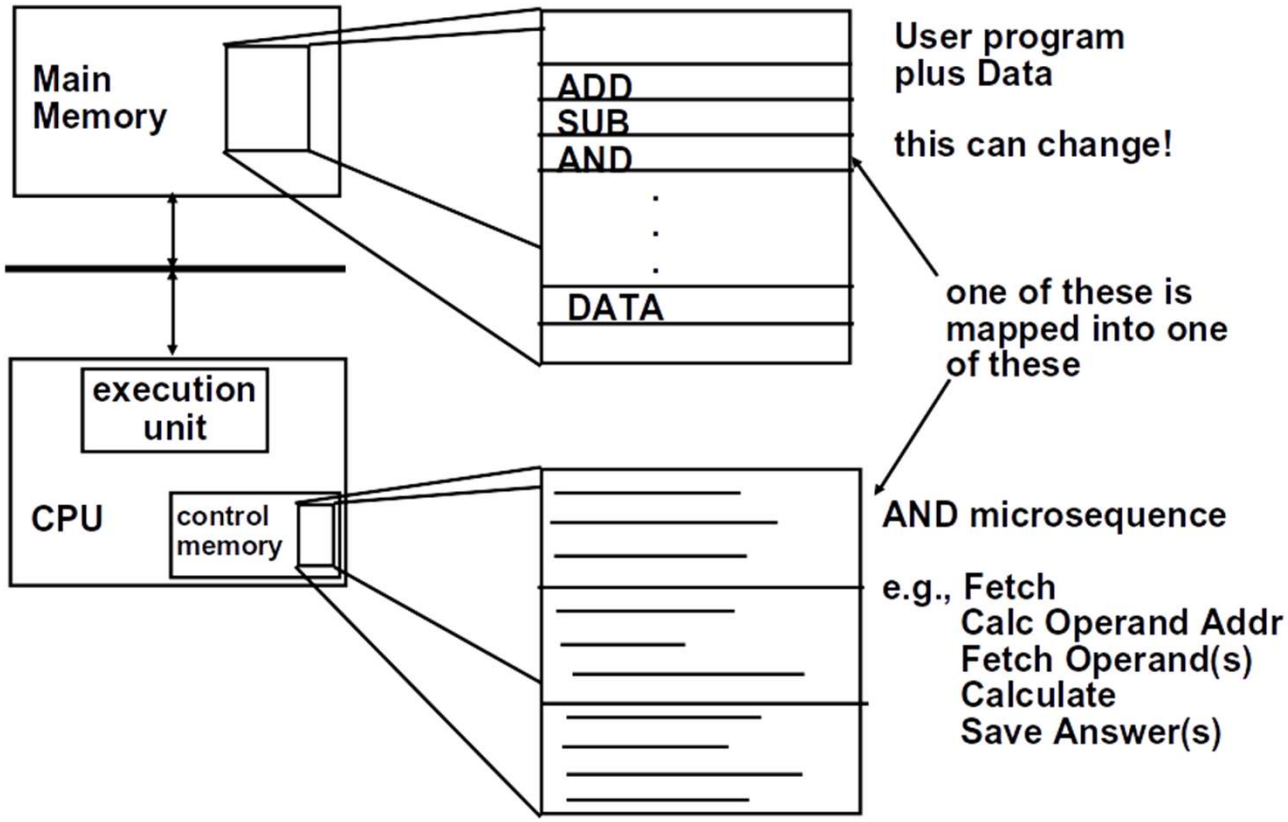
- ★ Microsequencer
- ★ Each microinstruction specify control signals





Microprograms (ctd.)

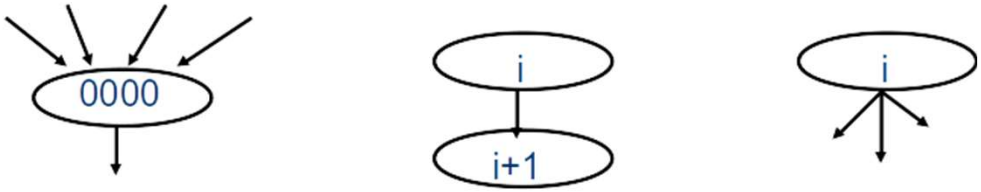
- ★ Map arbitrary Macroinstructions (Assembly) into several microinstructions.



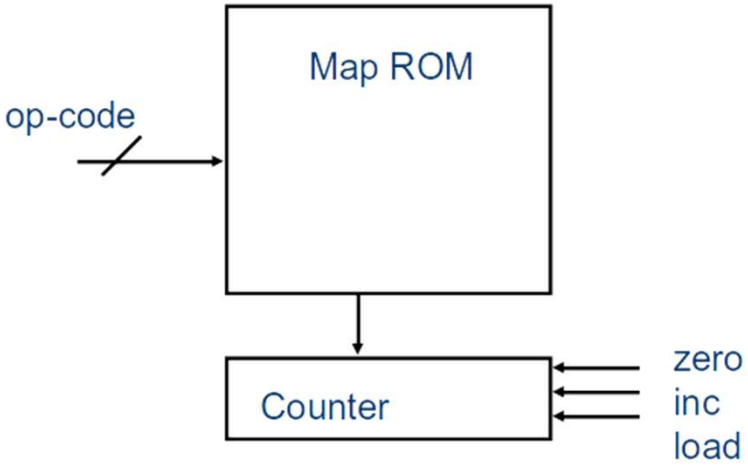


Microprograms (ctd.)

★ Map ROM for control signal



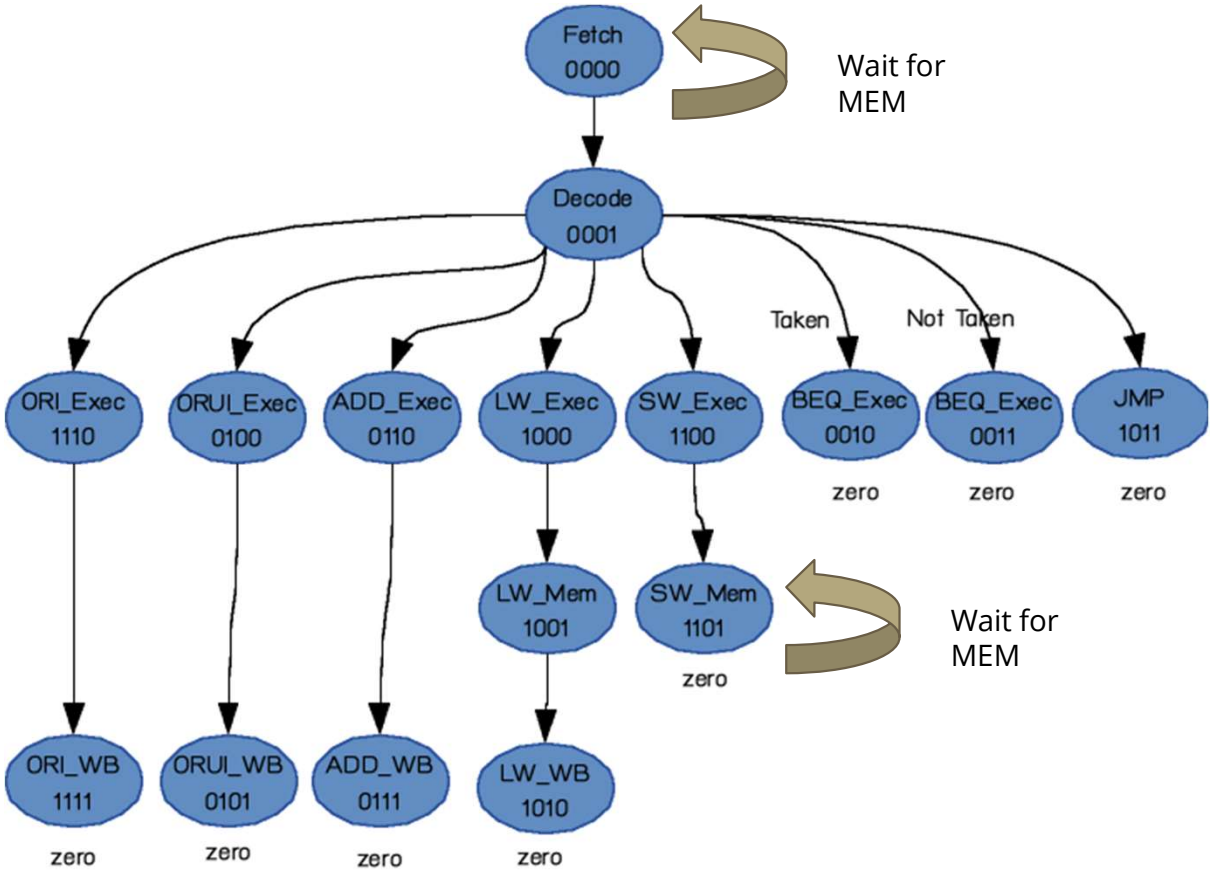
★ With appropriate hardware, we can change microprogram (Map ROM) to implement different ISA on the same hardware.





State Assignment

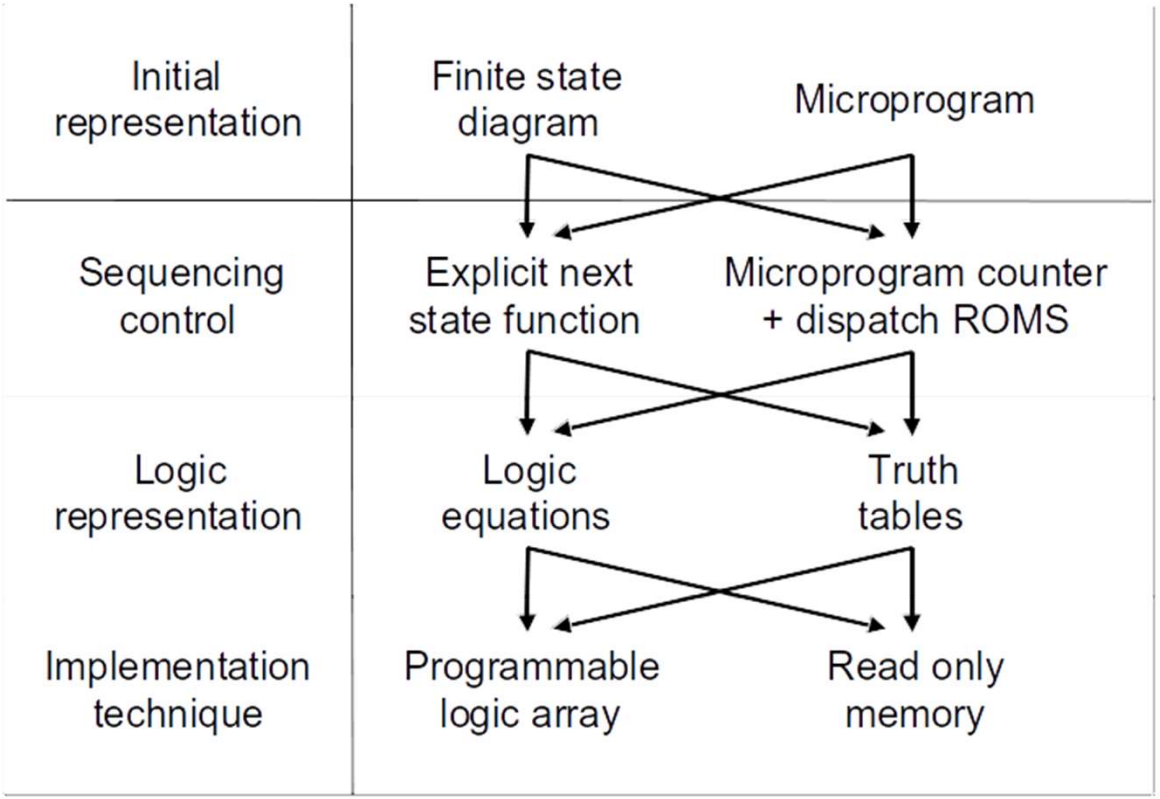
- ★ In reality, we may have to add wait state for the non-ideal memory





Big Picture

- ★ There are several ways to implement the processor.
- ★ For us, we will use VerilogHDL.





Summary

- ★ Disadvantages of the single cycle processor
  - Long cycle time
- ★ Advantages of multiple cycle processor
  - Divide the instructions into smaller steps
  - Execute each step (instead of the entire instruction) in one cycle
- ★ Partition datapath into equal size chunks to minimize cycle time
- ★ Control is a finite state machine.
- ★ CISC makes control more complicated.



Notes

- We do not take exceptions into the design at the moment.



Exercises





How may cycles?

- From the given code, calculate CPU Time.
  - LW $r2, 0($r3)
  - LW $r3, 4($r3)
  - BEQ $r2, $r3, end_program #assume not taken
  - ADD $r5, $r2, $r3
  - SW $r5, 8($r3)
  - end_program:.... # end of program
- Assuming that cycle time is 40ns.



Share Components

- ★ How to adjust the datapath so that we can share ALU with Adder (for PC)?



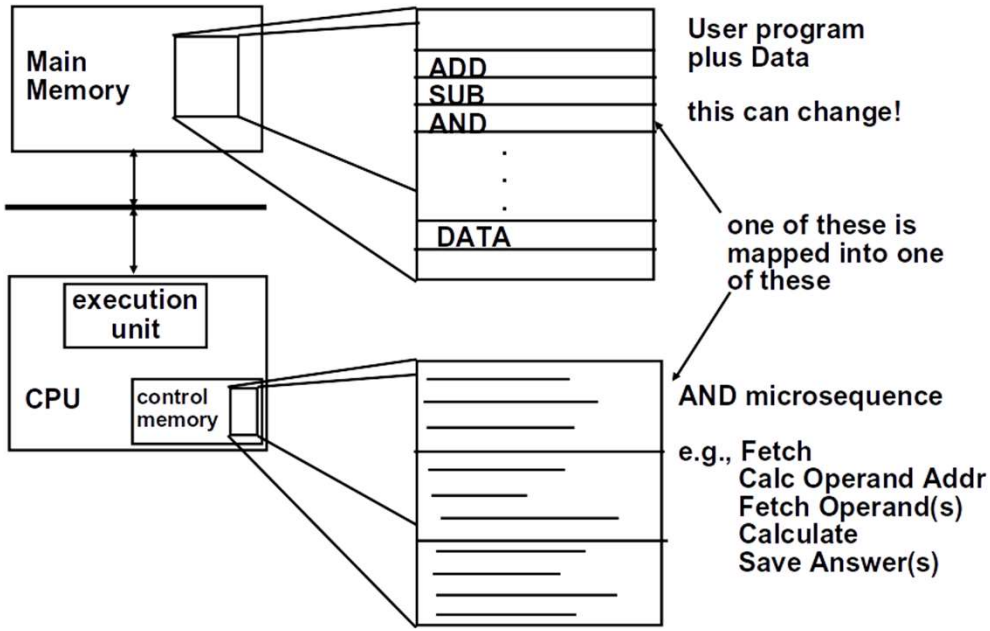
Interrupt and Exception

- ★ What do we have to do to have our processor support Interrupt and Exception?



MicroInstruction

- Given that a macroinstruction always translate into several microinstructions in the processor, why don't we skip macroinsturctions and program directly in microinstructions? This should take out a translation layer in the processor.





End of Chapter 5





Pipelining (part 1)

Chapter 6



Pipelining

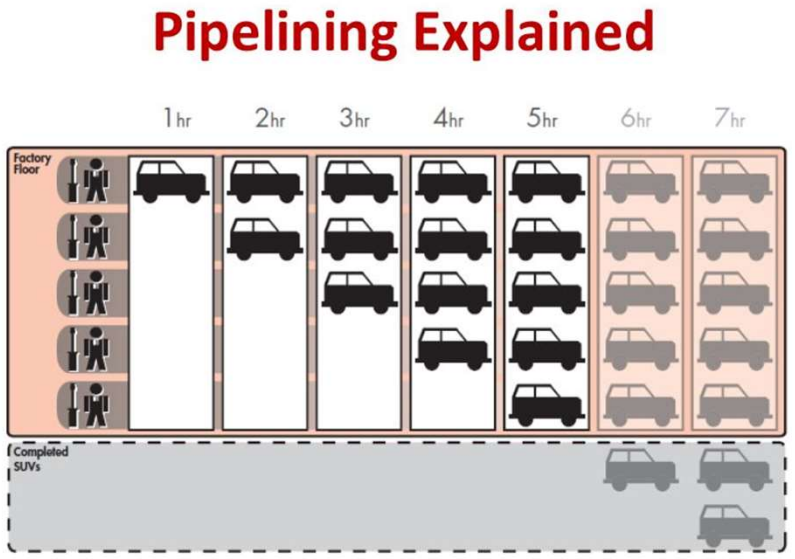
- ★ Pipelining Concept
  - Pipelining in real world
  - Lessons learned
- ★ Potential/Ideal SpeedUp
- ★ Pipelining in Processor
- ★ Representation of Pipeline
- ★ Potential Pipeline Hazards
  - Structural Hazards
  - Data Hazards
  - Control Hazards
- ★ Solutions to Pipeline Hazards



Pipelining in real world

- ★ In industrial, pipelining is used everywhere in the production line.
- ★ The example is the SUV car assembly plant.
- ★ The idea is to start a new task as soon as the resources are available.

- ★ Find more examples of pipelining in real life.



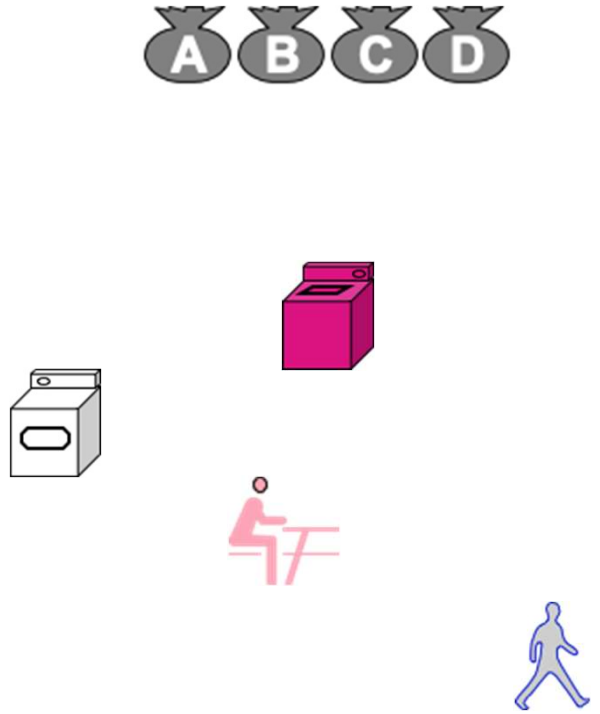
The lifecycle of an SUV in a pipelined factory

Picture taken from
https://images.slideplayer.com/24/7106280/slides/slide_6.jpg



Pipelining in Laundry

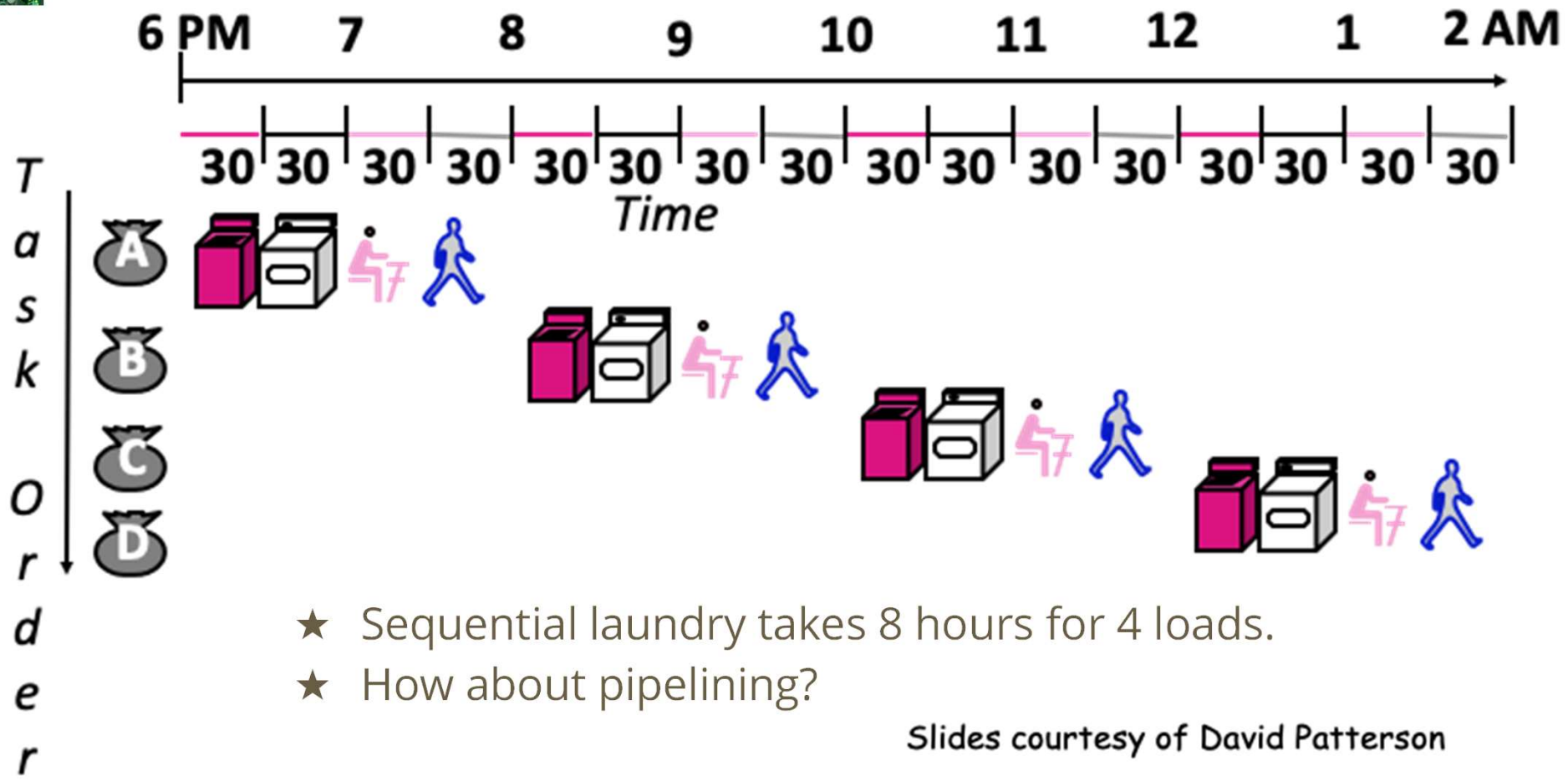
- ★ Laundry Example
- ★ Ann, Brian, Cathy, Dave
  each have one load of clothes
  to wash, dry, and fold
- ★ Washer takes 30 minutes
- ★ Dryer takes 30 minutes
- ★ “Folder” takes 30 minutes
- ★ “Stasher” takes 30 minutes
  to put clothes into drawers



Slides courtesy of David Patterson



Sequential Laundry



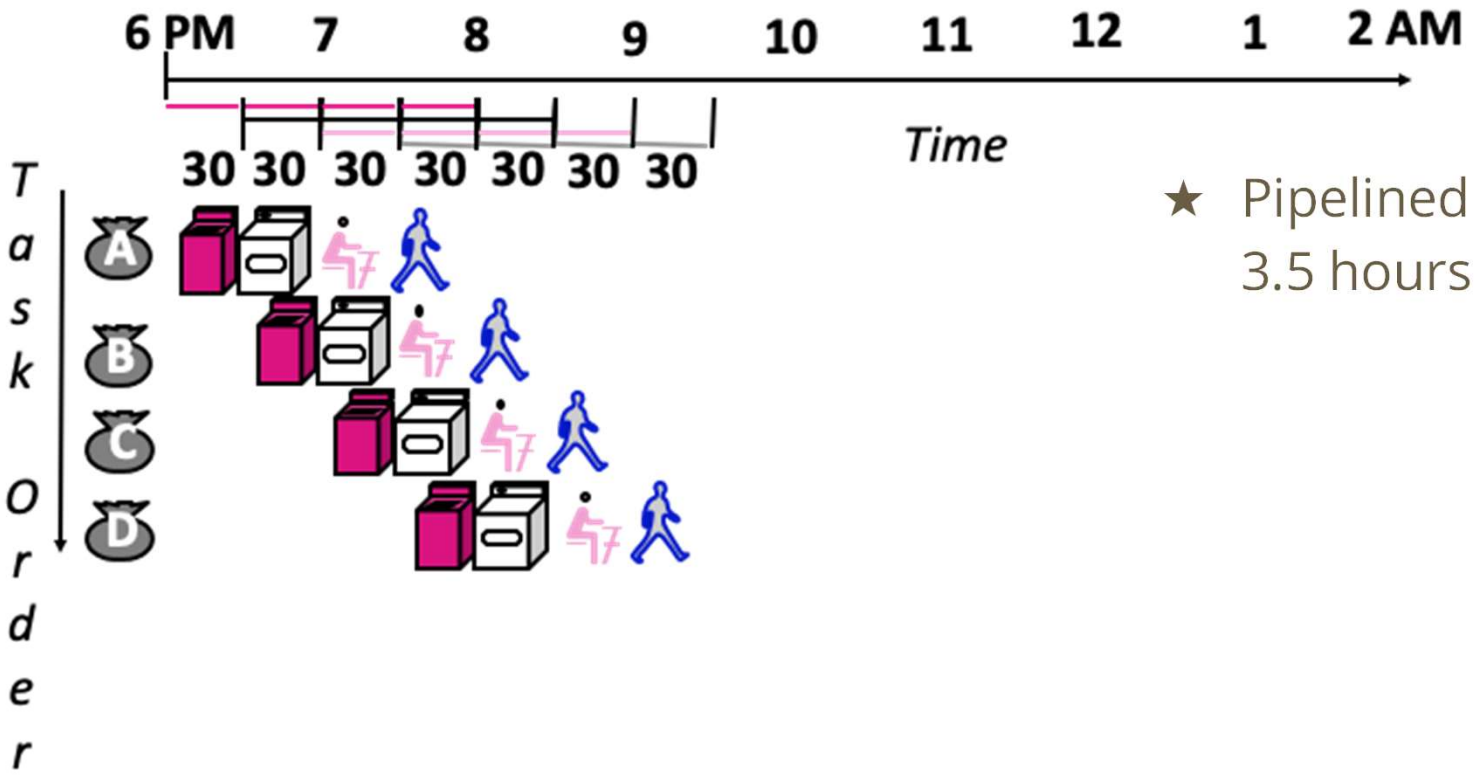
★ Sequential laundry takes 8 hours for 4 loads.

★ How about pipelining?

Slides courtesy of David Patterson



Pipelined Laundry (start work ASAP)



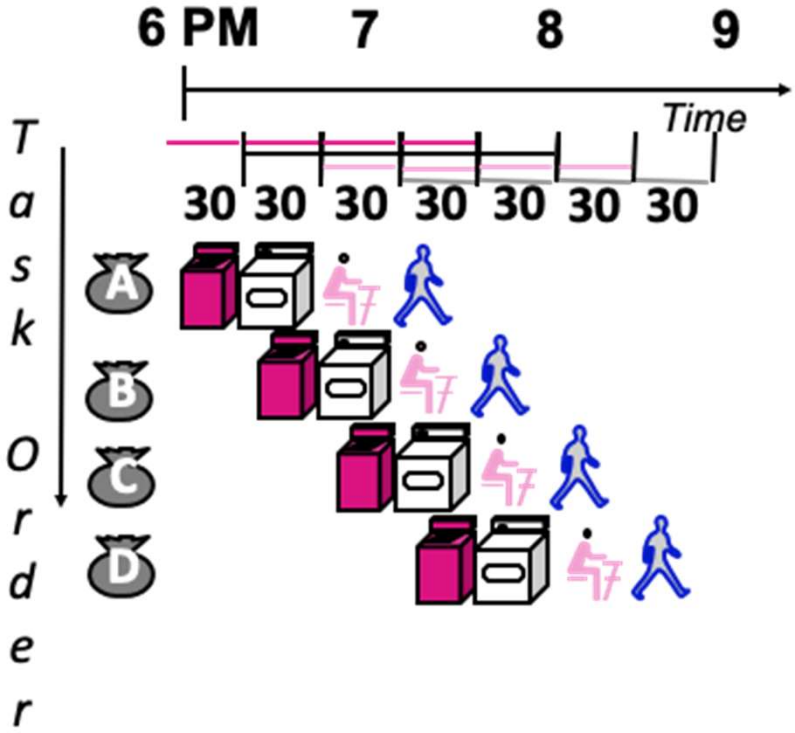
★ Pipelined Laundry taks only 3.5 hours for 4 loads.

Slides courtesy of David Patterson



Lessons Learned (1)

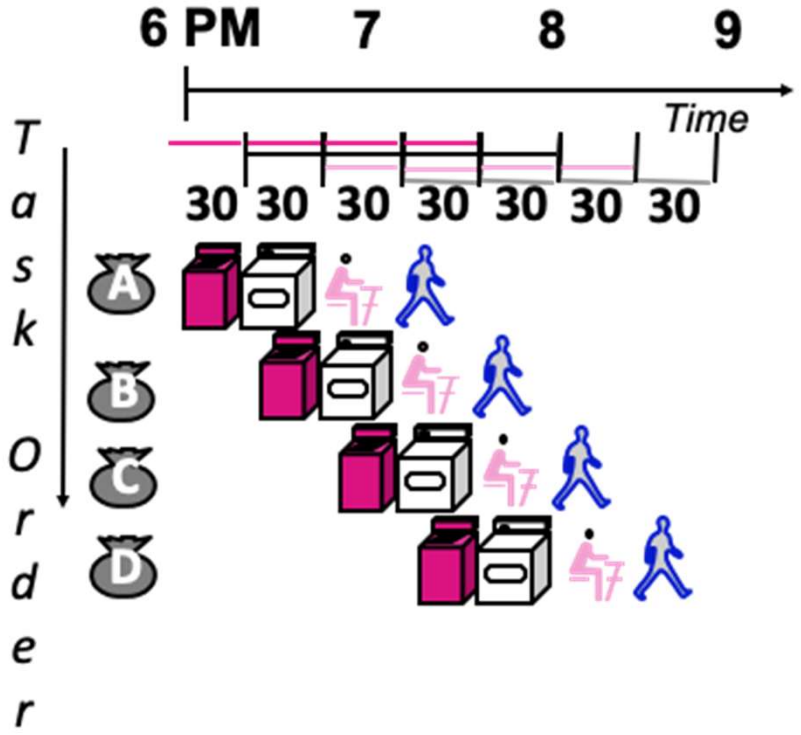
- ★ Pipelining does not improve **latency** of a task.
- ★ Pipelining helps **throughput** of entire workload.
- ★ Multiple tasks run simultaneously on different resources.



Slides courtesy of David Patterson



Lessons Learned (2)



- ★ Pipeline rate limited by the **slowest** stage.
- ★ Unbalanced lengths of pipe stages reduces speedup.
- ★ Time to **fill** pipeline and time to **drain** pipeline reduces speedup.

Slides courtesy of David Patterson



Pop Quiz

- ★ With pipelined Laundry, how long does it take to process k loads?
- ★ Hint: Time to fill the pipe and Time for each load



Potential Speedup

- ★ CPU Time = IC x CPI x Tc
- ★ Multiple Cycle Processor
  - CPU Time = k x 5 x [cycle time]
- ★ Pipelining
  - CPU Time = ([fill cycle] + (k-1) x 1) x [cycle time]
  - [fill cycle] is the number of stage
- ★ Assuming that k is large.
- ★ Potential Speedup is the **number of pipeline stages**

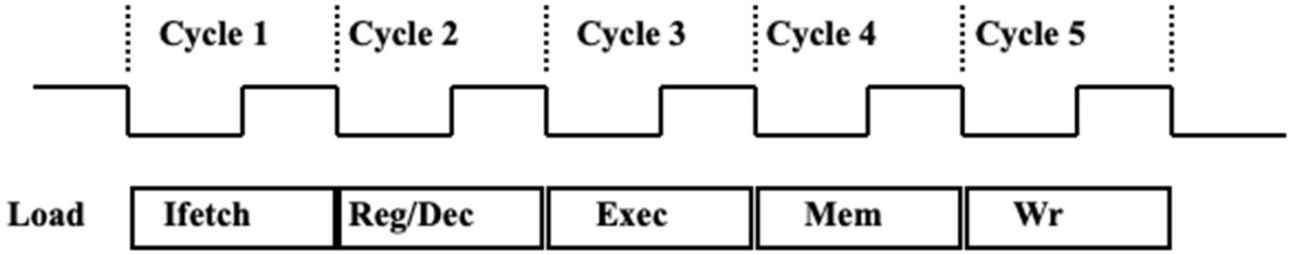
$$n = \frac{CPU\ TIME_{multiple\ cycle}}{CPU\ TIME_{pipeline}} = \frac{k \times 5 \times cycle\ time}{(5+(k-1)\times 1)\times cycle\ time}$$

$$n = \frac{k \times 5}{5+(k-1)\times 1} = \frac{5k}{5+k-1} = \frac{5k}{4+k}$$

$$n = \lim_{k \to \infty} \frac{5k}{5+k-1} = 5$$



The Five stages of Load

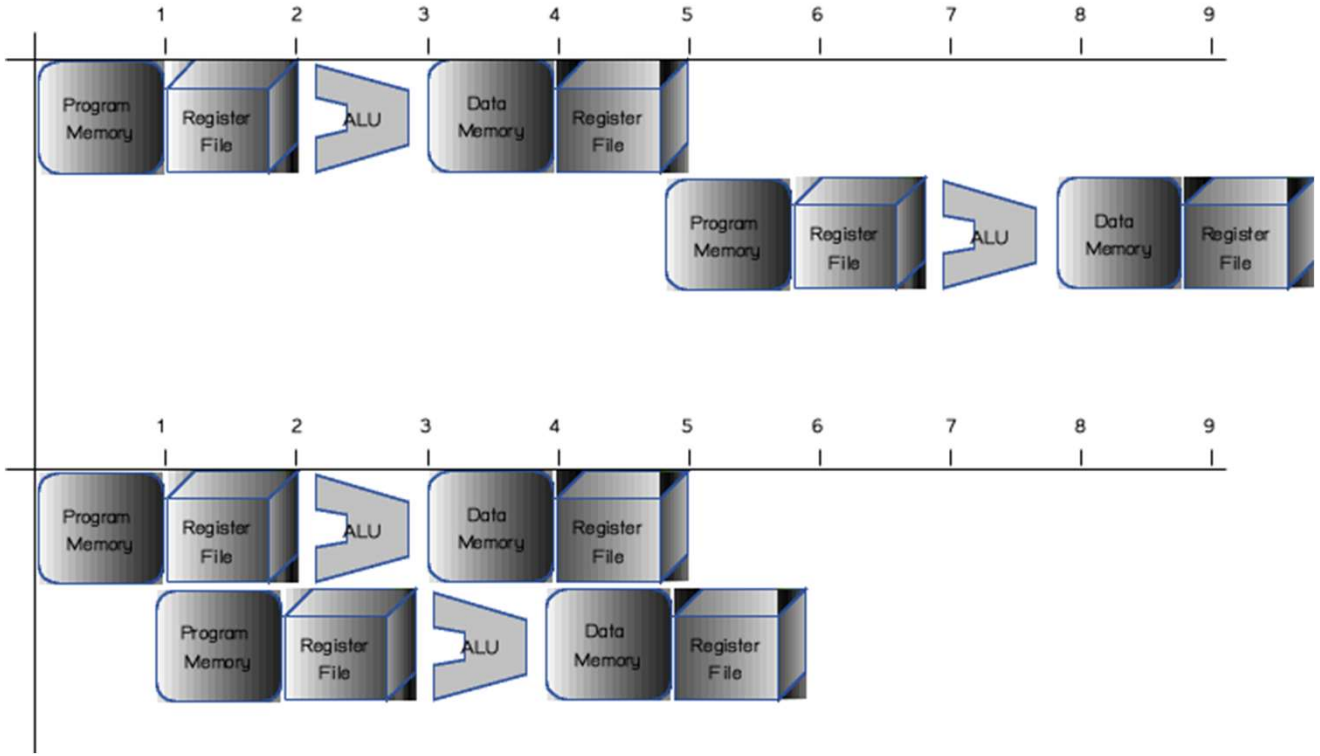


- ★ Ifetch: Instruction Fetch
- ★ Fetch the instruction from the Instruction Memory
- ★ Reg/Dec: Registers Fetch and Instruction Decode
- ★ Exec: Calculate the memory address
- ★ Mem: Read the data from the Data Memory
- ★ Wr: Write the data back to the register file



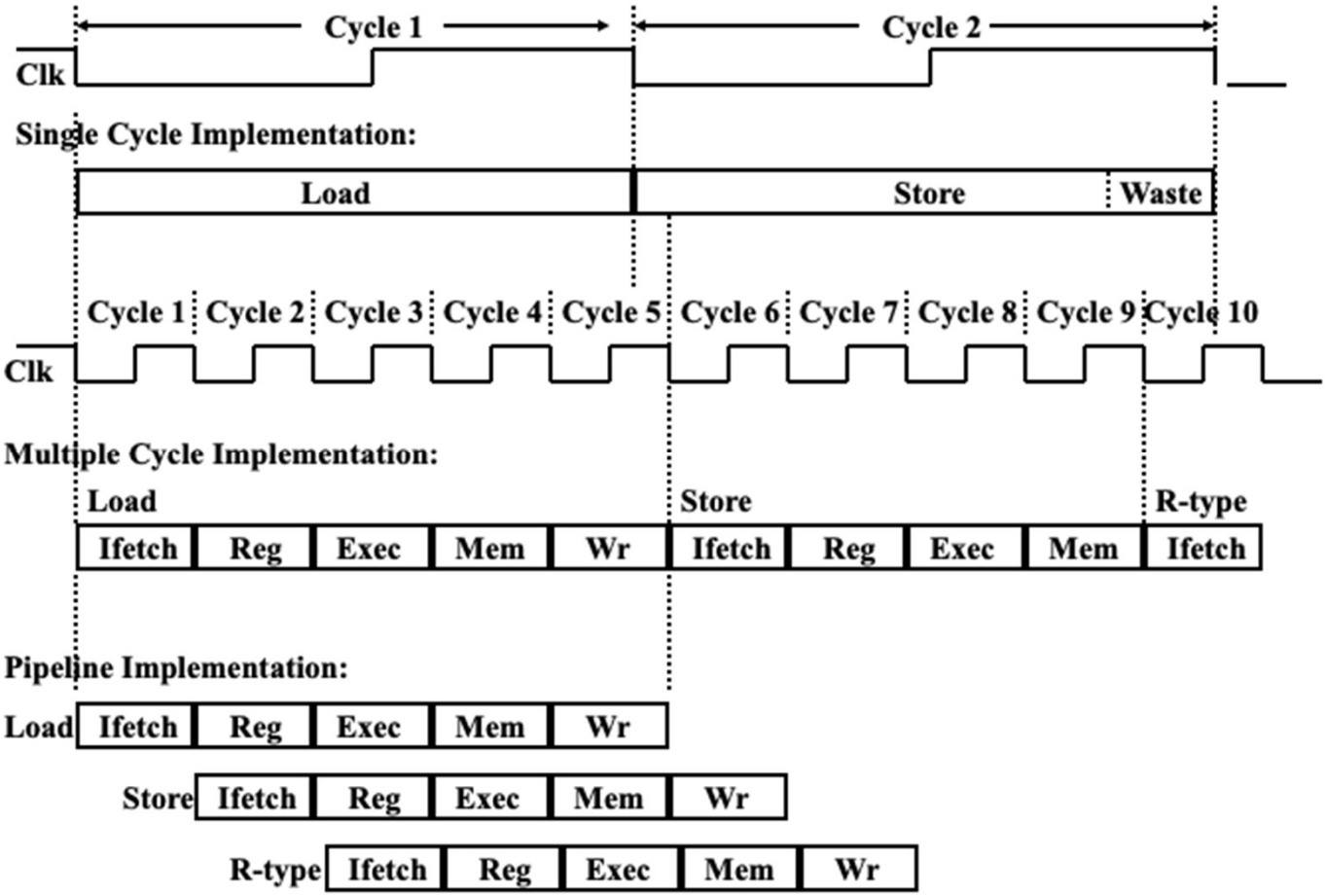
Representation

- Representation
  - Darker shade on the right-hand side means read.
  - Darker shard on the left-hand side means write.





Single Cycle vs. Multiple Cycle vs. Pipeline



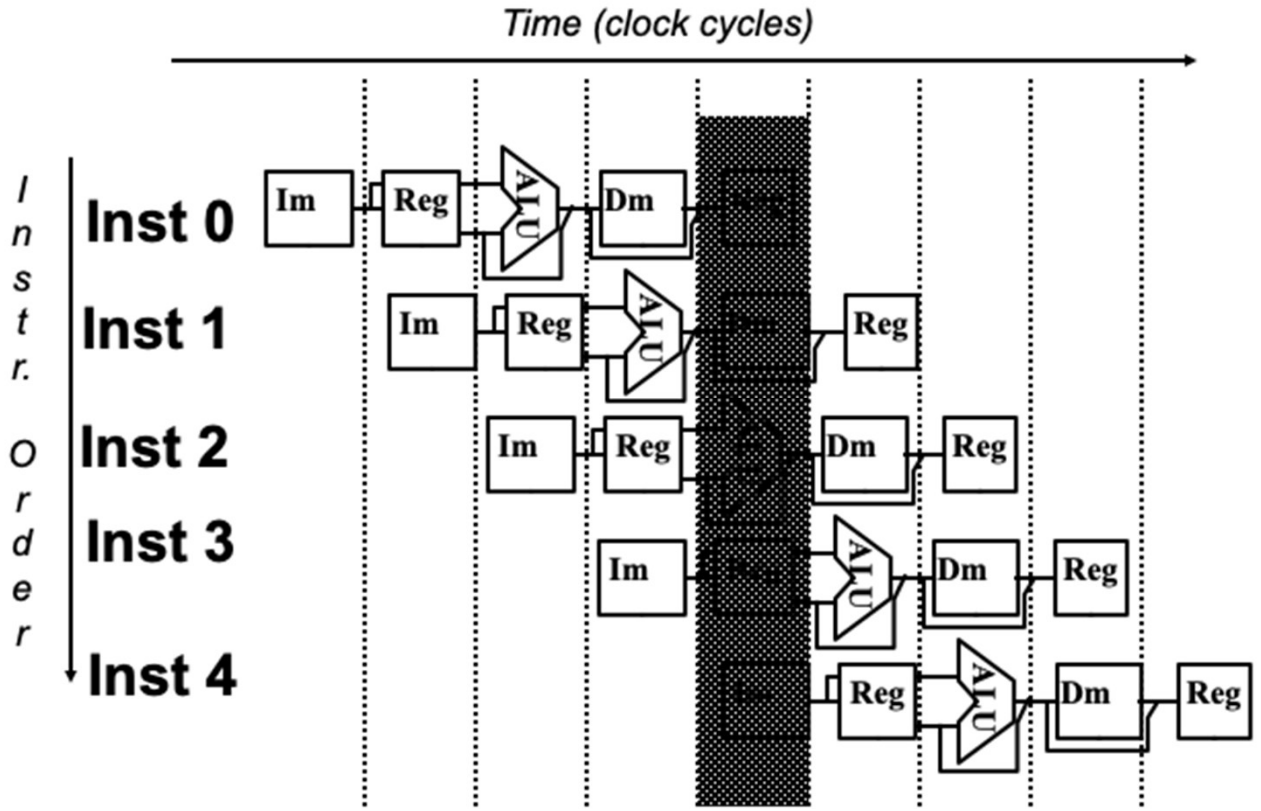


Processor Speedup of Processor

- ★ Suppose we execute 100 instructions
- ★ Single Cycle Machine has 45 ns/cycle
- ★ Multicycle Machine has 10 ns/cycle with 4.6 (averaged) CPI
- ★ Ideal pipelined machine has 10 ns/cycle with 4 stage to fill.
- ★ Which one is the fastest? (and Speedup?)



Since resources are there! Why not Pipeline?





Pipelining is not perfect.

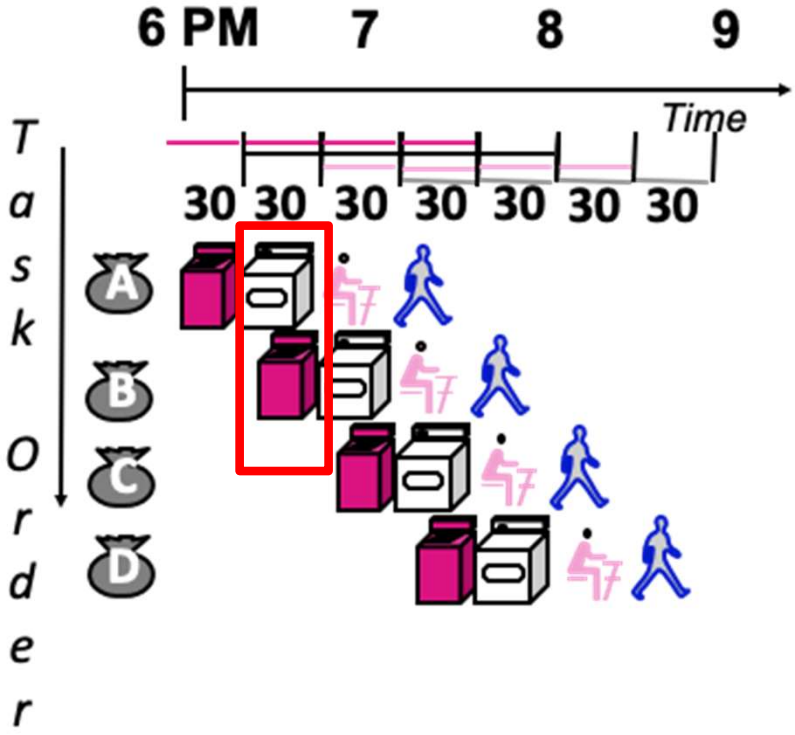
- ★ The world is not perfect. Pipelining can also get us into trouble.
- ★ Pipeline Hazards

- ★ Structural hazards - Limited in resources
- ★ Data hazards - dependency of works/data
- ★ Control hazards - don’t know what to do next



Structural Hazards

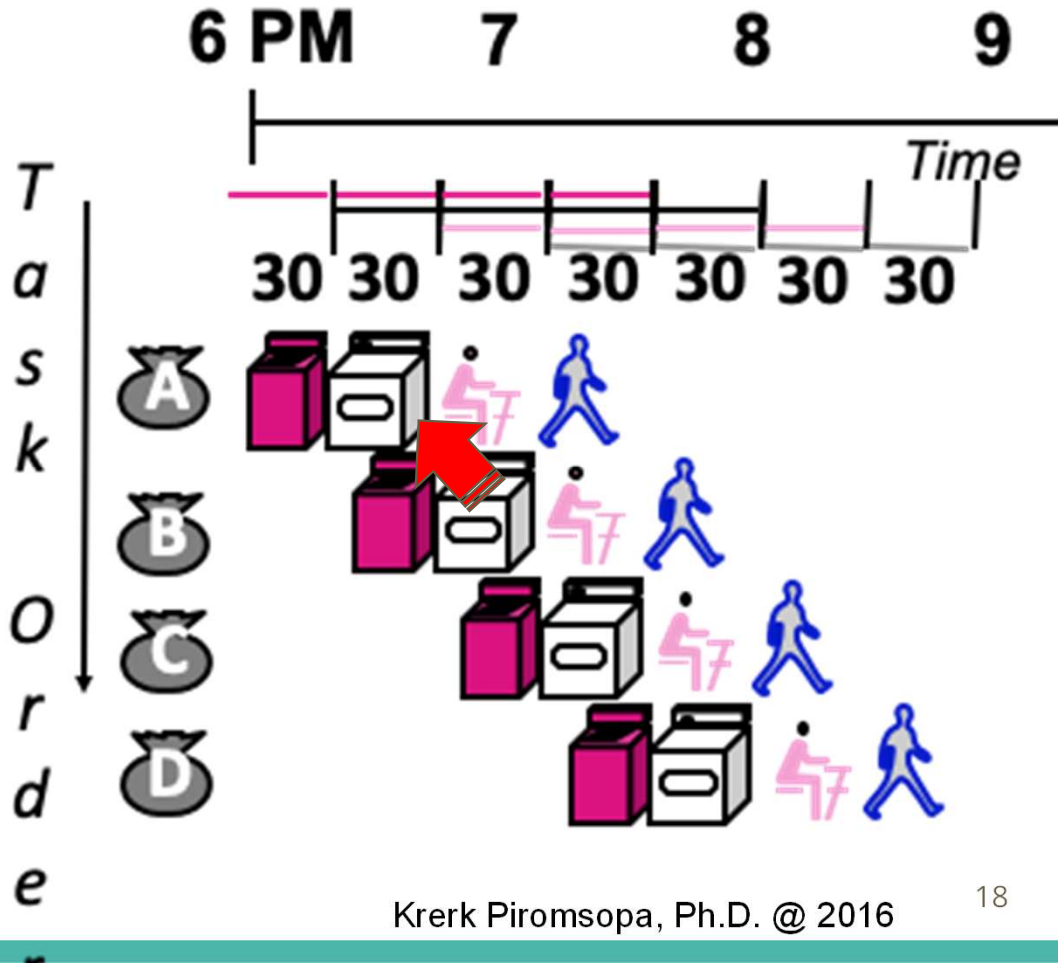
- ★ Structural hazards
  - Same resource is required in two (or more) different ways at the same time.
- ★ Example
  - (1) Washer and Dryer is now a same machine. This would be a structural hazard. It is not possible to wash and dry in the same cycle.
  - (2) Folder busy doing something else (e.g. watching TV).





Data Hazard

- ★ Data Hazards
  - Data is required before it is ready.
  - The data is in the future.
- ★ Example
  - (1) A uniform (shirt and pants) must be folded together. If they are washed and dried in different loads, folding is not possible.
  - (2) One sock of pair in dryer and one in washer; can't fold until get sock from washer through dryer





Control Hazards

- ★ Control Hazards
  - When the resource is available, we want to start a new load. However, the next load is not determined yet.
  - Cannot make a decision before the condition is determined.
- ★ Example
  - Washing football uniforms require proper level of detergent. We need to see it after washer/dryer before putting a next load in.
  - branch instructions



How to solve hazards

- ★ We can naively solve hazards by **waiting**.
- ★ However, waiting is bad for performance. Maybe there is a workaround solution.

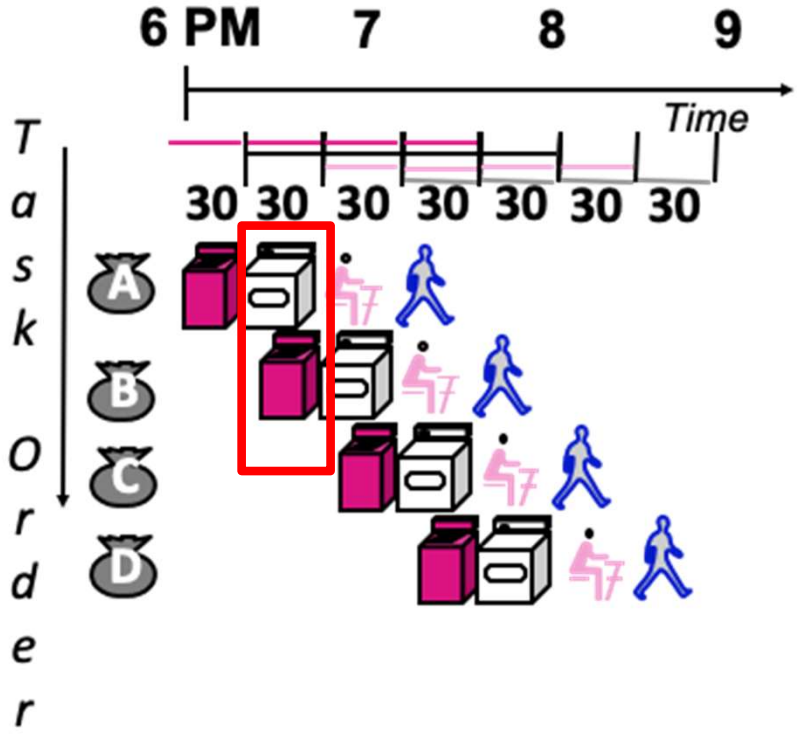
- ★ Pipeline control must detect the hazard
- ★ Take action (or delay action) to resolve hazards



Possible Solutions for Structural Hazards

- ★ Limited Resources
- ★ Add more instances (Solve by money?)
- ★ Solve by designs

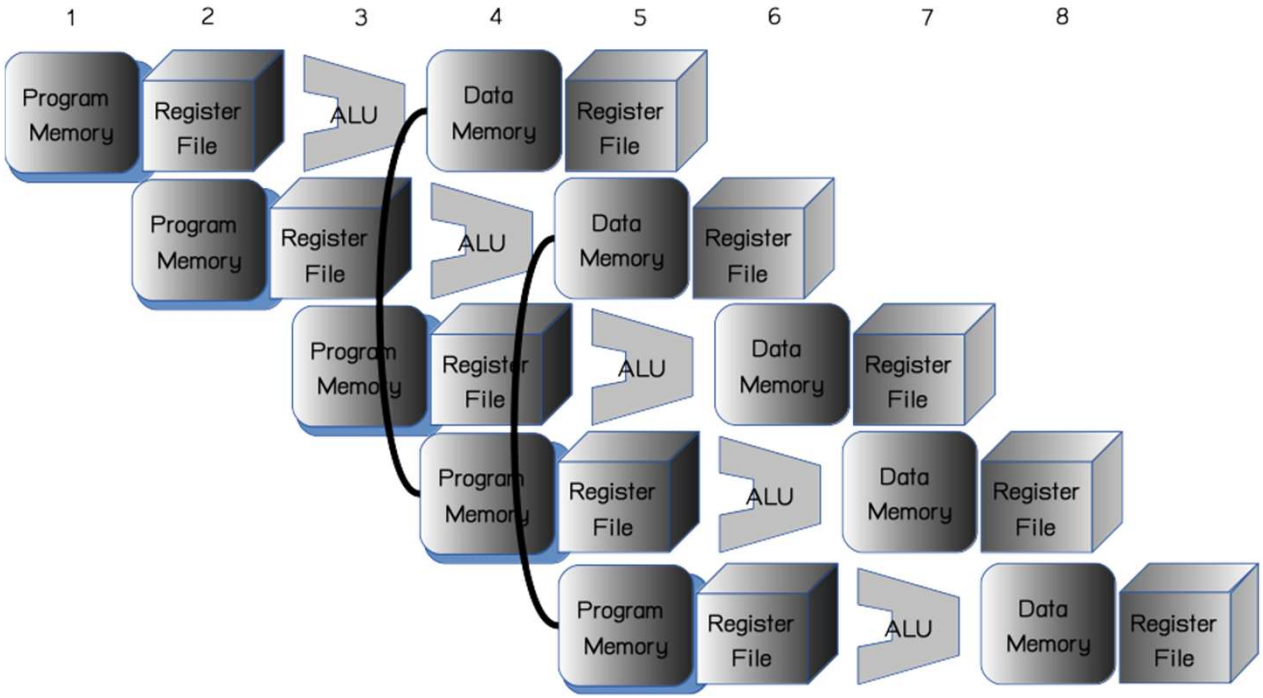
- ★ For our laundry, buy more machines.





Structural Hazards in Processor (1)

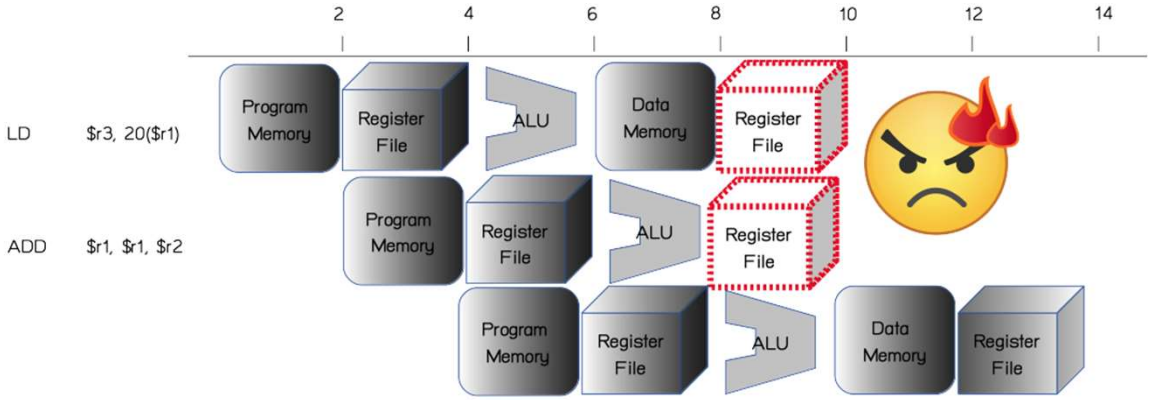
- ★ Single Memory: Data Memory and Program Memory are the same?
- ★ Solution: Make two copies (eg. separate CACHEs). One for program memory, another for data memory.





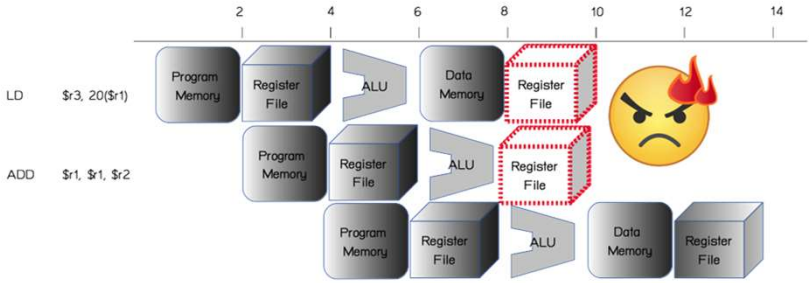
Structural Hazards in Processor (2)

- ★ Mixing 3, 4, 5 cycle instructions together, may lead to using the same resource at the same time.
- ★ LD (5 cycles) follows by ADD (4 cycles) will resulting in WB at the same cycle.
- ★ Solution:
  - (1) stall or
  - (2) redesign to avoid such condition.

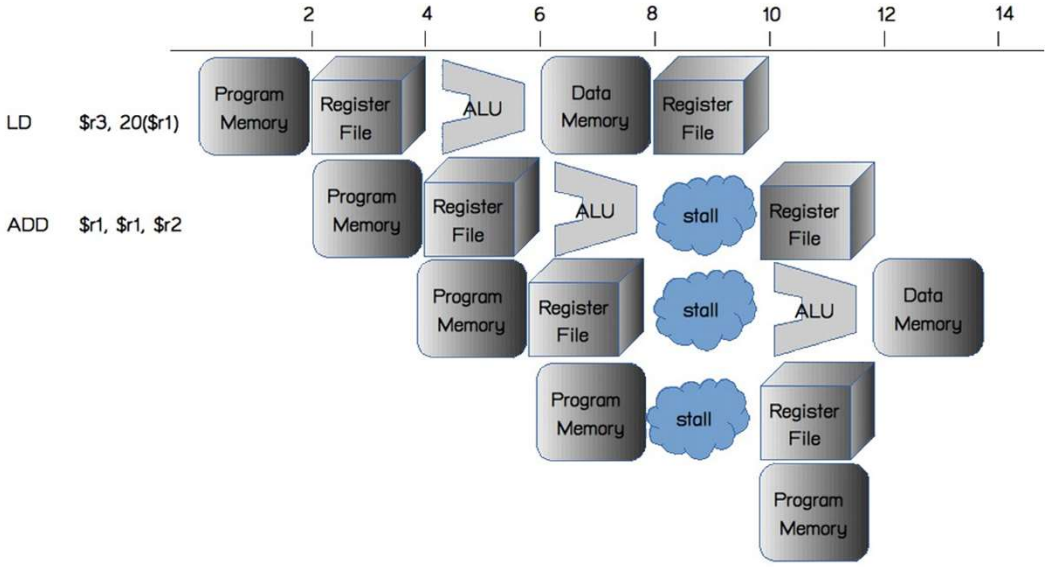




Pipeline Stalls



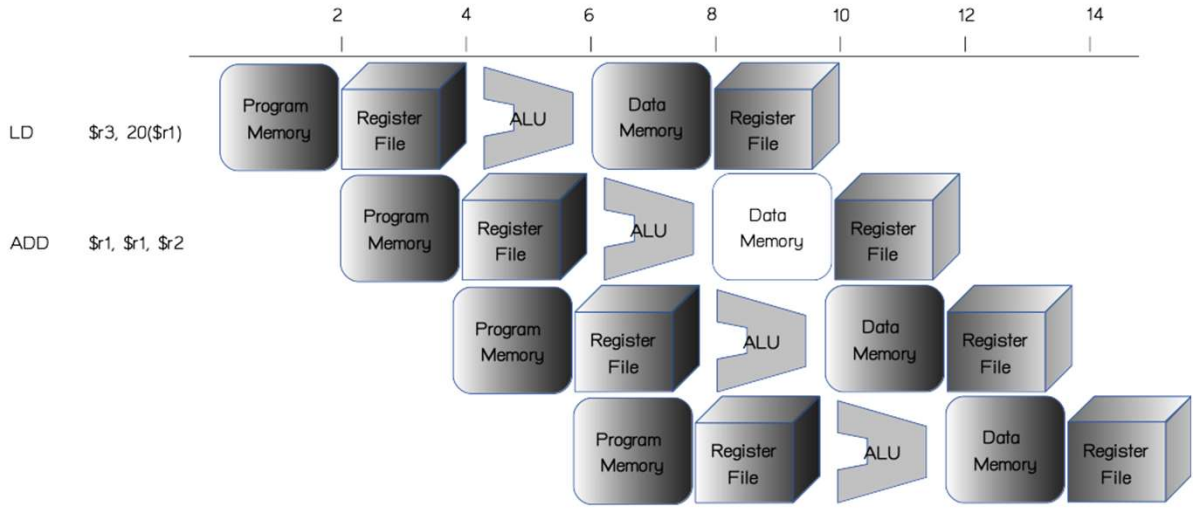
★ Now, pipelining is not ideal.





Redesign to avoid stall

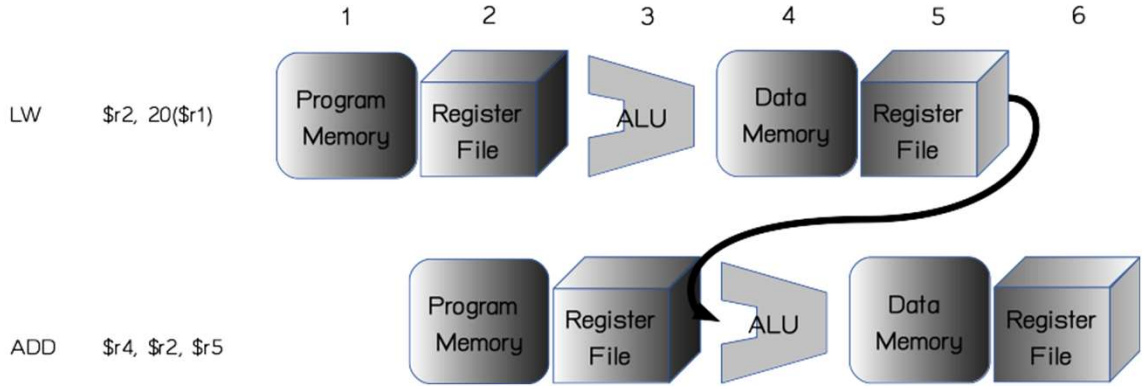
- ★ Make R-type to use 5 cycles (add a dummy MEM stage).
- ★ Now, we do not have hazard.
- ★ However, R-type instructions is now 1 cycle slower. (Design tradeoff)





Data Hazards in a Processor

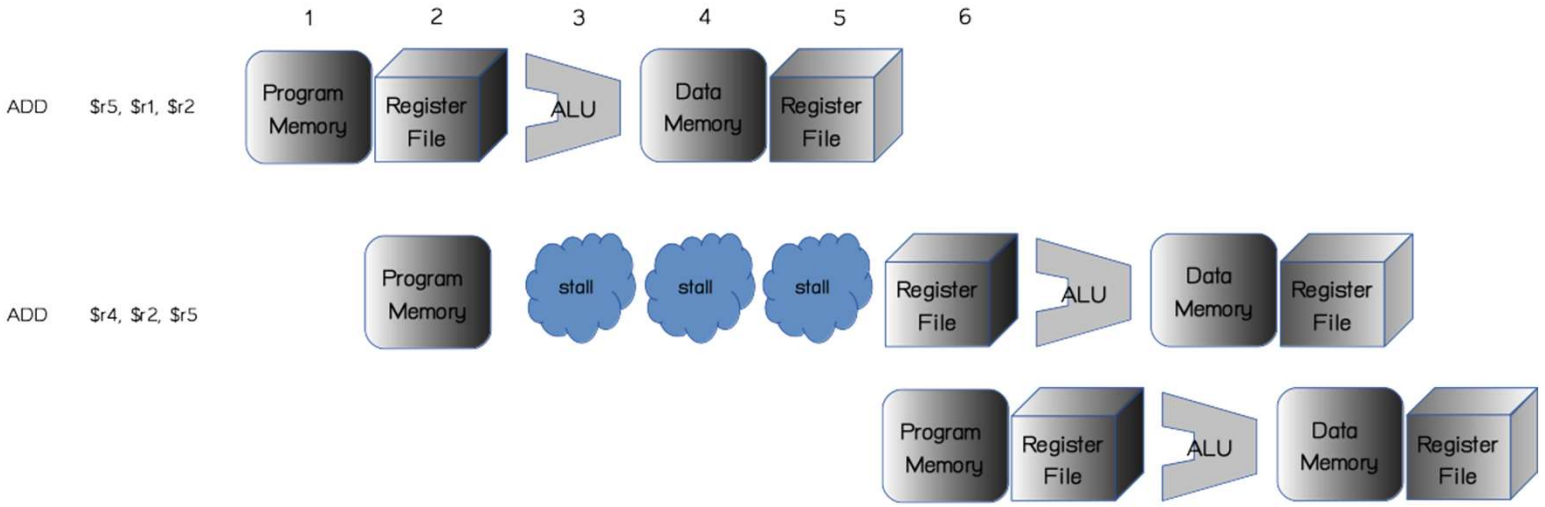
- ★ Reading a data that is a result of previous instruction results in data hazard (read after write).
- ★ ADD $r4, $r2, $r5 after LW $r2, 20($r1). $r2 will be ready after the 5th cycle. However, ADD wants to use $r2 in 4th cycle.





Possible Solutions to Data Hazards (1)

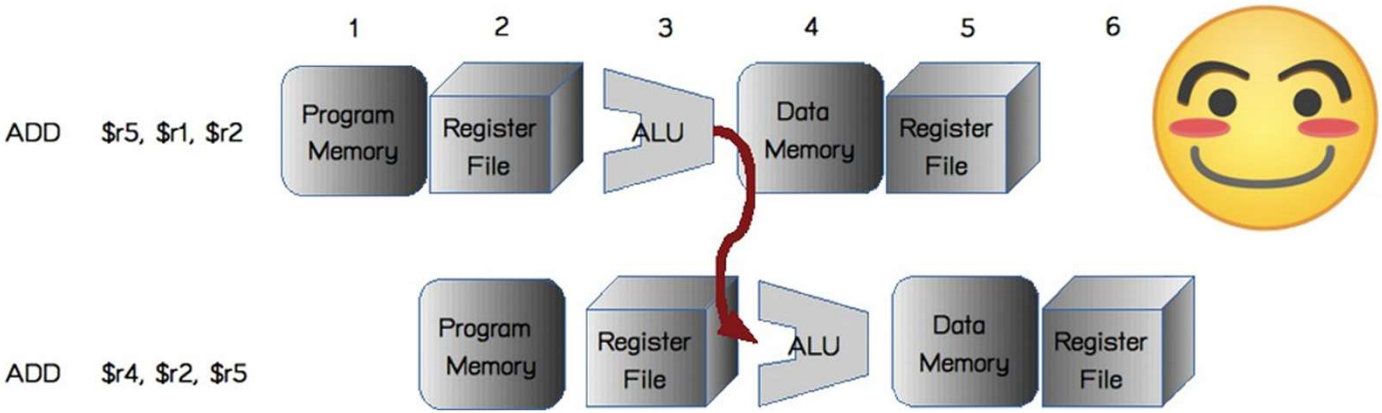
- ★ Wait ?
- ★ Stall ADD for 3 cycles until data is ready.





Possible Solutions to Data Hazards (2)

- ★ Hardware Bypass/Forward
- ★ In several cases, Data is there. Rather than waiting, we can just bypass data path.
- ★ Redesign datapath to support forwarding



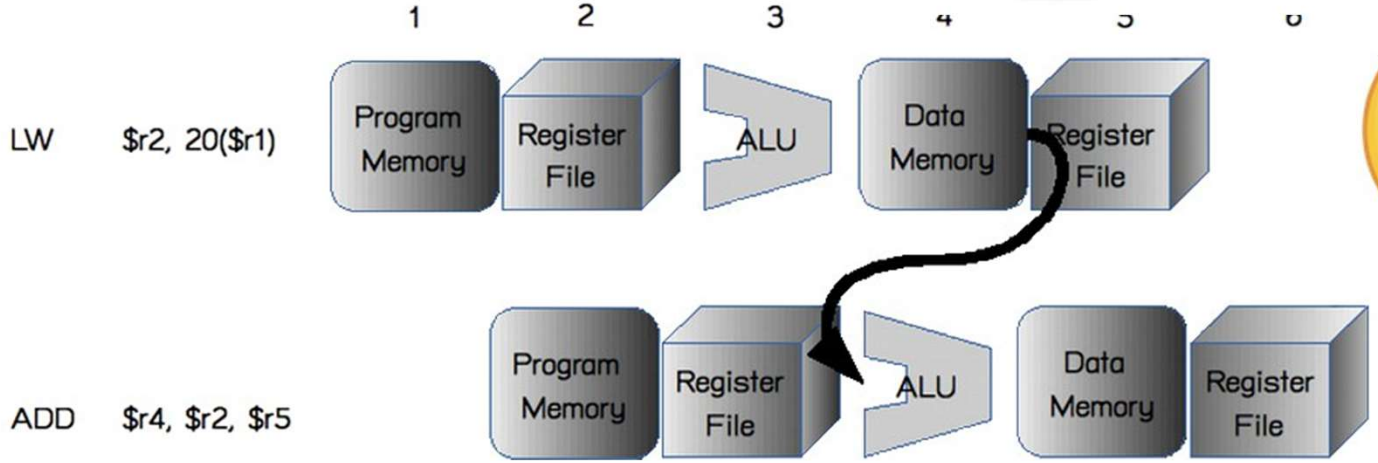


Possible Solutions to Data Hazards (3)



- ★ Forwarding is not always possible. (Unless we have a time machine.)
- ★ Consider ADD after LW.





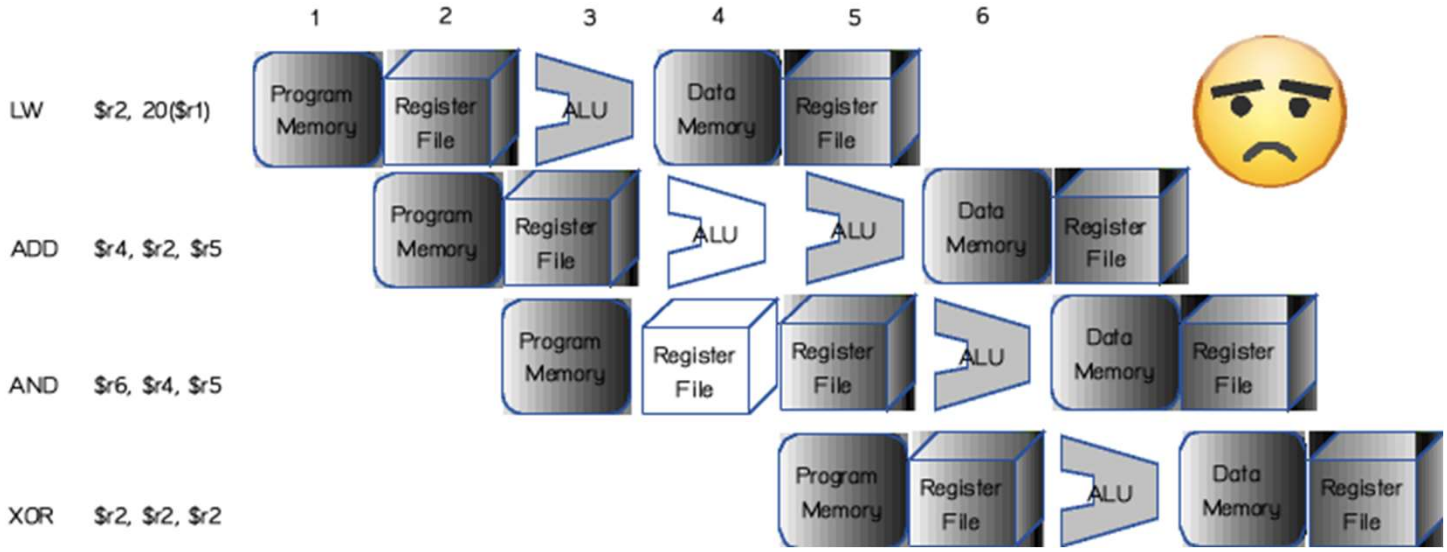


Pictures from
https://tshop.r10s.jp/hobbyzone/cabinet/02/003/4970381183851-j00.jpg?fitin=330:330
https://www.backtothefuture.store/v/vspfiles/photos/21011-2.jpg



Possible Solutions to Data Hazards (4)

- ★ Ask software for help by rescheduling code to avoid (data) hazards.
- ★ Adding/Moving instruction in between. (Delay Load)
- ★ If none is applicable, insert nop (no operation)





Possible Solutions to Data Hazard (5)

- ★ Drawing a diagram for the two code.
- ★ See how software can play a role in avoiding hazards.

```
LW      $r2,20($r1)
AND     $r4,$r2,$r5
```

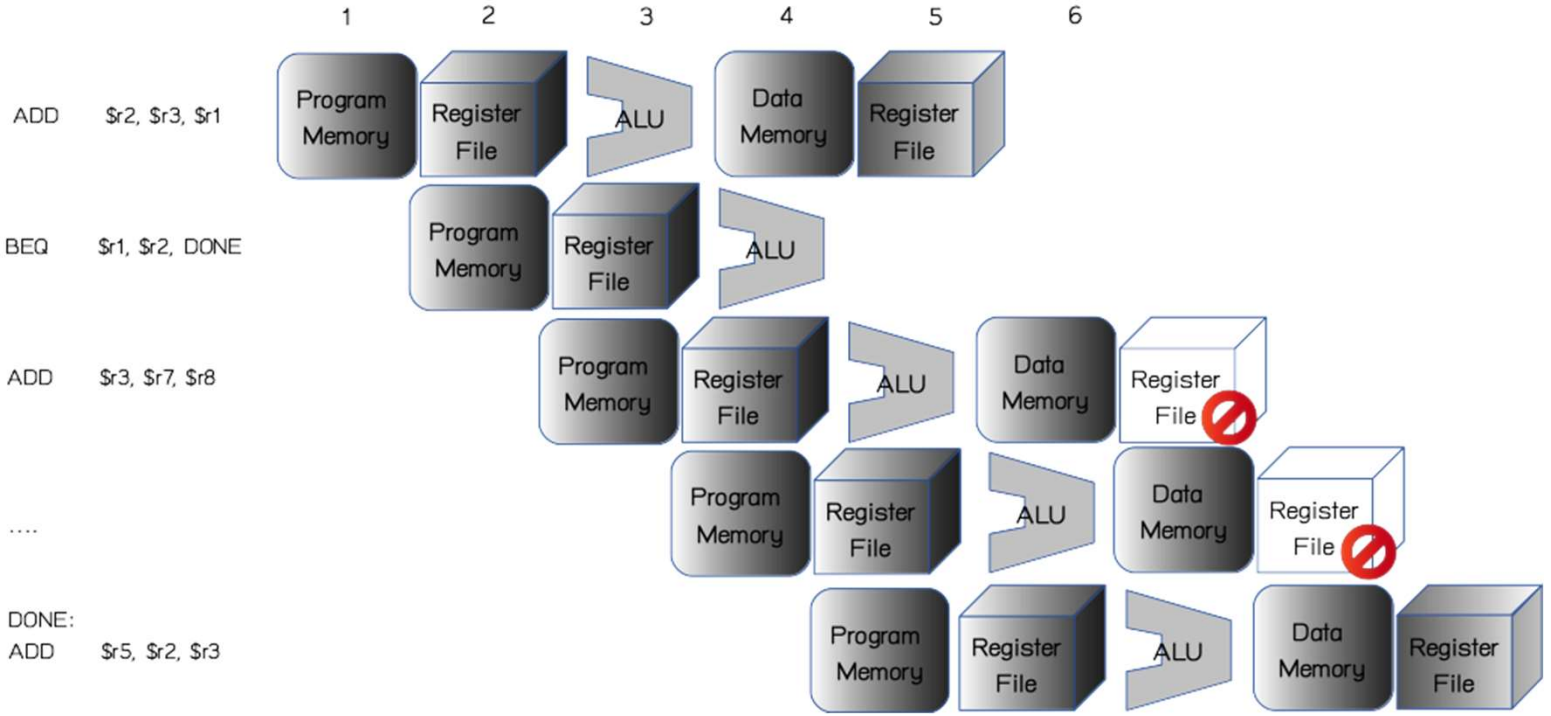
```
LW      $r2,20($r1)
NOP
NOP
NOP
AND     $r4,$r2,$r5
```

- ★ We will revisit data hazards later.



Control Hazards in a Processor

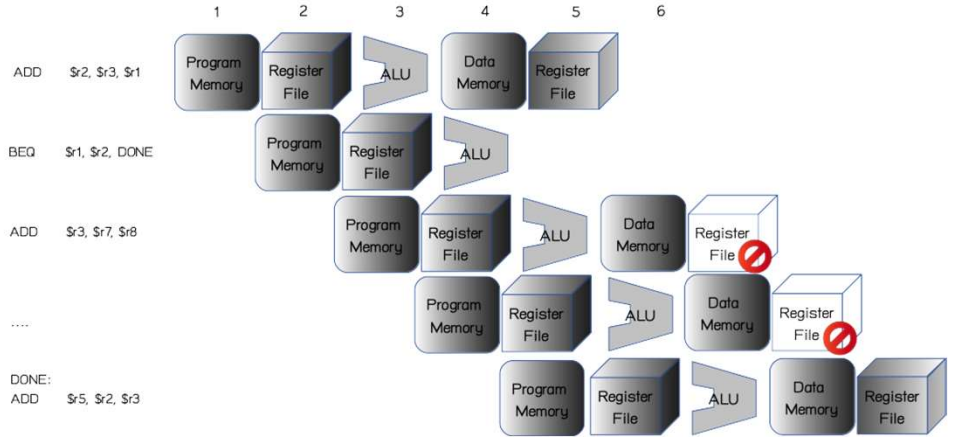
- ★ Control Hazard (aka. Branch Hazard)
- ★ Branch would resolve in 3 cycles. So, what should we do in the next cycle?





Possible Solutions to Control Hazards (1)

- ★ Stall?
  - 2 cycles slower
- ★ Guess (Branch Prediction)
  - Predict taken or not taken
  - If correct, no delay
  - If incorrect, 1 cycle delay?
  - At least 50% changes of being correct
  - More advance scheme 90% correct (e.g. Branch history)





Possible Solutions to Control Hazards (1)

- ★ Redefine branch with branch delay slot
- ★ A slot after is not related to branch. Branch is 1 cycle later.
- ★ Software/Compiler can take benefit of the extra slot.

Without delay slot

```
(1)       ADD     $r1, $r2, $r3
(2)       BEQ     $r2, $r3, BRANCH
(3)       ORI     $r4, $r5, #100
BRANCH:
(9)       SW      $r1, 100($r3)
```

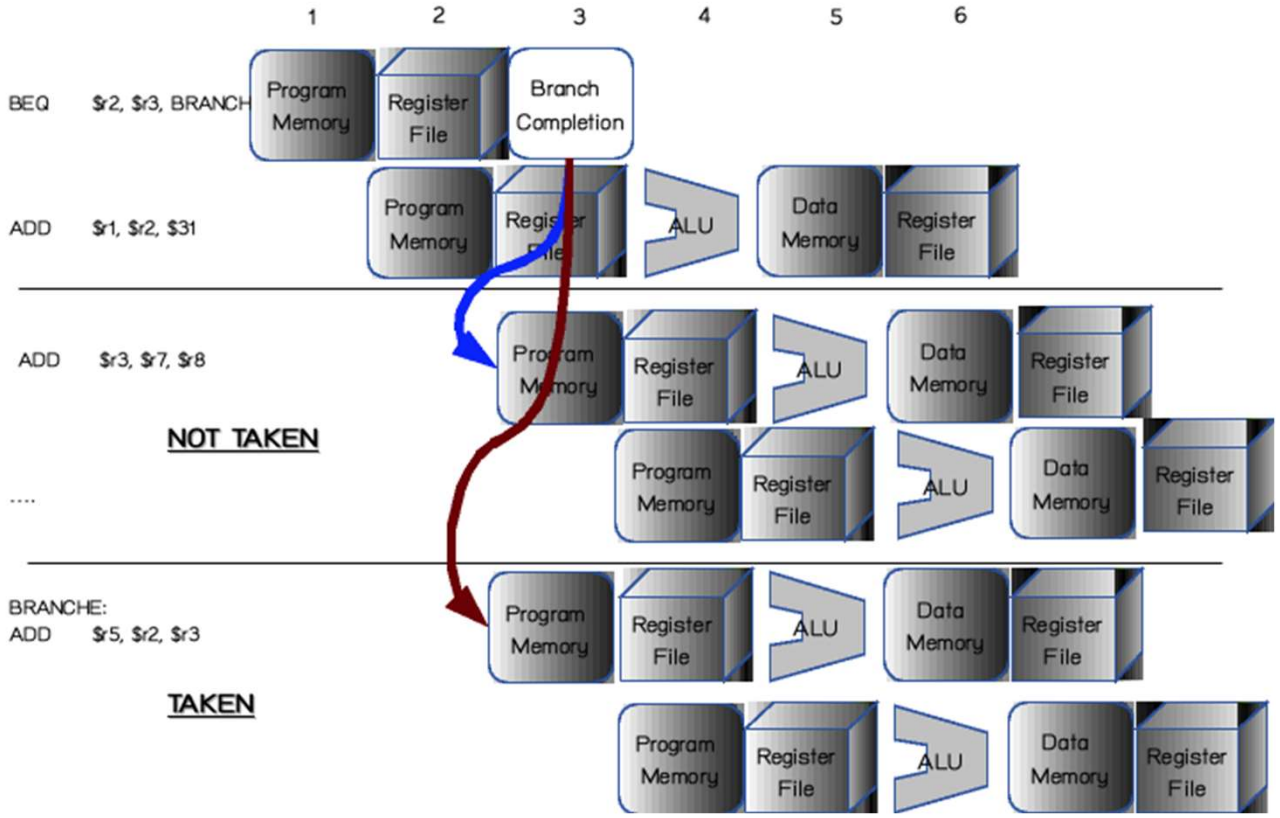
With delay slot

```
(2)       BEQ     $r2, $r3, BRANCH
(1)       ADD     $r1, $r2, $r3
(3)       ORI     $r4, $r5, #100
BRANCH:
(9)       SW      $r1, 100($r3)
```



Branch Delay Slot

★ Need support from software.





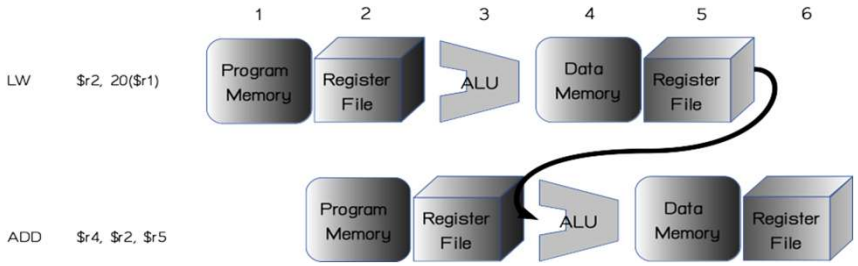
Exercises

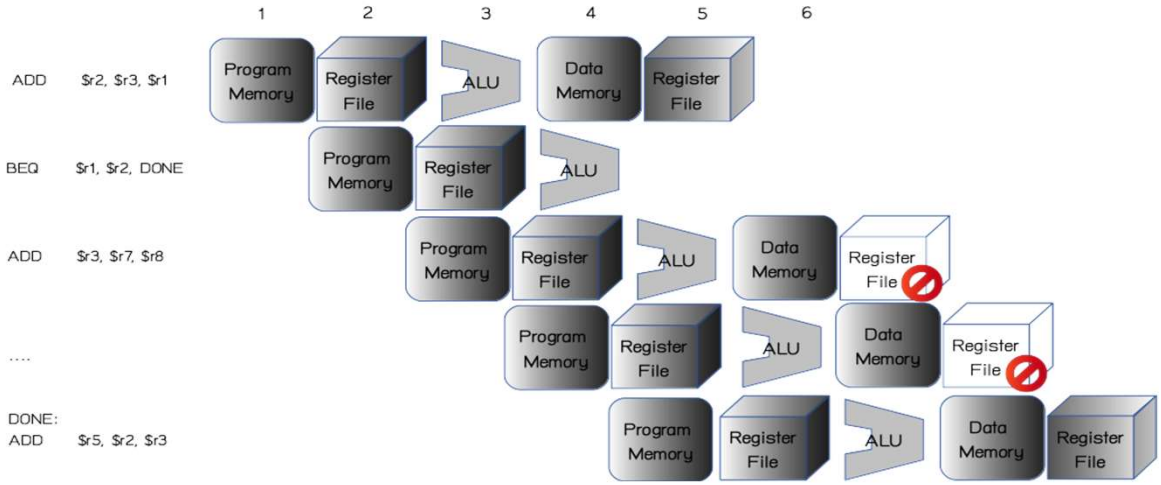




Mixing CPI and Structural Hazards

★ Why mixing SW, BEQ, and JMP with R-Type (ADD, ORI, ORUI) does not result in a structural hazard? Please provide your analysis.







Pipelining Representation

- ★ Please draw a diagram showing the pipelining of the following code. (Please also provide correct shade for reading and writing).
  - ADD $r10,$r0,$r1
  - LW $r12,20($r5)
  - ADD $r3,$r12,$r12
  - SW $r3,30($r10)
  - ADD $r10,$r10,$r1
- ★ There are some hazards in the given pipeline. Can you identify some of them?



End of Chapter 6 (part 1)





Pipelining (part 2)

Chapter 6



Pipelining (part 2)

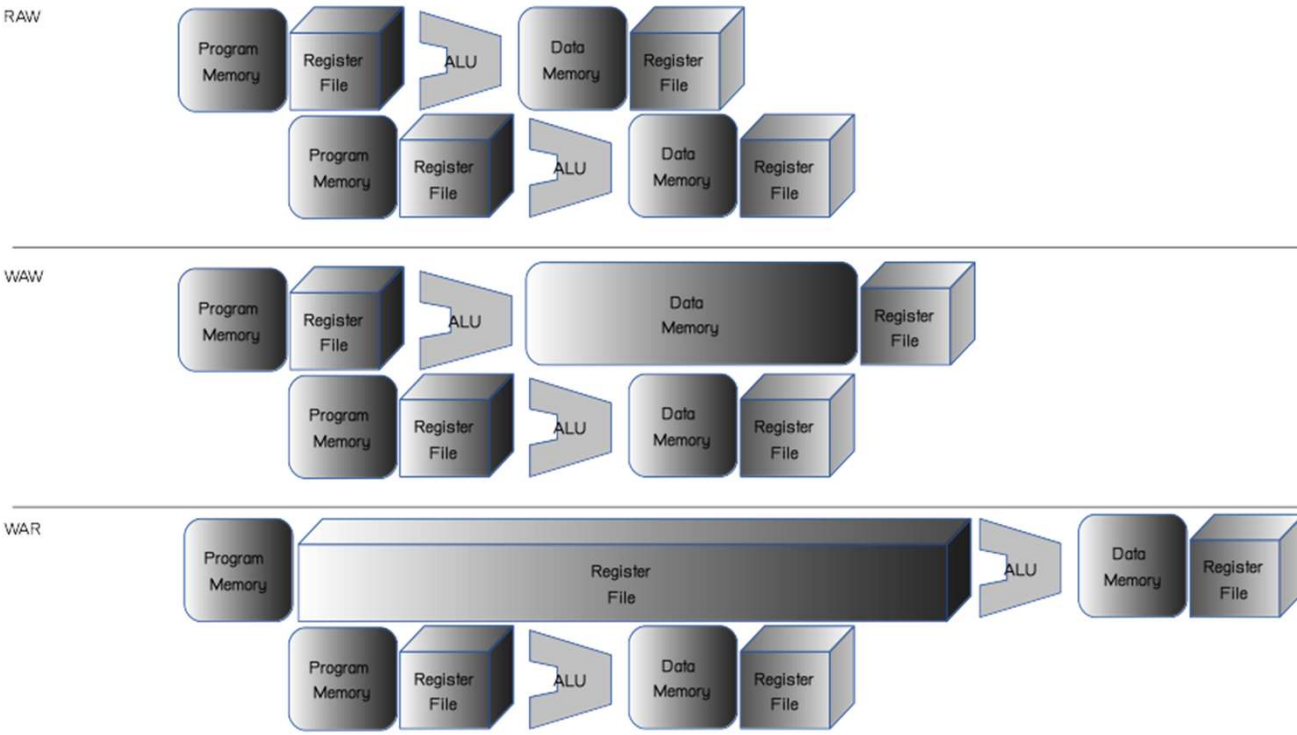
- ★ Data Hazard revisit
  - RAW
  - WAW
  - WAR
- ★ Designing a Pipelined Processor
  - Hazard Detection
  - Forwarding Circuit
- ★ Compiler avoiding hazard



Data Hazard revisit

- ★ Read after Write (RAW)
- ★ Write after Write (WAW)
- ★ Write after Read (WAR)

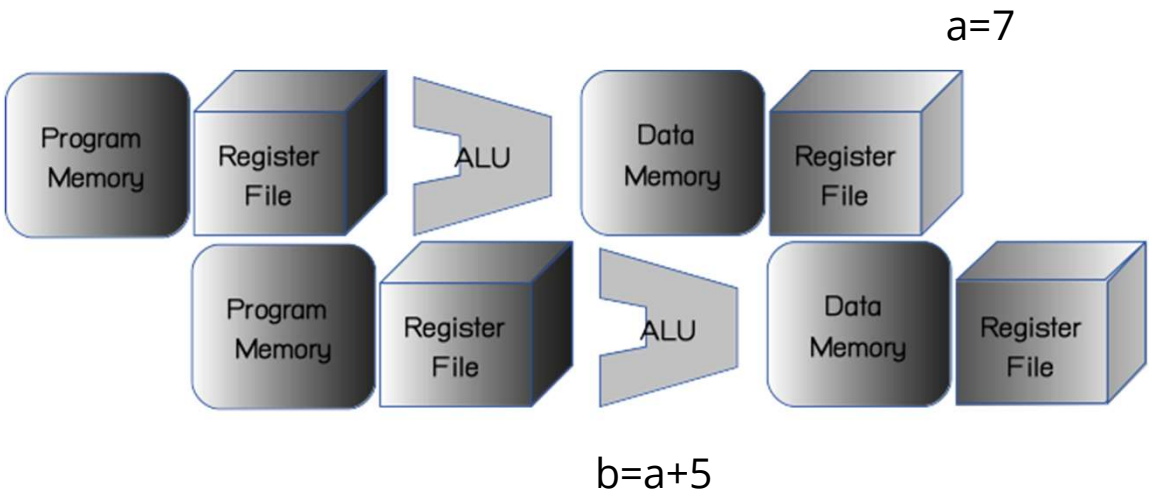
- ★ Why there is not Read after Read?





Read after Write

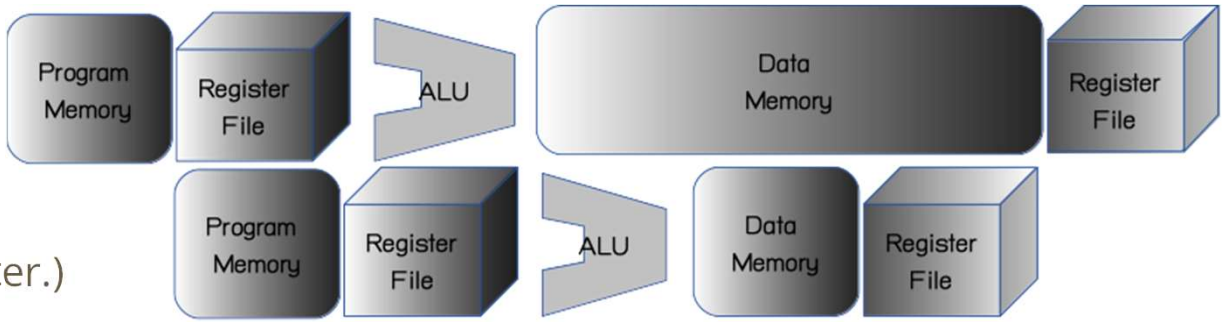
- ★ Consider the following code:
  - int a, b;
  - a = 7;
  - b = a + 5;
- ★ Solutions: Forwarding can solve most of the time.





Write after Write

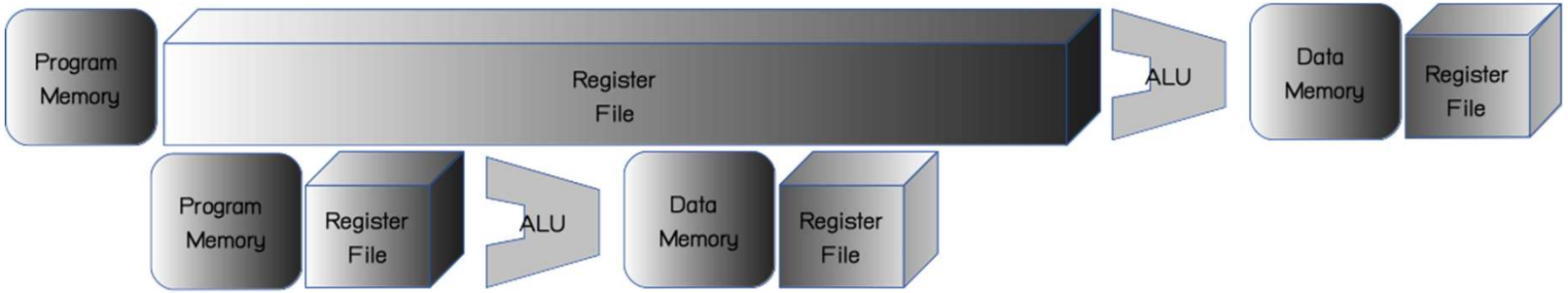
- ★ Consider the following code:
  - a = data[1000];
  - a = 10;
  - b = a + 3;
- ★ data[1000] may take several cycles (due to memory speed).
- ★ Solutions:
  - Can avoid by design.
  - Software solution?
  - Out-of-order execution? (We will learn about this later.)





Write after Read

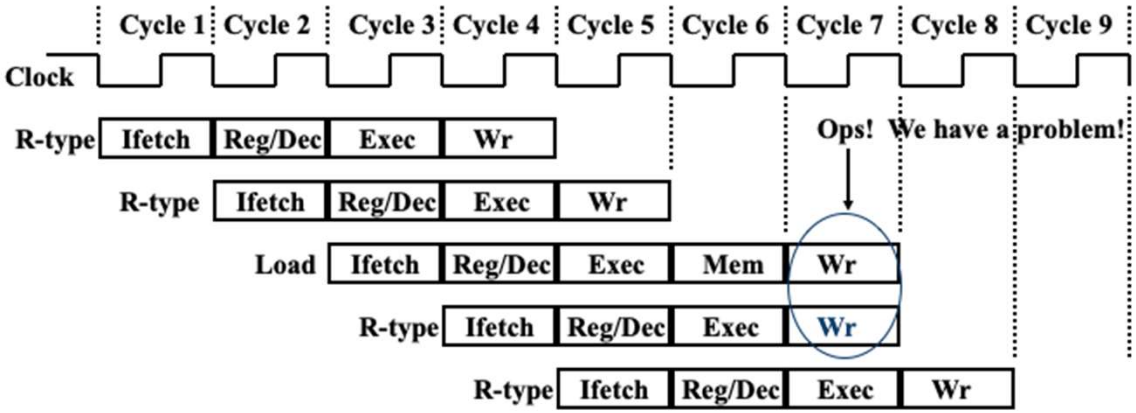
- ★ Hardly found in modern architecture.
- ★ Consider the following code:
  - a=sin(x);
  - x=10;
- ★ Assuming that sin function need special preparation. By the time we read x, it may be assigned.
- ★ Can avoid by design.





Mixing R-type with Load

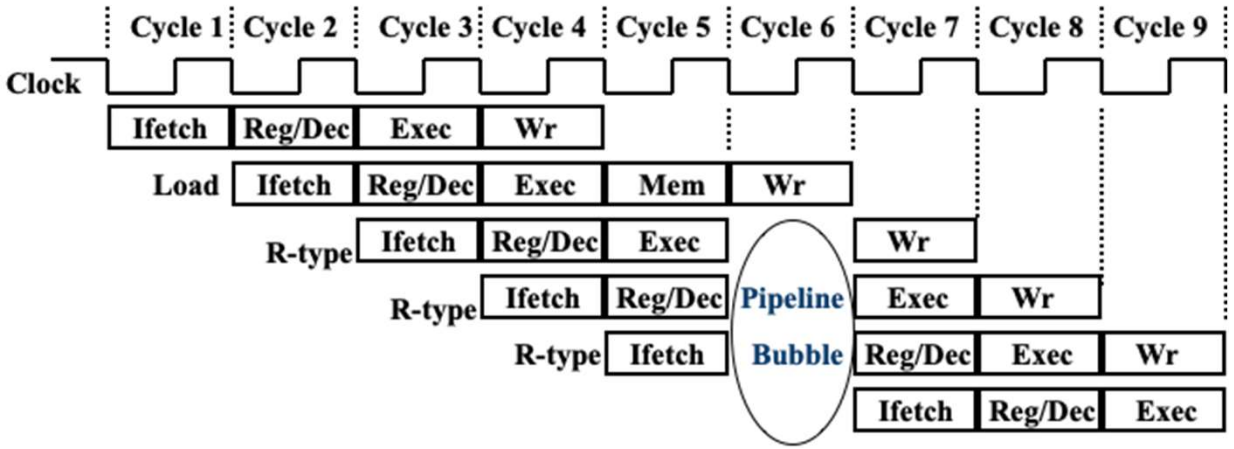
- ★ We have pipeline conflict or structural hazard:
- ★ Two instructions try to write to the register file at the same time!
- ★ Only one write port





Stall Cycle by Inserting Bubble

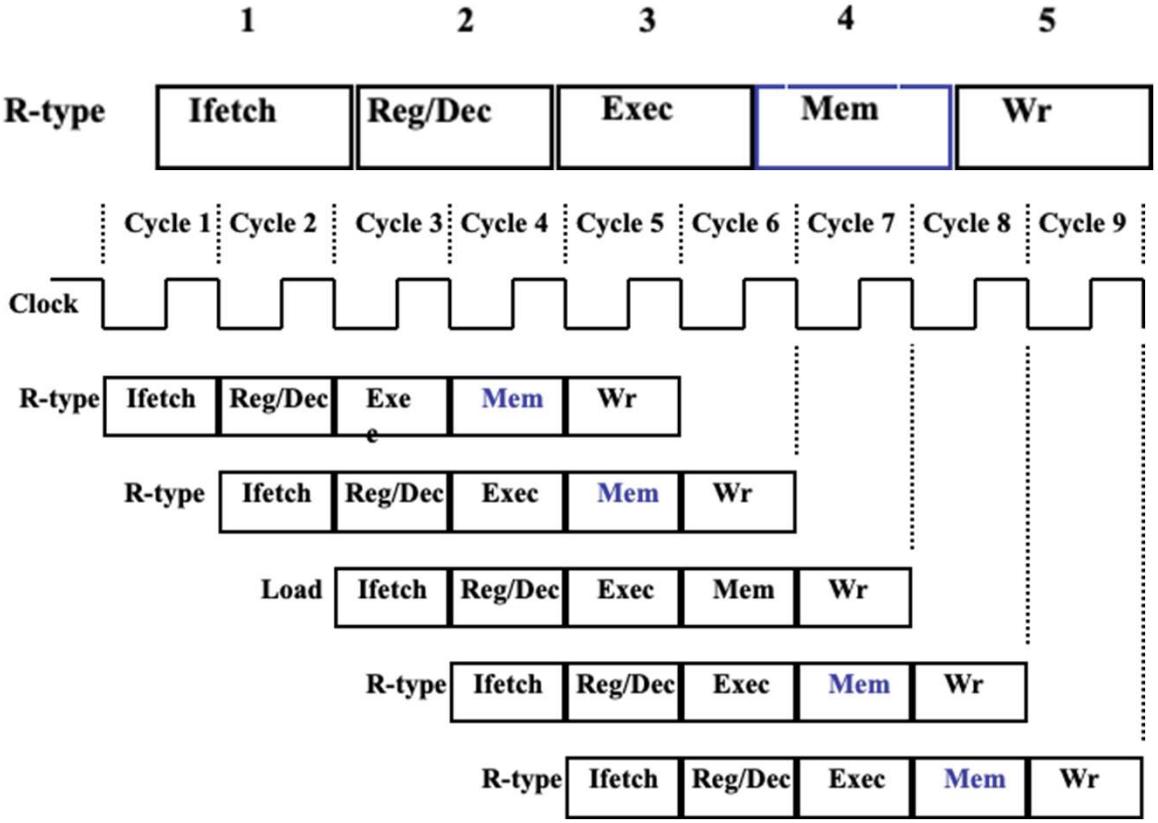
- ★ Insert a “bubble” into the pipeline to prevent 2 writes at the same cycle
- ★ The control logic can be complex.
- ★ Lose instruction fetch and issue opportunity.
- ★ No instruction is started in Cycle 6!





Delay R-type write by one cycle

- ★ Delay R-type's register write by one cycle:
- ★ Now R-type instructions also use Reg File's write port at Stage 5
- ★ Mem stage is a NOOP stage: nothing is being done.





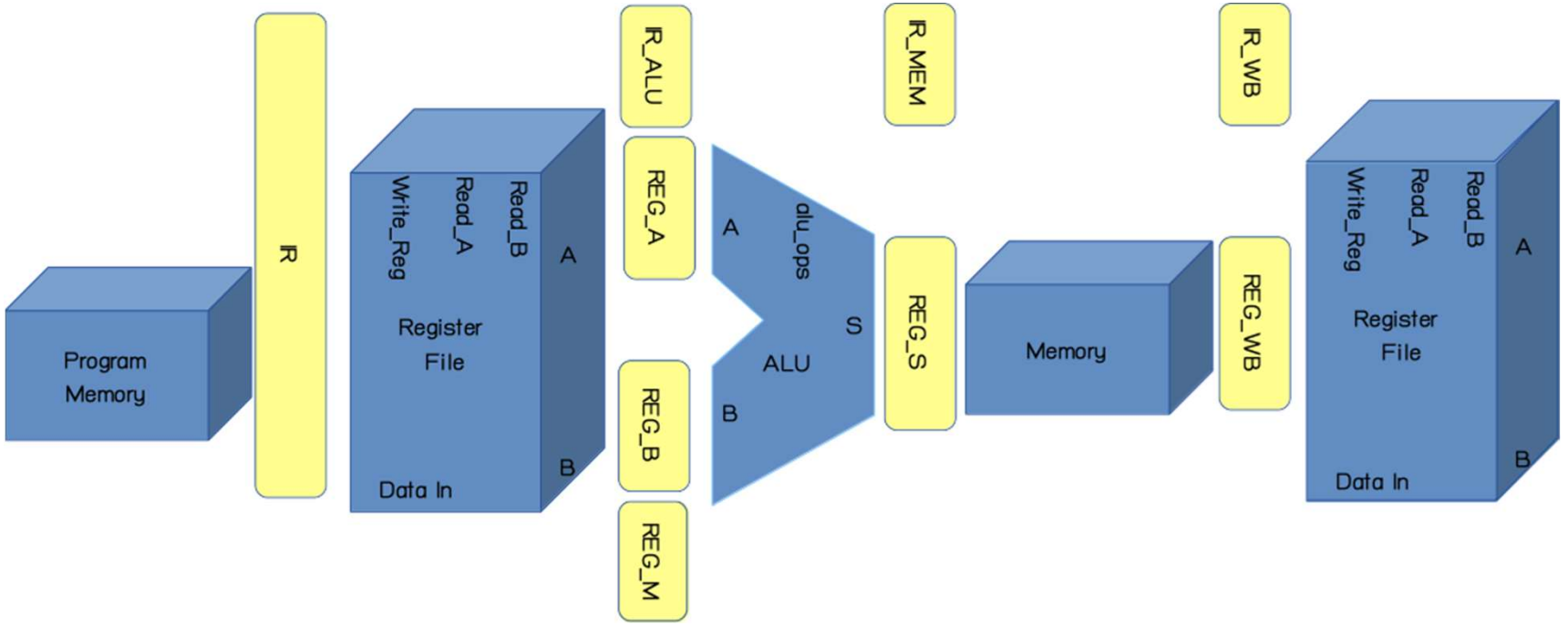
Designing a Pipelined Processor

- ★ Go back and examine your datapath and control diagram
- ★ associated resources with states
- ★ ensure that flows do not conflict, or figure out how to resolve
- ★ assert control in appropriate stage



Datapath for Pipelined Processor

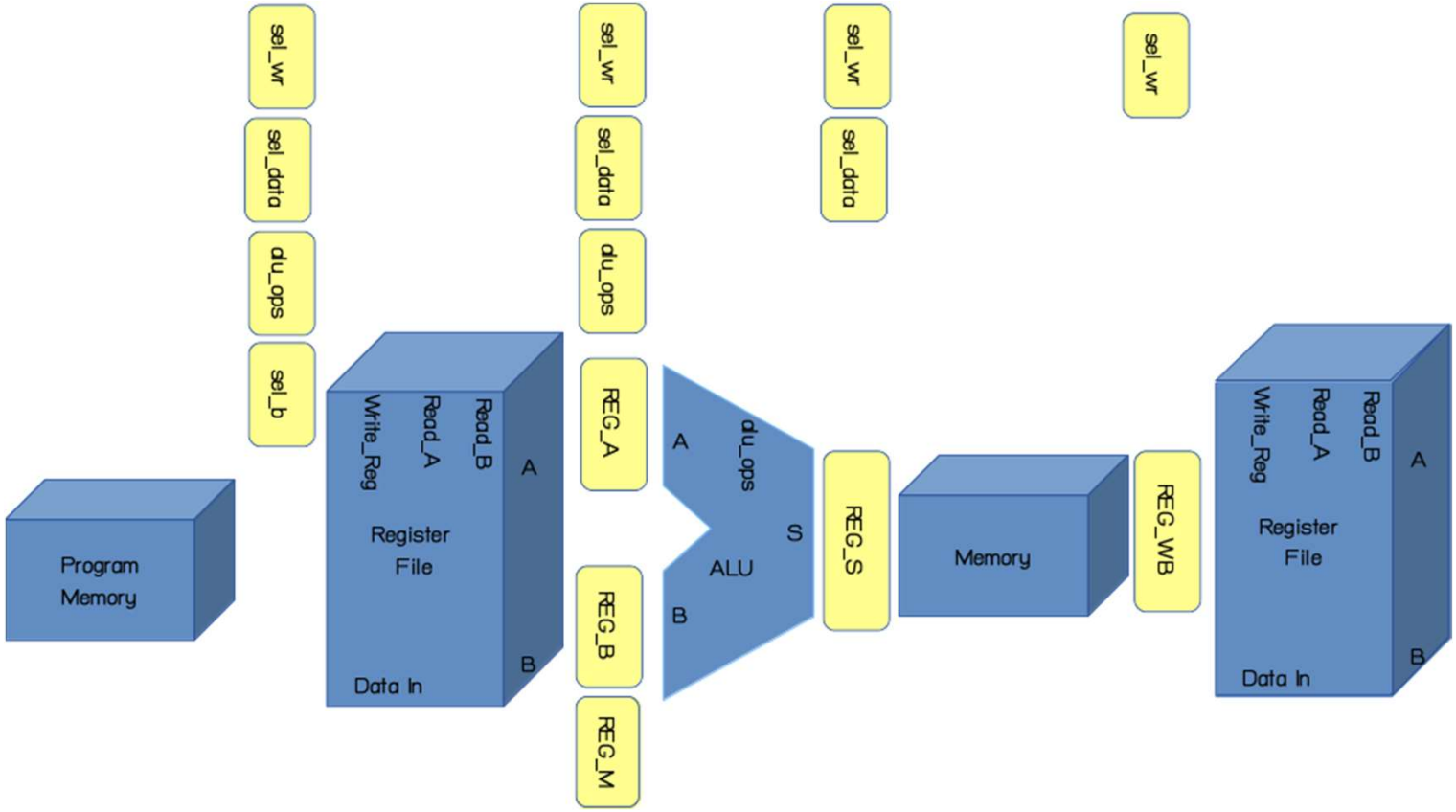
★ Each step has different instruction





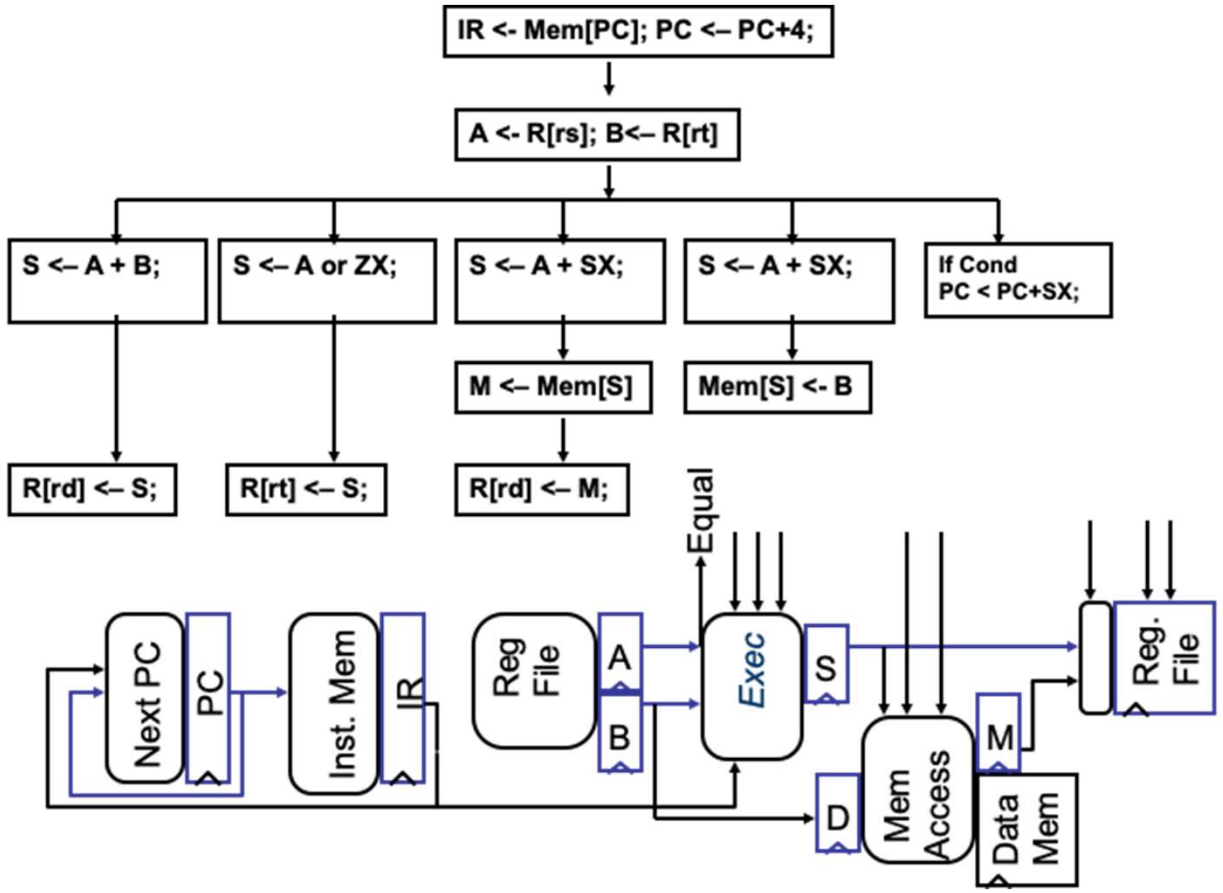
Unified Decoder

★ Alternatively, we can pass control signals only.





Datapath and Control Signal





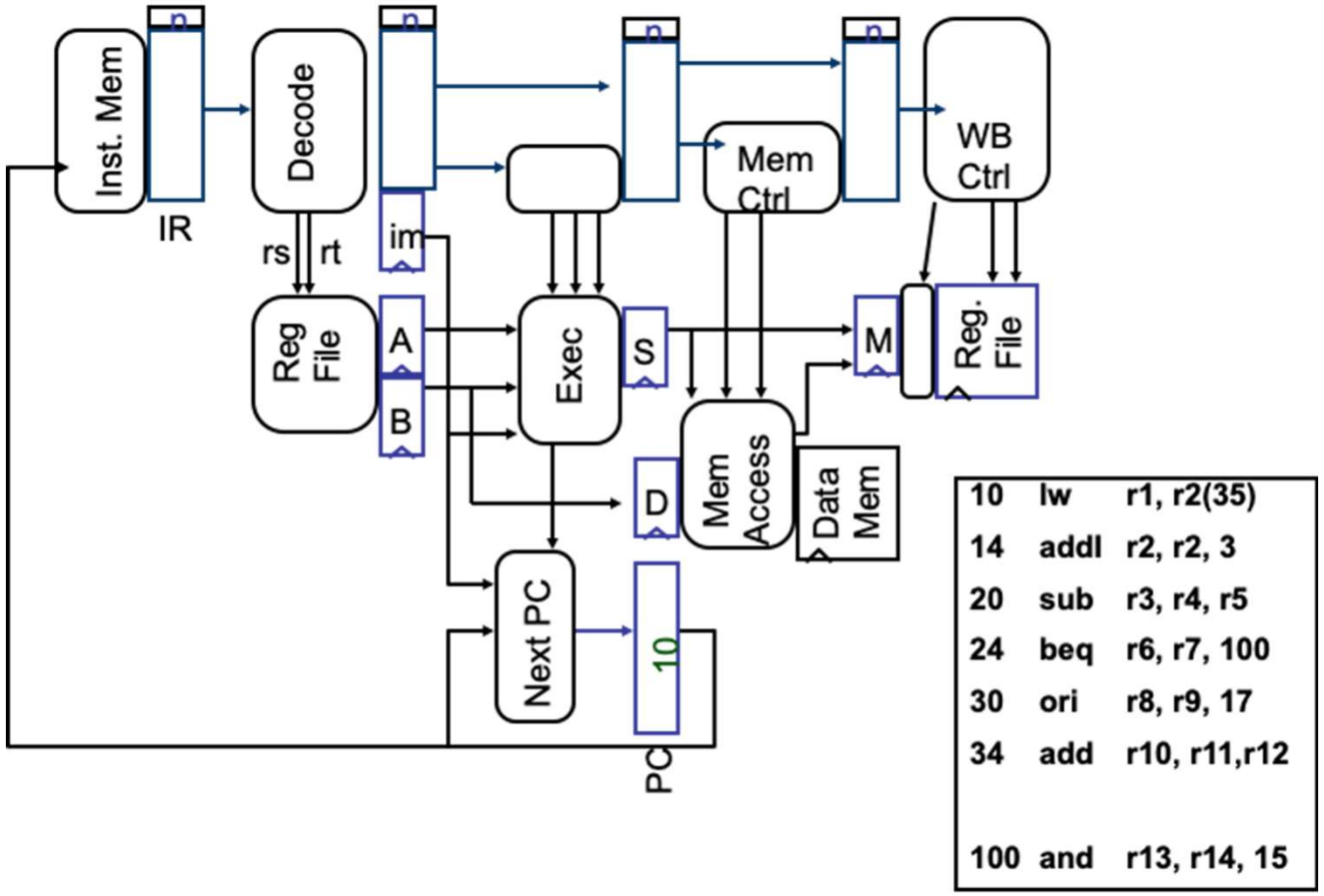
Give it a try

- ★ 10 lw r1, r2(35)
- ★ 14 addI r2, r2, 3
- ★ 20 sub r3, r4, r5
- ★ 24 beq r6, r7, 100
- ★ 30 ori r8, r9, 17
- ★ 34 add r10, r11, r12

- ★ 100 and r13, r14, 15

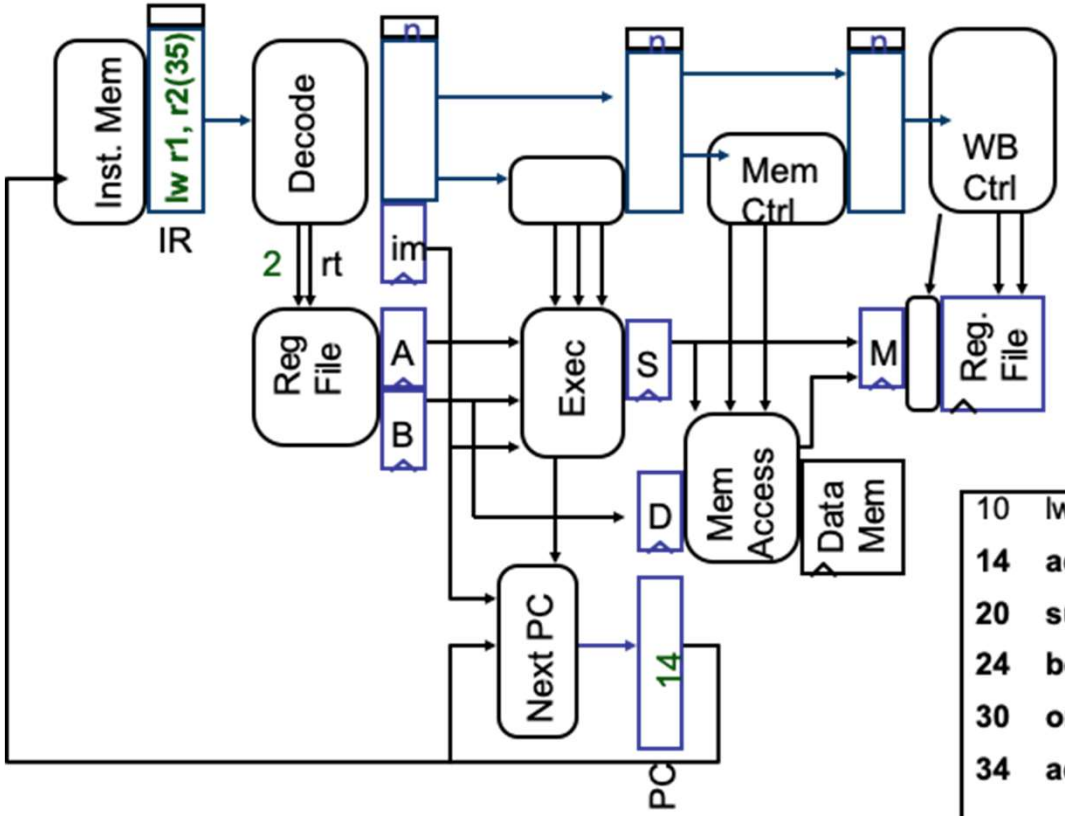


Fetch 10


Inst. Mem
IR
Decode
rs
rt
im
Reg File
A
B
Exec
S
Mem Ctrl
WB Ctrl
M
Reg. File
Mem Access
Data Mem
D
Next PC
10
PC
10 lw r1, r2(35)
14 addl r2, r2, 3
20 sub r3, r4, r5
24 beq r6, r7, 100
30 ori r8, r9, 17
34 add r10, r11,r12
100 and r13, r14, 15



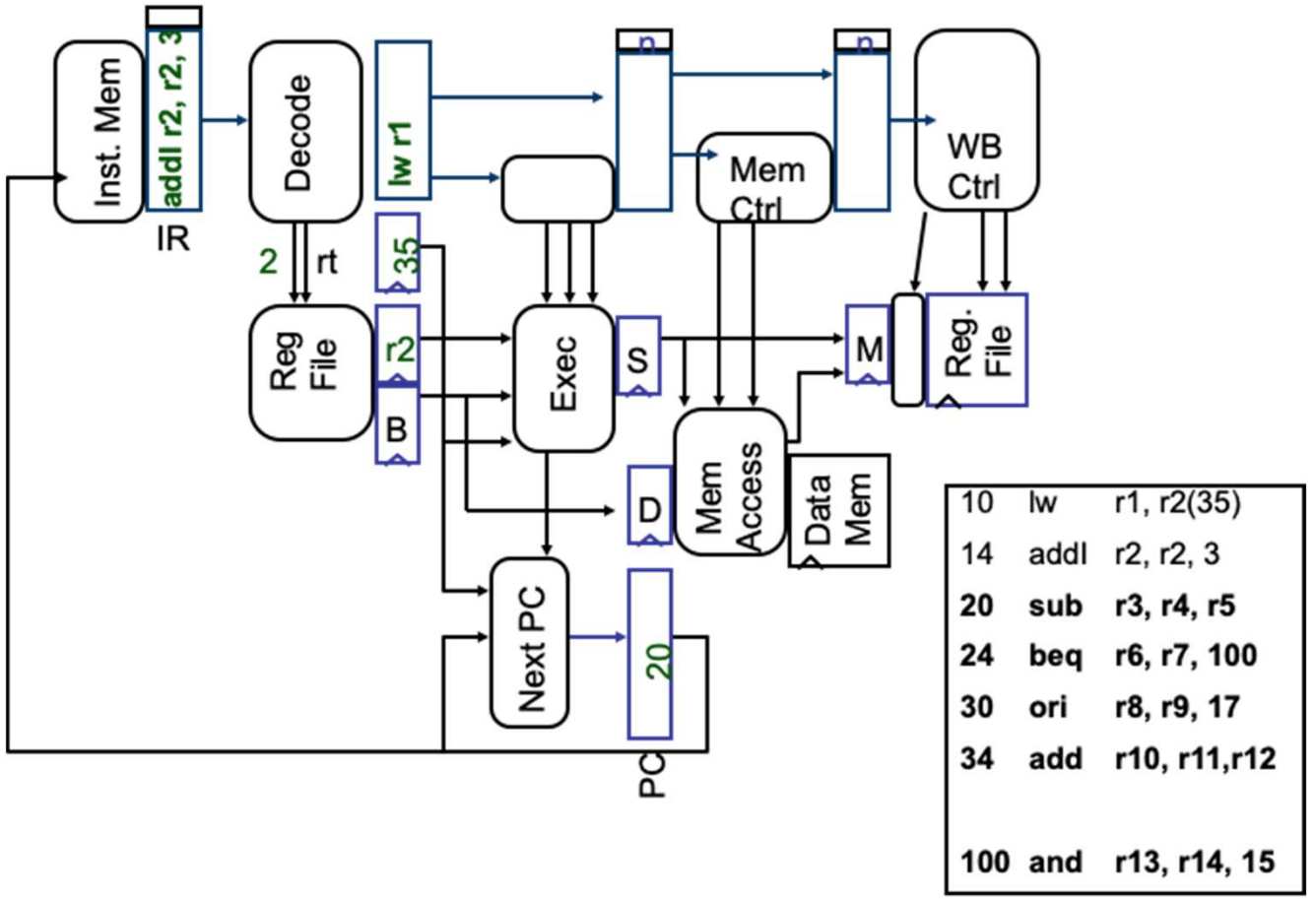
Fetch 14, Decode 10



10	lw	r1, r2(35)
14	**addl**	**r2, r2, 3**
20	**sub**	**r3, r4, r5**
24	**beq**	**r6, r7, 100**
30	**ori**	**r8, r9, 17**
34	**add**	**r10, r11,r12**
100	**and**	**r13, r14, 15**

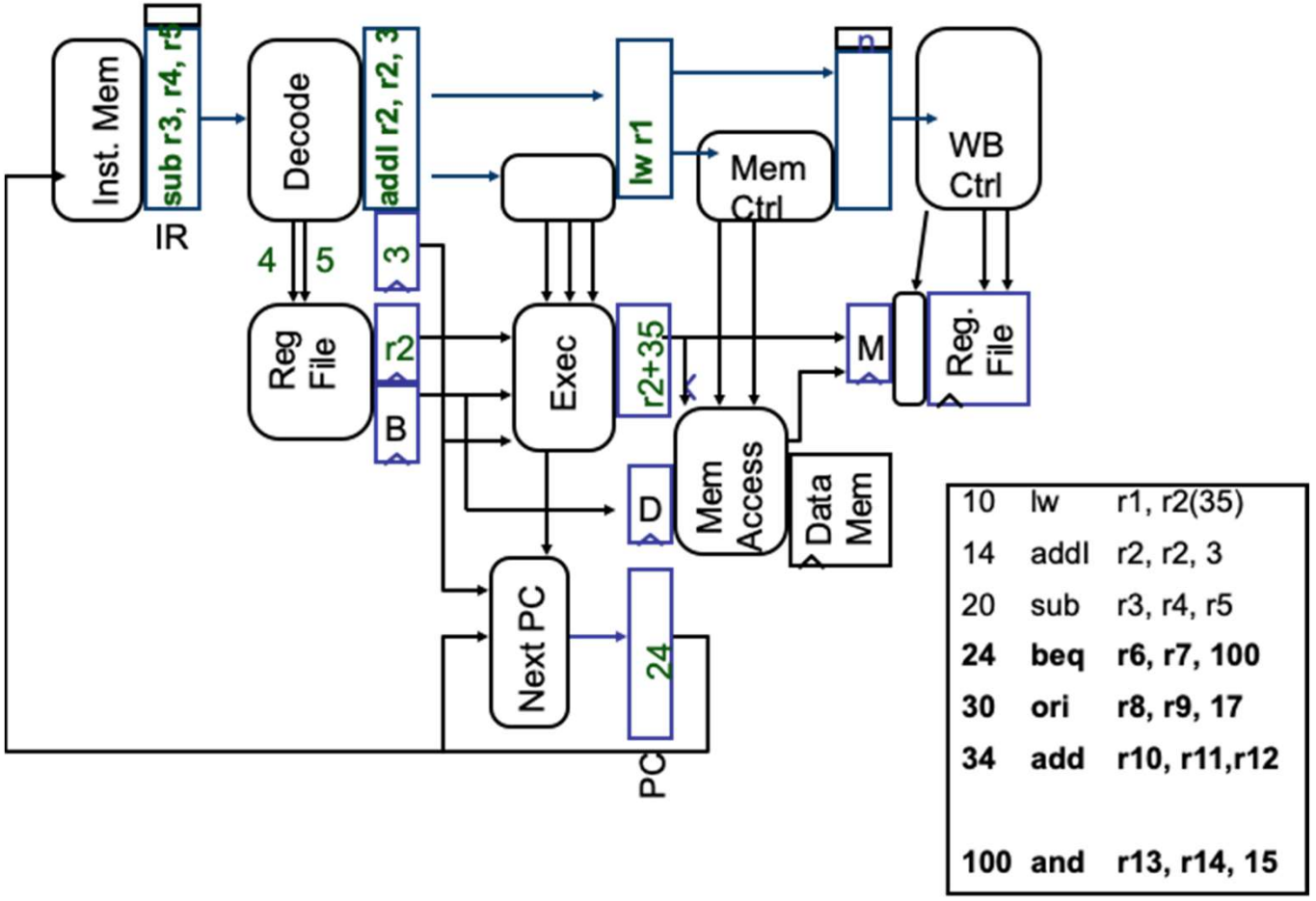


Fetch 20, Decode 14, Exec 10



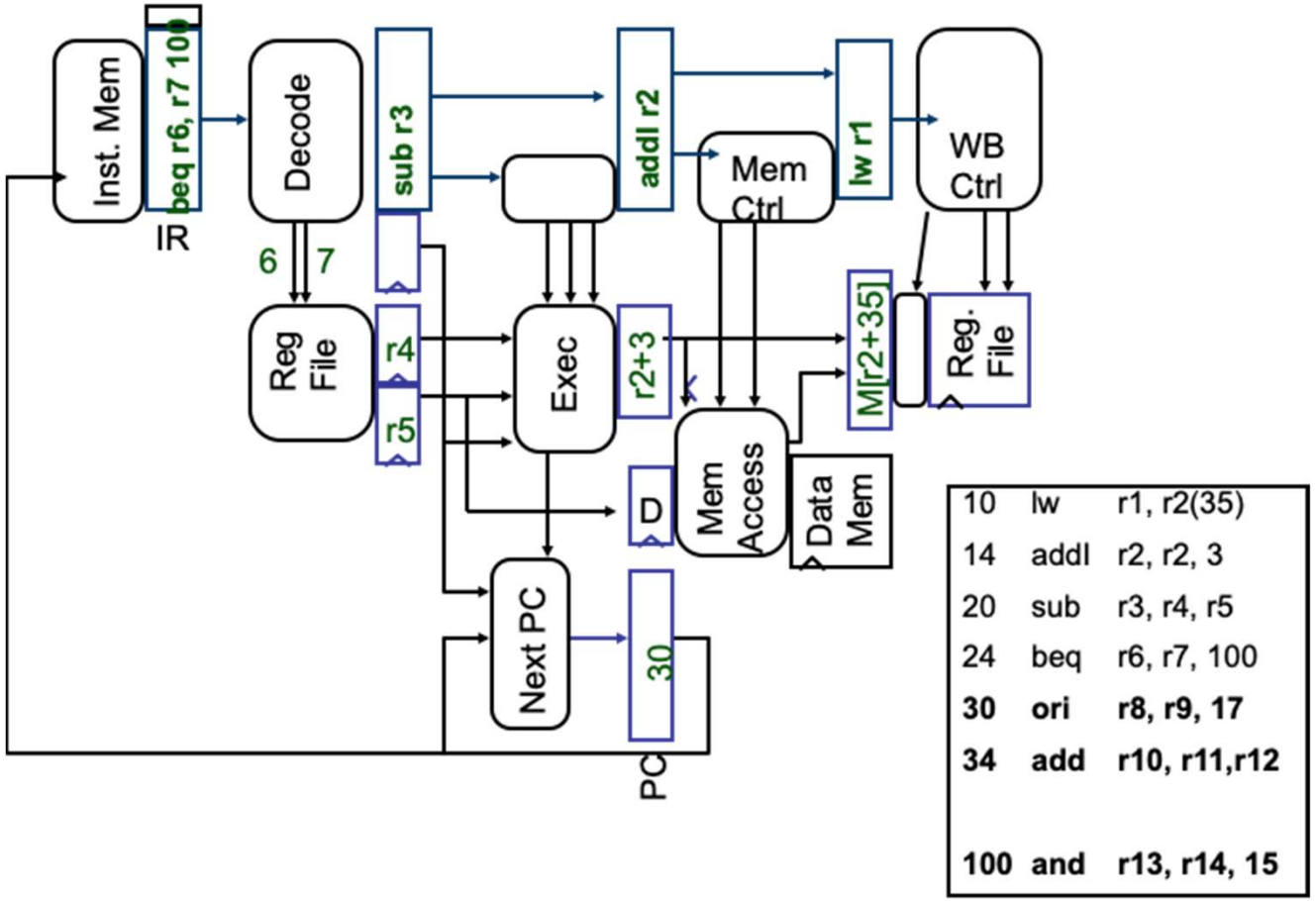


Fetch 24, Decode 20, Exec 14, Mem 10



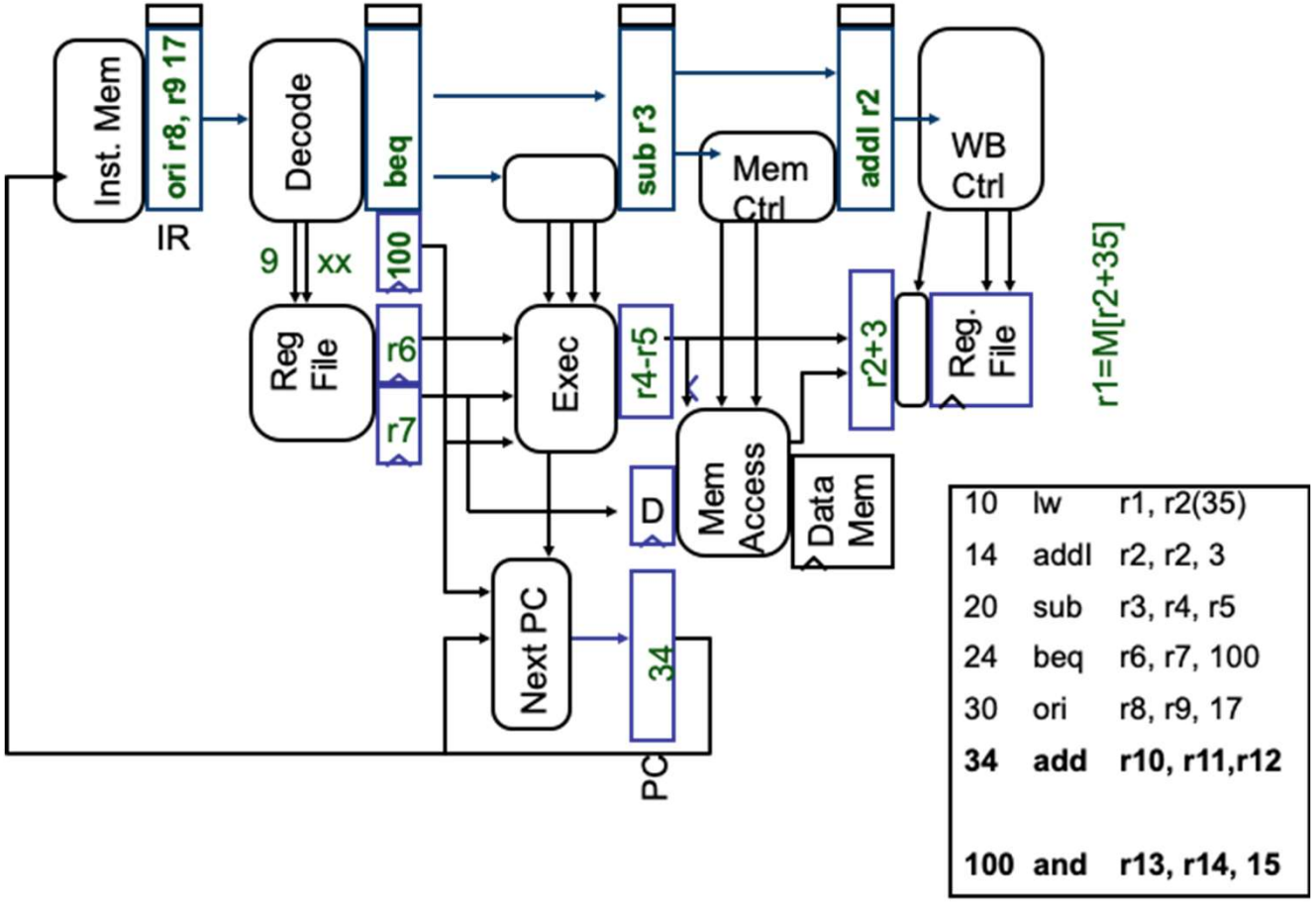


Fetch 30, Dcd 24, Ex 20, Mem 14, WB 10



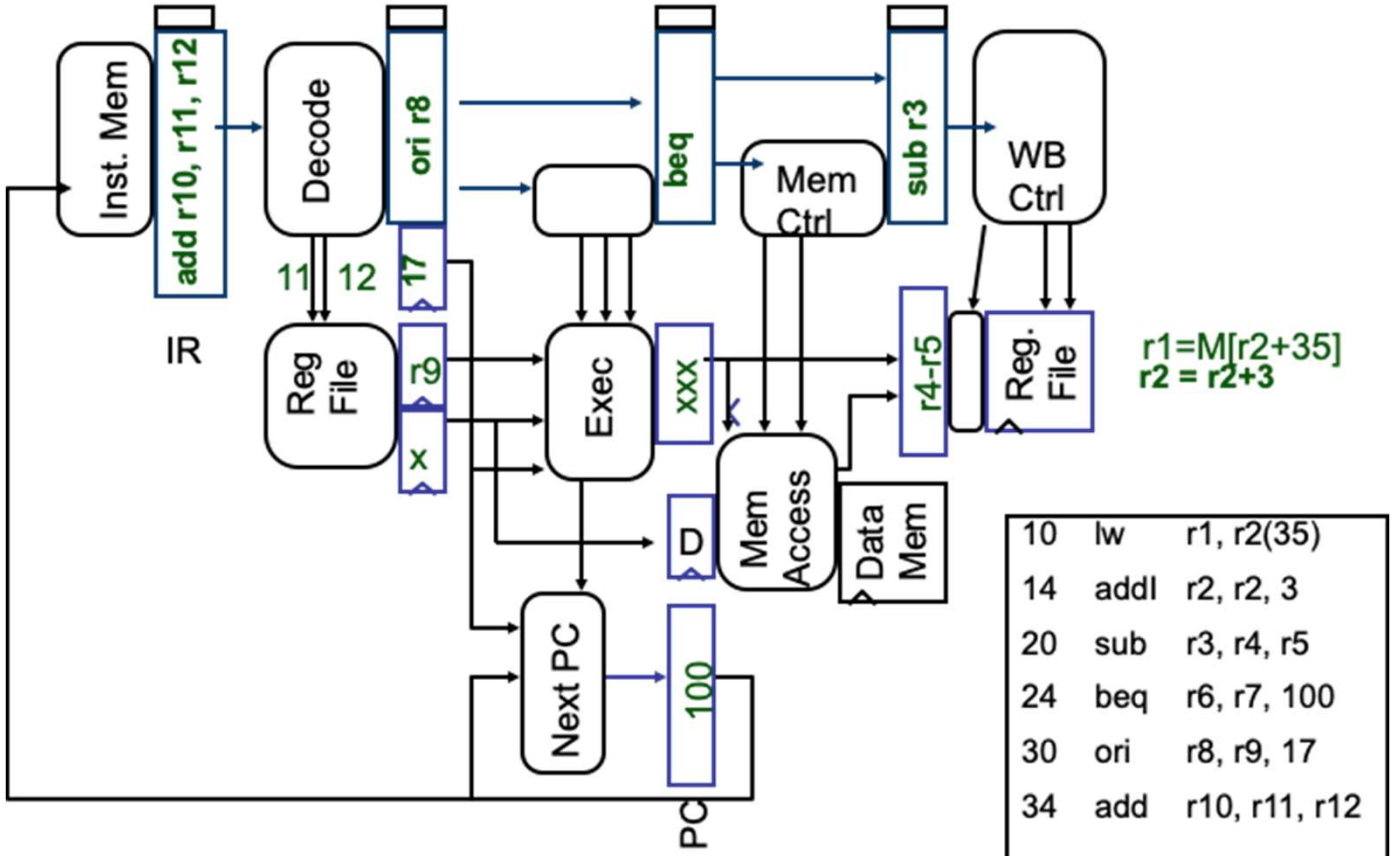


Fetch 34, Dcd 30, Ex 24, Mem 20, WB 14





Fetch 100, Dcd 34, Ex 30, Mem 24, WB 20



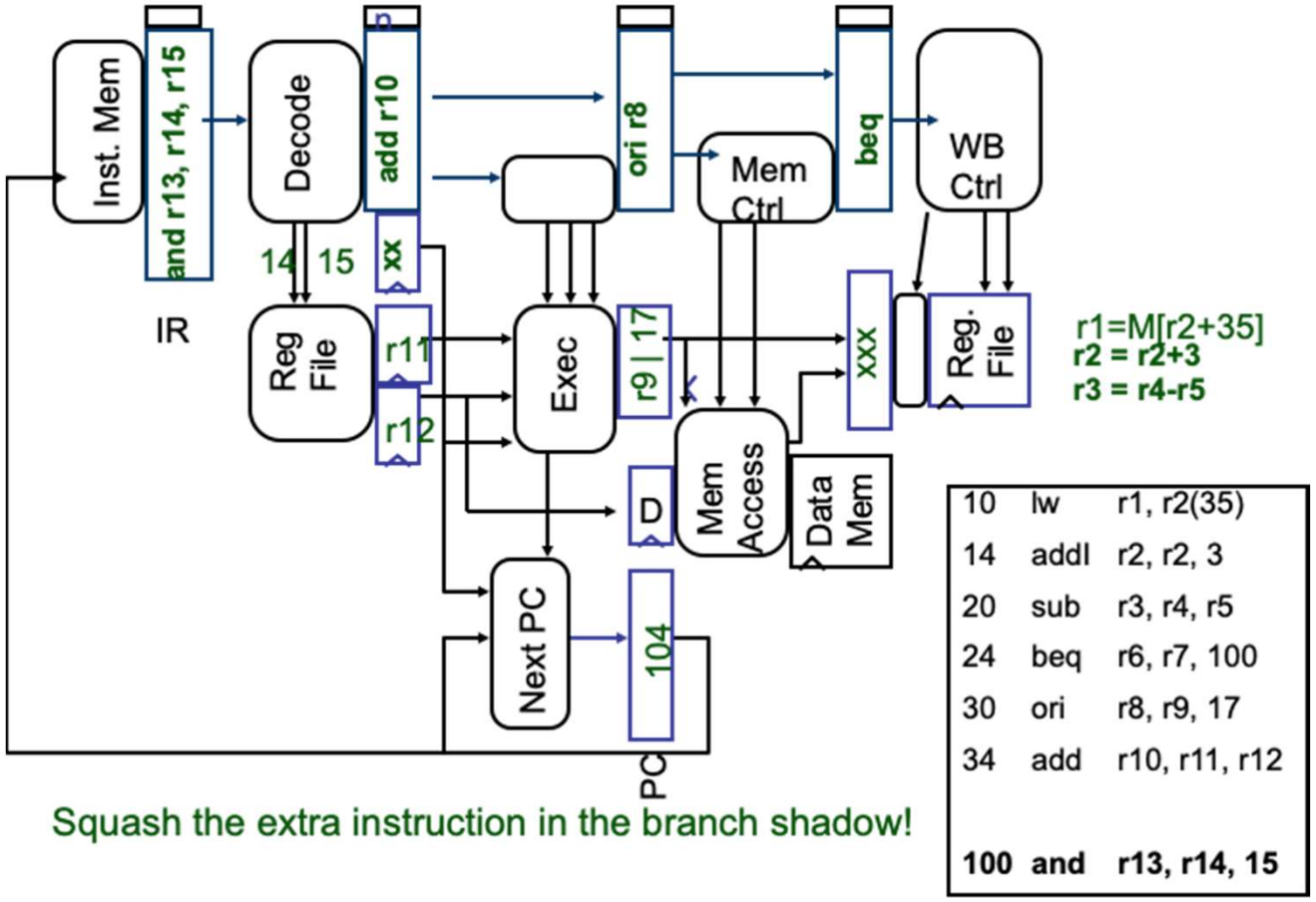
ooops, we should have only one delayed instruction

```
10   lw    r1, r2(35)
14   addI  r2, r2, 3
20   sub   r3, r4, r5
24   beq   r6, r7, 100
30   ori   r8, r9, 17
34   add   r10, r11, r12

100  and   r13, r14, 15
```

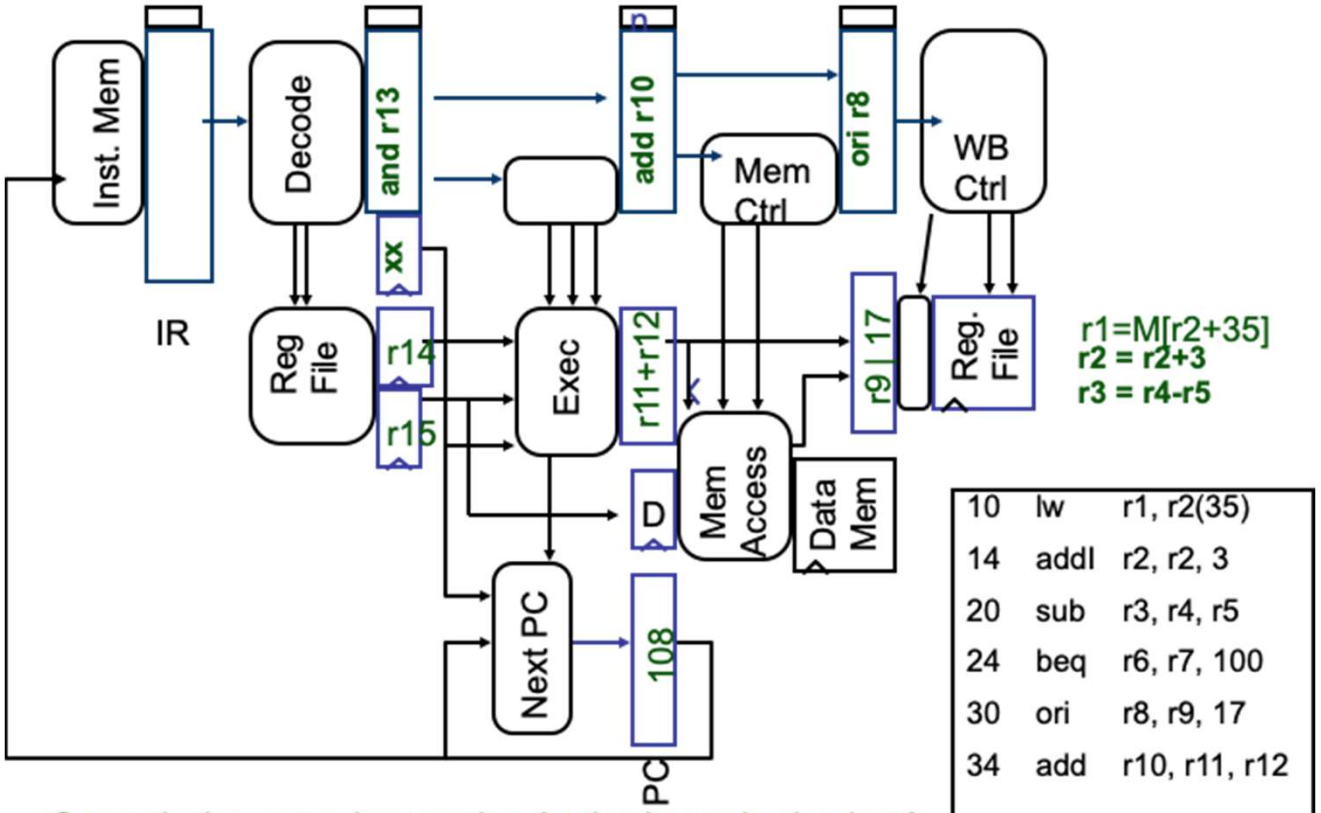


Fetch 104, Dcd 100, Ex 34, Mem 30, WB 24





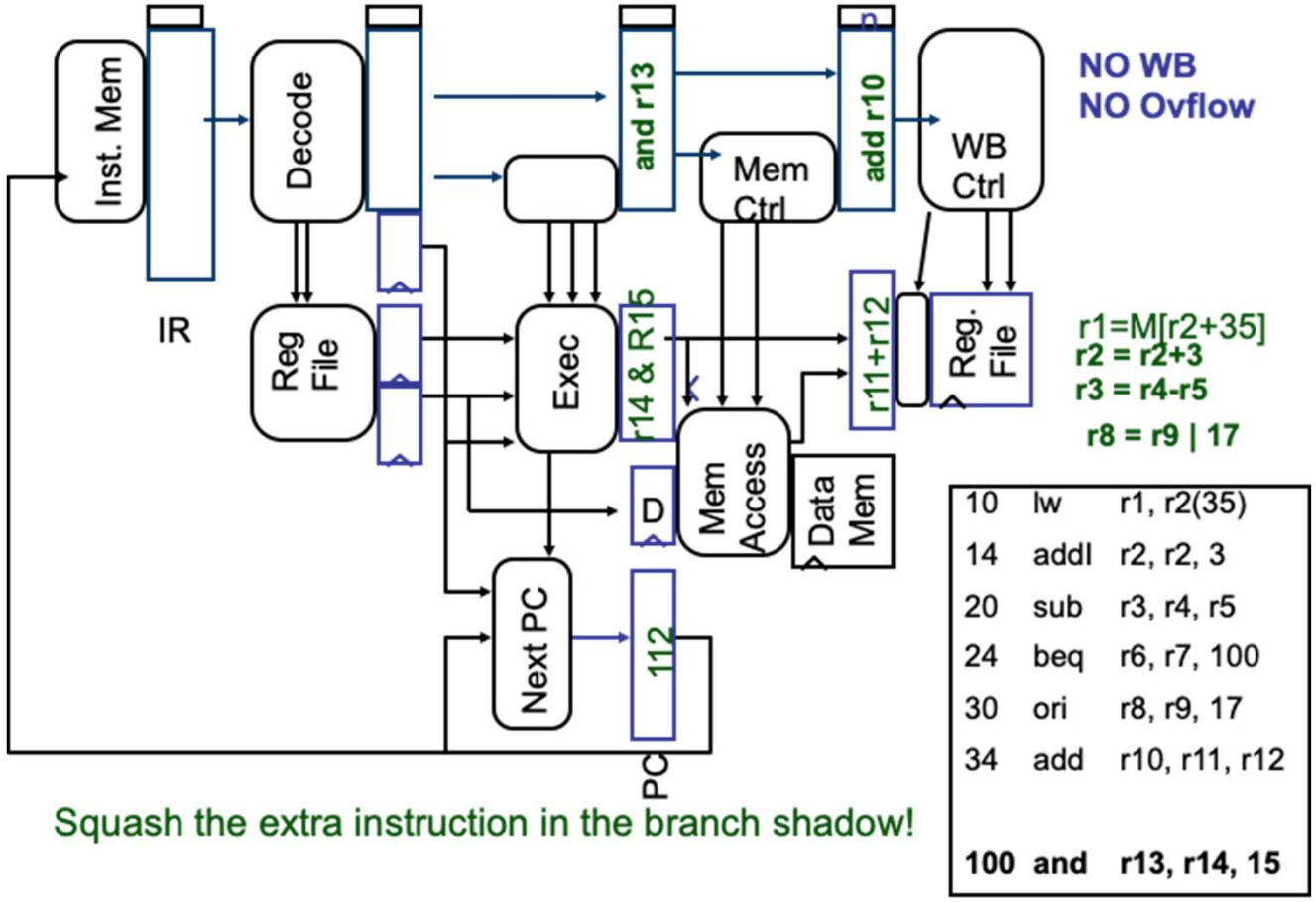
Fetch 108, Dcd 104, Ex 100, Mem 34, WB 30



10	lw	r1, r2(35)
14	addl	r2, r2, 3
20	sub	r3, r4, r5
24	beq	r6, r7, 100
30	ori	r8, r9, 17
34	add	r10, r11, r12
100	**and**	**r13, r14, 15**



Fetch 112, Dcd 108, Ex 104, Mem 100, WB 34



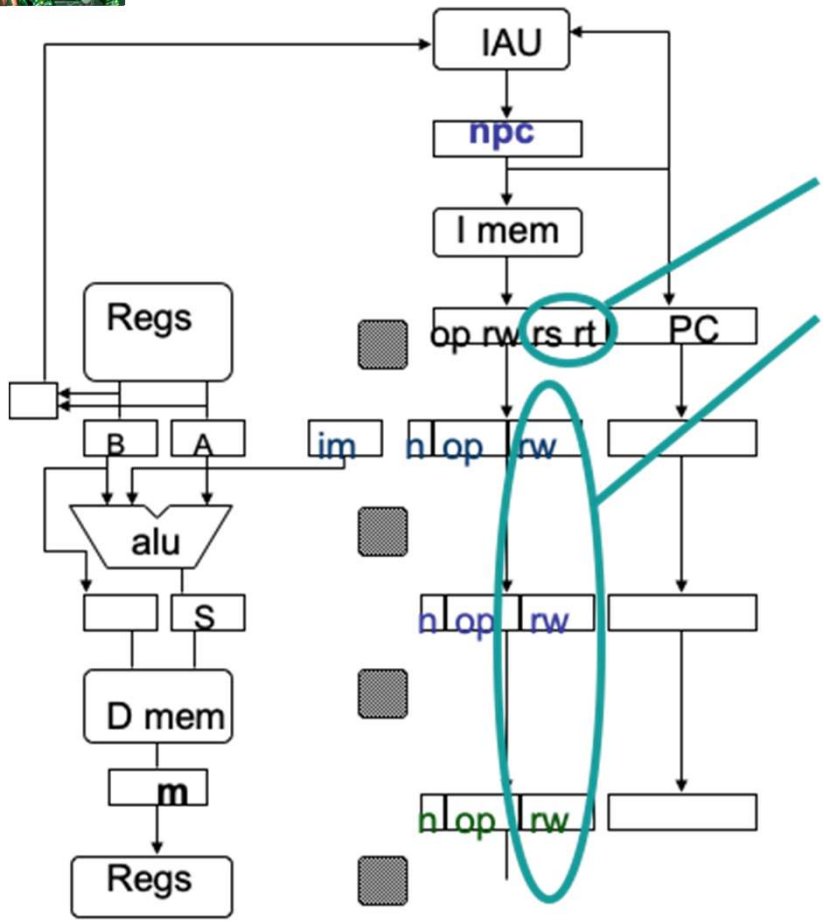


Hazard Detection

- ★ Suppose instruction i is about to be issued and a predecessor instruction j is in the instruction pipeline.
- ★ A RAW hazard exists on register r if r ∈ Rregs(i) ∩ Wregs(j)
  - Keep a record of pending writes (for inst's in the pipe) and compare with operand regs of current instruction.
  - When instruction issues, reserve its result register.
  - When on operation completes, remove its write reservation.
- ★ A WAW hazard exists on register r if r ∈ Wregs(i) ∩ Wregs(j)
- ★ A WAR hazard exists on register r if r ∈ Wregs(i) ∩ Rregs(j)



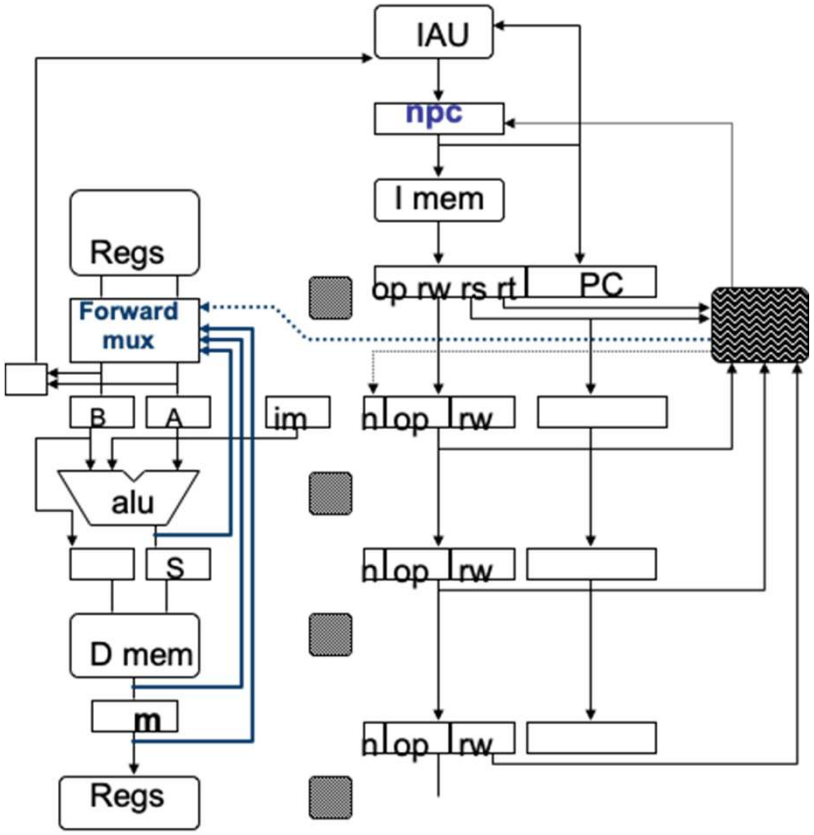
Hazard Detection Circuit



- Current operand registers
- Pending writes
- hazard <=
  - ((rs == rwex) & regWex) OR
  - ((rs == rwmem) & regWme) OR
  - ((rs == rwwb) & regWwb) OR
  - ((rt == rwex) & regWex) OR
  - ((rt == rwmem) & regWme) OR
  - ((rt == rwwb) & regWwb)



Forwarding Circuit

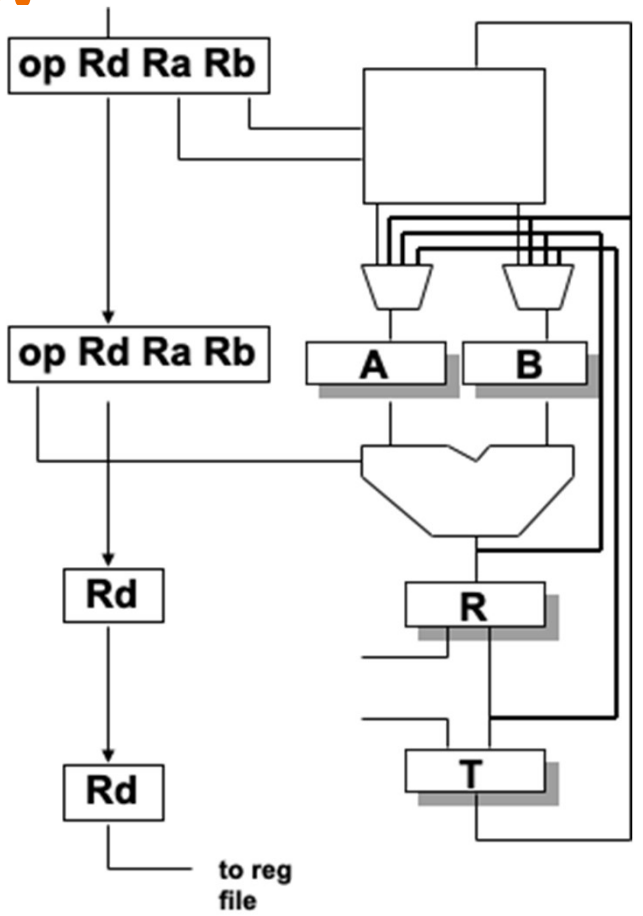


- ★ Detect nearest valid write op operand register and forward into op latches, bypassing remainder of the pipe
- ★ Increase muxes to add paths from pipeline registers
- ★ Data Forwarding = Data Bypassing



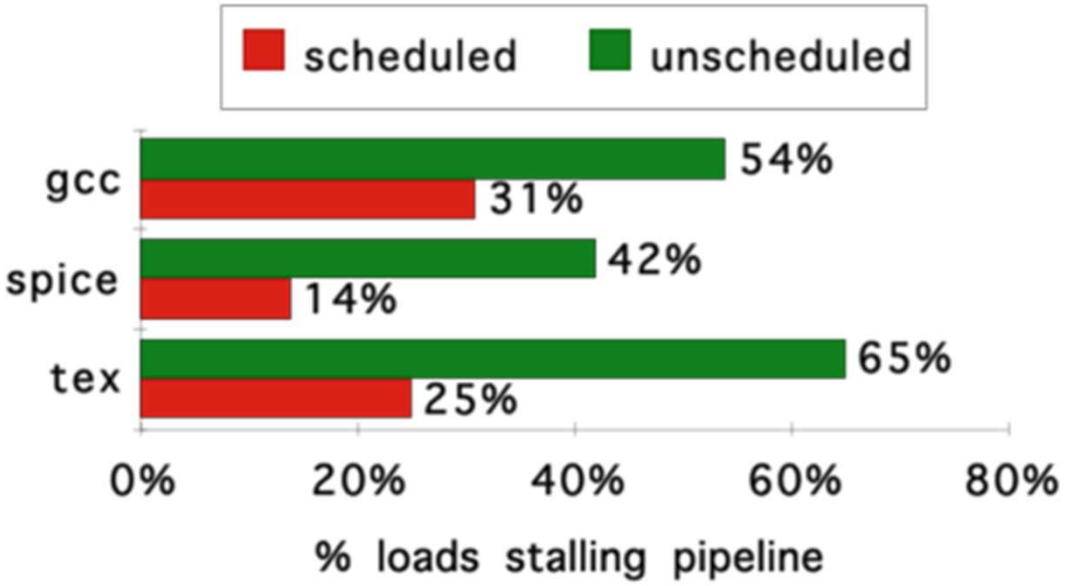
What about memory operations?

- If instructions are initiated in order and operations always occur in the same stage, there can be no hazards between memory operations!
- What does delaying WB on arithmetic operations cost?
  - cycles ?
  - hardware ?
- What about data dependence on loads?
  - R1 <- R4 + R5
  - R2 <- Mem[R2 + I]
  - R3 <- R2 + R1
- => “Delayed Loads”





Compiler scheduling to avoid load stalls





Summary

- ★ Pipelining is a fundamental concept
- ★ multiple steps using distinct resources
- ★ Utilize capabilities of the Datapath by pipelined instruction processing
- ★ start next instruction while working on the current one
- ★ limited by length of longest stage (plus fill/flush)
- ★ detect and resolve hazards



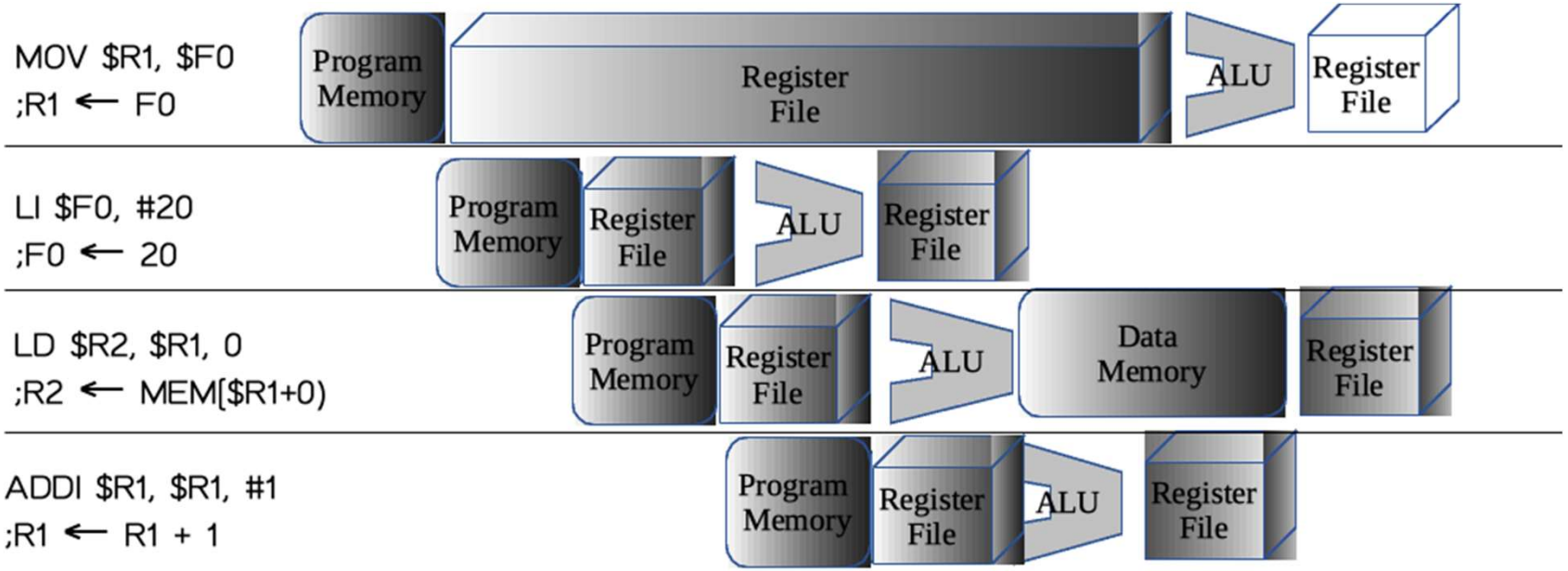
Exercises





Data Hazard

★ Identify all data hazards in this diagram. (WAR, WAW, RAW)





Rescheduling

- Assuming that the hardware has a full support for forwarding hardware, please reschedule the following code to take the benefit of delayed branch and delayed load. You answer must be free from stall cycle (if possible)
- Loop: LD $F0, 0($R1)
  - ADD $F4, $F0, $F2
  - SD 0($R1), $F4
  - LD $F1, 8($R1)
  - ADD $F5, $F1, $F2
  - SD 8($R1), $F5
  - SUB $R1, $R1, #16
  - BNEZ $R1, Loop



End of Chapter 6 (part 2)





Contemporary Architecture

Chapter 7



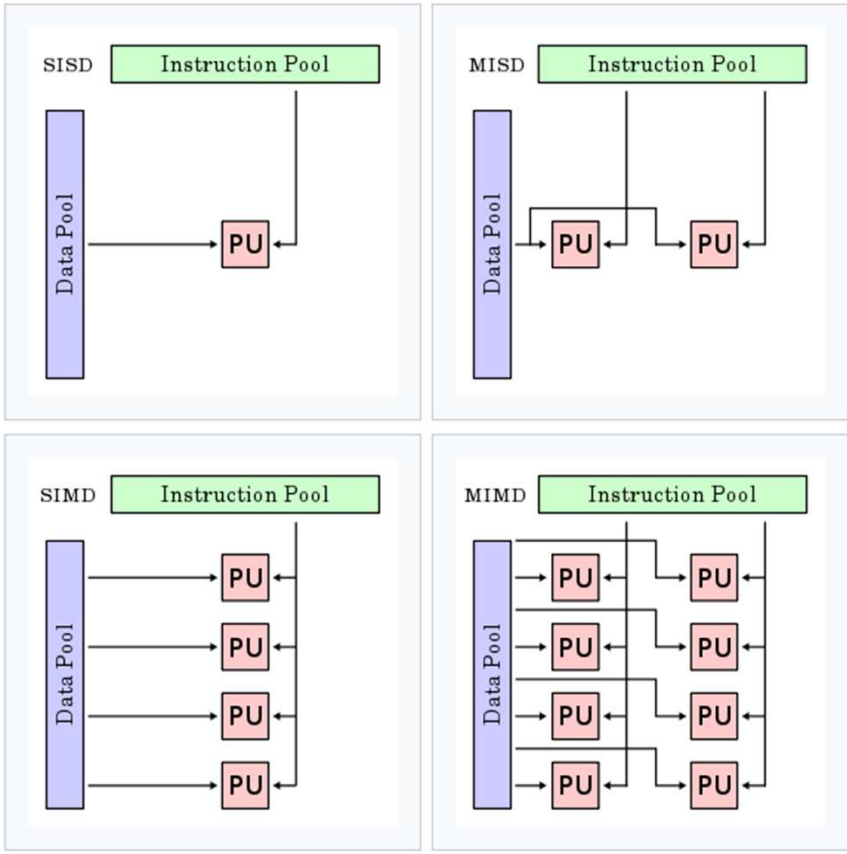
Contemporary Architecture

- ★ Flynn's Taxonomy
  - SISD, SIMD, MISD, MIMD
- ★ Instruction-Level Parallelism
  - Vector Processor, Vector Unit
  - Pipelining Processor, SuperPipeline
  - SuperScalar (Out-of-order execution, Register Renaming)
  - Very Long Instruction Word (VLIW)
- ★ SuperSymmetry
- ★ Dynamic vs. Static Scheduling
- ★ Compiler & ILP
  - Loop Unroll, Software Pipelining, LLVM
- ★ Thread-Level Parallelism



Flynn's Taxonomy

- ★ by Michael Flynn (1934-) in 1966
- ★ Partition by Instruction Stream and Data Stream
  - Single Instruction stream, Single Data stream (SISD) - **uniprocessor**
  - Single Instruction stream, Multiple Data stream (SIMD) - **vector processor**
  - Multiple Instruction stream, Single Data stream (MISD) - **Fault-Tolerant Computing** (The course is available as approved from A. Arthit.)
  - Multiple Instruction stream, Multiple Data stream (MISD) - **Multicore, Distributed Systems**



Pictures from wikipedia



Instruction-Level Parallelism

- ★ ILP - a measure of how many of the instructions in a computer program can be executed simultaneously
- ★ Several micro-architectural techniques involve:
  - Vectorization
  - Speculative Execution - allow (parts of) instructions to be executed before needed.
    - Branch Prediction - avoid stalling from control dependencies(speculative execution)
  - Instruction Pipelining and Super Pipeline
  - SuperScalar
    - Out-of-order execution
    - Register Renaming
  - VLIW



Vector Processor, Vector Unit

- ★ A variation of SIMD
- ★ Known products
  - Apple/Motorola/IBM Altivec
  - Intel MMX, SSE[2,3,4], AVX[2,512]
  - GPU - CUDA, OpenCL
- ★ Apply single operation to vectors (array of data)
- ★ We will revisit this on GPGPU topics.

```
// uniprocessor
int A[8];
int B[8];
int C[8];

for (int i=0;i<8;i++)
{
    C[i]=A[i]+B[i];
}
```

```
// vectorize code
// equivalent in C
int A[8];
int B[8];
int C[8];

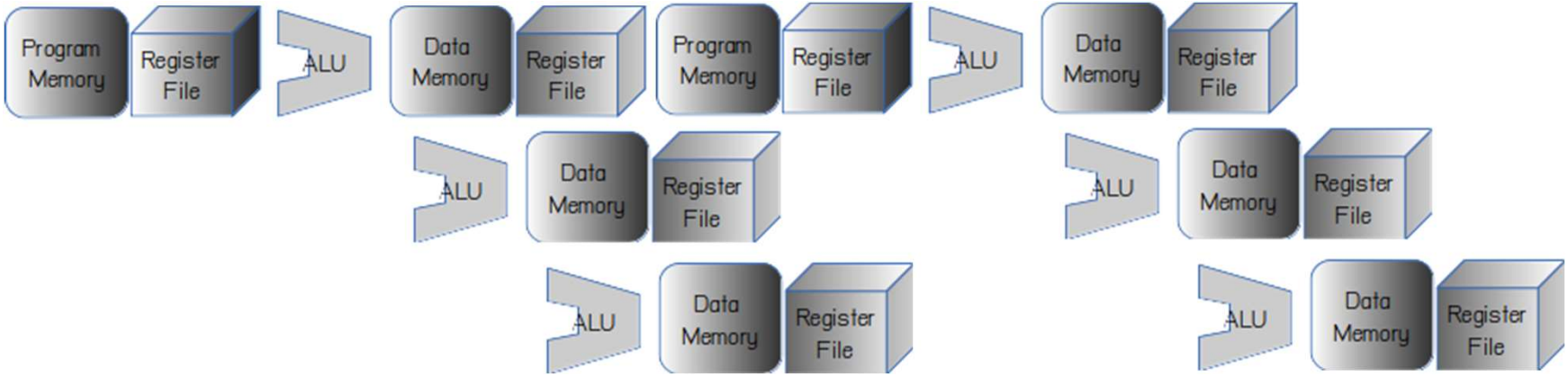
C=A+B;
```

```
; Optimize of -O3 on GCC 6.3
; vectorize with Intel AVX
….
movdqa    16+B(%rip), %xmm0
paddd    16+A(%rip), %xmm0
movaps    %xmm0, 16+C(%rip)
….
```



Vector Processor (ctd.)

★ Some vector processors will expand loops internally.

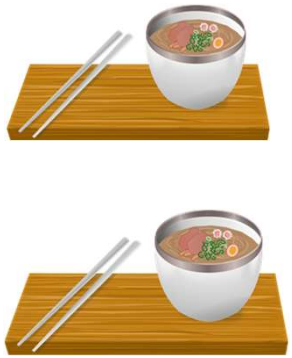




Vector processor in real world

★ Ordering several dishes of the same things, a cook receives an order once.





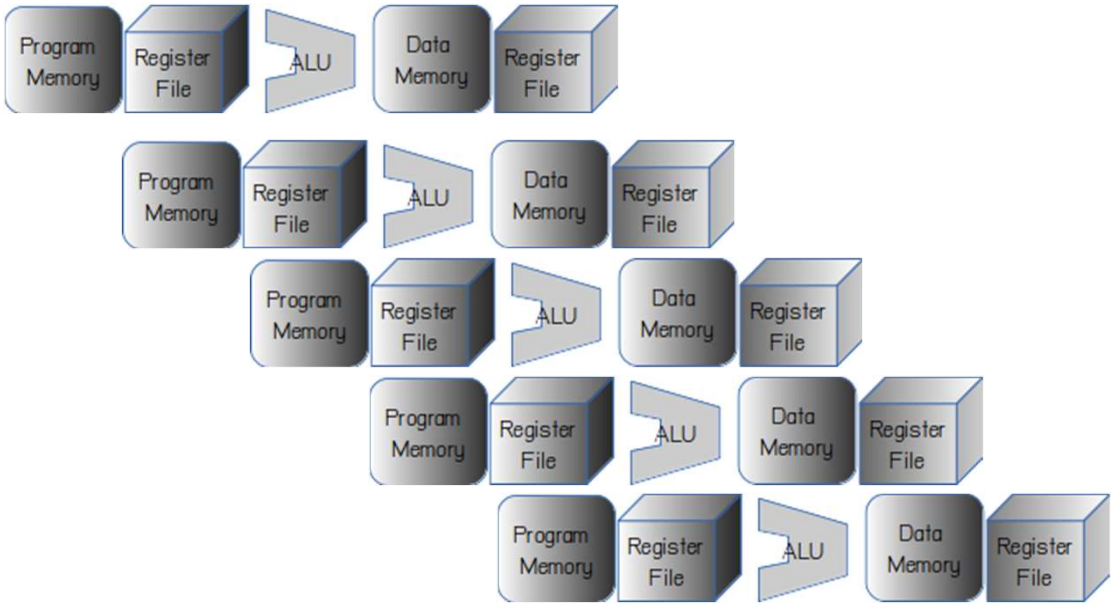




Pipelining Processor (revisit)

- ★ A pipeline with 5 states has 5 instructions in flight.
- ★ For k state pipelines, ILP = k.

- ★ CPU Time for n instructions
  T(n) = (k + (n-1)) Tc





Pipeline in real world

- ★ One line for buying food.
- ★ 5 steps







SuperScalar

- ★ SuperScalar with degree 2 and 5 states of pipeline has 10 instructions in flight.
- ★ For k state pipelines in superscalar x degrees, ILP = kx.
- ★ CPU Time for n instructions $T(n) = (k + (n-x)/x)\ Tc$

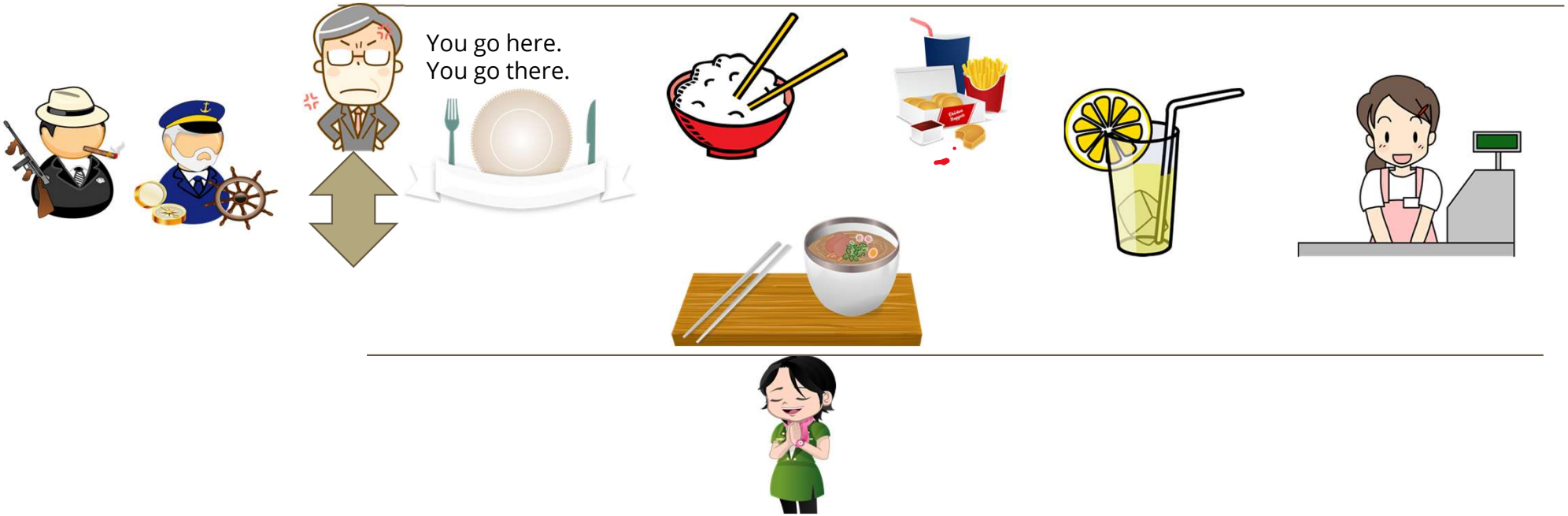




SuperScalar in real world

- ★ One line for buying food.
- ★ After getting in, a manager will separate customers into two lines (rice line and noodle line)







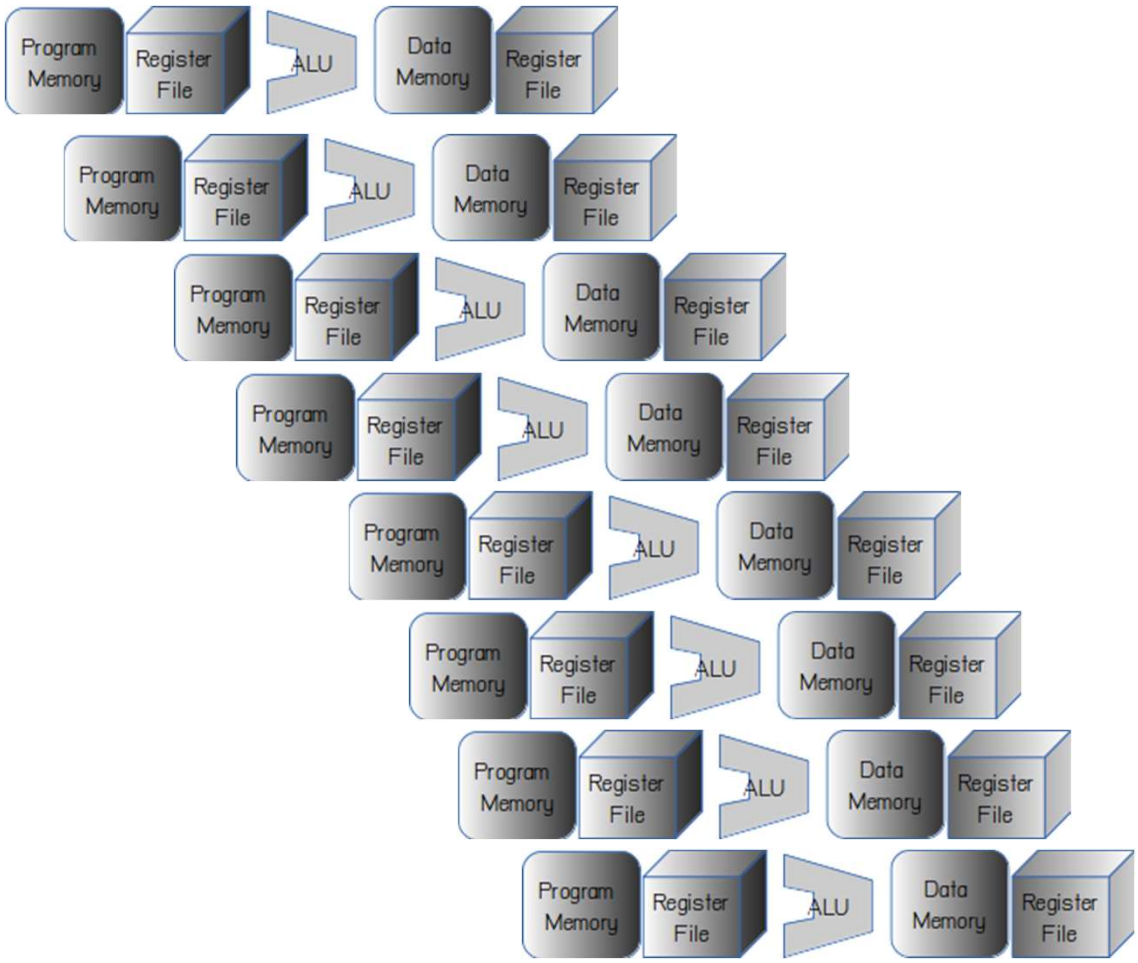
SuperScalar (ctd.)

- ★ With super scalar, we may have **out-of-order execution**.
- ★ A person that comes after (but order noodle) may finish earlier than a person that comes before (but order rice).
- ★ This may result in potential **WAR** and **WAW**.
- ★ To solve this -- **in-order completion** ?



SuperPipeline

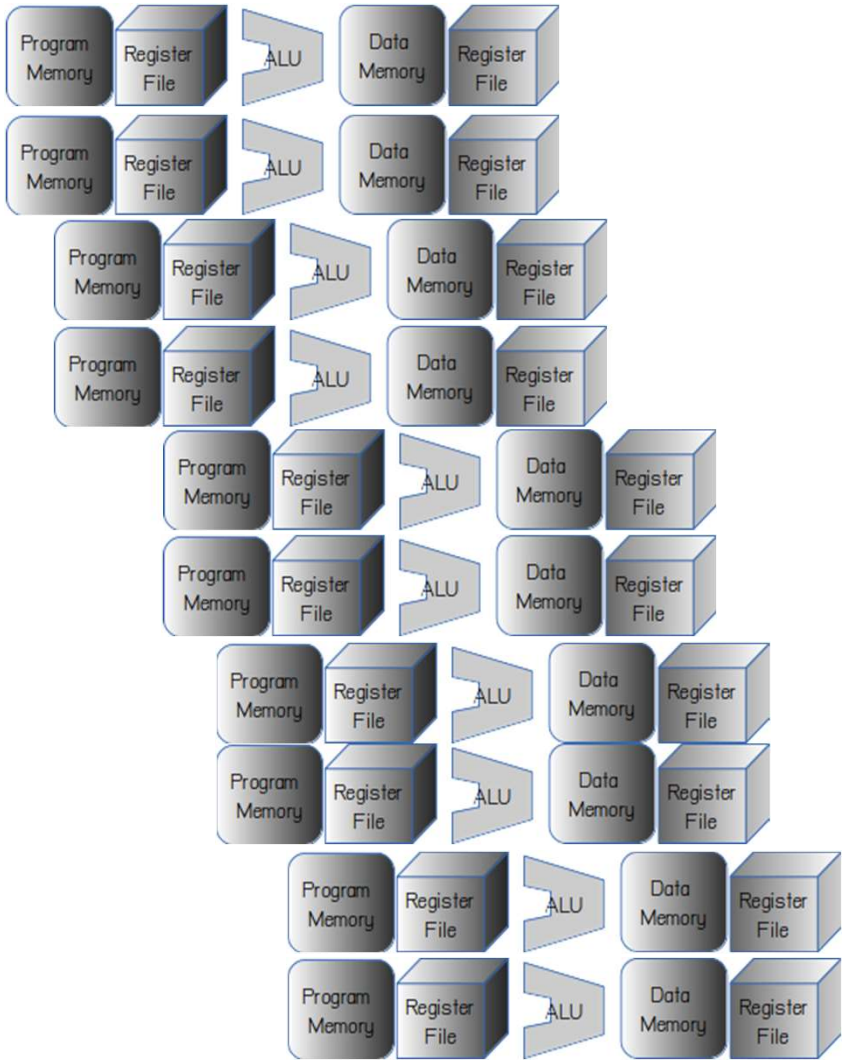
- ★ Use minor cycles.
- ★ In this example, 2 minor cycles is equivalent to 1 major cycle.
- ★ For m minor cycles to k state pipeline, ILP=mk
- ★ CPU Time for n instructions T(n) = (k + (n-1)/m) Tc





SuperPipeline SuperScalar

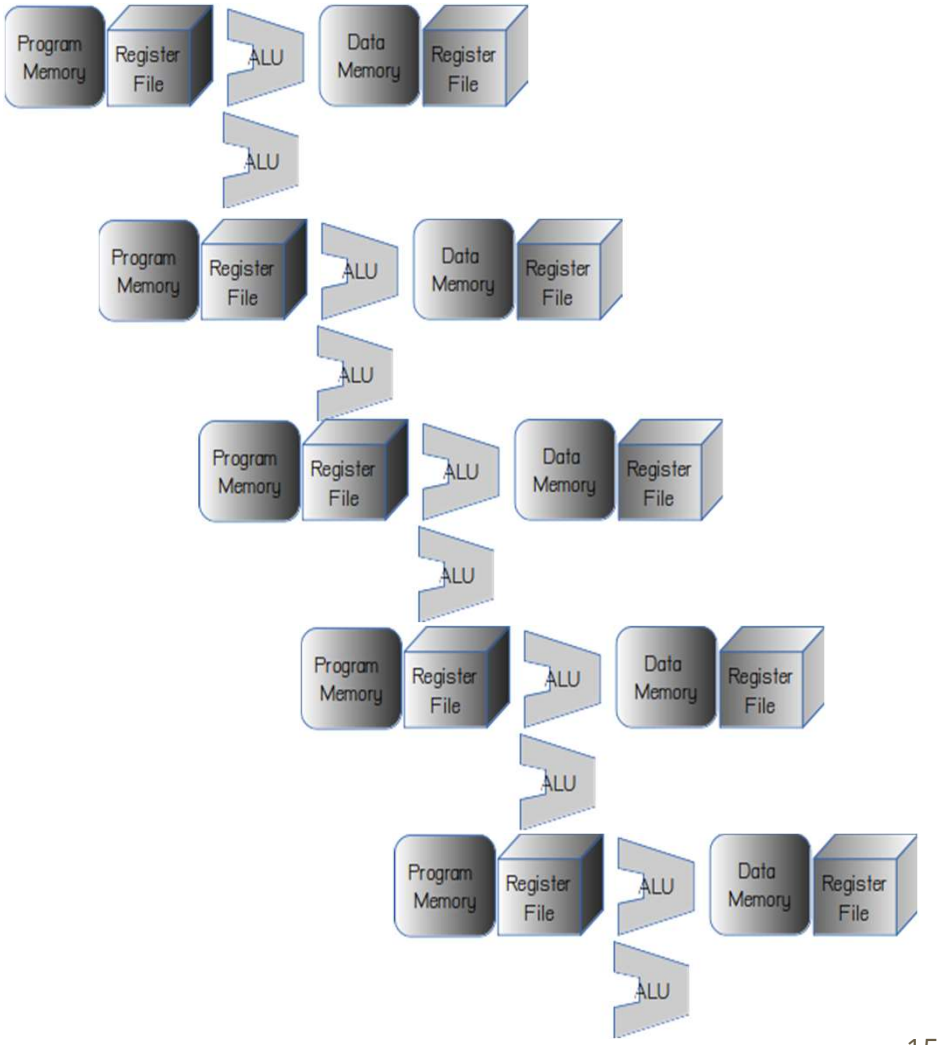
- ★ SuperScalar with minor cycles.
- ★ In this example, 2 minor cycles is equivalent to 1 major cycle.
- ★ For m minor cycles to k states pipeline in superscale degree x,
  ILP=mkx
- ★ CPU Time for n instructions
  $T(n) = (k + (n-x)/mx)\ Tc$





VLIW

- ★ VLIW with degree 2 and 5 states of pipeline has 10 instructions in flight.
- ★ For k state pipelines in VLIW x degrees,
  ILP = kx.
- ★ CPU Time for n major instructions
  T(n) = (k + (n/x-x)) Tc





VLIW in real world

- ★ Static scheduling.
- ★ Two lines for ordering
- ★ One for rice
- ★ Another for noodle



















SuperSymmetry

- ★ SuperScalar
- ★ Speedup=
  (k + (n-x)/x) Tc /
  (k + (n-1)) Tc
- ★ For large n,
  Speedup=x

- ★ SuperPipeline
- ★ Speedup=
  (k + (n-1)/m) Tc/
  (k + (n-1)) Tc
- ★ For large n,
  Speedup=m

- ★ VLIW
- ★ Speedup=
  (k + (n/x-x)) Tc/
  (k + (n-1)) Tc
- ★ For large n,
  Speedup=x

At the same degree, SuperScalar, SuperPipeline and VLIW yield the same performance.



Observations on VLIW

- ★ Software for Intel Itanium and Intel Itanium II (also the successors) are not binary compatible. **Why?** (Hint. static scheduling)
- ★ As of January 2019, Intel officially announced the end of life and product discontinuance of the Itanium CPU family.
- ★ Other implementations of VLIW exist (e.g. Transmeta Crusoe). They are not practically used today.





Speed Up (pipeline)

- ★ Assuming that there are two separate pipelines (integer unit and floating point unit), two instructions can be executed at the sametime.
- ★ CPU Time = Max (Int, FP)
- ★ Potential Speed Up
  =19/Max(Int,FP)
  =19/Max(7,12)
  =1.6

Basic Loop:	Cycles
load Ra <- Ai	1
load Ry <- Yi	1
fmult Rm <- Ra*Rx	7
fadd Rs <- Rm+Ry	5
store Ai <- Rs	1
inc Yi	1
dec i	1
inc Ai	1
branch	1



Performance (Revisit)

- ★ CPU Time = IC x CPI x TC
- ★ In practical, operation classes and latency are not balance.

	IC	CPI	TC	Potential Speed Up
Single Cycle Processor	n	1	L	1
Multiple Cycle Processor	n	3-5 (avg=4.2)	L/5	1.xx
Pipelining Processor	n	1	L/5	5
SuperScalar (degree k)	n	1/k	L/5	5k
SuperPipeline (m minor cycle)	n	1/m	L/5	5m
VLIW (x instructions in a word)	n/x	1	L/5	5x



Compiler & ILP

- ★ Compiler can play a critical role in exploding more ILP
- ★ Small basic block means less opportunity for filling available slots.
- ★ A larger basic block means more opportunities for filling slots.
- ★ To increase a basic block,
  - ○ Loop Unrolling
  - ○ Software Pipeline



Loop Unroll

- ★ Make the basic block in a loop larger by having more iterations in a loop.
- ★ The new code is potentially not stall at the cost of larger code size.

Normal loop.

```
for (i=MAX-1;i>=0;i--) {
    A[i] = A[i] + C;
}
```

Symbolically unroll 4 iterations

```
for (i=MAX-1;i>=3;i-=4) {
    A[i] = A[i] + C;
    A[i-1] = A[i-1] + C;
    A[i-2] = A[i-2] + C;
    A[i-3] = A[i-3] + C;
}
```



Loop Unroll (ctd.)

Normal Loop

```
LOOP:
              LW     $f0, $r1
(stall)
    ADD     $f2, $f0,$f1
(stall)
(stall)
    SW     $f2, $r1
    SUB     $r1, $r1,#4
    BNE     $r1, LOOP
```

8 cycles / Iteration

Register Renaming

- ★ To free unrolled iterations from the first iteration, a register would be renamed.
- ★ Each iteration is now resource independent.

Unroll (4 iterations)

```
LOOP:
   LW     $f0, 0($r1)
(stall)
   ADD     $f2, $f0, $f1
(stall)
(stall)
   SW     $f2, 0($r1)
   LW     $f3, 4($r1)
(stall)
   ADD     $f4, $f3, $f1
(stall)
(stall)
   SW     $f4, 4($r1)
   LW     $f5, 8($r1)
(stall)
   ADD     $f6, $f5, $f1
(stall)
(stall)
   SW     $f6, 8($r1)
   LW     $f7, 12($r1)
(stall)
   ADD     $f8, $f6, $f1
(stall)
(stall)
   SW     $f8, 12($r1)
   SUB     $r1, $r1,#16
   BNE     $r1, LOOP
```

Unroll + Rescheduling

```
LOOP:
    LW     $f0, 0($r1)
    LW     $f3, 4($r1)
    LW     $f5, 8($r1)
    LW     $f7, 12($r1)
    ADD     $f2, $f0, $f1
    ADD     $f4, $f3, $f1
    ADD     $f6, $f5, $f1
    ADD     $f8, $f6, $f1
    SW     $f2, 0($r1)
    SW     $f4, 4($r1)
    SW     $f6, 8($r1)
    SW     $f8, 12($r1)
    SUB     $r1, $r1,#16
    BNE     $r1, LOOP
```

14 cycles / 4 Iteration
= 3.5 cycles per iteration

Speed Up=8/3.5



Loop Unroll and Superscalar

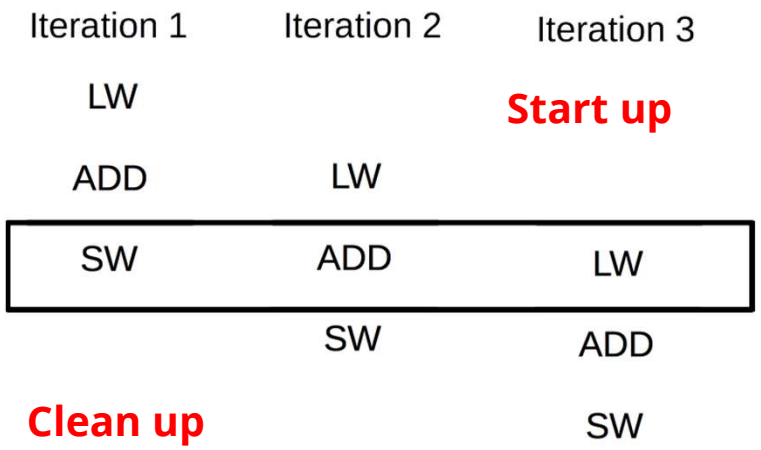
12 cycles / 5 Iteration
= 2.4 cycles per iteration

Integer Unit	Floating Unit	Clock Cycle (Issued)
LOOP:		
LW **$f0**, 0($r1)		1
LW $f3, 4($r1)		2
LW $f5, 8($r1)	ADD $f2, **$f0**, $f1	3
LW $f7, 12($r1)	ADD $f4, $f3, $f1	4
LW $f9, 16($r1)	ADD $f6, $f5, $f1	5
SW $f2, 0($r1)	ADD $f8, $f6, $f1	6
SW $f4, 4($r1)	ADD $f10, $9, $f1	7
SW $f6, 8($r1)		8
SW $f8, 12($r1)		9
SW $f10, 16($r1)		10
SUB $r1, $r1,#24		11
BNE $r1, LOOP		12



Software Pipeline

- ★ Unroll as needed only.
- ★ Good performance, appropriate code size.



Unroll (3 terations)

```
LOOP:
     LW     $f0, 0($r1)
     LW     $f3, 4($r1)
     LW     $f5, 8($r1)
     ADD     $f2, $f0, $f1
     ADD     $f4, $f3, $f1
     ADD     $f6, $f5, $f1
     SW     $f2, 0($r1)
     SW     $f4, 4($r1)
     SW     $f6, 8($r1)
     SUB     $r1, $r1, #12
     BNE     $r1, LOOP
```

Unroll + Rescheduling

Start up

```
     SUB     $r1, $r1, #4
     LW     $f0, 4($r1)
     ADD     $f2, $f0, $f1
     LW     $f0, 0($r1)
LOOP:
     SW     $f2, 0($r1)
     ADD     $f2, $f0, $f1
     LW     $f0, 8($r1)
     SUB     $r1, $r1, #4
     BNE     $r1, LOOP
```

Clean up

```
     SW     $f2, 4($r1)
     ADD     $f2, $f0, $f1
     SW     $f2, 0($r1)
```



Dynamic vs. Static Scheduling

- ★ **Dynamic Scheduling (Hardware)**
- ★ Hardware dynamically schedules code (out-of-order execution).
  - ○ Loop are dynamically unrolled in the processor.
  - ○ Multiple instructions are dynamically issued.
- ★ Legacy software gains better performance on new hardware.
- ★ Complex hardware design?

- ★ **Static Scheduling (Software/Compiler)**
- ★ Compiler can see larger scope.
  - ○ Loop are statically unrolled by compilers.
  - ○ Multiple instructions are statically scheduled for each cycle (see VLIW).
- ★ Poor compatibilities
- ★ Dependency cannot be determined at compile time.
- ★ Simple hardware
- ★ Lower Power requirements?

Who is the winner?



How about a chip with multicores?





Chip with multicores

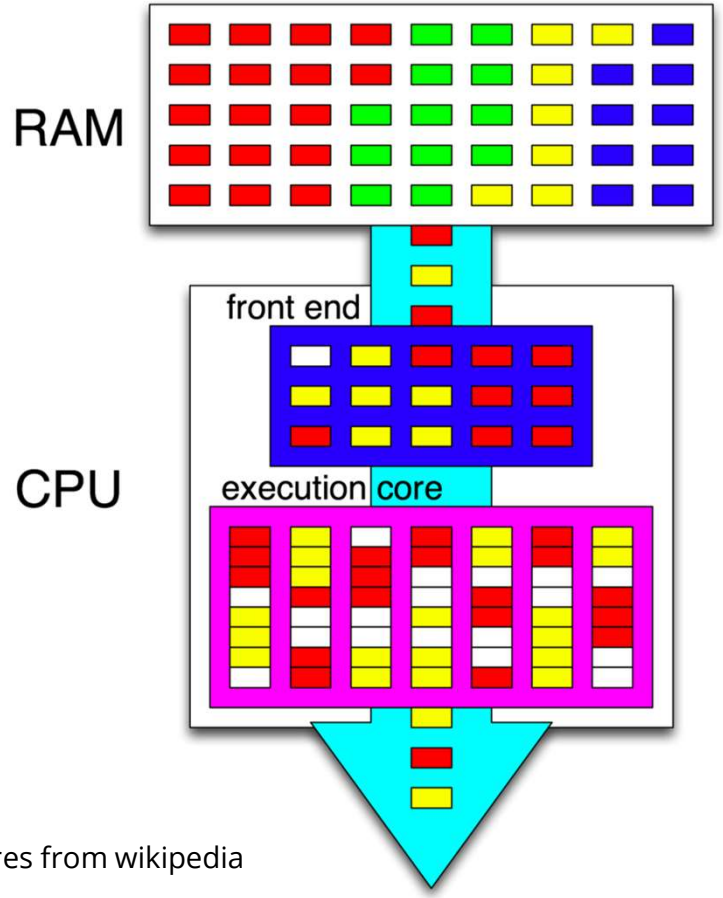
- ★ A chip with multicores is basically a chip with multiple superscalar cores inside.
- ★ Can also be viewed as Multiple Pipes.
- ★ Demands for more ILP. -- Thread-Level parallelism



Thread-Level Parallelism

The detail is beyond the scope of this course. To learn more, join **High Performance Architecture**.

- ★ Since a program (thread) cannot fill available slots in the processor, let's get instructions from several threads.
- ★ Instructions in pipeline are coming from several threads.
- ★ The technology is called simultaneous multithreading (SMT).
- ★ Intel implementation is officially called Hyper-Threading Technology.



Pictures from wikipedia



Summary

- ★ There exist several approaches to ILP.
- ★ At similar degrees, SuperScalar, SuperPipeline, VLIW will yield similar performance.
- ★ Dynamic scheduling allows legacy software to gain performance.
- ★ Static scheduling may gain more benefits if a compiler is more powerful.
  - Proven to be too difficult to have good compiler.



Exercises





Loop Unroll

★ For the given code, if MAX is a known/large value,
(1)how many iterations can we unroll this loop?
(2) What is the potential speed up of the unrolled (rescheduled) code comparing to the original one.
(3) What will be the size of the unrolled code?
(4) Which resources is limited to the number of iterations that can be unrolled?

```
for (i=MAX-1;i>=0;i--) {
    A[i] = A[i] + C;
}
```

```
LOOP:
            LW    $f0,
$r1
(stall)
    ADD    $f2, $f0,$f1
(stall)
(stall)
    SW    $f2, $r1
    SUB    $r1, $r1,#4
    BNE    $r1, LOOP
```



Software Pipeline

★ For the given code, if MAX is a known/large value, please use software pipeline to optimize the code.
(1) What is the potential speed up of the unrolled (rescheduled) code comparing to the loop unroll version.
(2) What will be the size of the code?

```
for (i=MAX-1;i>=0;i--) {
    A[i] = A[i] + C;
}
```

```
LOOP:
            LW    $f0,
$r1
(stall)
    ADD    $f2, $f0,$f1
(stall)
(stall)
    SW    $f2, $r1
    SUB    $r1, $r1,#4
    BNE    $r1, LOOP
```



Dynamic SuperScalar (out-of-order execution)

★ Given a super scalar at degree two, please draw a diagram showing the pipeline of the following code.

```
LOOP:
    LW     $f0, $r1
    ADD     $f2, $f0,$f1
    SW     $f2, $r1
    SUB     $r1, $r1,#4
    BNE     $r1, LOOP
```

		Issue	EX	MEM	WB	Notes
LW	$f0, $r1	1	2	3	4	
ADD	$f2, $f0,$f1	1	5,6,7		8	Wait $f0
SW	$f2, $r1	2	3	9		Wait $f2
SUB	$r1, $r1,#4	2	4		5	
BNE	$r1, LOOP	3	6			
LW	$f0, $r1	4	7	8	9	
ADD	$f2, $f0,$f1	4	10,11,12		13	Wait $f0
SW	$f2, $r1	5	8	14		
SUB	$r1, $r1,#4	5	9		10	
BNE	$r1, LOOP	6	11			



End of Chapter 7





Memory Organization (part 1)

Chapter 8



Memory Organization (part 1)

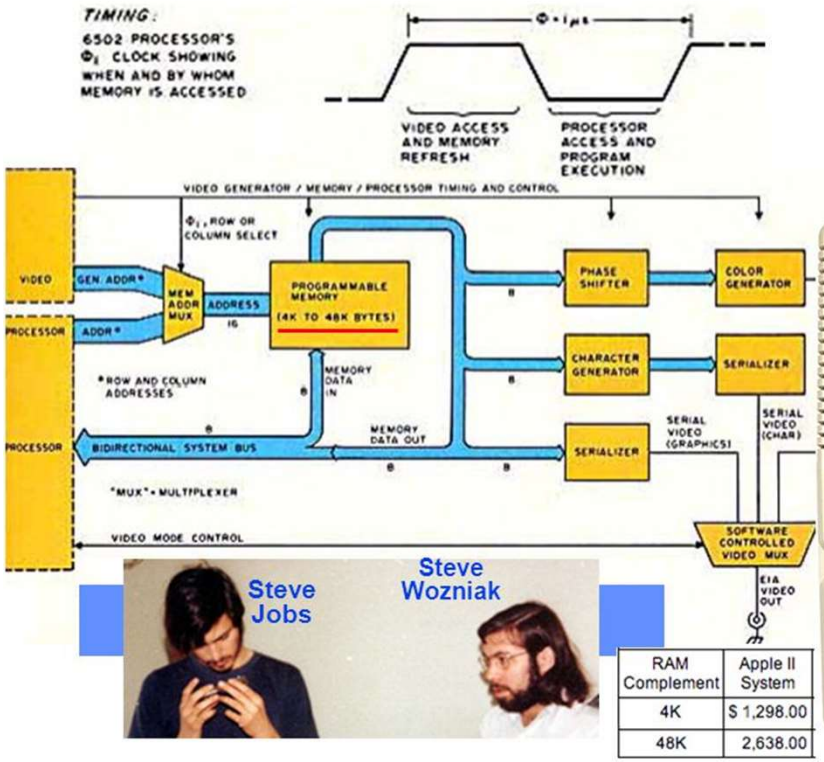
- ★ Technology Trends and Memory Issues
- ★ Design Goal
- ★ Principle of Locality
  - ○ Temporal, Spatial
- ★ Cache: Terminology and Organization
- ★ Performance
  - ○ Sources of Miss
- ★ Configuration
  - ○ Cache Size, Block Size, Miss Penalty
- ★ Write Management
  - ○ Write Buffer
- ★ Software and Cache



Technology Trends and Memory Issues

- ★ In 70s, memory is faster than a processor.
- ★ In the text mode/dos era, 640KB was more than enough.
- ★ Now, minimum requirement is 4-8 GB.

1977: At one time, DRAM was faster than microprocessors 7



RAM Complement	Apple II System
4K	$ 1,298.00
48K	2,638.00

Apple][(1977)

CPU: 1000 ns

DRAM: 400 ns

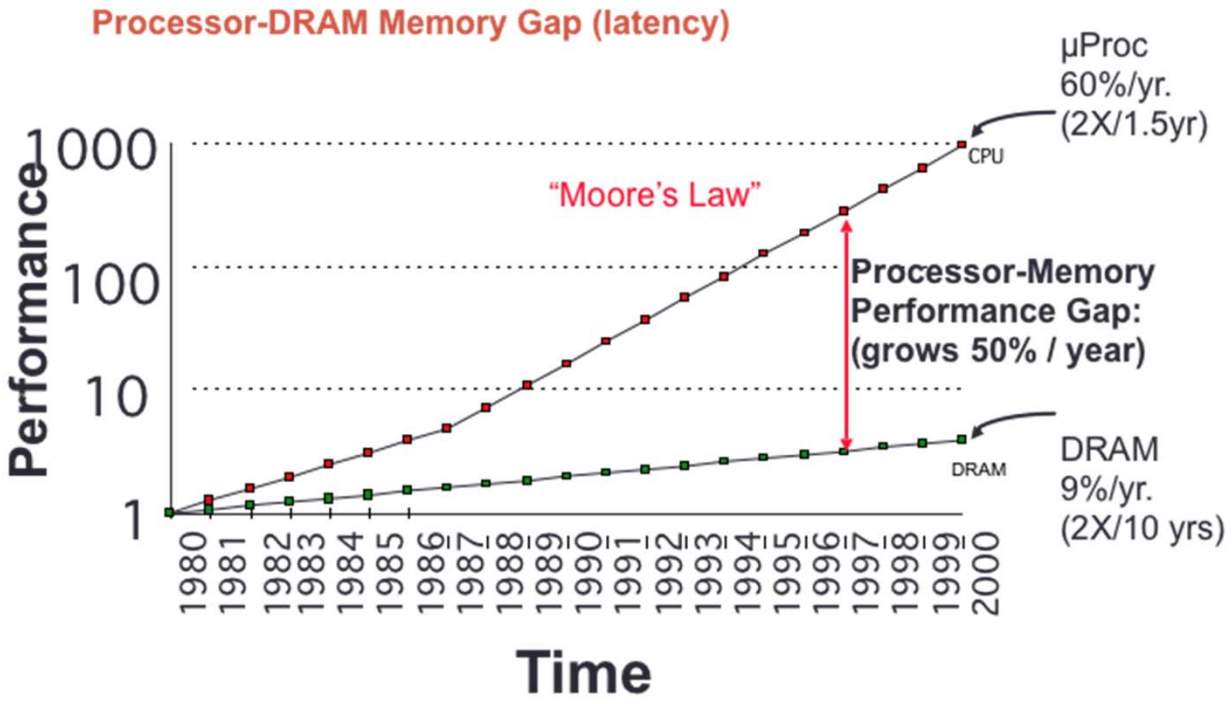


CSCI 620



Technology Trends and Memory Issues

- ★ Now, memory is relatively slow, comparing to processor.
- ★ In 1980, CPU has no cache.
- ★ In 1995, CPU has 2-level caches.





Issues on Performance

- ★ Assuming that a memory operation takes 50 cycles and other operations take 1 cycle (with pipeline). With 10% of memory operation, what is the percentage of time that a processor has to wait for memory operations?
- ★ CPU Time = IC x CPI x Tc
- ★ CPU Time = ((0.9 IC x 1) + (0.1 IC x 50)) x Tc
- ★ Memory Wait = (0.1 IC x 50) x Tc
- ★ Wait time = (0.1 IC x 50) / ((0.9 IC x 1) + (0.1 IC x 50)) = 5 / 5.9 ≅ 85%



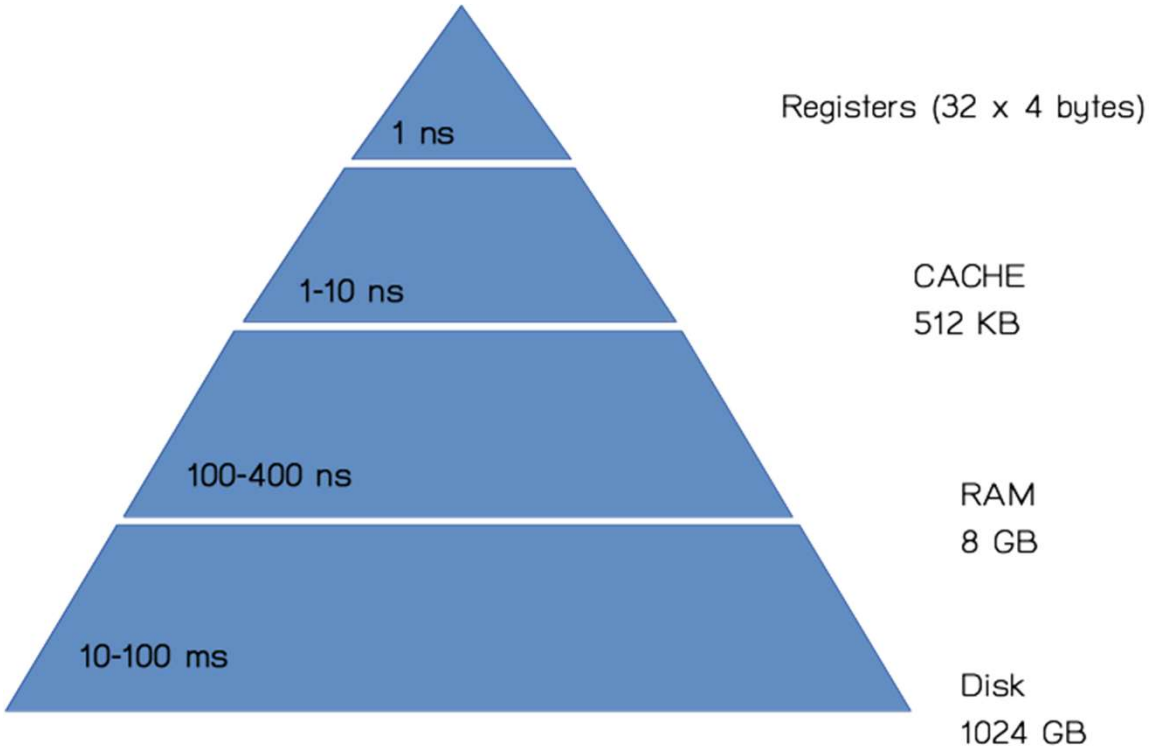
Design Goals

- ★ To have large, fast, (and cheap) memory for software
- ★ How do we create a memory that is large, cheap and fast (most of the time)?
  - Hierarchy
  - Parallelism
- ★ Keep in mind that large memory is slow. (Physical rules)



Memory Hierarchy

- ★ We have levels of memory.
- ★ Fewer fast memory and larger cheap memory.
- ★ Locality may make memory access faster.
- ★ Lower level is closer to the processor.



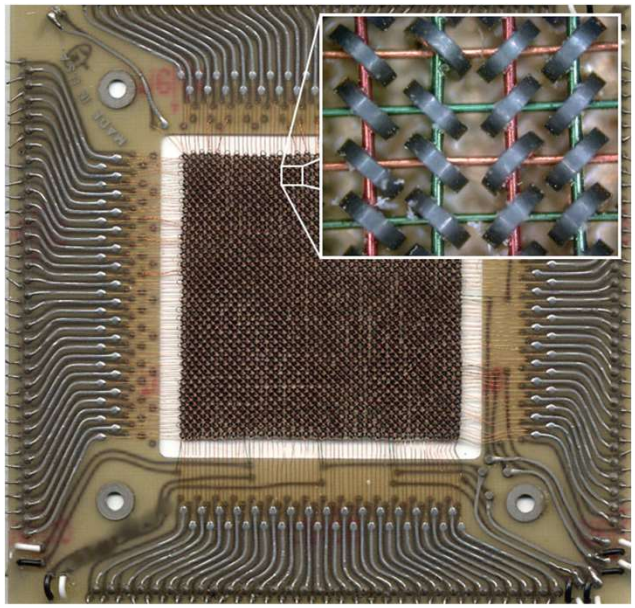


Management of Hierarchy

- ★ Registers ↔ Memory
  - By software (compiler, programmer?)
- ★ Cache ↔ Memory
  - By hardware
- ★ Memory ↔ Disks
  - By hardware with support from operating system (virtual memory/paging)
  - By the programmer (via files)



Memory Technology

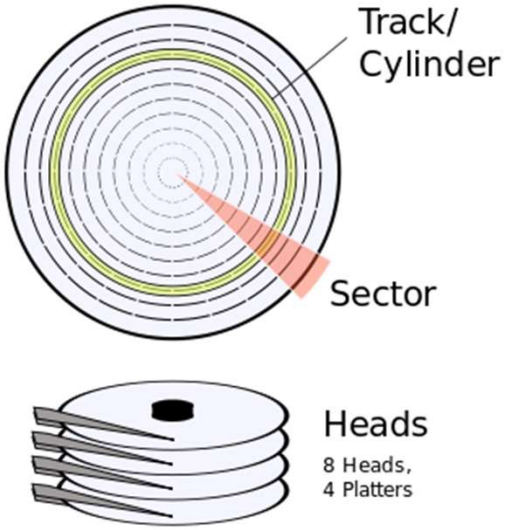


- ★ Random Access Memory
- ★ Random means constant time for all locations.
- ★ DRAM: Dynamic Random Access Memory
  - ○ Capacitor Technology
  - ○ High density, low power, cheap, slow
  - ○ Need to be refreshed regularly.
- ★ SRAM: Static Random Access Memory
  - ○ Transistor Technology
  - ○ Low density, high power, expensive, fast
  - ○ Content will last “forever” (while power on).



Memory Technology (ctd.)

- ★ Semi-random (Not-so-random) Access Technology
  - Seek, Rotate
  - Disk CDROM, DVDROM
- ★ Sequential Access Technology
  - Access time is linear to location
  - Tape



Picture from wikipedia



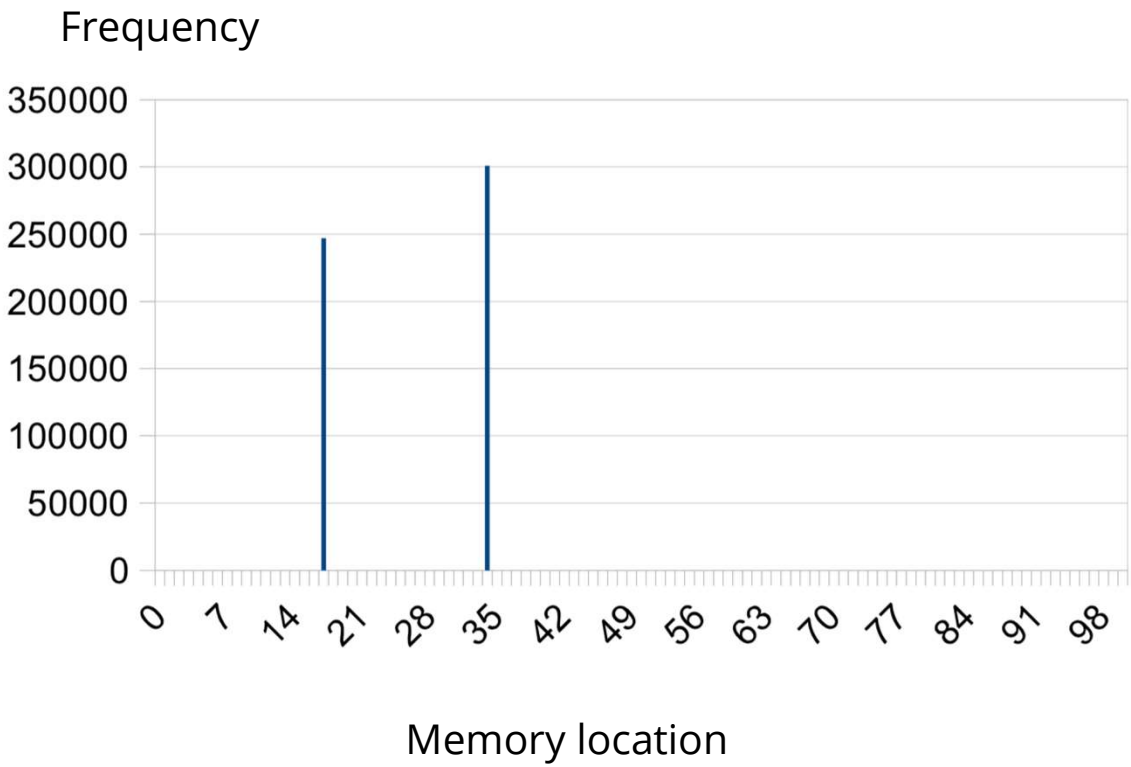
CACHE & Memory

- ★ DRAM is slow, but cheap and dense.
  - Good choice for memory
- ★ SRAM is fast, but expensive and not very dense.
  - Good choice for CACHE



Principle of Locality

- ★ At a particular time, software only access a small chunk of addresses.
- ★ If we can put this chunk in the lower level, we will feel that it is faster.
- ★ Two kinds of locality
  - Temporal
  - Spacial





Locality in real world

- ★ Temporal Locality- tempo is "time" in Italian.
- ★ Spatial Locality - Spatial means space.
- ★ If I just called you, chance is that I might call you again soon. Your number should be in a recent list (on the top). --- Temporal
- ★ If I just called you, chance is that I might want to call your friend as well. (Maybe you want to hit on a friend of your friend by using your friend as a bridge to her friend.)
  I should pick up your number and a number of your friend at the same time. --- Spatial
- ★ I always use pencil and rubber together. So I should carry them both in my pocket. -- Spatial



Cache: Terminology and Organization

- ★ Hit - when we find data in a Cache
  - Hit Rate (aka. Hit Ratio) - Probability that you will find data in a Cache
  - Hit Time - Time to access data in a Cache
- ★ Miss - when we cannot find data in a Cache
  - Miss Rate (aka. Miss Ratio) - Probability that you will not find data in a Cache
    Miss Rate = (1-Hit Rate)
  - Miss Penalty - Time for fetching data into a Cache
- ★ Block
  - A small chunk of data in a line of Cache.
  - We always take a whole block in/out of Cache.



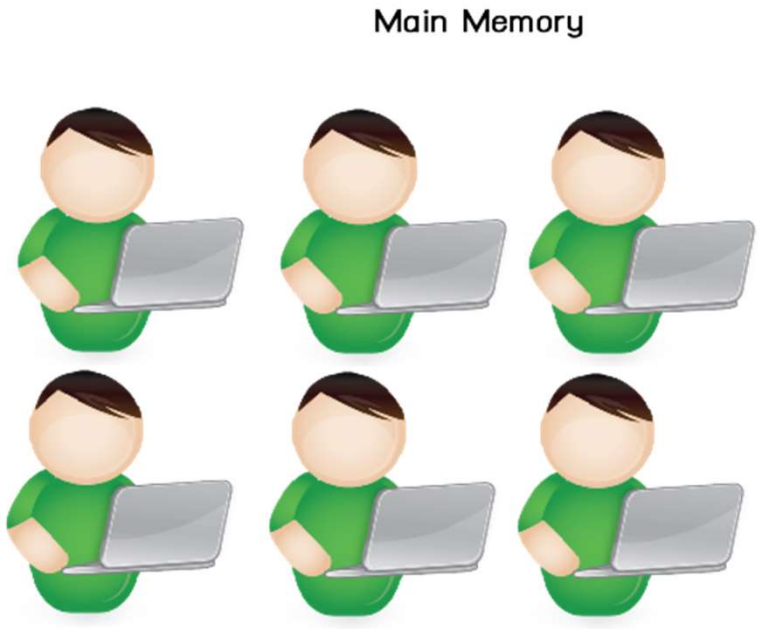
Cache Organization

- With memory hierarchy, there are 4 questions to address.
  - Q1. Where can a block be placed in the upper level?
  - Q2. How is a block found if it is in the upper level?
  - Q3. Which block should be replaced on a miss?
  - Q4. What happens on a write?
- Cache organization will address these issues.
- We will start with the most simple one.
  - Directed Mapped Cache



Directed Mapped Cache in real life

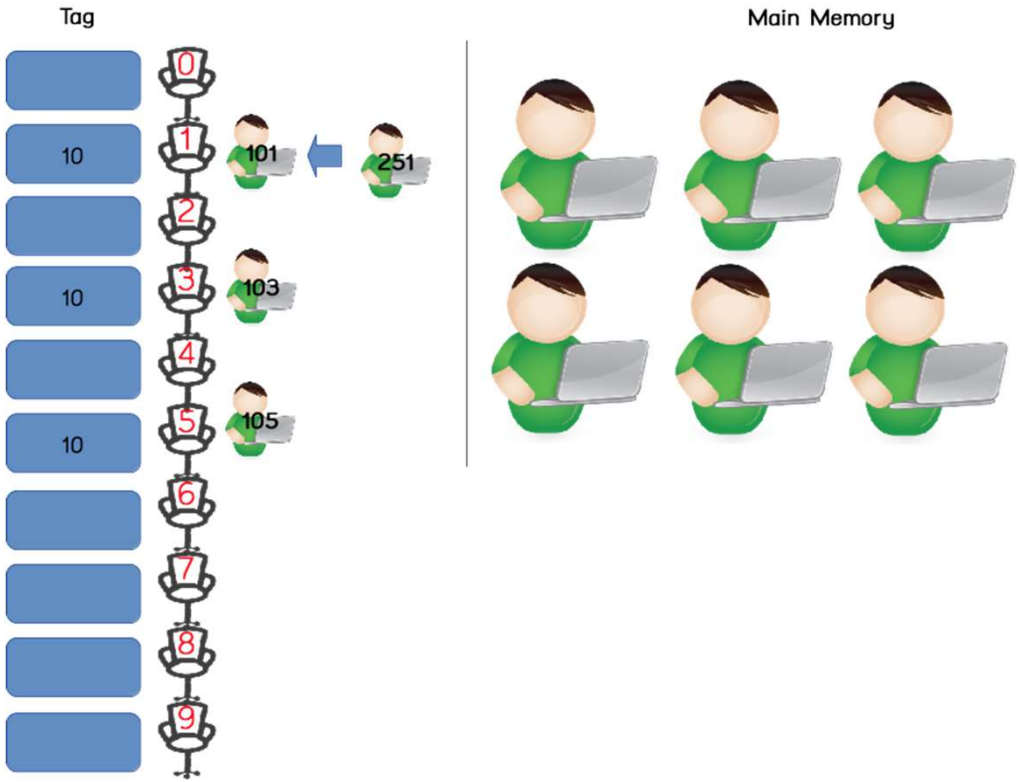
- ★ Suppose that a class of 100 students are in the auditorium (main memory). Each student got a seat number (e.g. id)
- ★ To work (e.g. exam), a student must first get into the office of a professor.
- ★ To avoid having students walked between the auditorium and the office, ten chairs are installed in the office.





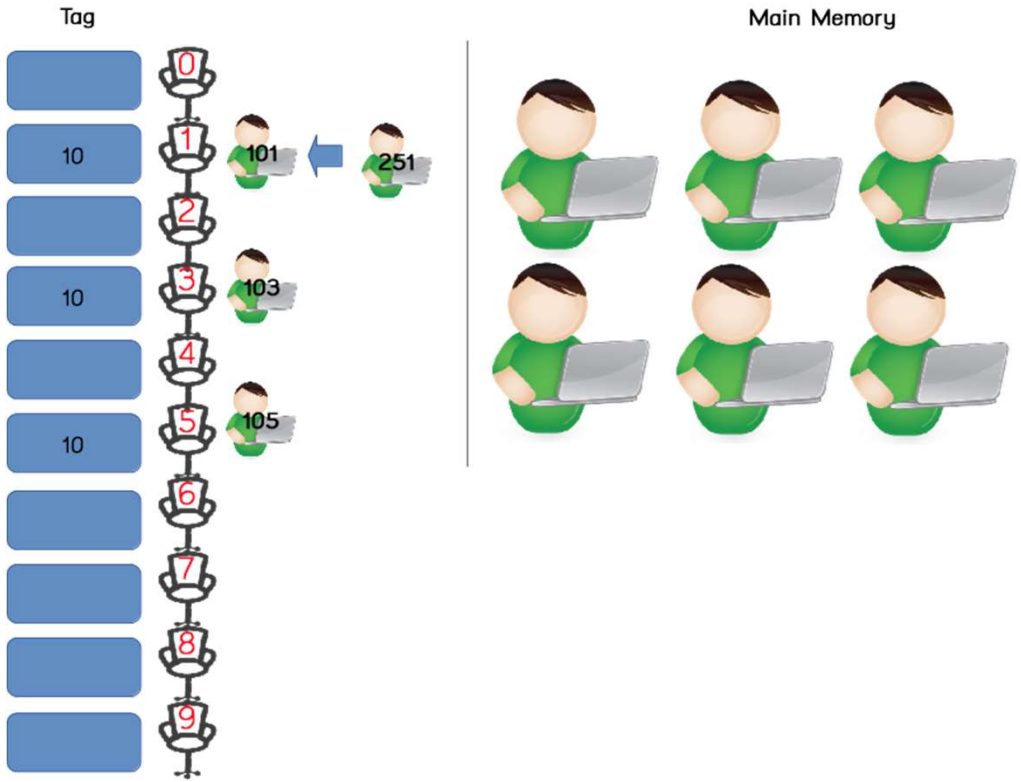
Directed Mapped Cache in real life (ctd)

- ★ A directed mapped cache means there is a place in locality for each person.
- ★ Given that each person has an address (id), we can use a digit to **index** a chair.
- ★ To figure out who is sitting at a particular chair, we put a **tag** at each seat.





Tag and Index



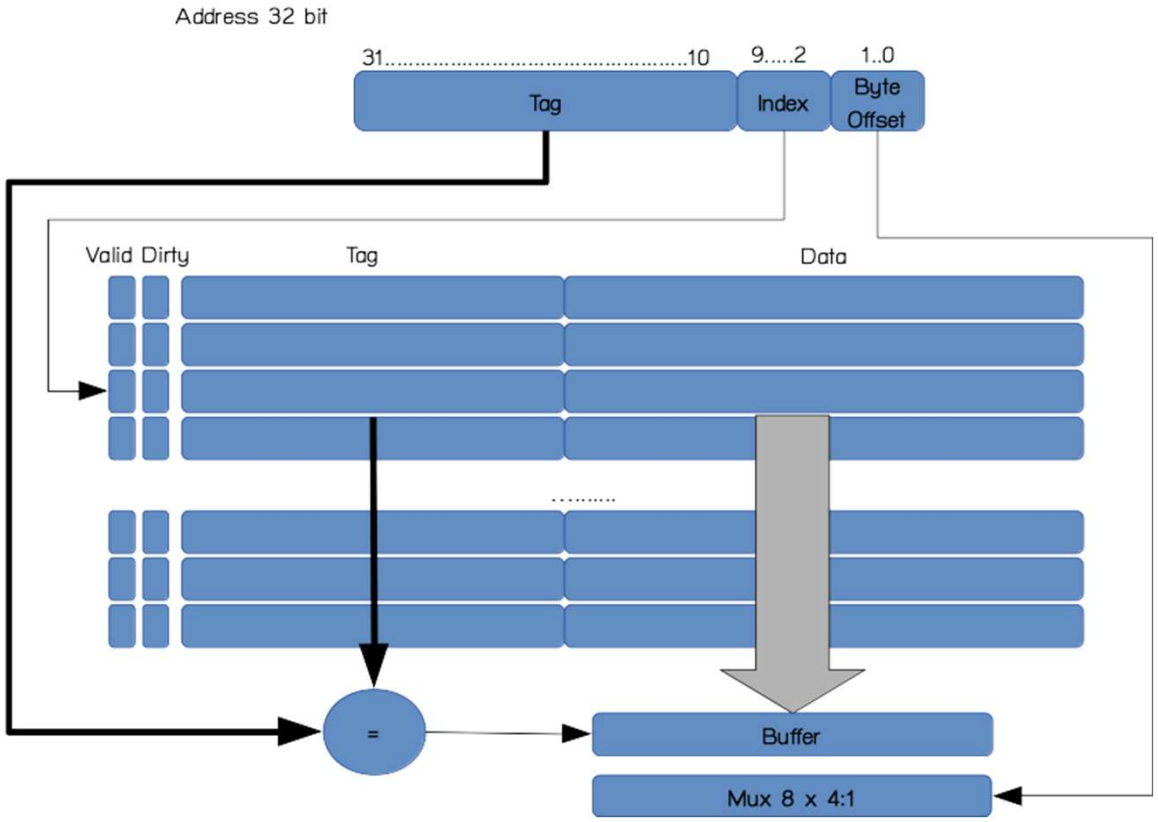
- ★ Index = a place to sit
  - = (address mod cache size) div block size.
- ★ Tag = an id of a person
  - = address div cache size.
- ★ With 10x1 chairs (10 rows, each with 1 chair) means cache size=10 and block size=1.
- ★ A person with id 105, will have to sit at seat index 5 (105 mod 10) and put up a tag 10 (105 div cache size).



Directed Mapped Cache in Memory

- ★ Find the following values
  - Block size
  - Cache size
- ★ What is
  - Valid bit?
  - Dirty bit?
- ★ Please locate 0x00004098 in this cache.

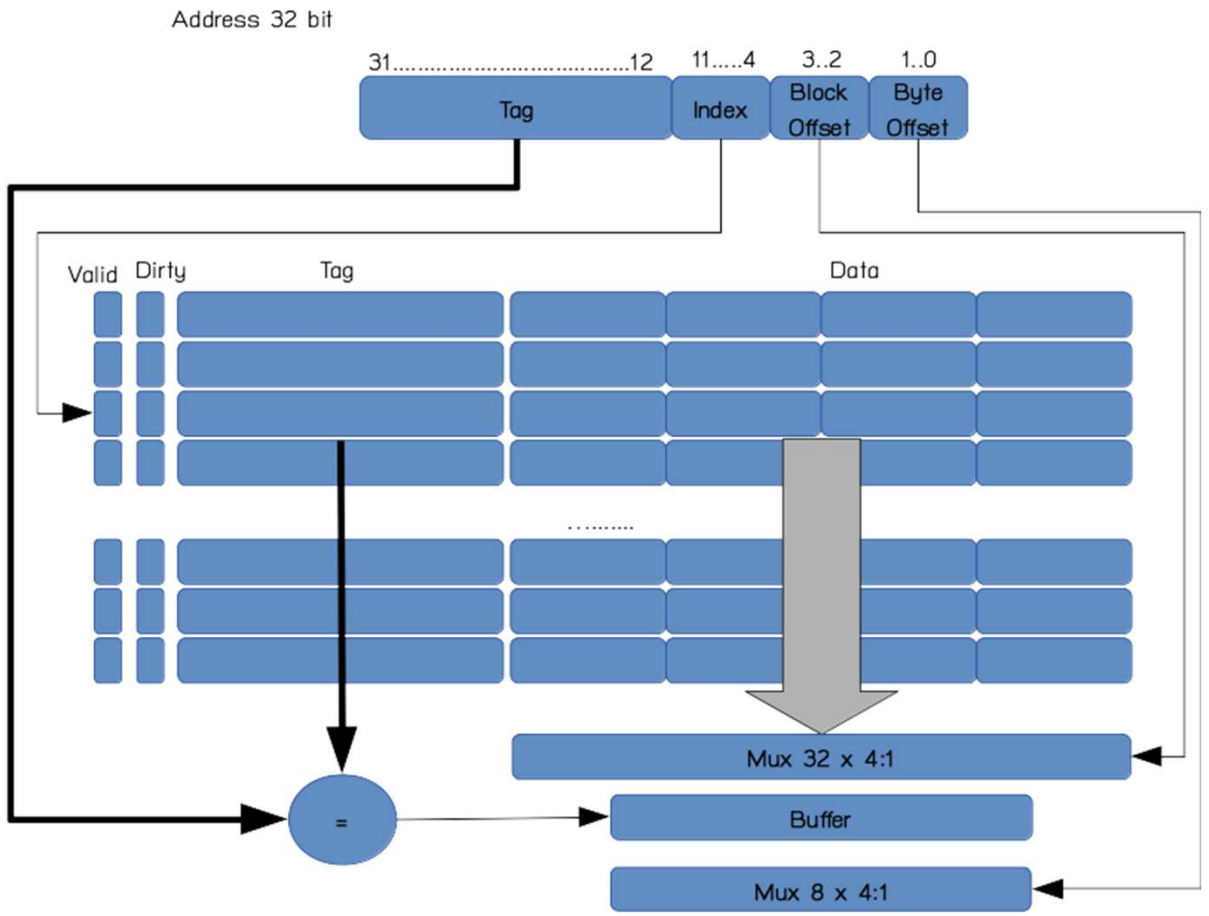
This is good for temporal.
Can we add more spatial?





Directed Mapped Cache with Spatial

- ★ A larger block size means more spatial locality.
- ★ Find the following values
  - Block size
  - Cache size





Performance (revisit)

- ★ Memory Access Time = Hit Time + (Miss Rate x Miss Penalty)
- ★ Hit Time and Miss Penalty usually report in cycle.
- ★ CPI = ideal CPI + (Miss Rate x Miss Penalty)
- ★ CPU Time = ??
- ★ Note. We previously assume that memory access time is 1 cycle.



Optimizing Cache

- ★ To get the best performance, we have to minimize memory access time.
- ★ Since Memory Access Time = Hit Time + (Miss Rate x Miss Penalty), How can we minimize:
  - Hit Time
  - Miss Rate
  - Miss Penalty
- ★ To minimize Hit Time, use small cache. Why?
- ★ How about Miss Rate and Miss Penalty?



Sources of Miss

- ★ To minimize Miss Rate, we have to first understand sources of miss.
- ★ 3 Sources of miss
- ★ Compulsory Miss (aka. Cold Miss)
  - Cold start. No data in cache. We can do nothing.
  - Maybe a prefetch can help. (Beyond the scope of this class.)
- ★ Capacity Miss
  - The cache size (locality) is too small for the workload.
  - See ping-pong effect in the next slide.
  - Naive solution is to increase the cache size.
- ★ Conflict Miss
  - There is a free space in a Cache. However, the mapping mechanism prevents us from using them.
  - Solutions: (1) Increase cache size, (2) multiple entries for the same index.



Ping-Pong effect (extreme capacity miss)





- ★ If there is only one line of cache, an access to a new address always result in a miss.
- ★ Assuming that we can only put a page of paper on a desk, every page changes will result in a miss.
- ★ The CPU is always busy swapping memory in and out.



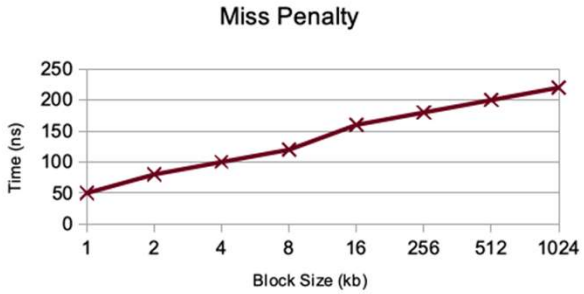
Cache Configurations

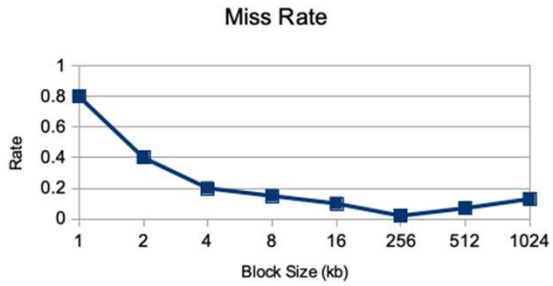
- 5 Configurations
  - Cache Size
  - Block Size
  - Associativity
  - Replacement Algorithm
  - Write Management (Write Buffer)
- Let's optimize Miss Rate

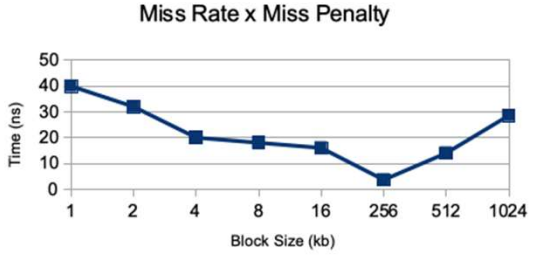


Block Size

- ★ Intuitively, larger block size means more spatial locality.
- ★ However, larger block also means higher miss penalty
- ★ Too large a block also means less temporal locality (when cache size is fixed.)
- ★ Finding an optimal block size requires a lots of benchmarking.
- ★ See assignment II.



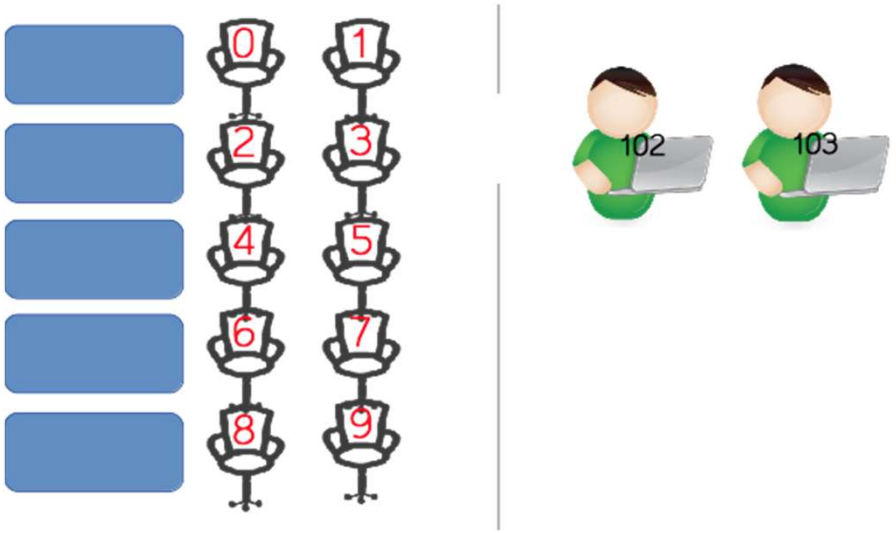






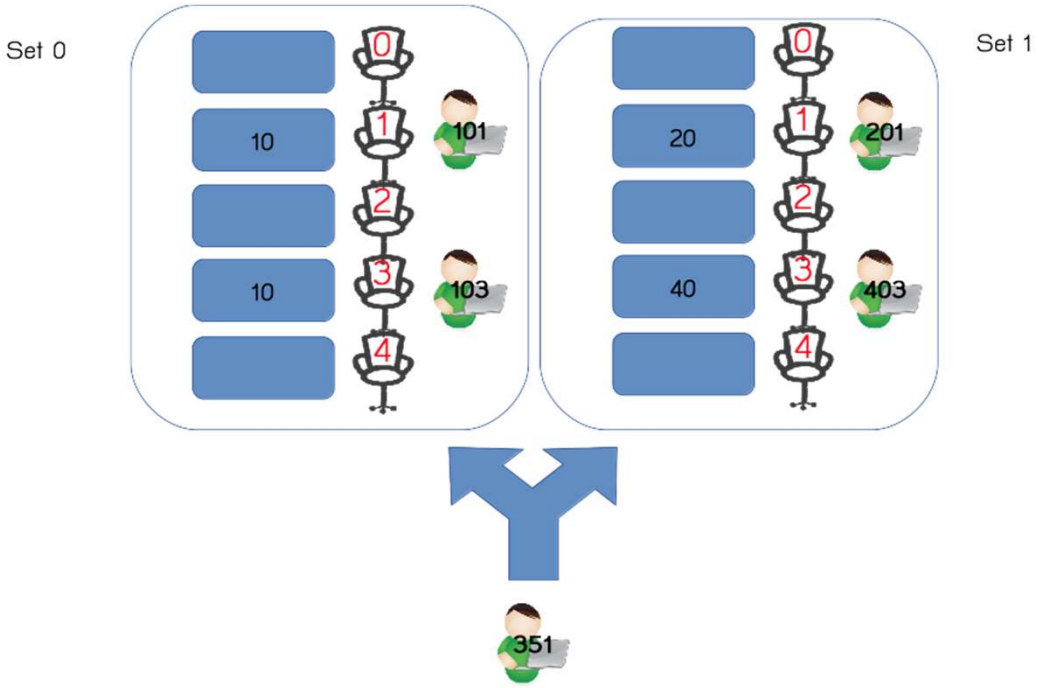
Block Size (ctd.)

★ Unless increasing a bus size, larger block means higher miss penalty. (Bringing several person into a room with small door.)





Two-way set Associativity

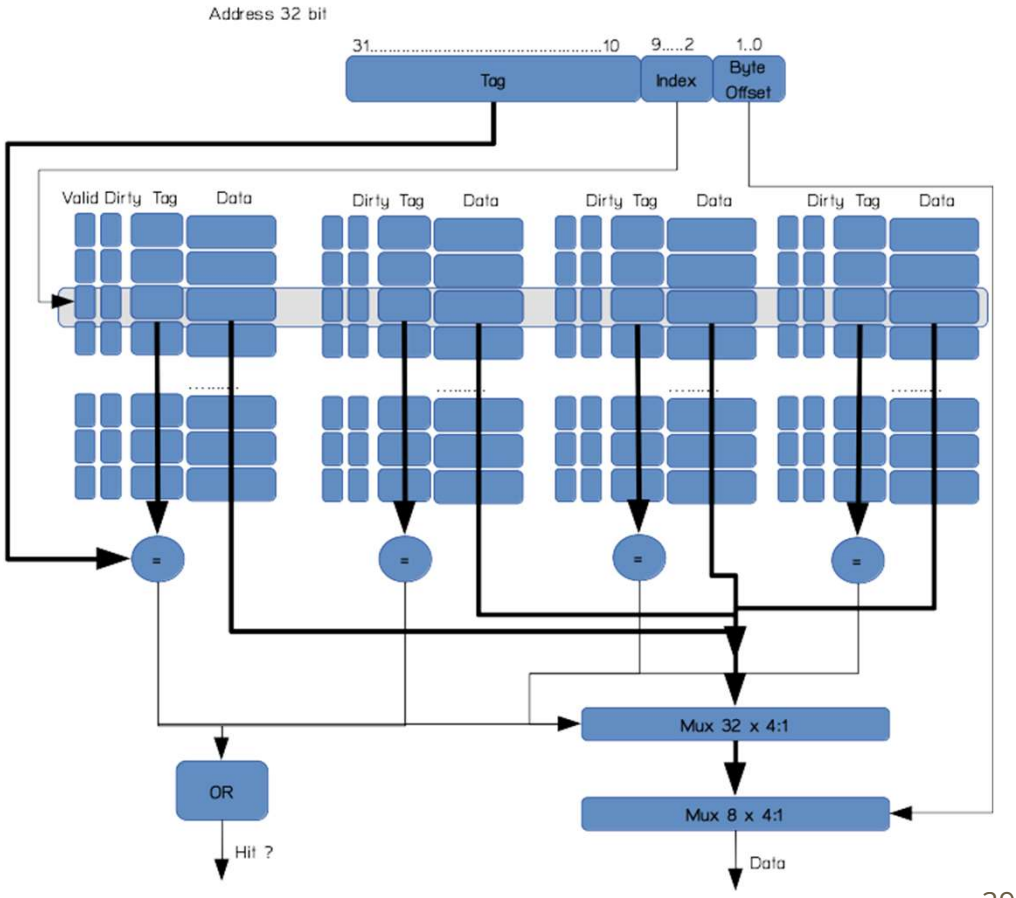


- ★ Two sets of chairs in an office.
- ★ Two entries for the same index.
- ★ Less conflict miss.
- ★ Can also increase to four ways, and eight ways.



4-way set Associativity Cache

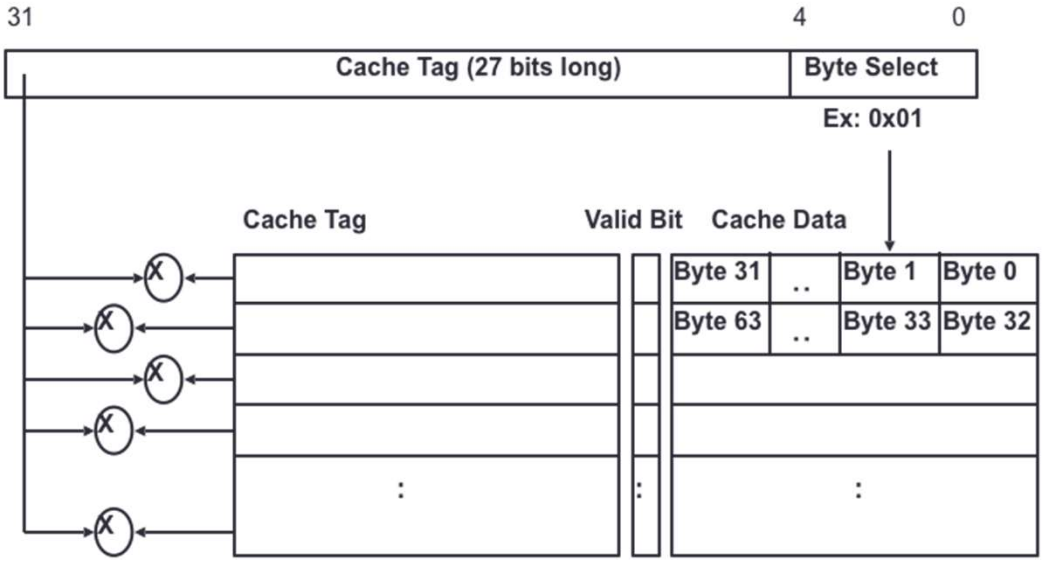
- ★ More complex circuit for accessing data
- ★ 4 comparators
- ★ Large multiplexer
- ★ Higher Hit Time? (Slower?)





Fully Associative (zero conflict miss)

- ★ Another extreme
- ★ No index. Data can be placed anywhere.
- ★ How many comparators do we need?
- ★ Conflict Miss = 0





Replacement Algorithm

- ★ Least Recently Used (LRU)
  - ○ Discards the least recently used items first
- ★ Round Robin
  - ○ Taking turn for replacement.
- ★ How about First In First Out (FIFO), and Random Replacement?
- ★ Practically, code and data use split caches. There is more spatial locality in code.

Rep.	bench	Data	
		Miss	Miss rate
RR	gcc	36805	0.018
LRU	gcc	33126	0.017
RR	go	7255	0.005
LRU	go	9073	0.006



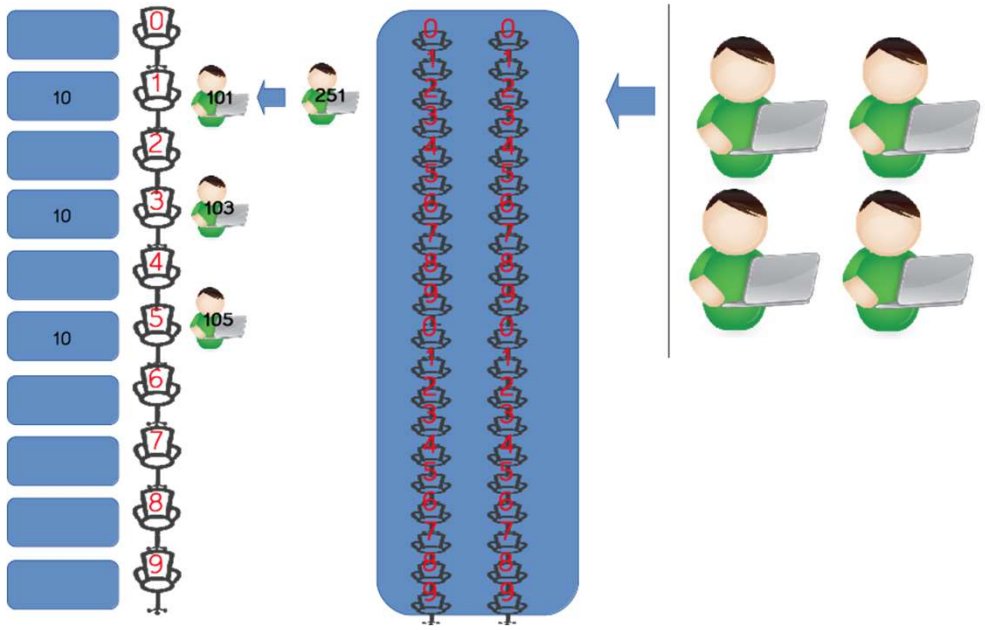
Pop Quiz

With directed map, what kind of replacement algorithm should we use?
Please provide your analysis.



Reducing Miss Penalty

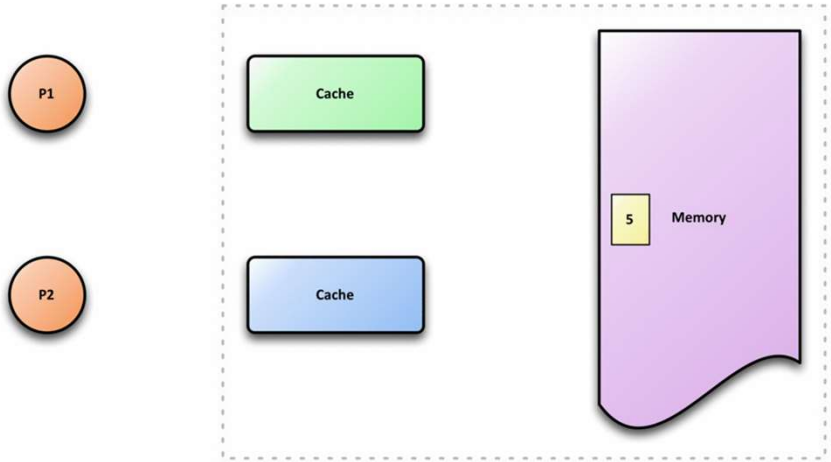
- ★ Between the professor's office and auditorium, we can install chairs in a hall.
- ★ A miss from an office may hit in the hall.
- ★ Less miss penalty comparing to going to the auditorium.





Write Management

- ★ Two choices
  - Write Back
  - Write Through
- ★ Write Back
  - Write to Cache only. Will update main memory on replacement.
  - Good for repeated write
  - Fast, but with incoherent data in multiprocessor system.
- ★ Write Through
  - Always write to cache and main memory
  - Bad for repeated write
  - Slow, but preferred for multiprocessor system





Write Management (ctd.)

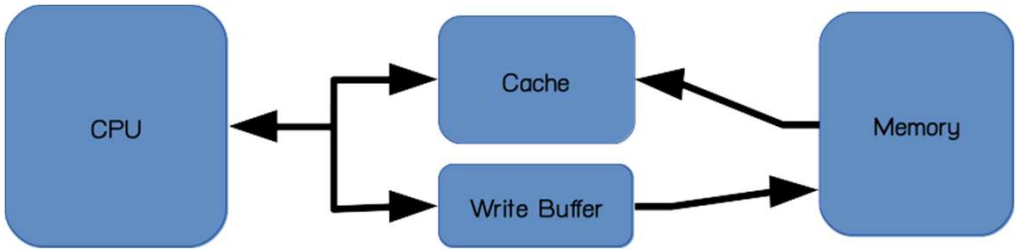
- ★ Write allocate vs. Write not allocate
- ★ Write not allocate means write directly without first loading.
- ★ On write miss, write allocate means loading data before write (equivalent to miss penalty + write time).
  - ○ How about writing to a part of the block?
  - ○ Use subblock concept? (beyond the scope of this class)

- ★ For simplicity, we will use write through and write allocate.



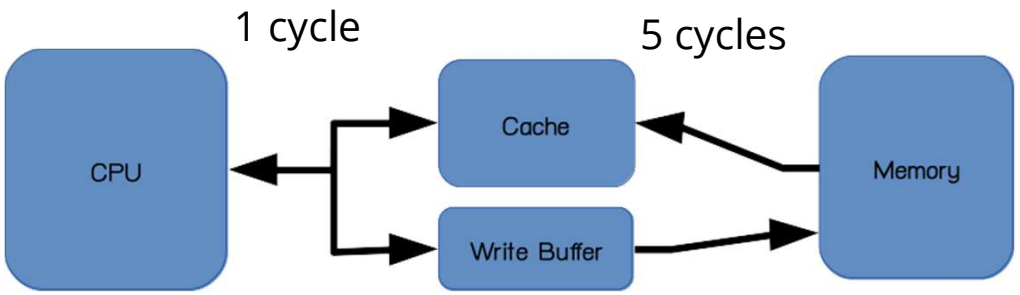
Write Buffer (for Write Through)

- ★ With write through, a write access is bad for performance. (equivalent to miss penalty?)
- ★ To make write faster, use write buffer.
- ★ Write Buffer
- ★ Since write through is like running to auditorium on each write. A write buffer is having someone to run to the auditorium to tell the main person for you.





Write Buffer Saturation

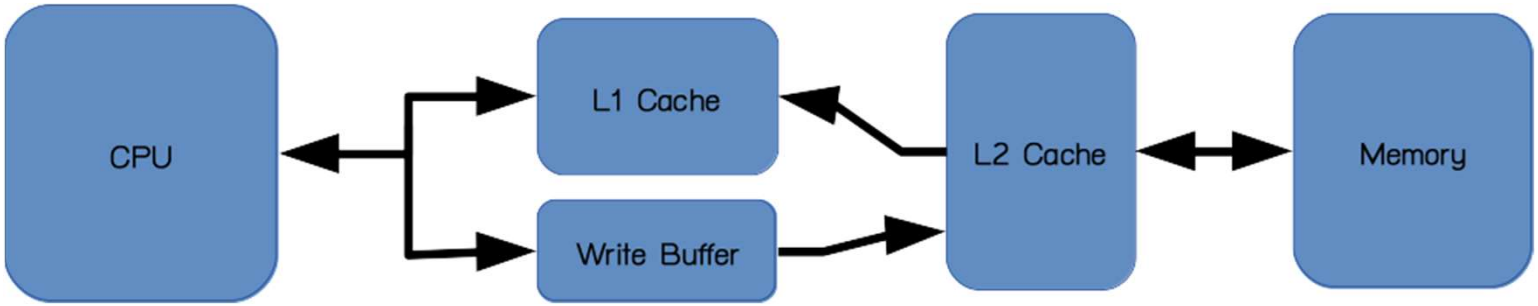


- ★ Write Buffer is basically a small FIFO (eg. 4 words)
- ★ If memory is much more slower than main memory, write buffer will soon be saturated (overflowed) regardless the size of write buffer. (Nobody to run to the auditorium for the professor.)
- ★ How to solve this problem?



Write buffer saturation (ctd.)

- ★ A naive solution is to introduce a second level cache (L2).
- ★ Analogy: Rather than running to the auditorium, a write means running to the hall. (less saturation)





Software and Cache

- ★ Cache Management is transparent to software.
  (We gain benefits from cache without any software modification.)
- ★ As a programmer, can we programmatically take advantage of Cache?
- ★ Can a program exploit more temporal and more spatial?
  - Continuously use variables in the scope (more temporal).
- ★ Can a program avoid conflict miss?



More Spatial

- ★ A software can programmatically take advantage of spatial locality.
- ★ These two snippets are logically equivalent.
- ★ However, one is faster Why?

```
// row-major order
int data [1000][1000];
int sum;
for (int r=0; r<1000; r++){
  for (int c=0; c<1000; c++){
      sum += data[r][c];
  }
}
```

real	0m0.010s
user	0m0.004s
sys	0m0.004s

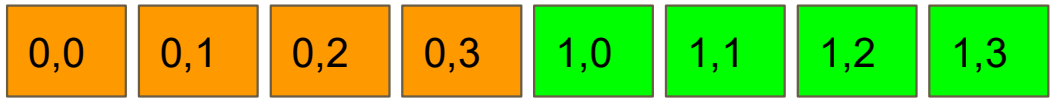
```
// column-major order
int data [1000][1000];
int sum;
for (int c=0; c<100; c++){
  for (int r=0; r<100; r++){
      sum += data[r][c];
  }
}
```

real	0m0.013s
user	0m0.007s
sys	0m0.004s

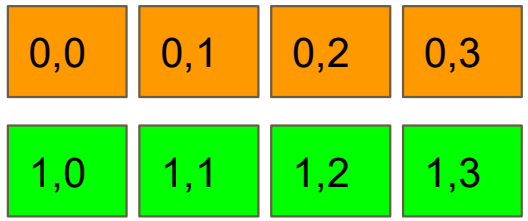


Row-Major Order vs. Column-Major Order

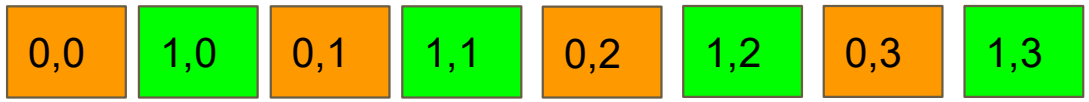
Row Major




```
int data [2][4];
```



Column Major





Reduce Conflict Miss

- ★ Likely, accessing different arrays with same index may result in a conflict miss. (same cache index)
- ★ To reduce conflict miss, an array of structure may help.

```
// three arrays
int a[100];
int b[100];
int c[100];
for (int i=0; i<100; i++) {
     c[i] = a[i] * b[i];
}
```

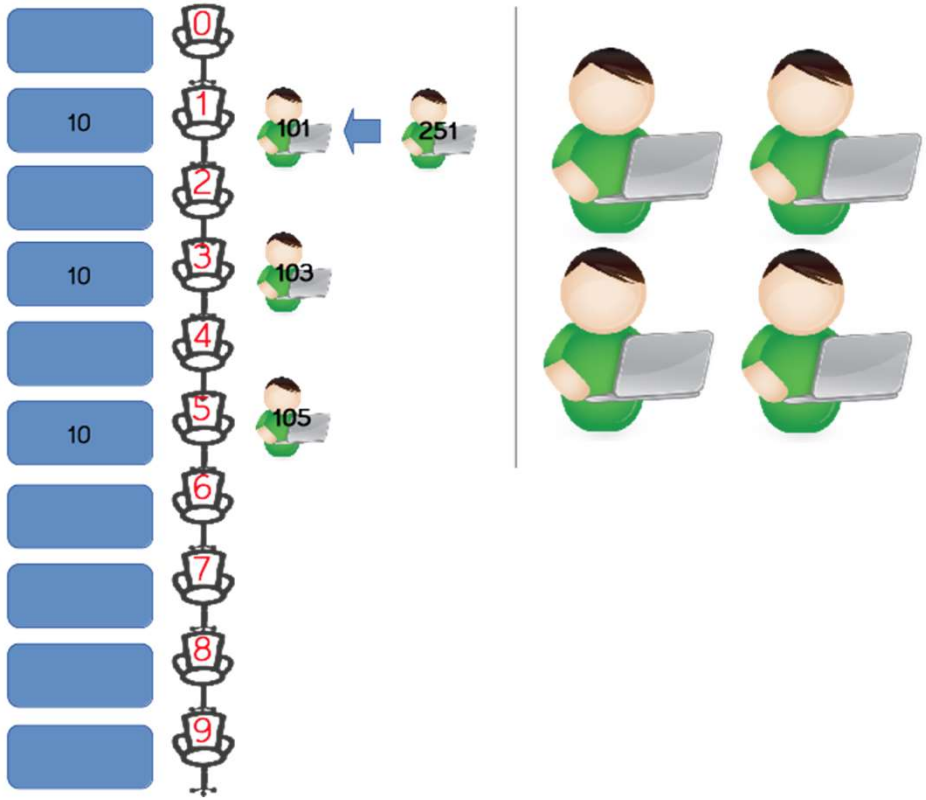
```
// array of struct
struct Data {
     int a;
     int b;
     int c;
}
data[100];
for (int i=0;i<100;i++) {
     data[i].c = data[i].a * data[i].b;
}
```



Reduce Conflict Miss (ctd.)

- Same index of an array is likely to share the same index of Cache.

```
int a[100];
int b[100];
int c[100];
for(i=0;i<100;i++) {
    c[i]=a[i]*b[i];
}
```





Summary

- ★ Principle of locality
  - Program likely accesses a relatively small chunk of memory at any instant of time.
  - Temporal locality - Locality in Time
  - Spatial locality - Locality in Space
- ★ Reducing Miss
  - Cold Miss, Capacity Miss, Conflict Miss
- ★ Cache organization
  - Cache size
  - Block size
  - Associativity
  - Replacement Algorithm
  - Write Management



Exercises





Assignment II

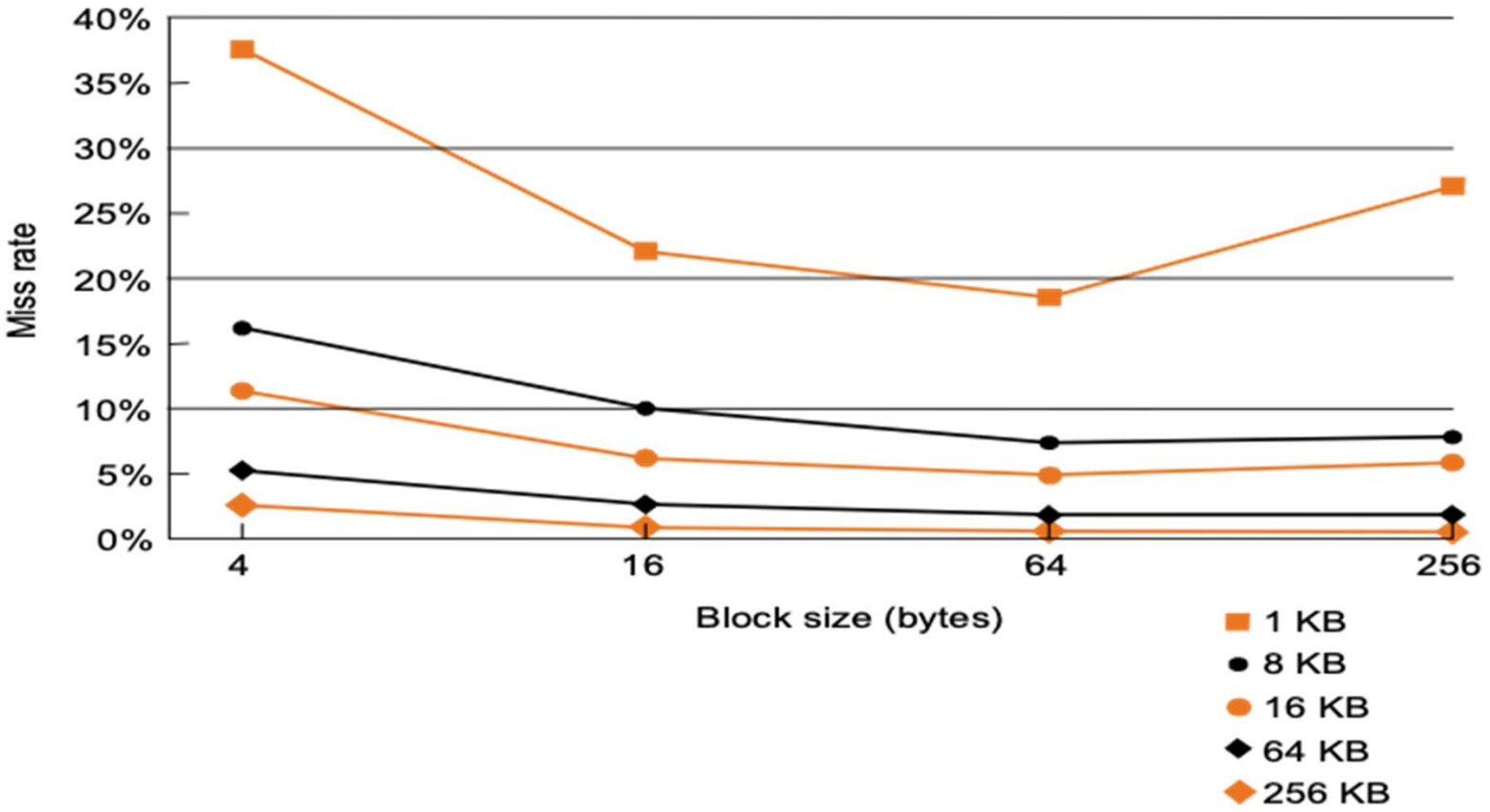
★ Use the cache simulator for studying the factors that affects the performance of cache accesses. The cache simulator will read an address trace file from gcc_ld_trace.txt or go_ld_trace.txt (You can use genAddr.c for generating more address traces). The simulator will show us a number of cache hit, a number of cache miss, miss rate, and access time. Please use only one address trace and modify the code of simulator for calculating the miss rate. Please fill your results and plot graph of each table.

Direct mapped				
Block Size (Bytes)	Cache Size (KB)			
	4	8	16	32
4				
8				
16				
32				

Associativity				
Cache Size (KB)	Two-way		Four-way	
	LRU	RR	LRU	RR
1				
4				
8				
32				
512				
1024				

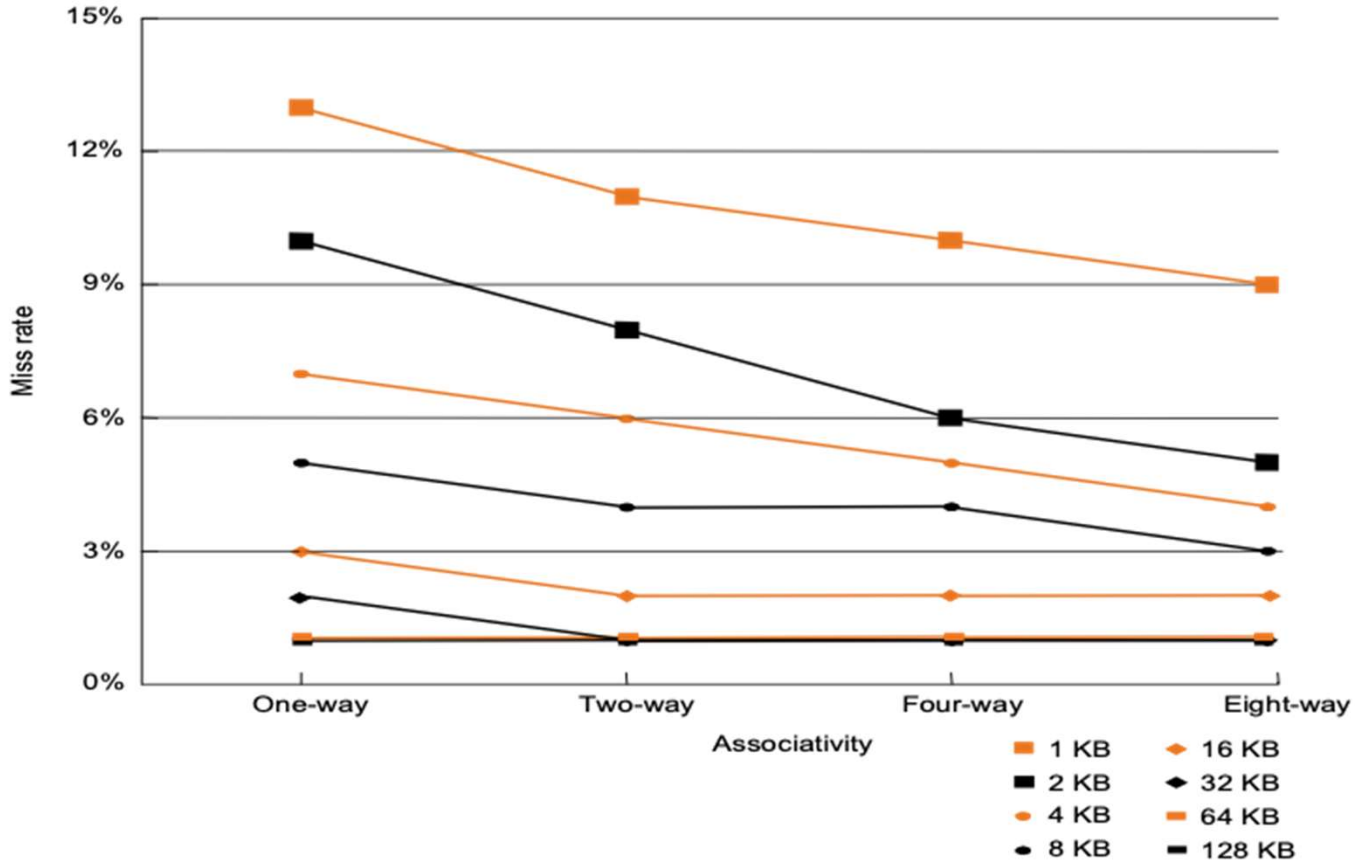


Expected Result (Block size)





Expected Result (Associativity)





Multilevel Cache

★ What is effective access time of multilevel Cache?

★ Access Time$_0$ = Hit Time$_0$ + (Miss Rate$_0$ x Miss Penalty$_0$)
★ Miss Penalty$_0$ = Access Time$_1$?



End of Chapter 8 (part 1)





Memory Organization (part 2)

Chapter 8



Memory Organization (part 2)

- ★ Virtual Memory
- ★ Paging Organization
- ★ Address Translation
- ★ Page Tables
  - ○ Translation Look-Aside Buffers
- ★ Virtual Memory and Cache
  - ○ Overlapped Cache & TLB Access



Virtual Memory

- ★ Cost (Run software that requires 4GB on 2GB hardware.)
  - ○ Illusion of more memory (borrowing from secondary storage)
  - ○ Demand loading and swap
- ★ Program Relocatable
  - ○ Different process has its own address space.
  - ○ Address 0x100 of Prog. A is not the same as address 0x100 of Prog. B.
- ★ Protection (Security)
  - ○ Prog. A may not access memory space of Prog. B directly.



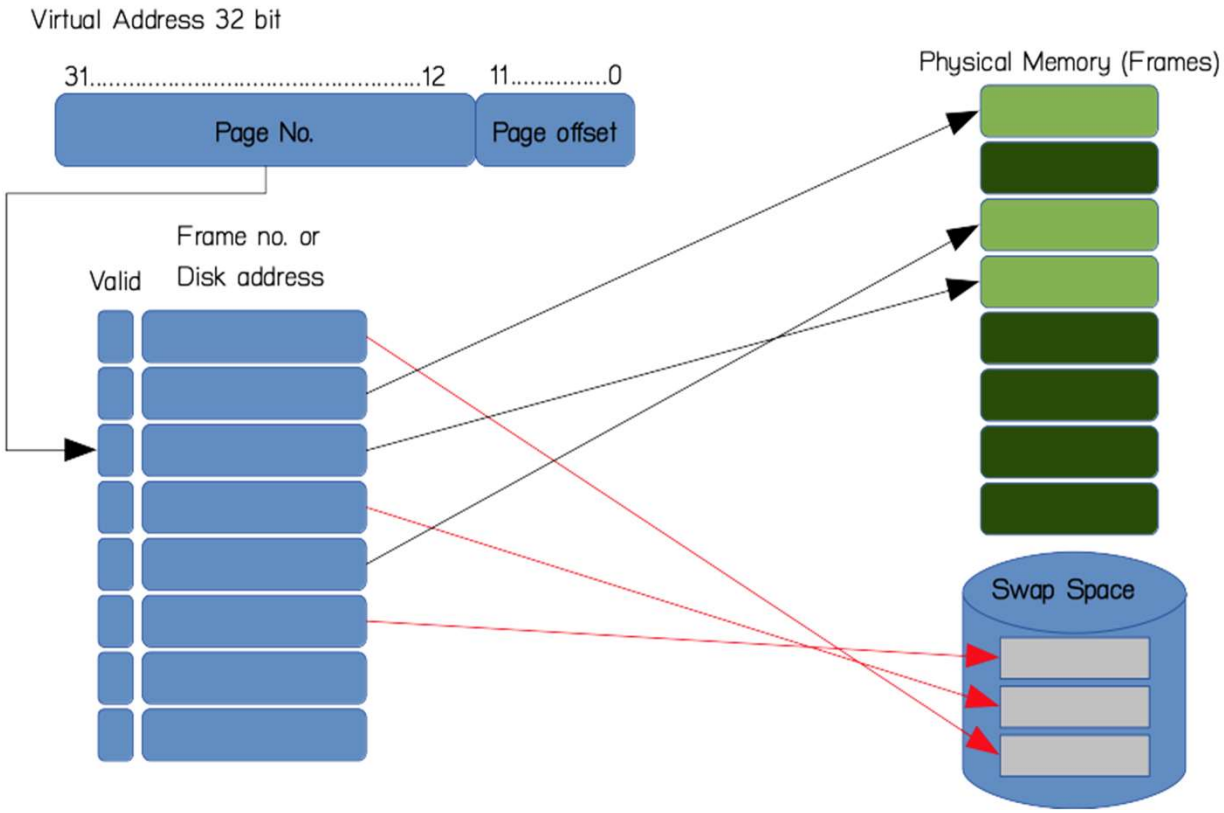
Paging Organization

- ★ A **virtual address** is the address that visible to software.
- ★ A virtual address is translated into a **physical address** by looking up from **page table**.
- ★ When physical address space is smaller than the amount of virtual address space, a **swap space** is borrowed from secondary storage (disk).
- ★ A virtual block is called a **page**. (Similar to block in cache)
- ★ A physical block is called a **page frame**.
- ★ A miss is called **page faults**.



Paging Organization (ctd.)

- ★ Physical Memory can act as a cache for secondary storage (file/disk).
- ★ Missing item is loaded on fault only (demand load policy)
- ★ Replacement policy is handled by operating system.





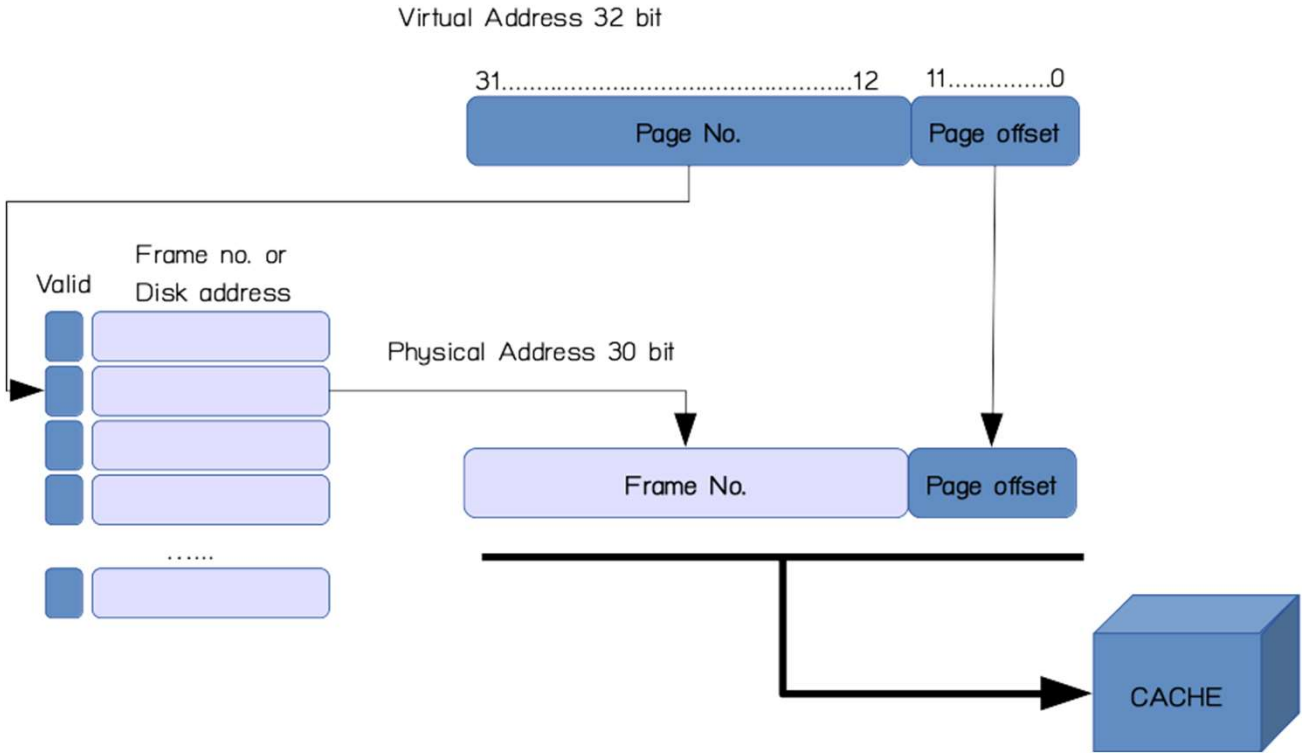
Pages: Virtual Memory Blocks

- ★ Virtual and physical address space partitioned into blocks of equal size
- ★ Virtual -- pages, Physical -- page frames
- ★ Page faults -- data is not in memory, retrieve from disk
  - Hugh miss penalty. A page is usually large (eg. 4KB).
  - A write through is too expensive. Only uses **write back** (page out).
  - A fault is handled by software.
    (Hardware does not know file system or swap management.)



Address Translation

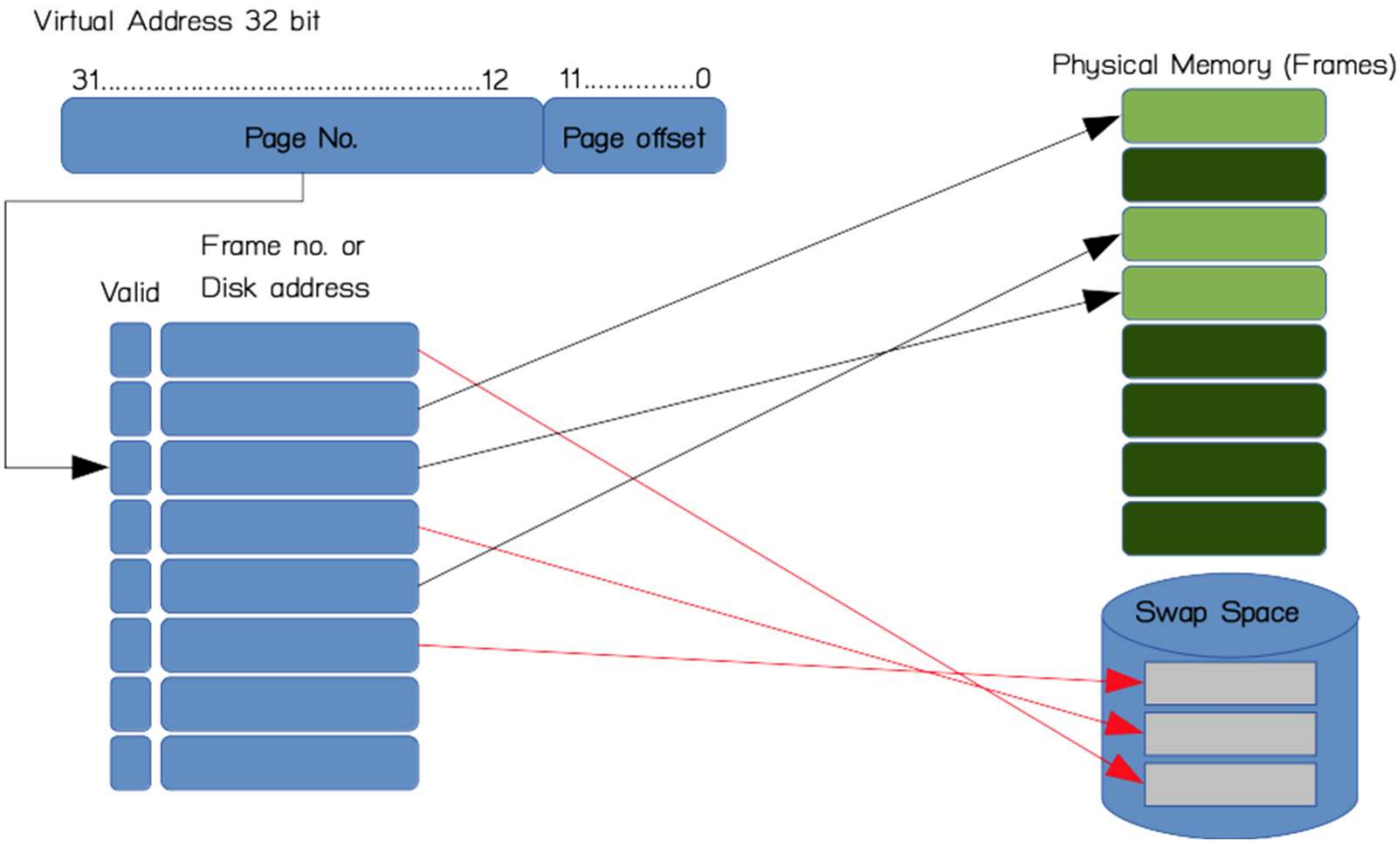
- ★ Virtual address space can be larger than Physical address space. (Run 4GB software on 2GB hardware)





Page Tables

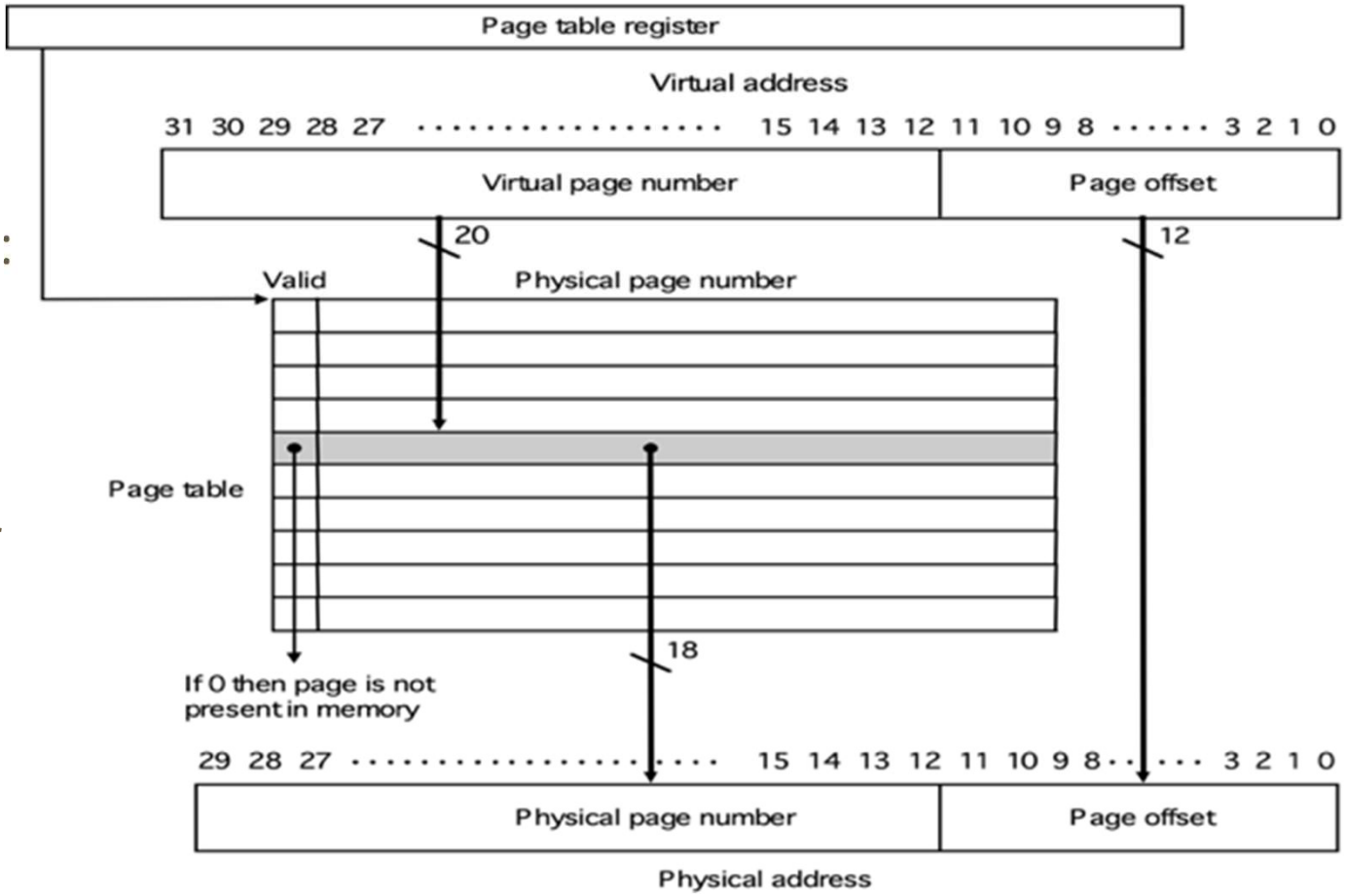
- In reality, page table also include access permission (eg. readonly)





Page Table (ctd.)

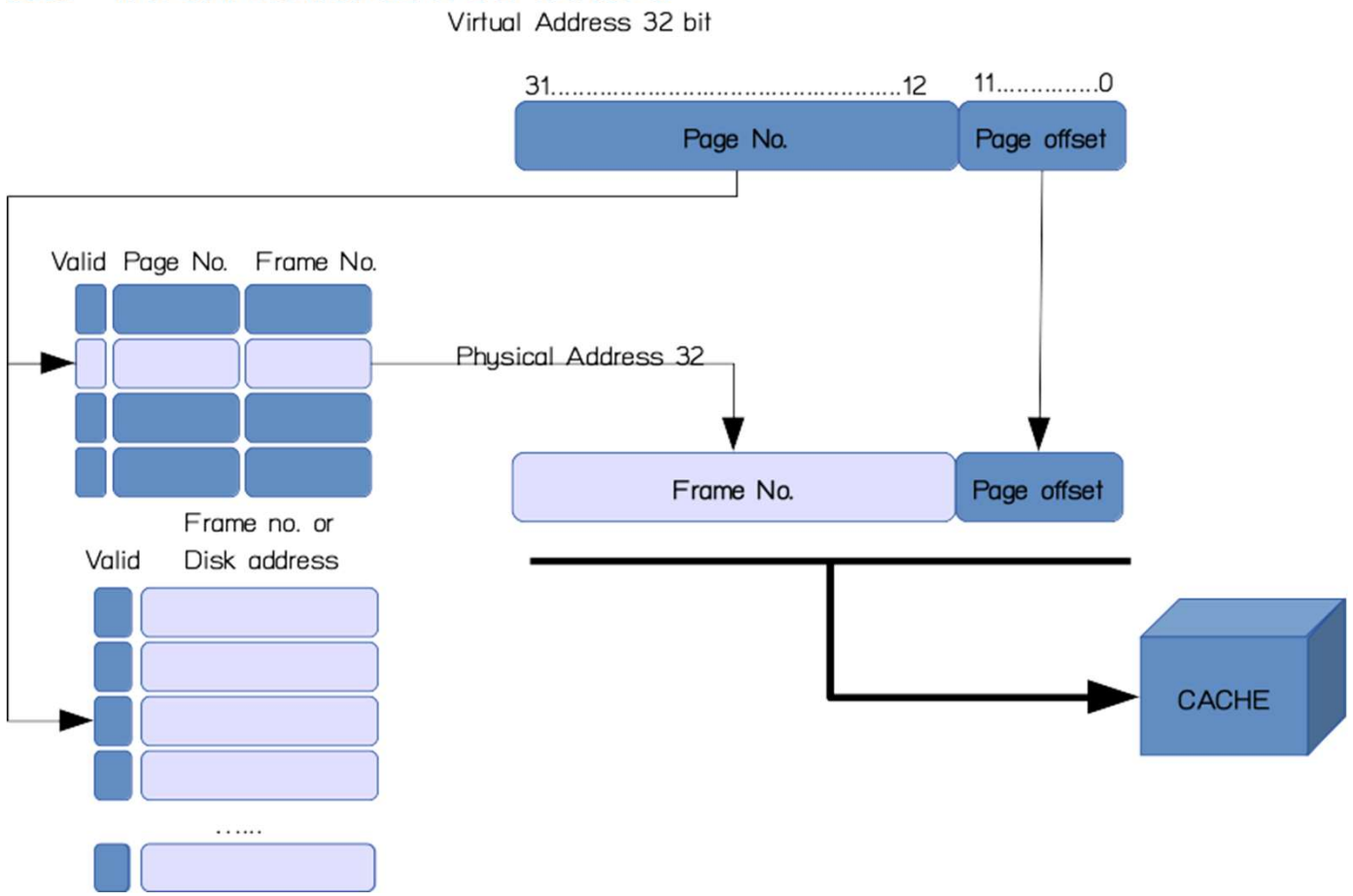
- From this organization, what is the size of :
  - Page
  - Physical memory
  - A page table (4 bytes x 2^{20})
- A page table is per process





Make Address Translation fast

- A cache for address translations: translation lookaside buffer





TLB (ctd.)

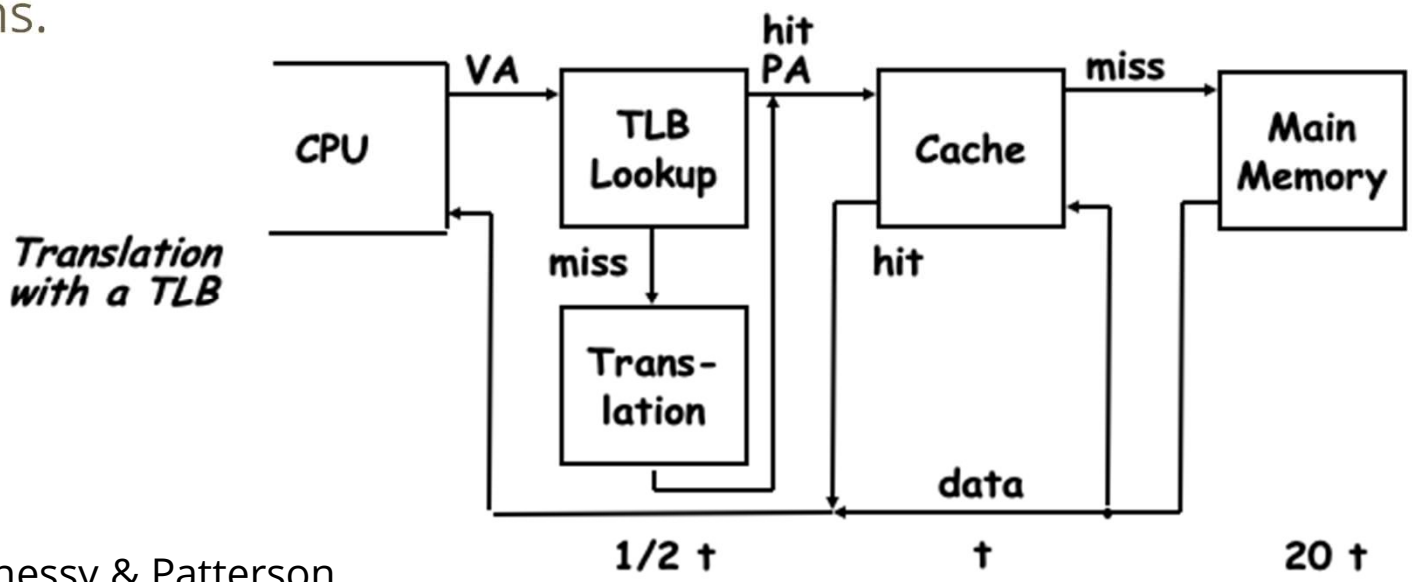
- ★ A way to speed up translation is to use a special cache of recently used page table entries -- this has many names, but the most frequently used is Translation Lookaside Buffer or TLB
- ★ TLB access time comparable to cache access time (much less than main memory access time)

Virtual Address	Physical Address	Dirty	Ref	Valid	Access



Translation Look-Aside Buffers

★ TLBs are usually small, typically not more than 128 - 256 entries even on high-end machines. This permits **fully associative** lookup on these machines. Most mid-range machines use **small n-way set associative** organizations.

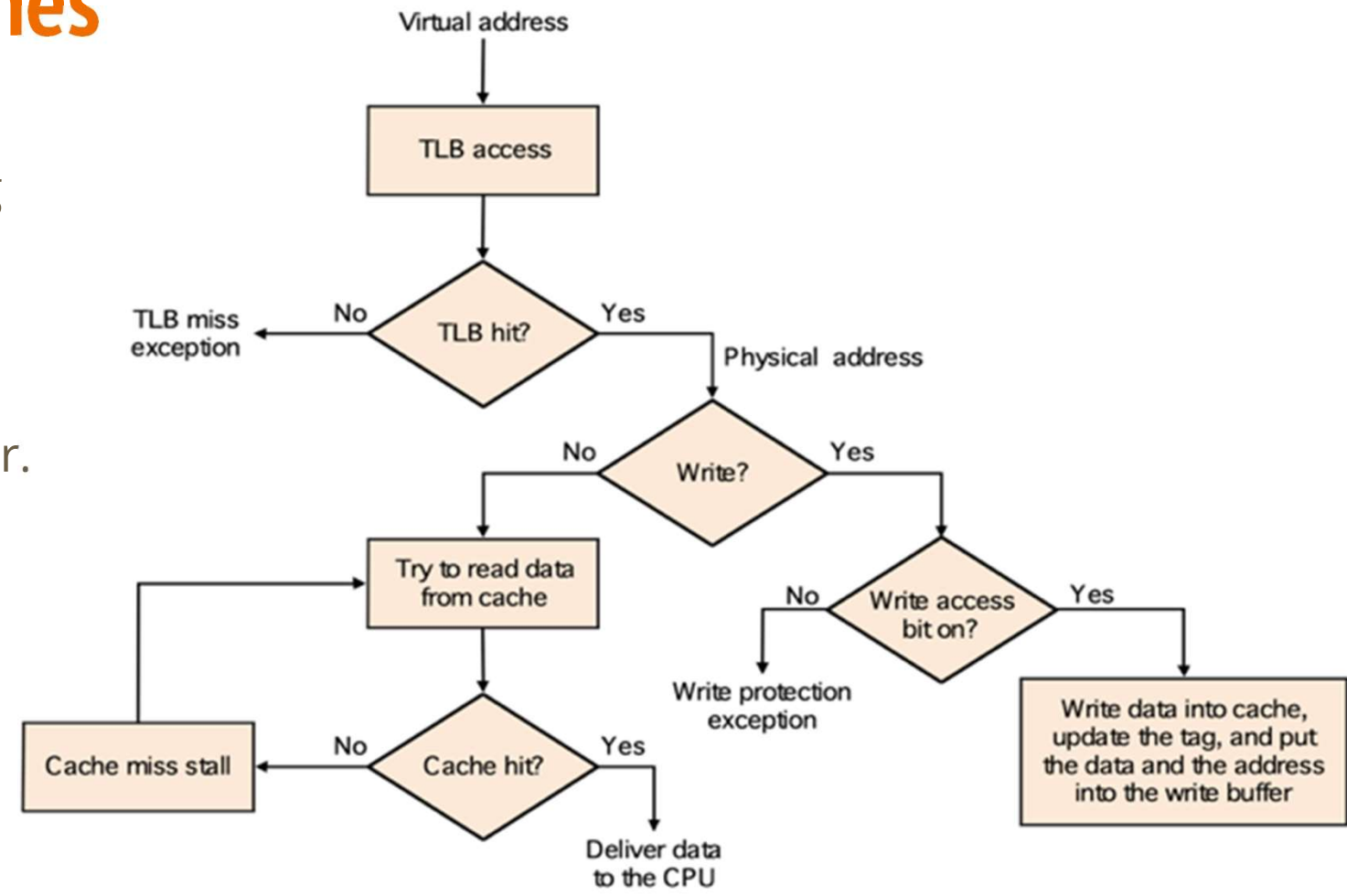


Picture from Hennessy & Patterson



TLB & Caches

- ★ Steps in accessing data (taken from Intel x86)
- ★ This is slow.
- ★ Let's make it faster.



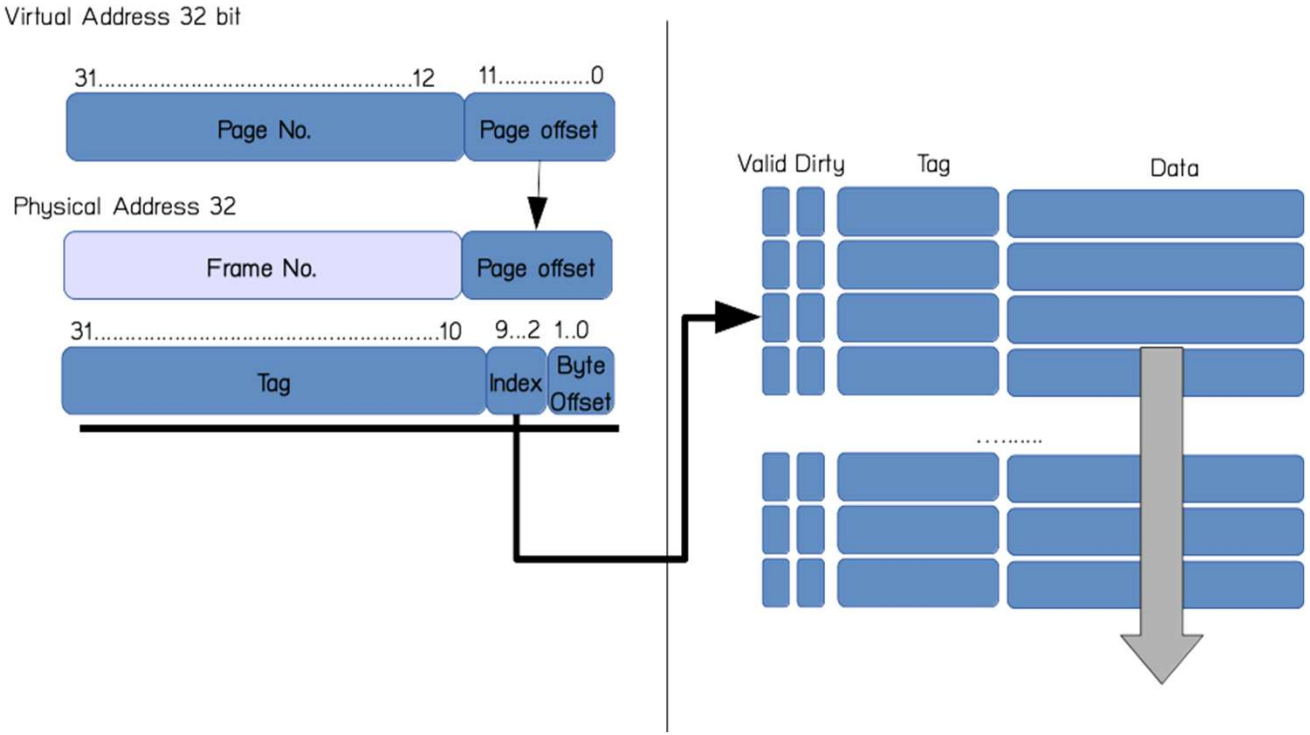


Reduce Translation Time

- ★ Machines with TLBs go one step further to reduce # cycles/cache access
- ★ They overlap the cache access with the TLB access
- ★ Works because high order bits of the VA are used to look in the TLB while low order bits are used as index into cache



Overlapped Cache & TLB Access



- ★ If we can parallel (overlap) the cache look up (index) and translation from virtual address to physical address at the same time.
- ★ Possible when **index** is **not changed** in address translation



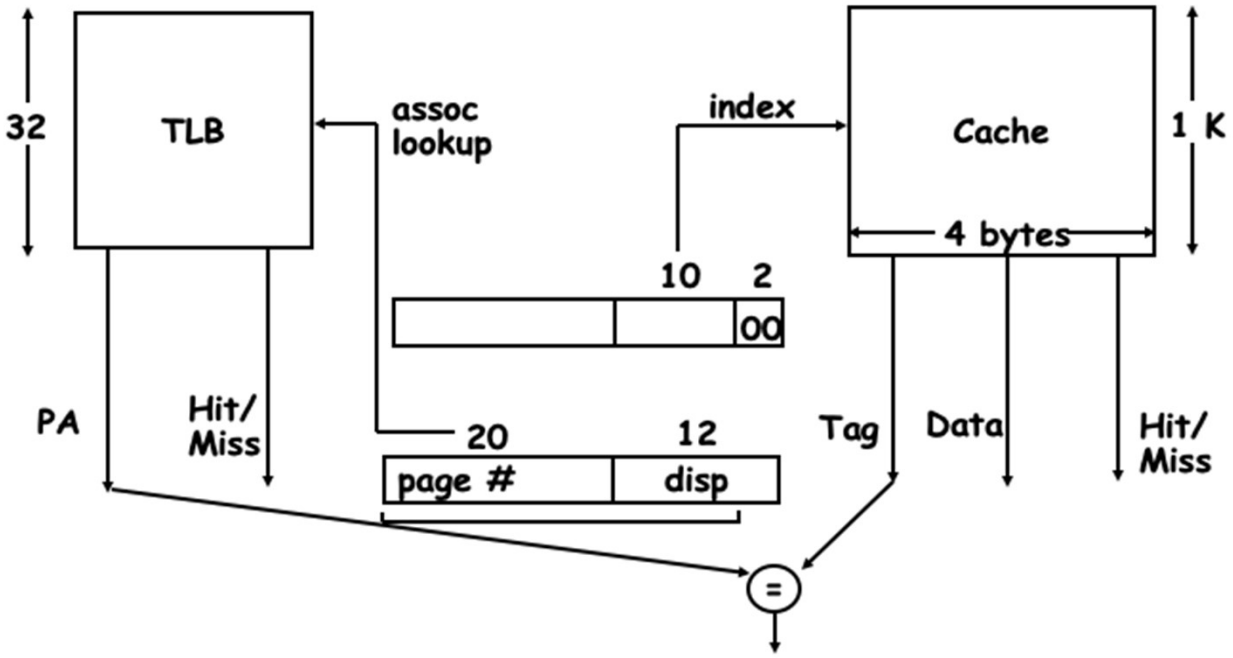
Overlapped Cache & TLB Access (ctd.)

IF cache hit AND (cache tag = PA) then deliver data to CPU

ELSE IF [cache miss OR (cache tag != PA)] and TLB hit

THEN access memory with the PA from the TLB

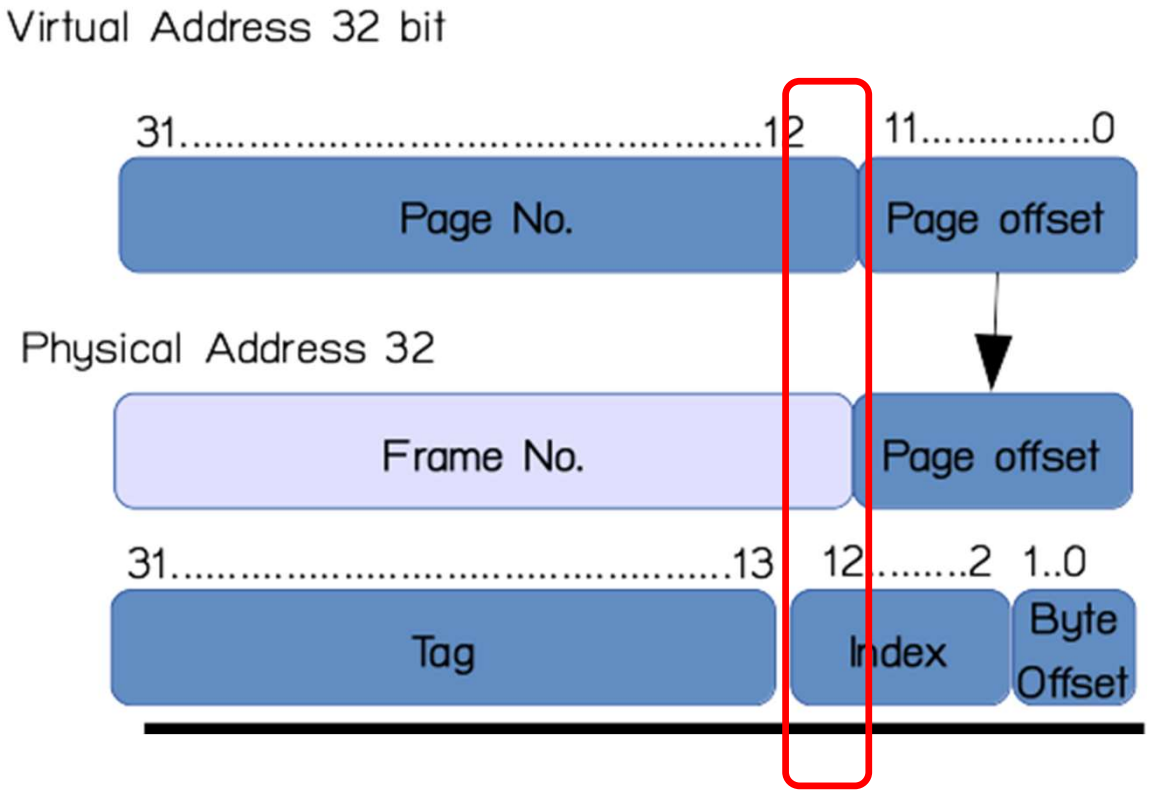
ELSE do standard VA translation





When overlapped access is not possible

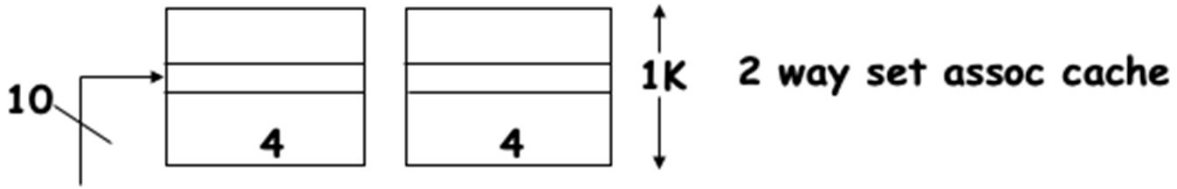
- ★ Larger cache (e.g. 8KB) in 4KB page.
- ★ 13 bits cache vs. 12 bits page offset





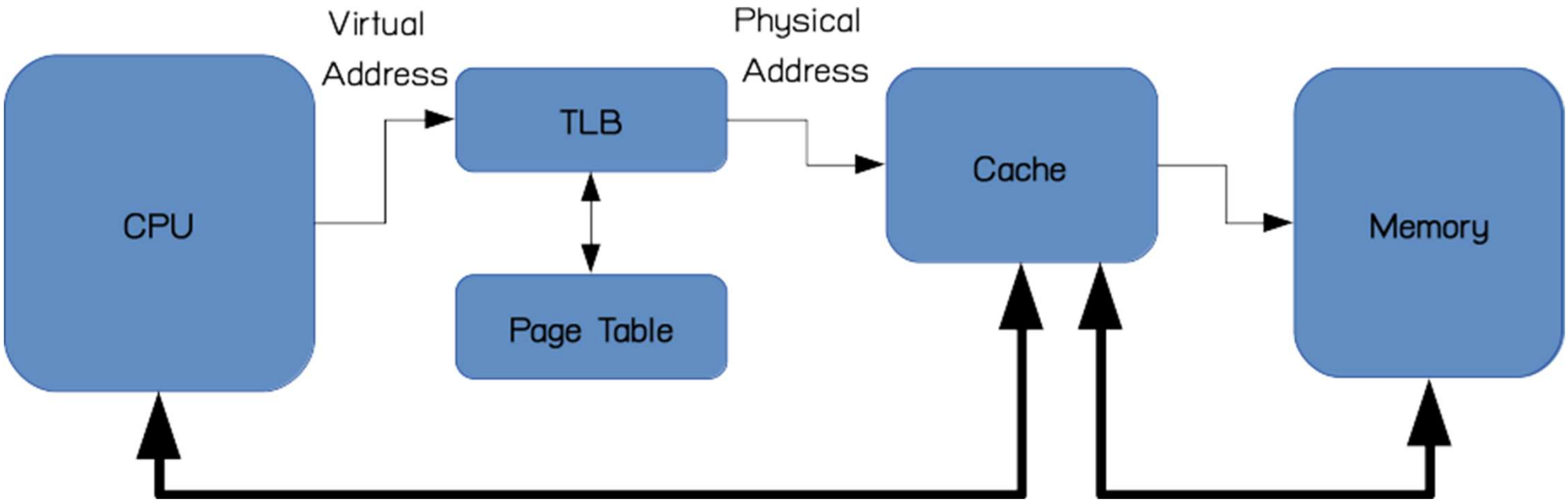
When overlapped access is not possible

- Solutions:
  - go to 8K byte page sizes;
  - go to 2 way set associative cache; or
  - SW guarantee VA[13]=PA[13]





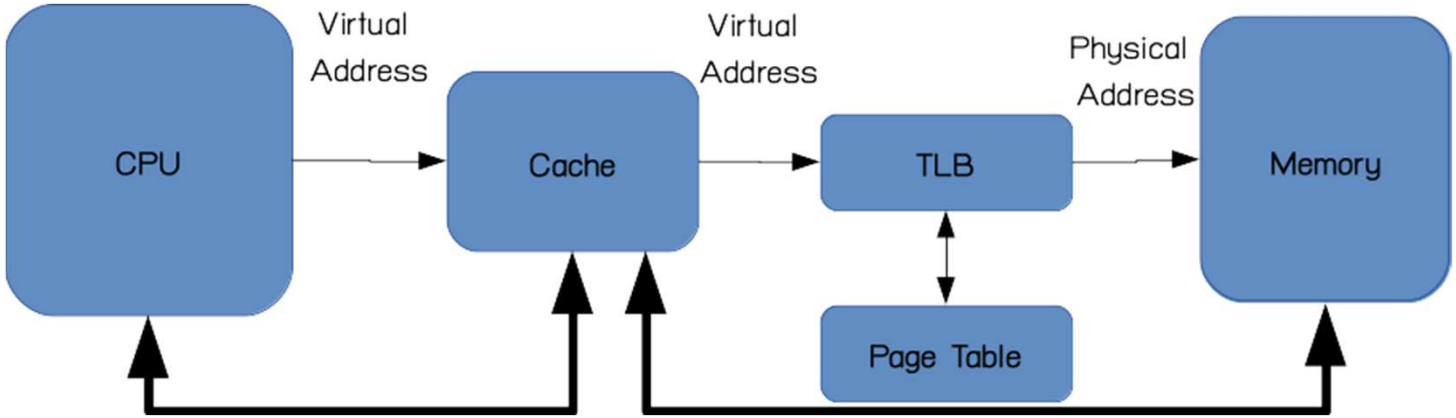
Virtual Memory and Cache





Can we access cache with virtual address?

- ★ Will there be any benefit (or issue) if we move cache before address translation?
- ★ Issues with shared memory (where different virtual addresses from different processes are mapped to same physical address.)





Beyond the scope of this class

- ★ There exists several technologies for performing memory relocation and memory management (such as overlay and segmentation).
- ★ There are also several methods for managing page table. (Inverted Page Table, Multilevel page table)
  With virtualization, several processors now also support virtualized page table.



Summary

- ★ Caches, TLBs, Virtual Memory all understood by examining how they deal with 4 questions:
  - 1) Where can block be placed?
  - 2) How is block found?
  - 3) What block is replaced on miss?
  - 4) How are writes handled?
- ★ Page tables map virtual address to physical address
- ★ TLBs are important for fast translation
- ★ TLB misses are significant in processor performance

https://gabrieletolomei.wordpress.com/miscellanea/operating-systems/virtual-memory-paging-and-swapping/



Exercises





Memory Design

★ With 4KB page, how should an architect design 32KB cache to the advantage of overlapped access? Please provide your analysis.



Process switch

★What should happen to the following elements when the OS performs a process switch

- ★TLB
- ★Page table
- ★Cache



End of Chapter 8 (part 2)







Storage Architecture

Chapter 9



Storage Architecture

- ★ Storage Technology
  - Protocol vs. Storage
- ★ Storage Virtualization
- ★ RAID
- ★ Network Storage
  - Network Attached Storage (NAS)
    - NFS
    - CIFS (SMB)
  - Storage Area Network (SAN)
    - iSCSI



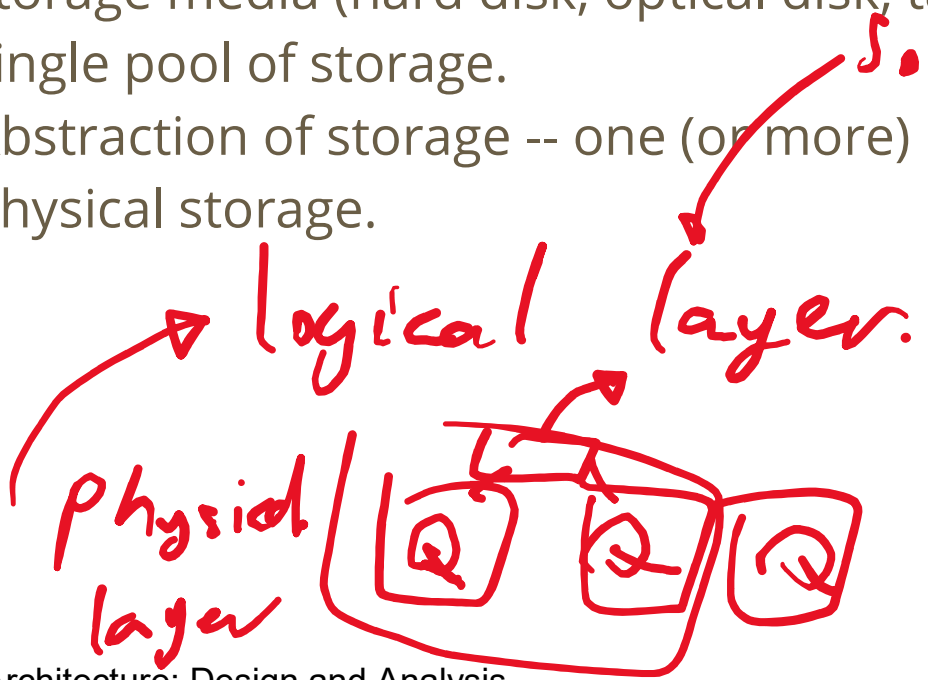
Storage Technology

- Storage Technology
  - SSD (Solid State Drive) - (Flash) Memory
  - HDD (Hard Disk Drive) - Mechanical Disk
- Interface Technology
  - Serial attached SCSI - Typically on server (SAS-3, 12.0 Gbit/s)
  - Serial ATA - (SATA 3.0, 6.0 Gbit/s)
  - PCI Express - (PCIe 3.0 ×4, 31.5 Gbit/s)
  - Fibre Channel - almost exclusively found on servers (128 Gbit/s)
  - USB - (10 Gbit/s)
  - ~~Parallel ATA - (UDMA, 1064 Mbit/s)~~
  - ~~(Parallel) SCSI - (40 Mbit/s- 2560 Mbit/s)~~



Storage Virtualization

- ★ Storage virtualization is "the process of presenting a logical view of the physical storage resources to" a host computer system, "treating all storage media (hard disk, optical disk, tape, etc.) (in the enterprise) as a single pool of storage.
- ★ Abstraction of storage -- one (or more) logical storage from several physical storage.

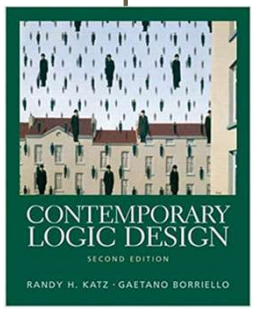


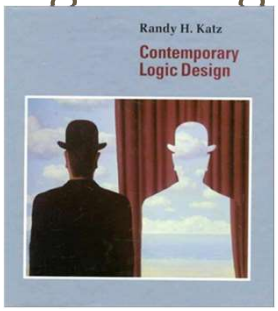


RAID

- ★ RAID (Redundant Array of Independent Disks) was originally (Redundant Array of Inexpensive Disks)
- ★ storage virtualization technology
- ★ "RAID" was invented by David Patterson, Garth A. Gibson, and Randy Katz at the University of California, Berkeley in 1987

David Patterson is the author of several Computer Architecture books. Randy Katz is the author of Contemporary Logic Design used in our Digital Computer Logic class.







RAID

- ★ In the past (60s, 70s), high performance means expensive disk (e.g. battery-backed RAM disk).
- ★ RAID allows commodity hardware to meet the performance of expensive disk.
- ★ RAID level, provides a different balance among the key goals: reliability, availability, performance, and capacity.

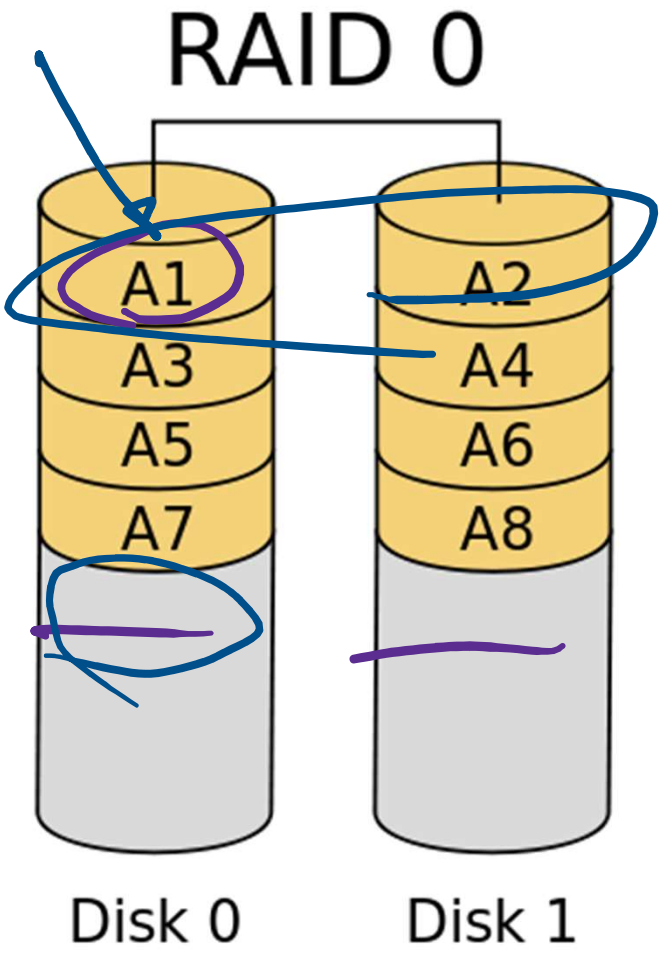




RAID 0 (Stripe)

★ Good for high performance, but low reliability



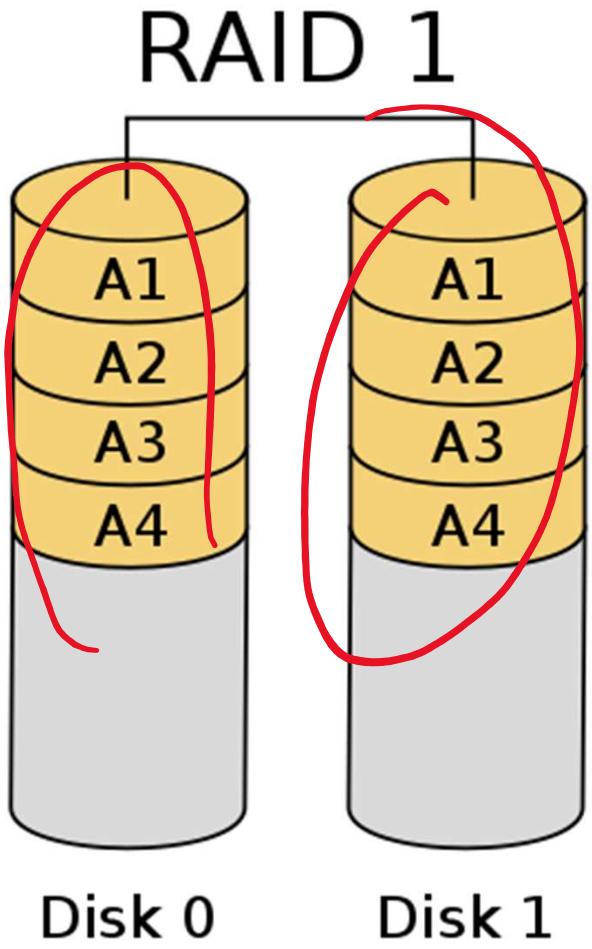


Pictures from wikipedia



RAID 1 (Mirror)

- ★ Mirror
- ★ Useful when read and reliability is important
- ★ Tolerant to failure of a disk



p^2

2 → 1

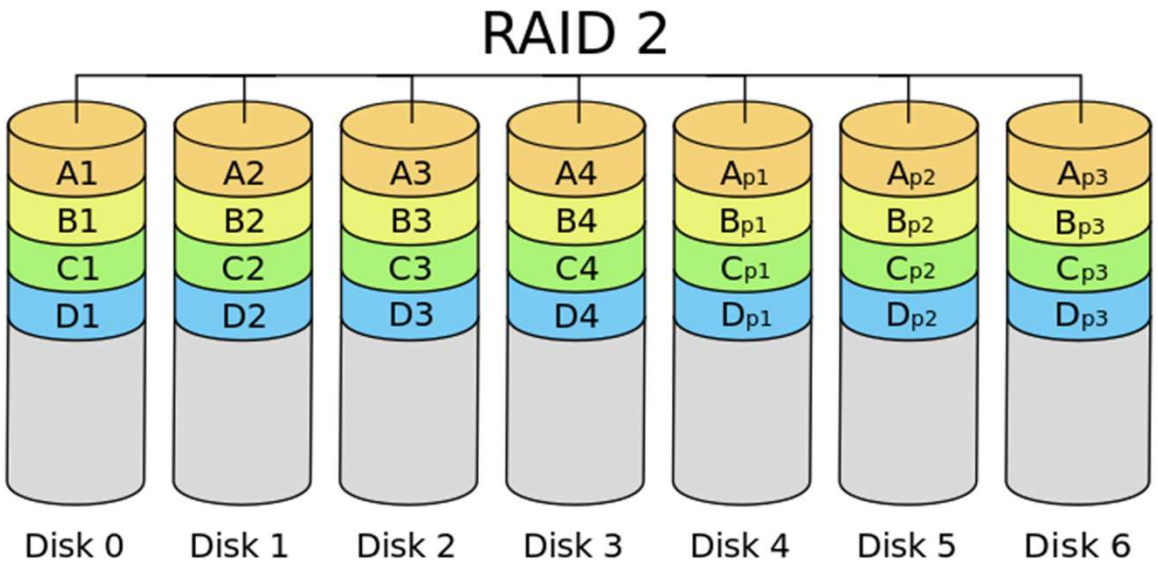
2 : 1

Pictures from wikipedia



RAID 2

- ★ Rarely used in practice
- ★ Bit-level parity
- ★ Use humming code for error correction.
- ★ Tolerant to failure of a disk

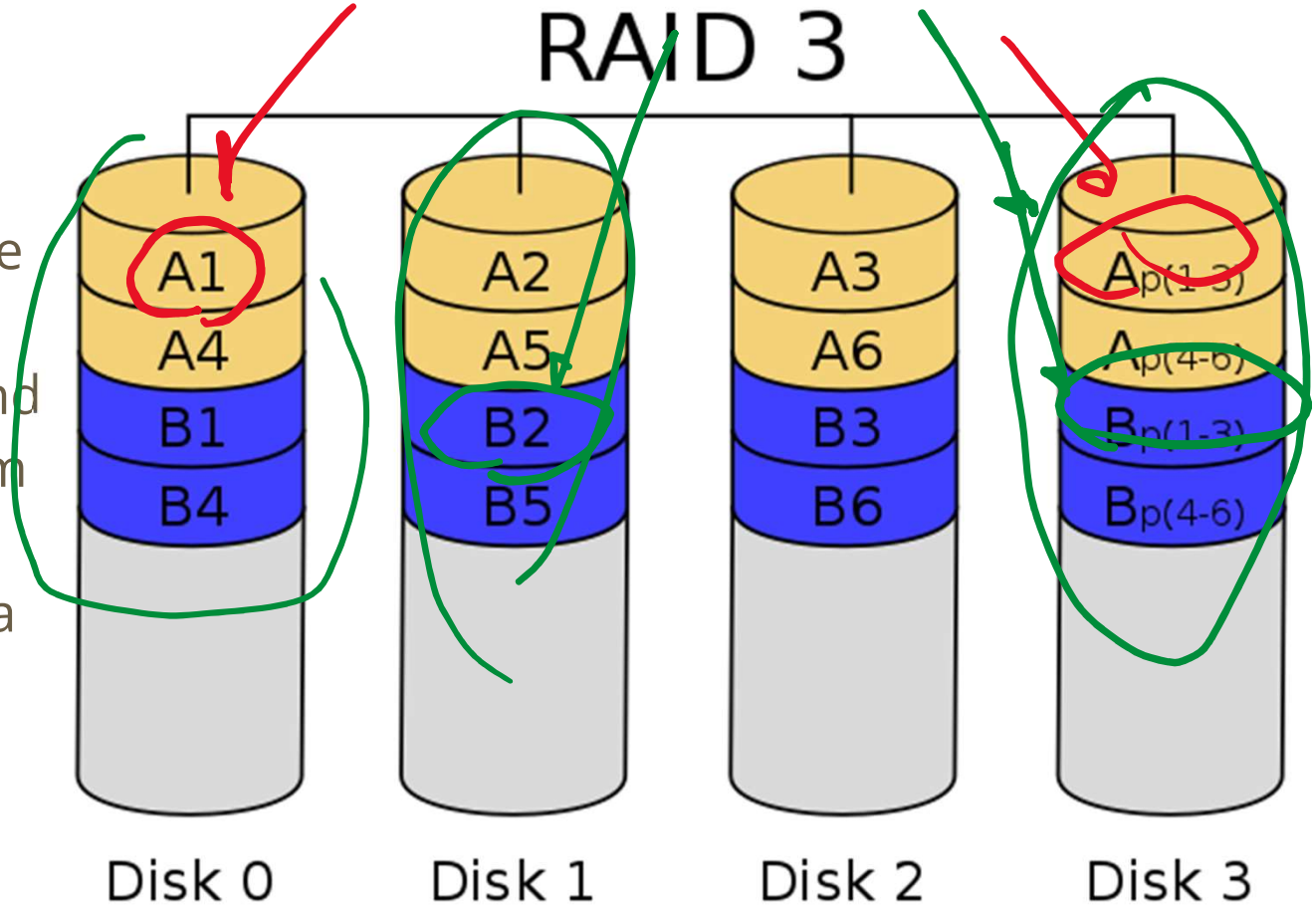


Pictures from wikipedia



RAID 3

- ★ Rarely used in practice
- ★ Byte-level parity
- ★ Good for long read and write. (Bad for random read and write)
- ★ Tolerant to failure of a disk

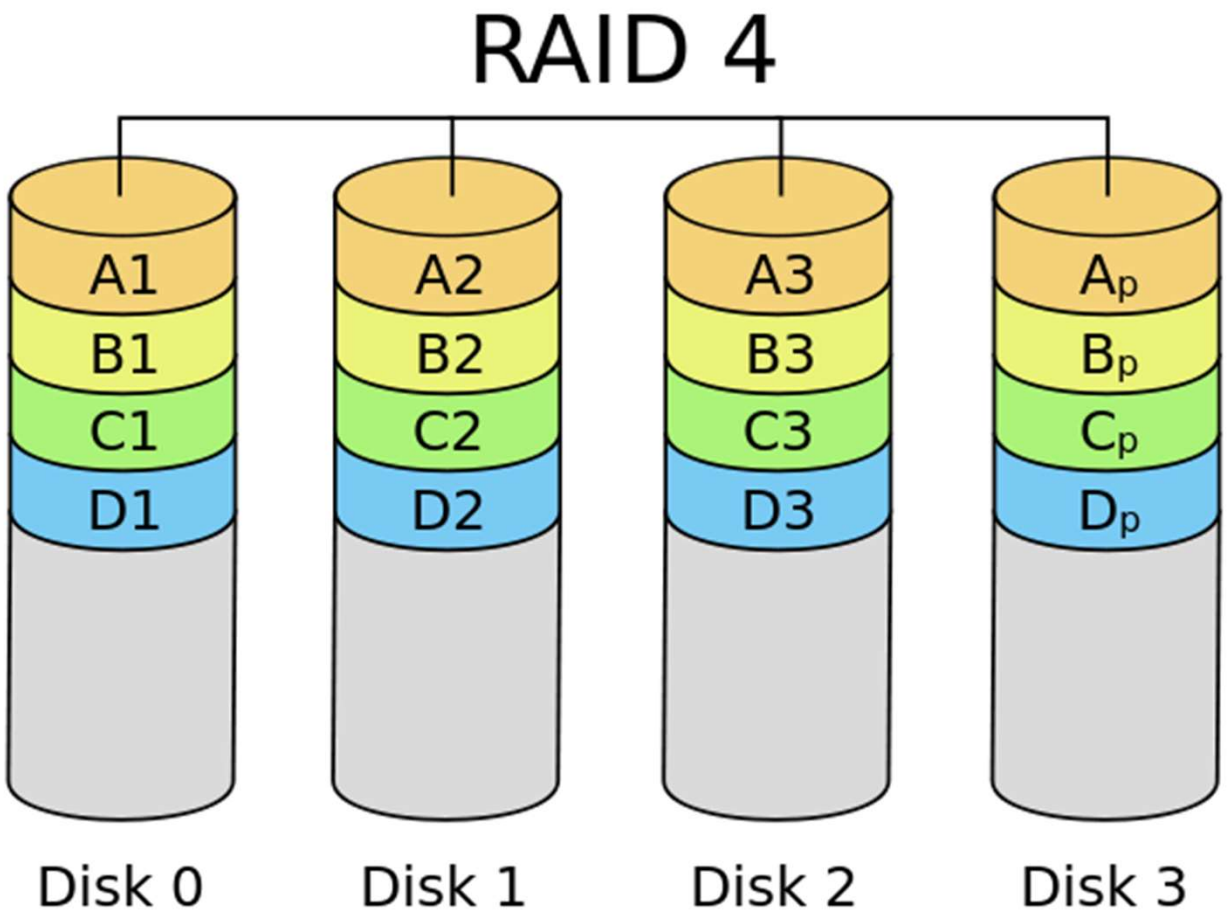


Pictures from wikipedia



RAID 4

- ★ Rarely used in practice
- ★ Block-level parity to a dedicate disk
- ★ Good read, slow write (parity disk is a bottleneck.)

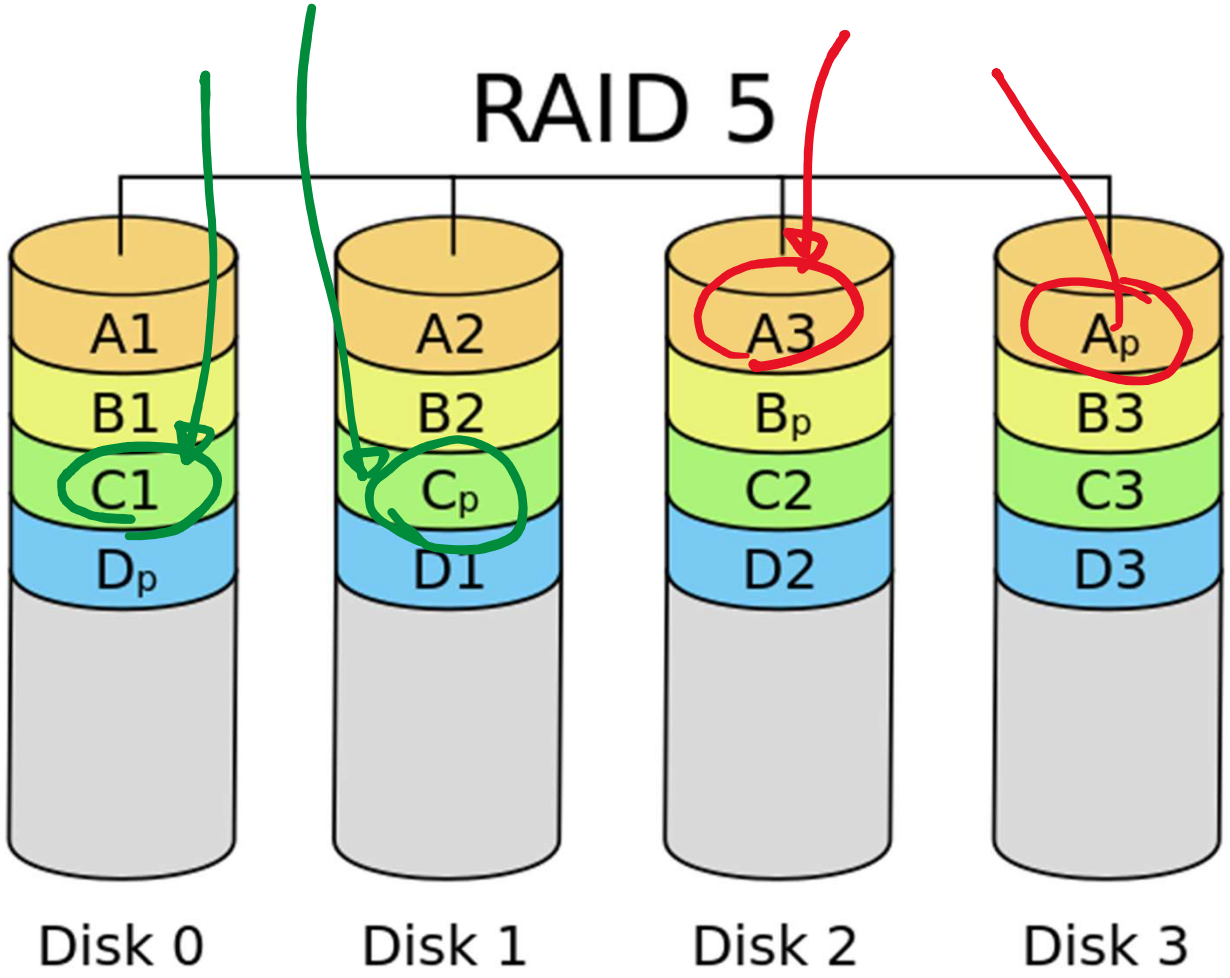


Pictures from wikipedia



RAID 5

- ★ Block-level parity
- ★ Balance between performance and redundancy
- ★ Tolerant to failure of a disk

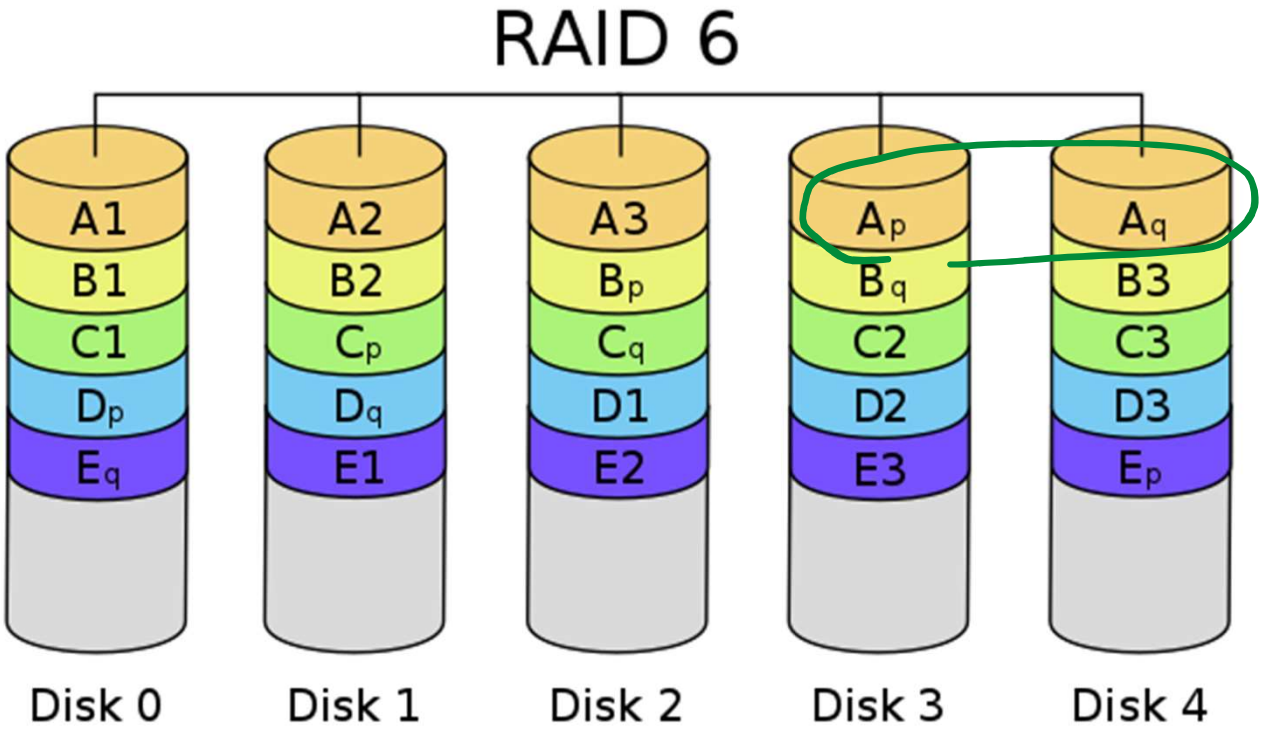


Pictures from wikipedia



RAID 6

- ★ Two block-level parities
- ★ Tolerant up to failure of 2 disks



Pictures from wikipedia



Pop Quiz

What is the minimum number of disks required for RAID 0,1, and 5?

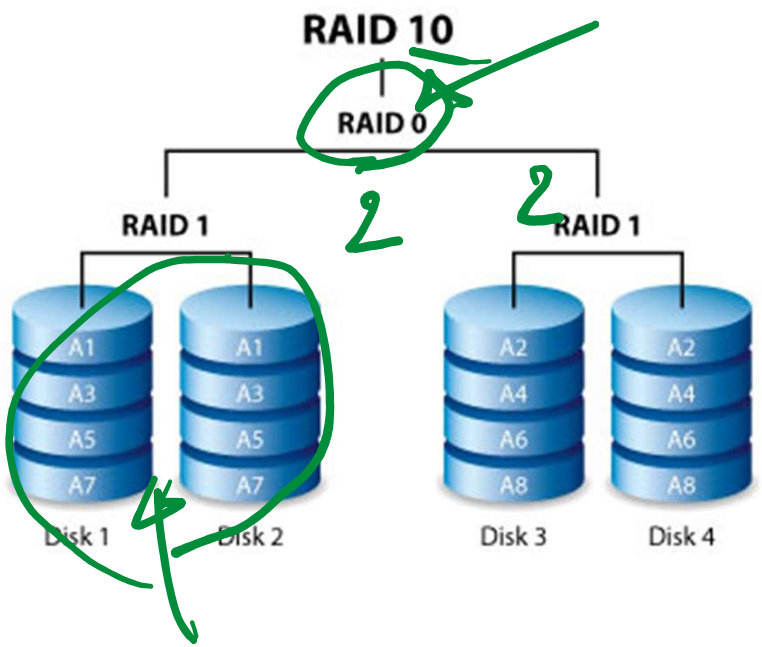
2 2 3

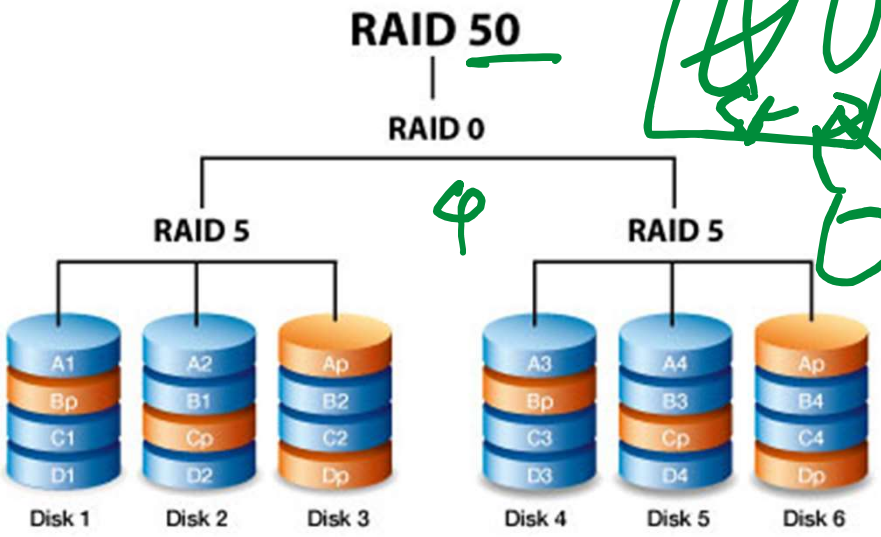


Nested RAID

Hot swapable

- Combined levels of RAID. see RAID 01, RAID 50, RAID 10



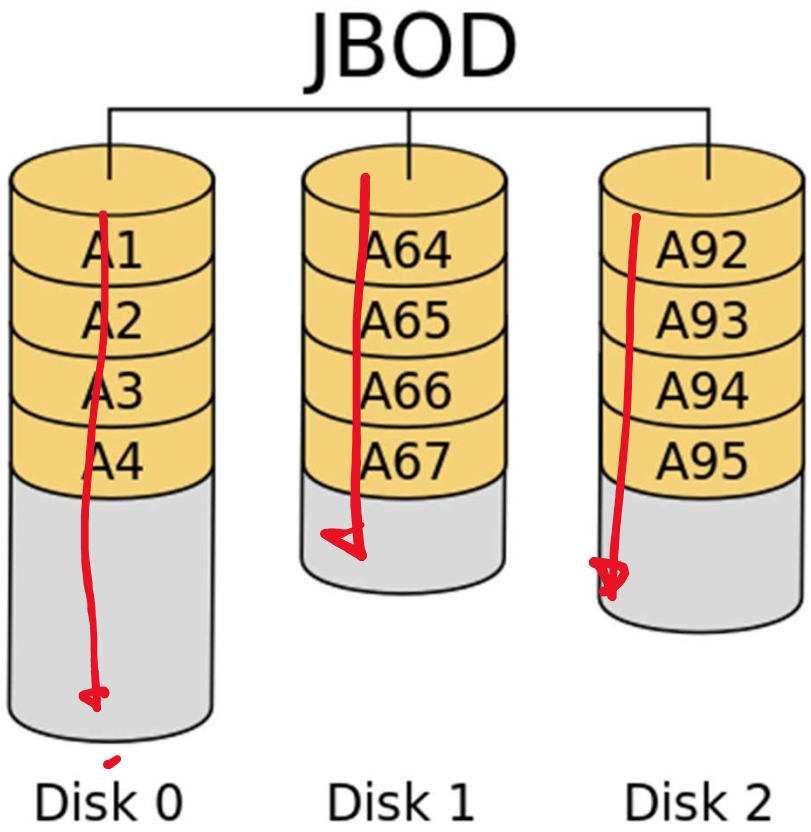


Pictures from https://www.lacie.com/as/en/manuals/lrm/raid/



Other non-RAID

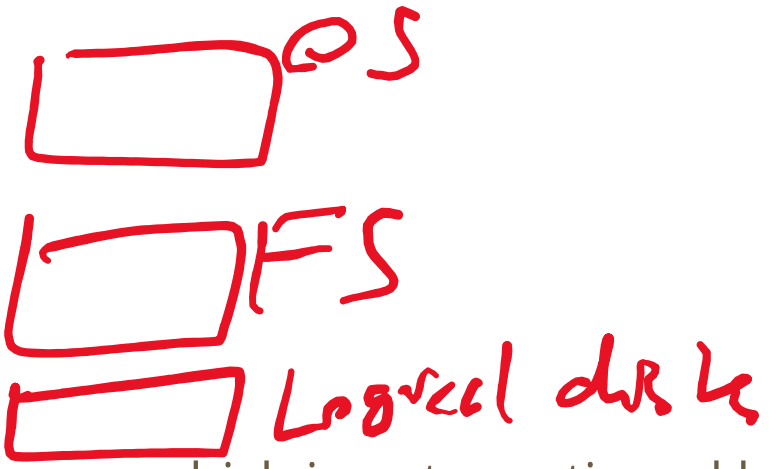
- ★ JBOD (just a bunch of disks/drives)
  - Span
  - Allows disk with different size to be combined.
- ★ Redundant?





Network Storage

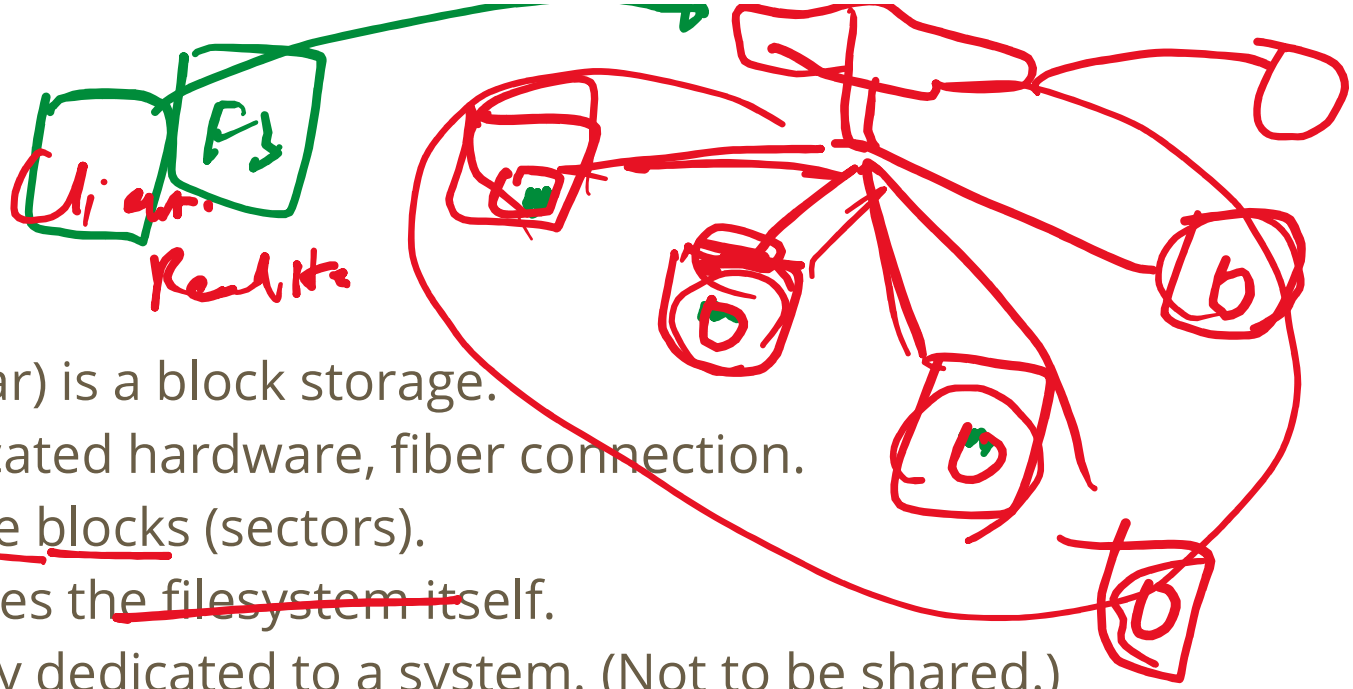
- ★ Block vs. File
- ★ SAN vs. NAS



- ★ There exists object (e.g. document) storage, which is not mentioned here.
- ★ Similar concepts on cloud services
  - Amazon Elastic Block Store vs. Amazon S3
  - Google Persistent Disk vs. Google Cloud Filestore



SAN



- ★ Storage Area Network
- ★ SAN (via iSCSI or similar) is a block storage.
- ★ Usually required dedicated hardware, fiber connection.
- ★ (Operating) System see blocks (sectors).
- ★ (Client) System manages the filesystem itself.
- ★ Block storage is usually dedicated to a system. (Not to be shared.)

- ★ There exists similar technology such as iATA and AoE. However, it is not widely used.



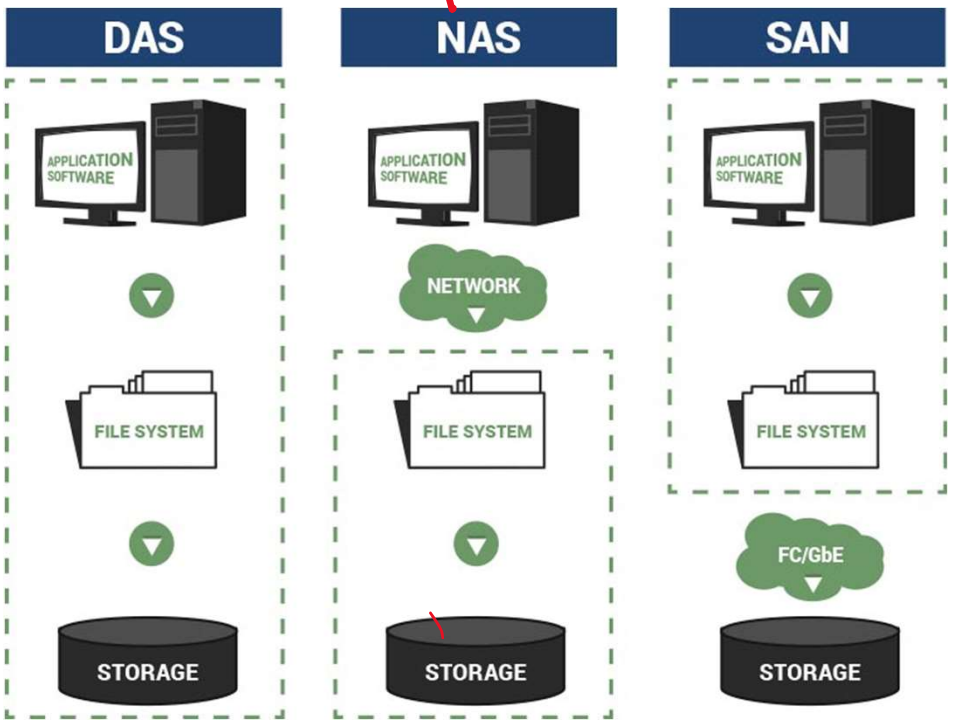
NAS

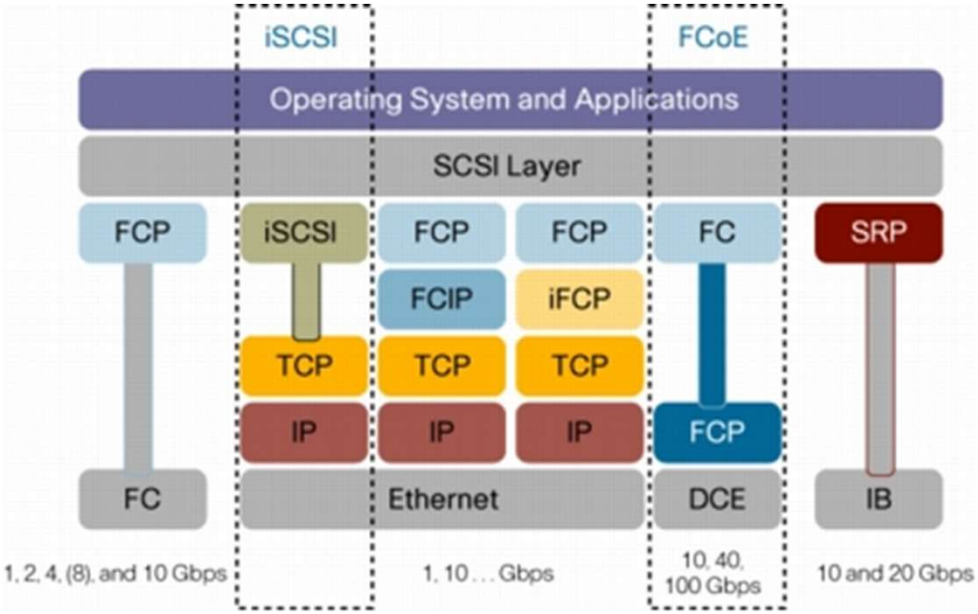
- Network Attached Storage
- NAS is a file storage.
  Think Network Drive. (e.g. Windows map drive)
- Can easily be shared among clients.
- Host (Server) manages file system (and permissions).
- Several protocols for sharing.
  - NFS
  - CIFS (aka. SMB)
  - AFP (Apple File Protocol, deprecated.)
  - FTP, SFTP
  - etc.



Comparisons

Network attached Storage
Storage area network



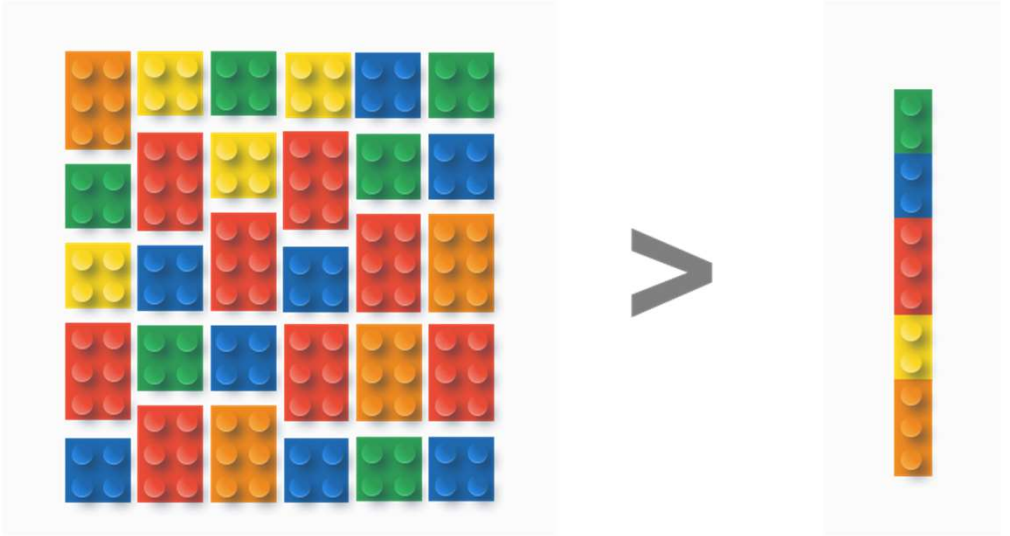


Picture from
https://sites.google.com/site/andreanguyencis4397/module-1/nas-and-san-introduction
https://www.cisco.com/c/en/us/products/collateral/switches/nexus-5000-series-switches/white_paper_c11-495142.html



Other Technologies

- ★ Deduplication
  - Several systems usually have similar files/blocks.
  - Can we share files/blocks to save (physical) spaces?



Picture from https://kaminario.com/company/blog/data-dedupe-all-flash-arrays/



Real-World Consideration

- ★ Performance
- ★ Price
- ★ Block vs. File (vs. Object)
- ★ Sharing
- ★ Reliability
- ★ Redundancy
- ★ Deduplication



Summary

- ★ Several Storage Technology
- ★ RAID allows independent disks to be used for large storage system.
- ★ SAN provides block access for clients.
- ★ NAS provides file access for clients



Exercises





Cloud Storage

- ★ Nowaday, cloud storage is available at reasonably low price. Please name type of storage (e.g. block vs. file vs. object) provided by each technology.
  - Amazon S3
  - Amazon EBS
  - Amazon Dynamo
  - Google Cloud Storage Bucket
  - Google Drive
  - One Drive
  - Dropbox
  - Apple iCloud



Raid Performance

- Why is RAID 5 faster when comparing to RAID 4?
  - Hint. Parity



Can you explain why the user experience for Google Drive is like this?

[redacted]
April 6 at 3:55 PM ·

ยิ่งใช้ Google Drive มากขึ้น ก็เริ่มรู้สึกว่า มันไม่ได้ออกแบบมาเพื่อให้เราทำงานบน browser ได้จริงๆ (อ่านคำบ่นต่อใน comments)

[redacted]. ไม่สามารถดูได้ว่าในโฟลเดอร์มีไฟล์จำนวนกี่ไฟล์
2. ไม่สามารถ drag & drop เพื่อย้ายหรือคัดลอกไฟล์ได้
3. UI ของการย้ายไฟล์ คือกล่อง pop-up อันนิดเดียว ไม่สามารถพิมพ์ path ของปลายทางได้ ต้องค่อยๆ คลิกทีละจิ๊ก
4. การดาวน์โหลดโฟลเดอร์ใหญ่ ที่มีไฟล์จำนวนมากลงมาให้เครื่องตนเองจากเว็บเบราเซอร์ ต้องลุ้นว่าไฟล์จะมาครบหรือไม่
4. URL ของโฟลเดอร์บน google drive อ่านไม่รู้เรื่อง และดูไม่ออกว่ามาจาก path ใด
5. ไม่สามารถเปิดดูโฟลเดอร์ย่อยในหน้าต่างใหม่ได้ ต้อง double click แล้วกด back กลับไปกลับมา

บ่นเพราะรักนะ Google Drive ยังไงก็ยังใช้เธอต่อไป

Like · Reply · 1w 6



End of Chapter 9

